THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms. **A.VATHANA** Reg. No:(**20524UCSC051)**, a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course Incharge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: **.Khan Academy** **.Hacker Rank** **.Code Chef**		

EX. NO:01 Creation of DDL operations using Laptops Table

DATE:21.08.2026

Aim

To create a **Laptops** table, modify its structure using ALTER, insert laptop records, display all records, and retrieve laptops priced from 50000 onwards.

Software Used

Oracle / MySQL / sqlite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects such as tables, views and indexes.

DDL COMMANDS:

- CREATE
- ALTER
- DROP
- TRUNCATE
- RENAME

CREATE: The CREATE command is used to create a new table and to define constraints on its columns.

Syntax

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
SQL> CREATE TABLE laptops (
   id INTEGER PRIMARY KEY,
   model TEXT,
   price INTEGER
);
```

Table Created

Structure of the Table

SQL> DESC laptops;

Column	Data Type	Constraint
id	INTEGER	PRIMARY KEY
model	TEXT	-
price	INTEGER	-

ALTER: The ALTER command is used to change the structure of an existing table. It can be used with ADD, MODIFY, DROP and RENAME options.

ADD: The ADD option inserts a new column with its data type into an existing table.

Syntax

ALTER TABLE table_name ADD column_name datatype;

Query

SQL> ALTER TABLE laptops ADD brand VARCHAR(50);

Table Altered

MODIFY: The MODIFY option changes the data type or increases the size of an existing column.

Syntax

ALTER TABLE table_name MODIFY column_name datatype;

Query

SQL> ALTER TABLE laptops MODIFY model VARCHAR(120);

ALTER TABLE laptops ALTER COLUMN model VARCHAR(120);

Table Altered

DROP COLUMN: This option removes a column from a table.

Syntax

ALTER TABLE table_name DROP COLUMN column_name;

Query

SQL> ALTER TABLE laptops DROP COLUMN brand;

Table Altered

RENAME: The RENAME option is used to change the name of a table or a column.

SQL> RENAME TABLE laptops TO computers;

Table Renamed

(OR)

ALTER TABLE laptops RENAME TO computers;

Column Rename

ALTER TABLE laptops RENAME COLUMN model TO model_name;

Column Renamed

TRUNCATE: The TRUNCATE command removes all the records from a table but keeps the table structure for future use.

Syntax

TRUNCATE TABLE table_name;

Query

SQL> TRUNCATE TABLE laptops;

Table Truncated

DROP: The DROP command deletes the table along with all of its data permanently.

Syntax

DROP TABLE table_name;

Query

SQL> DROP TABLE laptops;

Table Dropped

Explanation

- Create a table named **laptops**.
- Insert laptop details into the table.
- Display all laptop records.
- Retrieve laptops priced at or above **50000** and sort them by price.

Create Table

```
CREATE TABLE laptops (
    id INTEGER PRIMARY KEY,
    model TEXT,
    price INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO laptops VALUES (1, 'Inspiron 15', 52000);
INSERT INTO laptops VALUES (2, 'ThinkPad E14', 61000);
INSERT INTO laptops VALUES (3, 'Pavilion x360', 58000);
INSERT INTO laptops VALUES (4, 'VivoBook 14', 39000);
INSERT INTO laptops VALUES (5, 'MacBook Air', 92000);
INSERT INTO laptops VALUES (6, 'IdeaPad Slim 3', 35000);
```

6 rows inserted.

SQL Query

Query 1: Display all laptop records

```
SQL> SELECT * FROM laptops;
```

Resultant Query

id	model	price
1	Inspiron 15	52000
2	ThinkPad E14	61000
3	Pavilion x360	58000
4	VivoBook 14	39000
5	MacBook Air	92000
6	IdeaPad Slim 3	35000

Query 2: Display laptops priced from 50000 onwards

```
SQL> SELECT *
FROM laptops
WHERE price >= 50000
ORDER BY price;
```

Resultant Query

id	model	price
1	Inspiron 15	52000
3	Pavilion x360	58000
2	ThinkPad E14	61000
5	MacBook Air	92000

Query 3: Count the number of laptops available

SQL> SELECT COUNT(*) AS total_laptops FROM laptops;

Resultant Query

total_laptops
6

RESULT:

Thus, the Laptops table was created and altered successfully using DDL commands, records were inserted, all laptop details were displayed, and the laptops priced from 50000 onwards were retrieved successfully.

EX. NO:02 Creation of DML Operations using Players Table

DATE: 25.08.2026

Aim

To create a **Players** table, insert player records, display all records, update player details, delete a player record, and retrieve players based on their runs.

Software Used

Oracle / MySQL / sqlite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete and retrieve the data stored in database tables.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

The INSERT command is used to add new records into a table.

Syntax

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

Query

```
SQL> INSERT INTO players VALUES (1, 'Karthik', 540);
```

1 row inserted.

UPDATE

The UPDATE command is used to modify the existing records of a table.

Syntax

```
UPDATE table_name
SET column_name = value
WHERE condition;
```

Query

```
SQL> UPDATE players
```

SET runs = 610

WHERE id = 1;

1 row updated.

DELETE

The DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM players

WHERE id = 4;

1 row deleted.

SELECT

The SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM players;

Explanation

- Create a table named **players**.
- Insert player details into the table.
- Display all player records.
- Update the runs scored by a player.
- Delete a player record.
- Retrieve players who scored **500 runs or more** and sort them by runs.

Create Table

```
CREATE TABLE players (
    id INTEGER PRIMARY KEY,
    name TEXT,
    runs INTEGER
);
```

Table Created

Sample Data

INSERT INTO players VALUES (1, 'Karthik', 540);

INSERT INTO players VALUES (2, 'Lakshmi', 480);

INSERT INTO players VALUES (3, 'Vignesh', 655);

INSERT INTO players VALUES (4, 'Janani', 390);

INSERT INTO players VALUES (5, 'Harish', 520);

INSERT INTO players VALUES (6, 'Nandhini', 710);

Records Inserted

SQL Query

Query 1: Display all player records

SQL> SELECT * FROM players;

Resultant Query

id	name	runs
1	Karthik	540
2	Lakshmi	480
3	Vignesh	655
4	Janani	390
5	Harish	520
6	Nandhini	710

Query 2: Update the runs of Karthik

SQL> UPDATE players

SET runs = 610

WHERE id = 1;

1 row updated.

Updated Record

id	name	runs
1	Karthik	610

Query 3: Delete Janani's record

SQL> DELETE FROM players

WHERE id = 4;

1 row deleted.

Remaining Records

id	name	runs
1	Karthik	610
2	Lakshmi	480
3	Vignesh	655
5	Harish	520
6	Nandhini	710

Query 4: Display players who scored 500 runs or more

SQL> SELECT *

FROM players

WHERE runs >= 500

ORDER BY runs;

Resultant Query

id	name	runs
5	Harish	520
1	Karthik	610
3	Vignesh	655
6	Nandhini	710

Query 5: Insert a new player and display the total number of players

SQL> INSERT INTO players VALUES (7, 'Surya', 575);

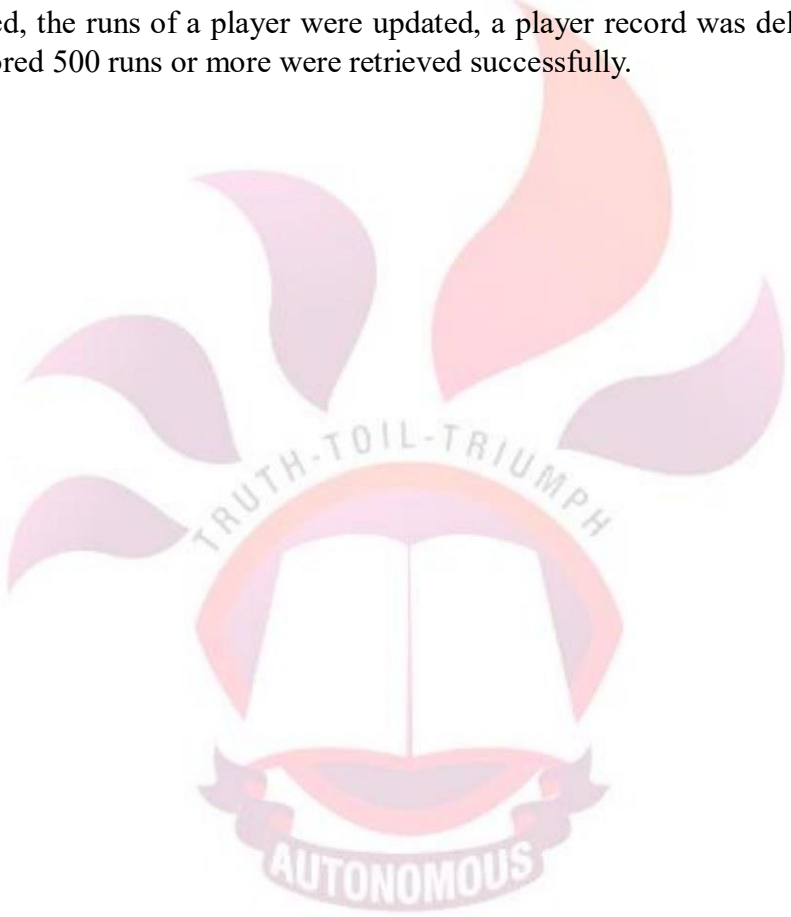
SQL> SELECT COUNT(*) AS total_players FROM players;

1 row inserted.

total_players
6

RESULT:

Thus, the Players table was created successfully, records were inserted, player details were displayed, the runs of a player were updated, a player record was deleted, and the players who scored 500 runs or more were retrieved successfully.



EX. NO:03 Creation of DQL Operations using Staff Table

DATE: 27.08.2026

Aim

To create a **Staff** table, insert staff records, and retrieve data using the SELECT, WHERE, ORDER BY, GROUP BY and HAVING clauses.

Software Used

Oracle / MySQL / sqlite3

Description

DQL (Data Query Language) is used to retrieve data from database tables. The SELECT statement is the main DQL command, and it can be combined with several clauses to filter, sort and group the data.

DQL CLAUSES:

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING

SELECT

The SELECT statement is used to fetch the required columns from a table.

Syntax

```
SELECT column1, column2
FROM table_name;
```

WHERE

The WHERE clause is used to filter the records that satisfy a given condition.

Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

ORDER BY

The ORDER BY clause is used to sort the result in ascending (ASC) or descending (DESC) order.

Syntax

```
SELECT column1, column2
FROM table_name
ORDER BY column1 DESC;
```

GROUP BY

The GROUP BY clause groups the rows that have the same values and is normally used with aggregate functions.

Syntax

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1;
```

HAVING

The HAVING clause is used to filter the groups created by the GROUP BY clause.

Syntax

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1
HAVING condition;
```

Explanation

- Create a table named **staff**.
- Insert staff details into the table.
- Display all staff records.
- Display the staff of a particular department using WHERE.
- Sort the staff by salary using ORDER BY.
- Find the total salary of every department using GROUP BY.
- Display the departments whose total salary is above 100000 using HAVING.

Create Table

```
CREATE TABLE staff (
    id INTEGER PRIMARY KEY,
```

```
name TEXT,
    dept TEXT,
    salary INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO staff VALUES (1, 'Selvi', 'Accounts', 42000);
INSERT INTO staff VALUES (2, 'Mohan', 'Sales', 38000);
INSERT INTO staff VALUES (3, 'Pavithra', 'Accounts', 47000);
INSERT INTO staff VALUES (4, 'Dinesh', 'Technical', 65000);
INSERT INTO staff VALUES (5, 'Revathi', 'Sales', 41000);
INSERT INTO staff VALUES (6, 'Gokul', 'Technical', 72000);
INSERT INTO staff VALUES (7, 'Sangeetha', 'Support', 30000);
INSERT INTO staff VALUES (8, 'Bala', 'Technical', 58000);
```

Records Inserted

SQL Query

Query 1: Display all staff records

```
SQL> SELECT * FROM staff;
```

Resultant Query

id	name	dept	salary
1	Selvi	Accounts	42000
2	Mohan	Sales	38000
3	Pavithra	Accounts	47000
4	Dinesh	Technical	65000
5	Revathi	Sales	41000
6	Gokul	Technical	72000
7	Sangeetha	Support	30000
8	Bala	Technical	58000

Query 2: Display only the name and department of every staff member

SQL> SELECT name, dept FROM staff;

Resultant Query

name	dept
Selvi	Accounts
Mohan	Sales
Pavithra	Accounts
Dinesh	Technical
Revathi	Sales
Gokul	Technical
Sangeetha	Support
Bala	Technical

Query 3: Display the staff of the Technical department (WHERE)

SQL> SELECT *

FROM staff

WHERE dept = 'Technical';

Resultant Query

id	name	dept	salary
4	Dinesh	Technical	65000
6	Gokul	Technical	72000
8	Bala	Technical	58000

Query 4: Display staff with salary of 40000 or more in descending order of salary (ORDER BY)

SQL> SELECT name, salary

FROM staff

WHERE salary >= 40000

ORDER BY salary DESC;

Resultant Query

name	salary
Gokul	72000
Dinesh	65000
Bala	58000
Pavithra	47000
Selvi	42000
Revathi	41000

Query 5: Display the total salary of each department (GROUP BY)

SQL> SELECT dept, SUM(salary) AS total_salary

FROM staff

GROUP BY dept;

Resultant Query

dept	total_salary
Accounts	89000
Sales	79000
Support	30000
Technical	195000

Query 6: Display the number of staff in each department (GROUP BY)

SQL> SELECT dept, COUNT(*) AS staff_count

FROM staff

GROUP BY dept;

Resultant Query

dept	staff_count
Accounts	2
Sales	2
Support	1
Technical	3

Query 7: Display departments whose total salary is above 100000 (HAVING)

SQL> SELECT dept, SUM(salary) AS total_salary

FROM staff

GROUP BY dept

HAVING SUM(salary) > 100000;

Resultant Query

dept	total_salary
Technical	195000

Query 8: Display the average salary of departments having more than one staff member

SQL> SELECT dept, AVG(salary) AS avg_salary

FROM staff

GROUP BY dept

HAVING COUNT(*) > 1

ORDER BY avg_salary DESC;

Resultant Query

dept	avg_salary
Technical	65000
Accounts	44500
Sales	39500

RESULT:

Thus, the Staff table was created successfully and the records were retrieved using the SELECT, WHERE, ORDER BY, GROUP BY and HAVING clauses.

EX. NO:04 Creation of Transaction Operations using Wallets Table

DATE: 31.08.2026

Aim

To create a **Wallets** table and to perform transaction control operations using the COMMIT, ROLLBACK and SAVEPOINT commands.

Software Used

Oracle SQL / SQL Command Line

Description

A **transaction** is a group of SQL statements that is executed as a single unit of work. Transaction Control Language (TCL) commands are used to save or undo the changes made by DML statements.

TRANSACTION COMMANDS:

- COMMIT
- ROLLBACK
- SAVEPOINT
- ROLLBACK TO SAVEPOINT

COMMIT

The COMMIT command saves all the changes made in the current transaction permanently in the database.

Syntax

COMMIT;

Query

SQL> UPDATE wallets

SET balance = 6000

WHERE wallet_id = 11;

SQL> COMMIT;

Commit complete.

ROLLBACK

The ROLLBACK command undoes all the changes made after the last COMMIT.

Syntax

ROLLBACK;

Query

SQL> DELETE FROM wallets

WHERE wallet_id = 15;

SQL> ROLLBACK;

Rollback complete.

SAVEPOINT

The SAVEPOINT command marks a point inside a transaction to which the transaction can be rolled back later.

Syntax

SAVEPOINT savepoint_name;

Query

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

This command undoes only the changes made after the specified savepoint.

Syntax

ROLLBACK TO savepoint_name;

Query

SQL> ROLLBACK TO sp1;

Rollback complete.

Explanation

- Create a table named **wallets**.
- Insert wallet details and make them permanent using COMMIT.
- Update a wallet balance and save it using COMMIT.
- Create a SAVEPOINT and update another wallet balance.
- Undo the second update using ROLLBACK TO SAVEPOINT.
- Undo an uncommitted change using ROLLBACK.

Create Table

CREATE TABLE wallets (

wallet_id NUMBER PRIMARY KEY,

holder_name VARCHAR2(50),

balance NUMBER

);

Table Created

Sample Data

INSERT INTO wallets VALUES (11, 'Karthika', 5000);

INSERT INTO wallets VALUES (12, 'Suresh', 7500);

INSERT INTO wallets VALUES (13, 'Yamini', 3200);

INSERT INTO wallets VALUES (14, 'Ganesh', 9000);

INSERT INTO wallets VALUES (15, 'Deepa', 4100);

Records Inserted

SQL Query

Query 1: Display all wallet records

SQL> SELECT * FROM wallets;

Resultant Query

wallet_id	holder_name	balance
11	Karthika	5000
12	Suresh	7500
13	Yamini	3200
14	Ganesh	9000
15	Deepa	4100

Query 2: Commit the inserted records

SQL> COMMIT;

Commit complete.

Query 3: Update Karthika's wallet balance

SQL> UPDATE wallets

SET balance = 6000

WHERE wallet_id = 11;

1 row updated.

Updated Record

wallet_id	holder_name	balance
11	Karthika	6000

SQL> COMMIT;

Commit complete.

Query 4: Create a SAVEPOINT

SQL> SAVEPOINT before_suresh;

Savepoint created.

Query 5: Update Suresh's wallet balance

SQL> UPDATE wallets

SET balance = 8800

WHERE wallet_id = 12;

1 row updated.

Updated Record

wallet_id	holder_name	balance
12	Suresh	8800

Query 6: Rollback to SAVEPOINT

SQL> ROLLBACK TO before_suresh;

Rollback complete.

Resultant Query

SQL> SELECT * FROM wallets WHERE wallet_id = 12;

wallet_id	holder_name	balance
12	Suresh	7500

The balance of Suresh is restored to 7500 because the update was made after the savepoint, while the committed update of Karthika is not affected.

Query 7: Rollback an uncommitted change

SQL> DELETE FROM wallets

WHERE wallet_id = 15;

1 row deleted.

SQL> ROLLBACK;

Rollback complete.

Resultant Query

SQL> SELECT * FROM wallets;

wallet_id	holder_name	balance
11	Karthika	6000
12	Suresh	7500
13	Yamini	3200
14	Ganesh	9000
15	Deepa	4100

Transaction Properties (ACID)

Property	Description
Atomicity	All the statements of a transaction are executed or none of them is executed
Consistency	A transaction moves the database from one valid state to another valid state
Isolation	Concurrent transactions do not interfere with each other
Durability	Committed changes remain permanent even after a system failure

RESULT:

Thus, the Wallets table was created successfully, and the transaction control commands COMMIT, ROLLBACK, SAVEPOINT and ROLLBACK TO SAVEPOINT were executed successfully.

EX. NO:05 Creation of join operations using Patients and Doctors Table

DATE: 01.09.2026

Aim

To create the **Patients** and **Doctors** tables, insert records into them, and combine the data using INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN and SELF JOIN.

Software Used

Oracle / MySQL / sqlite3

Description

A **JOIN** is used to combine the rows of two or more tables based on a related column between them.

JOIN COMMANDS:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- SELF JOIN

INNER JOIN

INNER JOIN returns only the rows that have matching values in both tables.

Syntax

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

LEFT JOIN

LEFT JOIN returns all the rows from the left table and the matching rows from the right table. If there is no match, NULL is displayed for the right table.

Syntax

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

RIGHT JOIN

RIGHT JOIN returns all the rows from the right table and the matching rows from the left table. If there is no match, NULL is displayed for the left table.

Syntax

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

FULL JOIN

FULL JOIN returns all the rows from both tables, whether or not a match exists.

Syntax

```
SELECT columns
FROM table1
FULL JOIN table2
ON table1.column = table2.column;
```

SELF JOIN

SELF JOIN is used to join a table with itself by using two different aliases.

Syntax

```
SELECT a.column, b.column
FROM table_name a
JOIN table_name b
ON a.column = b.column;
```

Explanation

- Create the tables named **doctors** and **patients**.
- Insert doctor and patient details into the tables.
- Display the patients along with their doctors using INNER JOIN.
- Display all patients using LEFT JOIN.
- Display all doctors using RIGHT JOIN.
- Display all patients and all doctors using FULL JOIN.
- Display each team member with the team leader using SELF JOIN.

Create Tables

```
CREATE TABLE doctors (
```

```
  doctor_id INTEGER PRIMARY KEY,
    doctor_name VARCHAR(50),
    speciality VARCHAR(50)
);

CREATE TABLE patients (
    patient_id INTEGER PRIMARY KEY,
    patient_name VARCHAR(50),
    doctor_id INTEGER
);
```

Tables Created

Sample Data

```
INSERT INTO doctors VALUES (1, 'Dr. Ramesh', 'Cardiology');
INSERT INTO doctors VALUES (2, 'Dr. Anitha', 'Neurology');
INSERT INTO doctors VALUES (3, 'Dr. Suresh', 'Orthopedics');
INSERT INTO doctors VALUES (4, 'Dr. Kavitha', 'Pediatrics');

INSERT INTO patients VALUES (101, 'Muthu', 1);
INSERT INTO patients VALUES (102, 'Aarthi', 2);
INSERT INTO patients VALUES (103, 'Bharath', 1);
INSERT INTO patients VALUES (104, 'Chitra', 3);
INSERT INTO patients VALUES (105, 'Dhanush', NULL);
```

Records Inserted

SQL Query

Doctors Table

doctor_id	doctor_name	speciality
1	Dr. Ramesh	Cardiology
2	Dr. Anitha	Neurology
3	Dr. Suresh	Orthopedics
4	Dr. Kavitha	Pediatrics

Patients Table

patient_id	patient_name	doctor_id
101	Muthu	1
102	Aarthi	2
103	Bharath	1
104	Chitra	3
105	Dhanush	NULL

Query 1: Inner Join

```
SQL> SELECT p.patient_id, p.patient_name, d.doctor_name, d.speciality
FROM patients p
INNER JOIN doctors d
ON p.doctor_id = d.doctor_id;
```

Resultant Query

patient_id	patient_name	doctor_name	speciality
101	Muthu	Dr. Ramesh	Cardiology
102	Aarthi	Dr. Anitha	Neurology
103	Bharath	Dr. Ramesh	Cardiology
104	Chitra	Dr. Suresh	Orthopedics

Dhanush is not displayed because no doctor is assigned, and Dr. Kavitha is not displayed because she has no patients.

Query 2: Left Join

SQL> SELECT p.patient_id, p.patient_name, d.doctor_name

FROM patients p

LEFT JOIN doctors d

ON p.doctor_id = d.doctor_id;

Resultant Query

patient_id	patient_name	doctor_name
101	Muthu	Dr. Ramesh
102	Aarthi	Dr. Anitha
103	Bharath	Dr. Ramesh
104	Chitra	Dr. Suresh
105	Dhanush	NULL

Query 3: Right Join

SQL> SELECT p.patient_name, d.doctor_id, d.doctor_name

FROM patients p

RIGHT JOIN doctors d

ON p.doctor_id = d.doctor_id;

Resultant Query

patient_name	doctor_id	doctor_name
Muthu	1	Dr. Ramesh
Bharath	1	Dr. Ramesh
Aarthi	2	Dr. Anitha
Chitra	3	Dr. Suresh
NULL	4	Dr. Kavitha

Query 4: Full Join

SQL> SELECT p.patient_name, d.doctor_name

FROM patients p

FULL JOIN doctors d

ON p.doctor_id = d.doctor_id;

Resultant Query

patient_name	doctor_name
Muthu	Dr. Ramesh
Aarthi	Dr. Anitha
Bharath	Dr. Ramesh
Chitra	Dr. Suresh
Dhanush	NULL
NULL	Dr. Kavitha

Query 5: Self Join

A separate table named team_members is created, in which every member stores the ID of his or her team leader.

Create Table

```
CREATE TABLE team_members (
    member_id INTEGER PRIMARY KEY,
    member_name VARCHAR(50),
    leader_id INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO team_members VALUES (1, 'Arjun', NULL);
INSERT INTO team_members VALUES (2, 'Bhuvana', 1);
INSERT INTO team_members VALUES (3, 'Charan', 1);
INSERT INTO team_members VALUES (4, 'Devi', 2);
INSERT INTO team_members VALUES (5, 'Elango', 2);
```

Records Inserted

```
SQL> SELECT m.member_name AS member, l.member_name AS leader
```

FROM team_members m

JOIN team_members l

ON m.leader_id = l.member_id;

Resultant Query

member	leader
Bhuvana	Arjun
Charan	Arjun
Devi	Bhuvana
Elango	Bhuvana

Query 6: Count the patients of each doctor

SQL> SELECT d.doctor_name, COUNT(p.patient_id) AS total_patients

FROM doctors d

LEFT JOIN patients p

ON d.doctor_id = p.doctor_id

GROUP BY d.doctor_name;

Resultant Query

doctor_name	total_patients
Dr. Anitha	1
Dr. Kavitha	0
Dr. Ramesh	2
Dr. Suresh	1

RESULT:

Thus, the Doctors and Patients tables were created successfully, and the INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN and SELF JOIN operations were executed successfully.

EX. NO:06 Creation of SQL functions using Items Table

DATE: 02.09.2026

Aim

To create an **Items** table and to apply aggregate functions, string functions and date functions on its records.

Software Used

Oracle SQL / SQL Command Line

Description

SQL functions are built-in routines that perform a specific operation on the data and return a result. They make it easier to process and present the data.

SQL FUNCTION TYPES:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions work on a group of rows and return a single value.

Function	Description
COUNT()	Returns the number of rows
SUM()	Returns the total of a column
AVG()	Returns the average of a column
MAX()	Returns the highest value
MIN()	Returns the lowest value

STRING FUNCTIONS

String functions are used to process character data.

Function	Description
UPPER()	Converts text to uppercase
LOWER()	Converts text to lowercase
LENGTH()	Returns the number of characters
SUBSTR()	Extracts part of a string
INITCAP()	Converts the first letter of every word to uppercase

DATE FUNCTIONS

Date functions are used to process date values.

Function	Description
SYSDATE	Returns the current system date
ADD_MONTHS()	Adds the given number of months to a date
MONTHS_BETWEEN( )	Returns the number of months between two dates
TO_CHAR()	Converts a date into a formatted string
LAST_DAY()	Returns the last day of the month

Explanation

- Create a table named **items**.
- Insert item details into the table.
- Apply aggregate functions to find the count, total, average, maximum and minimum price.
- Apply string functions to display the item names in different forms.
- Apply date functions to work with the purchase dates.

Create Table

```
CREATE TABLE items (
    item_id NUMBER PRIMARY KEY,
    item_name VARCHAR2(50),
    price NUMBER,
    quantity NUMBER,
    purchase_date DATE
);
```

Table Created

Sample Data

```
INSERT INTO items VALUES (1, 'Basmati Rice', 120, 10, DATE '2026-01-12');
INSERT INTO items VALUES (2, 'Sunflower Oil', 165, 6, DATE '2026-02-03');
INSERT INTO items VALUES (3, 'Toor Dal', 142, 8, DATE '2026-02-20');
INSERT INTO items VALUES (4, 'Tea Powder', 310, 3, DATE '2026-03-14');
INSERT INTO items VALUES (5, 'Sugar', 46, 15, DATE '2026-04-02');
```

INSERT INTO items VALUES (6, 'Wheat Flour', 58, 12, DATE '2026-04-25');

COMMIT;

Records Inserted

SQL Query

Query 1: Display all item records

SQL> SELECT * FROM items;

Resultant Query

item_id	item_name	price	quantity	purchase_date
1	Basmati Rice	120	10	12-JAN-26
2	Sunflower Oil	165	6	03-FEB-26
3	Toor Dal	142	8	20-FEB-26
4	Tea Powder	310	3	14-MAR-26
5	Sugar	46	15	02-APR-26
6	Wheat Flour	58	12	25-APR-26

AGGREGATE FUNCTIONS

Query 2: Count the total number of items

SQL> SELECT COUNT(*) AS total_items FROM items;

Resultant Query

total_items
6

Query 3: Calculate the total price of all items

SQL> SELECT SUM(price) AS total_price FROM items;

Resultant Query

total_price
841

Query 4: Calculate the average price of items

SQL> SELECT ROUND(AVG(price), 2) AS average_price FROM items;

Resultant Query

average_price
140.17

Query 5: Find the highest and lowest item price

SQL> SELECT MAX(price) AS highest_price,

MIN(price) AS lowest_price

FROM items;

Resultant Query

highest_price	lowest_price
310	46

Query 6: Calculate the total stock value of all items

SQL> SELECT SUM(price * quantity) AS stock_value FROM items;

Resultant Query

stock_value
5642

STRING FUNCTIONS

Query 7: Display item names in uppercase

SQL> SELECT item_name, UPPER(item_name) AS upper_name

FROM items;

Resultant Query

item_name	upper_name
Basmati Rice	BASMATI RICE
Sunflower Oil	SUNFLOWER OIL
Toor Dal	TOOR DAL
Tea Powder	TEA POWDER
Sugar	SUGAR
Wheat Flour	WHEAT FLOUR

Query 8: Display item names in lowercase

SQL> SELECT item_name, LOWER(item_name) AS lower_name

FROM items;

Resultant Query

item_name	lower_name
Basmati Rice	basmati rice
Sunflower Oil	sunflower oil
Toor Dal	toor dal
Tea Powder	tea powder
Sugar	sugar
Wheat Flour	wheat flour

Query 9: Display the length of each item name

SQL> SELECT item_name, LENGTH(item_name) AS name_length

FROM items;

Resultant Query

item_name	name_length
Basmati Rice	12
Sunflower Oil	13
Toor Dal	8
Tea Powder	10
Sugar	5
Wheat Flour	11

Query 10: Display the first three characters of each item name

SQL> SELECT item_name, SUBSTR(item_name, 1, 3) AS short_name

FROM items;

Resultant Query

item_name	short_name
Basmati Rice	Bas
Sunflower Oil	Sun
Toor Dal	Too
Tea Powder	Tea
Sugar	Sug
Wheat Flour	Whe

DATE FUNCTIONS

Query 11: Display the current date

SQL> SELECT SYSDATE FROM dual;

Resultant Query

SYSDATE
04-OCT-26

Query 12: Display the purchase date in a formatted style

SQL> SELECT item_name,

TO_CHAR(purchase_date, 'DD-MON-YYYY') AS purchased_on

FROM items;

Resultant Query

item_name	purchased_on
Basmati Rice	12-JAN-2026
Sunflower Oil	03-FEB-2026
Toor Dal	20-FEB-2026
Tea Powder	14-MAR-2026
Sugar	02-APR-2026
Wheat Flour	25-APR-2026

Query 13: Display the date after six months of purchase

SQL> SELECT item_name,

ADD_MONTHS(purchase_date, 6) AS review_date

FROM items;

Resultant Query

item_name	review_date
Basmati Rice	12-JUL-26
Sunflower Oil	03-AUG-26
Toor Dal	20-AUG-26
Tea Powder	14-SEP-26
Sugar	02-OCT-26
Wheat Flour	25-OCT-26

Query 14: Display items purchased after March 2026

SQL> SELECT item_name, price

FROM items

WHERE purchase_date > DATE '2026-03-31';

Resultant Query

item_name	price
Sugar	46
Wheat Flour	58

RESULT:

Thus, the Items table was created successfully, and the aggregate functions, string functions and date functions were executed successfully.

EX. NO:07 Creation of Subqueries and Nested Queries using Faculty Table

DATE: 07.09.2026

Aim

To create a **Faculty** table and to retrieve data using single row subqueries, multiple row subqueries and nested subqueries.

Software Used

Oracle / MySQL / sqlite3

Description

A **subquery** is a query written inside another query. The inner query is executed first and its result is used by the outer query. Subqueries are written inside parentheses.

SUBQUERY TYPES:

- Single Row Subquery
- Multiple Row Subquery
- Nested Subquery

SINGLE ROW SUBQUERY

A single row subquery returns only one row, so it is used with comparison operators such as =, >, <, >= and <=.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name > (SELECT column_name
                     FROM table_name
                     WHERE condition);
```

MULTIPLE ROW SUBQUERY

A multiple row subquery returns more than one row, so it is used with operators such as IN, ANY and ALL.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name IN (SELECT column_name
                      FROM table_name
```

WHERE condition);

NESTED SUBQUERY

A nested subquery is a subquery placed inside another subquery.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT MAX(column_name)
                     FROM table_name
                     WHERE column_name < (SELECT MAX(column_name)
                                          FROM table_name));
```

Explanation

- Create a table named **faculty**.
- Insert faculty details into the table.
- Display the faculty whose salary is greater than the average salary.
- Display the faculty who belong to the department of the highest paid members.
- Display the faculty who earn the highest salary.
- Display the faculty who earn more than a particular faculty.
- Find the second highest salary using a nested subquery.

Create Table

```
CREATE TABLE faculty (
   fid INTEGER PRIMARY KEY,
   fname VARCHAR(50),
   dept VARCHAR(30),
   salary INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO faculty VALUES (1, 'Anand', 'CS', 56000);
INSERT INTO faculty VALUES (2, 'Bhavani', 'Maths', 48000);
INSERT INTO faculty VALUES (3, 'Chandru', 'CS', 62000);
INSERT INTO faculty VALUES (4, 'Dhivya', 'Physics', 45000);
```

INSERT INTO faculty VALUES (5, 'Eshwar', 'Maths', 52000);

INSERT INTO faculty VALUES (6, 'Fathima', 'CS', 70000);

INSERT INTO faculty VALUES (7, 'Gowtham', 'Physics', 50000);

Records Inserted

SQL Query

Query 1: Display all faculty records

SQL> SELECT * FROM faculty;

Resultant Query

fid	fname	dept	salary
1	Anand	CS	56000
2	Bhavani	Maths	48000
3	Chandru	CS	62000
4	Dhivya	Physics	45000
5	Eshwar	Maths	52000
6	Fathima	CS	70000
7	Gowtham	Physics	50000

Query 2: Display faculty whose salary is greater than the average salary

SQL> SELECT *

FROM faculty

WHERE salary > (SELECT AVG(salary)

FROM faculty);

Resultant Query

fid	fname	dept	salary
1	Anand	CS	56000
3	Chandru	CS	62000
6	Fathima	CS	70000

The average salary of all the faculty is 54714.29, so only the faculty earning more than this value are displayed.

Query 3: Display faculty who belong to the department having a salary above 60000

```
SQL> SELECT *
FROM faculty
WHERE dept IN (SELECT dept
        FROM faculty
        WHERE salary > 60000);
```

Resultant Query

fid	fname	dept	salary
1	Anand	CS	56000
3	Chandru	CS	62000
6	Fathima	CS	70000

Query 4: Display the faculty with the highest salary

```
SQL> SELECT *
FROM faculty
WHERE salary = (SELECT MAX(salary)
        FROM faculty);
```

Resultant Query

fid	fname	dept	salary
6	Fathima	CS	70000

Query 5: Display faculty whose salary is greater than Bhavani's salary

```
SQL> SELECT fname, salary
FROM faculty
WHERE salary > (SELECT salary
        FROM faculty
        WHERE fname = 'Bhavani');
```

Resultant Query

fname	salary
Anand	56000
Chandru	62000
Eshwar	52000
Fathima	70000
Gowtham	50000

Query 6: Display faculty who earn more than all the Physics faculty

```
SQL> SELECT fname, dept, salary
FROM faculty
WHERE salary > ALL (SELECT salary
                    FROM faculty
                    WHERE dept = 'Physics');
```

Resultant Query

fname	dept	salary
Anand	CS	56000
Chandru	CS	62000
Eshwar	Maths	52000
Fathima	CS	70000

Query 7: Display the second highest salary (Nested Subquery)

```
SQL> SELECT MAX(salary) AS second_highest
FROM faculty
WHERE salary < (SELECT MAX(salary)
                FROM faculty);
```

Resultant Query

second_highest
62000

Query 8: Display faculty earning more than the average salary of their own department

```
SQL> SELECT f.fname, f.dept, f.salary
FROM faculty f
WHERE f.salary > (SELECT AVG(salary)
                  FROM faculty
                  WHERE dept = f.dept);
```

Resultant Query

fname	dept	salary
Fathima	CS	70000
Eshwar	Maths	52000
Gowtham	Physics	50000

RESULT:

Thus, the Faculty table was created successfully, and the single row subqueries, multiple row subqueries and nested subqueries were executed successfully.

EX. NO:08 Creation of views, indexes and sequences using Members Table

DATE: 08.09.2026

Aim

To create a **Members** table and to implement views, indexes and sequences using SQL.

Software Used

Oracle SQL / SQL Command Line

Description

View:

A View is a virtual table created from the result of a SELECT query. It does not store data by itself.

Index:

An Index is a database object that is used to speed up the searching of records in a table.

Sequence:

A Sequence is a database object that generates a series of unique numbers automatically.

1. VIEW

Definition

A view is a stored query that displays selected rows and columns of one or more tables. It is useful for hiding unwanted columns and for simplifying complex queries.

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Create Table

```
CREATE TABLE members (
    member_id NUMBER PRIMARY KEY,
    member_name VARCHAR2(50),
    city VARCHAR2(30),
    join_year NUMBER,
    fee NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO members VALUES (1, 'Kalai', 'Chennai', 2022, 1200);
INSERT INTO members VALUES (2, 'Raja', 'Madurai', 2023, 1500);
INSERT INTO members VALUES (3, 'Saranya', 'Chennai', 2021, 1000);
INSERT INTO members VALUES (4, 'Tamil', 'Trichy', 2024, 1800);
INSERT INTO members VALUES (5, 'Uma', 'Madurai', 2022, 1300);
COMMIT;
```

Records Inserted

Create View

The following view displays only the members who belong to Chennai.

```
CREATE VIEW chennai_members AS
SELECT member_id, member_name, fee
FROM members
WHERE city = 'Chennai';
```

View Created

Display View

```
SELECT * FROM chennai_members;
```

Resultant Query

member_id	member_name	fee
1	Kalai	1200
3	Saranya	1000

Explanation

The view chennai_members contains only the members whose city is Chennai. Whenever the members table changes, the view shows the latest data automatically.

2. INDEX

Definition

An index is created on one or more columns of a table. It allows the database to locate the required rows quickly without scanning the entire table.

Syntax

CREATE INDEX index_name

ON table_name (column_name);

Create Index

CREATE INDEX idx_member_city

ON members (city);

Index Created

Use Index in Search

SELECT member_name, city

FROM members

WHERE city = 'Madurai';

Resultant Query

member_name	city
Raja	Madurai
Uma	Madurai

Display Index Details

SELECT index_name, table_name

FROM user_indexes

WHERE table_name = 'MEMBERS';

Resultant Query

index_name	table_name
IDX_MEMBER_CITY	MEMBERS
SYS_C007421	MEMBERS

Explanation

The index idx_member_city is created on the city column, so searches that use the city column are executed faster.

3. SEQUENCE

Definition

A sequence generates unique numeric values automatically. It is mainly used to create primary key values.

Syntax

CREATE SEQUENCE sequence_name

START WITH value

INCREMENT BY value

MAXVALUE value

NOCYCLE;

Create Sequence

CREATE SEQUENCE member_seq

START WITH 6

INCREMENT BY 1

MAXVALUE 9999

NOCYCLE;

Sequence Created

Insert Record Using Sequence

INSERT INTO members

VALUES (member_seq.NEXTVAL, 'Vimal', 'Salem', 2025, 1600);

1 row created.

Display Sequence Value

SELECT member_seq.CURRVAL FROM dual;

Resultant Query

CURRVAL
6

Insert Another Record Using Sequence

INSERT INTO members

VALUES (member_seq.NEXTVAL, 'Yazhini', 'Chennai', 2025, 1700);

1 row created.

Display All Member Records

SELECT * FROM members;

Resultant Query

member_id	member_name	city	join_year	fee
1	Kalai	Chennai	2022	1200
2	Raja	Madurai	2023	1500
3	Saranya	Chennai	2021	1000
4	Tamil	Trichy	2024	1800
5	Uma	Madurai	2022	1300
6	Vimal	Salem	2025	1600
7	Yazhini	Chennai	2025	1700

Explanation

The sequence member_seq started from 6 and increased by 1 for every new record. Hence the new members received the IDs 6 and 7 automatically.

Updated View

Since Yazhini also belongs to Chennai, she is displayed automatically when the view is queried again.

SELECT * FROM chennai_members;

member_id	member_name	fee
1	Kalai	1200
3	Saranya	1000
7	Yazhini	1700

Important Commands

Drop View

DROP VIEW chennai_members;

View Dropped

Drop Index

DROP INDEX idx_member_city;

Index Dropped

Drop Sequence

DROP SEQUENCE member_seq;

Sequence Dropped

RESULT:

Thus, the Members table was created successfully, and the view, index and sequence were created and executed successfully using SQL.

EX. NO:09 Creation of stored procedures and functions using Employee Pay Table

DATE: 10.09.2026

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

Description

Stored Procedure:

A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:

A Function is a named PL/SQL program that performs a specific operation and always returns a single value.

1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can display or modify the data of a table.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
(parameter_name IN datatype)
IS
BEGIN
   statements;
END;
/
```

Create Table

```
CREATE TABLE employee_pay (
   emp_id NUMBER PRIMARY KEY,
   emp_name VARCHAR2(50),
   basic_pay NUMBER
```

);

Table Created

Insert Records

```
INSERT INTO employee_pay VALUES (201, 'Anbu', 32000);
INSERT INTO employee_pay VALUES (202, 'Barani', 45000);
INSERT INTO employee_pay VALUES (203, 'Chithra', 28000);
INSERT INTO employee_pay VALUES (204, 'Dhamu', 52000);
INSERT INTO employee_pay VALUES (205, 'Elavarasi', 39000);
COMMIT;
```

Records Inserted

Display Employee Records

```
SELECT * FROM employee_pay;
```

Resultant Query

emp_id	emp_name	basic_pay
201	Anbu	32000
202	Barani	45000
203	Chithra	28000
204	Dhamu	52000
205	Elavarasi	39000

Create Stored Procedure

The following procedure displays the details of an employee based on the employee ID.

```
CREATE OR REPLACE PROCEDURE show_pay (
   p_emp_id IN NUMBER
)
IS
   v_name employee_pay.emp_name%TYPE;
   v_pay employee_pay.basic_pay%TYPE;
BEGIN
   SELECT emp_name, basic_pay
```

```
    INTO v_name, v_pay
     FROM employee_pay
     WHERE emp_id = p_emp_id;

     DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
     DBMS_OUTPUT.PUT_LINE('Basic Pay: ' || v_pay);
END;
/
```

Procedure Created

Execute Stored Procedure

```
SET SERVEROUTPUT ON;
EXEC show_pay(204);
```

Resultant Query

Employee Name	Basic Pay
Dhamu	52000

Output

Employee Name: Dhamu

Basic Pay: 52000

Explanation

The procedure show_pay accepts the employee ID as an input parameter and retrieves the corresponding employee name and basic pay from the employee_pay table. The values are then displayed using DBMS_OUTPUT.

Procedure to Update Data

The following procedure increases the basic pay of an employee by a given percentage.

```
CREATE OR REPLACE PROCEDURE raise_pay (
   p_emp_id IN NUMBER,
   p_percent IN NUMBER
)
IS
```

```
BEGIN
   UPDATE employee_pay
   SET basic_pay = basic_pay + (basic_pay * p_percent / 100)
   WHERE emp_id = p_emp_id;

   COMMIT;
   DBMS_OUTPUT.PUT_LINE('Pay updated successfully.');
END;
/
```

Procedure Created

Execute Procedure

```
SET SERVEROUTPUT ON;
EXEC raise_pay(203, 10);
```

Output

Pay updated successfully.

Display Updated Record

```
SELECT * FROM employee_pay
WHERE emp_id = 203;
```

Resultant Query

emp_id	emp_name	basic_pay
203	Chithra	30800

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value to the calling program.

Syntax

```
CREATE OR REPLACE FUNCTION function_name
(parameter_name IN datatype)
RETURN datatype
```

```
IS
BEGIN
   statements;
   RETURN value;
END;
/
```

Create Function

The following function calculates the bonus amount based on the basic pay.

```
CREATE OR REPLACE FUNCTION calc_bonus (
   p_pay IN NUMBER
)
RETURN NUMBER
IS
   v_bonus NUMBER;
BEGIN
   IF p_pay >= 50000 THEN
      v_bonus := p_pay * 0.10;
   ELSIF p_pay >= 40000 THEN
      v_bonus := p_pay * 0.08;
   ELSIF p_pay >= 30000 THEN
      v_bonus := p_pay * 0.06;
   ELSE
      v_bonus := p_pay * 0.04;
   END IF;
   RETURN v_bonus;
END;
/
```

Function Created

Execute Function

```
SELECT emp_id, emp_name, basic_pay,
    calc_bonus(basic_pay) AS bonus
FROM employee_pay;
```

Resultant Query

emp_id	emp_name	basic_pay	bonus
201	Anbu	32000	1920
202	Barani	45000	3600
203	Chithra	30800	1848
204	Dhamu	52000	5200
205	Elavarasi	39000	2340

Explanation

The function calc_bonus accepts the basic pay as an input and returns the bonus amount. The function is called inside the SELECT statement, so the bonus is calculated for every employee.

Calling the Function from a PL/SQL Block

```
SET SERVEROUTPUT ON;
DECLARE
   v_result NUMBER;
BEGIN
   v_result := calc_bonus(45000);
   DBMS_OUTPUT.PUT_LINE('Bonus Amount: ' || v_result);
END;
/
```

Output

Bonus Amount: 3600

Difference between Procedure and Function

Procedure	Function
May or may not return a value	Always returns a single value
Called using EXEC or inside a PL/SQL block	Can be called inside a SELECT statement
Used to perform an action	Used to compute a value

Important Commands

Display Stored Procedure

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'SHOW_PAY';
```

Resultant Query

object_name	object_type
SHOW_PAY	PROCEDURE

Display Stored Function

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'CALC_BONUS';
```

Resultant Query

object_name	object_type
CALC_BONUS	FUNCTION

Drop Procedure

```
DROP PROCEDURE show_pay;
DROP PROCEDURE raise_pay;
```

Procedure Dropped

Drop Function

```
DROP FUNCTION calc_bonus;
```

Function Dropped

RESULT:

Thus, the Stored Procedures and Function were created and executed successfully using PL/SQL. The Stored Procedures were used to retrieve and update employee pay details, and the Function was used to calculate the bonus amount.

EX. NO:10 Creation of Triggers method using medicine stock table

DATE: 11.09.2026

Aim

To create and execute BEFORE and AFTER Triggers using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL block that is executed automatically when a specified event such as INSERT, UPDATE or DELETE occurs on a table.

Triggers are mainly used to maintain data integrity, to validate data and to keep an audit record of the changes.

Types of Triggers:

- BEFORE Trigger
- AFTER Trigger
- Row Level Trigger (FOR EACH ROW)
- Statement Level Trigger

1. CREATE TABLE

```
CREATE TABLE medicine_stock (
   med_id NUMBER PRIMARY KEY,
   med_name VARCHAR2(50),
   quantity NUMBER,
   price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO medicine_stock VALUES (1, 'Paracetamol', 200, 12);
INSERT INTO medicine_stock VALUES (2, 'Amoxicillin', 150, 45);
INSERT INTO medicine_stock VALUES (3, 'Cetirizine', 120, 8);
INSERT INTO medicine_stock VALUES (4, 'Ibuprofen', 90, 20);
INSERT INTO medicine_stock VALUES (5, 'Vitamin C', 300, 6);
COMMIT;
```

Records Inserted

Display Medicine Records

SELECT * FROM medicine_stock;

Resultant Query

med_id	med_name	quantity	price
1	Paracetamol	200	12
2	Amoxicillin	150	45
3	Cetirizine	120	8
4	Ibuprofen	90	20
5	Vitamin C	300	6

2. BEFORE INSERT TRIGGER

Definition

A BEFORE INSERT trigger is executed before a new record is inserted. It is used to validate or modify the data before it is stored.

Syntax

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
   statements;
END;
/
```

Create Trigger

The following trigger converts the medicine name into uppercase and does not allow a negative quantity.

```
CREATE OR REPLACE TRIGGER trg_med_before_insert
BEFORE INSERT ON medicine_stock
FOR EACH ROW
BEGIN
   :NEW.med_name := UPPER(:NEW.med_name);
```

```
    IF :NEW.quantity < 0 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Quantity cannot be negative'
        );
    END IF;
END;
/
```

Trigger Created

Execute Trigger

```
INSERT INTO medicine_stock
VALUES (6, 'dolo 650', 100, 3);
```

1 row created.

```
SELECT * FROM medicine_stock
WHERE med_id = 6;
```

med_id	med_name	quantity	price
6	DOLO 650	100	3

Execute Trigger with Invalid Data

```
INSERT INTO medicine_stock
VALUES (7, 'Aspirin', -5, 10);
```

Output

```
ERROR at line 1:
ORA-20001: Quantity cannot be negative
```

Explanation

The medicine name dolo 650 was converted into uppercase automatically, and the record having a negative quantity was rejected by the trigger.

3. BEFORE UPDATE TRIGGER

Definition

A BEFORE UPDATE trigger is executed before an existing record is modified. It can compare the old and new values of a column.

Create Trigger

The following trigger does not allow the price of a medicine to be reduced.

```
CREATE OR REPLACE TRIGGER trg_med_before_update
BEFORE UPDATE OF price ON medicine_stock
FOR EACH ROW
BEGIN
   IF :NEW.price < :OLD.price THEN
      RAISE_APPLICATION_ERROR(
         -20002,
         'Price cannot be reduced'
      );
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
UPDATE medicine_stock
SET price = 10
WHERE med_id = 2;
```

Output

```
ERROR at line 1:
ORA-20002: Price cannot be reduced
```

Valid Update

```
UPDATE medicine_stock
SET price = 48
WHERE med_id = 2;
```

1 row updated.

```
SELECT * FROM medicine_stock
```

WHERE med_id = 2;

med_id	med_name	quantity	price
2	Amoxicillin	150	48

4. AFTER UPDATE TRIGGER

Definition

An AFTER UPDATE trigger is executed after the record has been modified. It is commonly used to maintain an audit table that stores the history of changes.

Create Audit Table

```
CREATE TABLE stock_audit (
    audit_id NUMBER PRIMARY KEY,
    med_id NUMBER,
    old_quantity NUMBER,
    new_quantity NUMBER,
    changed_on DATE
);
```

Table Created

Before executing the trigger, create the required sequence:

```
CREATE SEQUENCE audit_seq START WITH 1 INCREMENT BY 1;
```

Sequence Created

Create Trigger

```
CREATE OR REPLACE TRIGGER trg_med_audit
AFTER UPDATE OF quantity ON medicine_stock
FOR EACH ROW
BEGIN
    INSERT INTO stock_audit
    VALUES (
        audit_seq.NEXTVAL,
        :OLD.med_id,
        :OLD.quantity,
```

```
    :NEW.quantity,
        SYSDATE
    );
END;
/
```

Trigger Created

Update the Stock

```
UPDATE medicine_stock
SET quantity = quantity - 20
WHERE med_id = 1;
```

1 row updated.

Display Audit Records

```
SELECT * FROM stock_audit;
```

Resultant Query

audit_id	med_id	old_quantity	new_quantity	changed_on
1	1	200	180	04-OCT-26

5. DISPLAY FINAL MEDICINE RECORDS

```
SELECT * FROM medicine_stock;
```

Resultant Query

med_id	med_name	quantity	price
1	Paracetamol	180	12
2	Amoxicillin	150	48
3	Cetirizine	120	8
4	Ibuprofen	90	20
5	Vitamin C	300	6
6	DOLO 650	100	3

Important Commands

Display Triggers

SELECT trigger_name, trigger_type, triggering_event

FROM user_triggers

WHERE table_name = 'MEDICINE_STOCK';

Resultant Query

trigger_name	trigger_type	triggering_event
TRG_MED_BEFORE_INSERT	BEFORE EACH ROW	INSERT
TRG_MED_BEFORE_UPDATE	BEFORE EACH ROW	UPDATE
TRG_MED_AUDIT	AFTER EACH ROW	UPDATE

Drop Trigger

DROP TRIGGER trg_med_before_insert;

DROP TRIGGER trg_med_before_update;

DROP TRIGGER trg_med_audit;

Trigger Dropped

RESULT:

Thus, the BEFORE INSERT, BEFORE UPDATE and AFTER UPDATE triggers were created and executed successfully using PL/SQL. The triggers were used to validate the data and to maintain an audit record of the stock changes.

EX. NO:11 Creation of cursor implementation using vehicle rental table

DATE: 15.09.2026

Aim

To implement and execute Cursors using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer to the memory area that holds the rows returned by a SQL query. It is used to process the rows one at a time.

Cursors are useful when each record has to be processed individually.

There are two types of cursors:

- Implicit Cursor
- Explicit Cursor

An Implicit Cursor is created automatically by Oracle for every DML statement, whereas an Explicit Cursor is declared and controlled by the programmer.

1. CREATE TABLE

```
CREATE TABLE vehicles (
   vehicle_id NUMBER PRIMARY KEY,
   vehicle_name VARCHAR2(50),
   owner_name VARCHAR2(50),
   rent_per_day NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO vehicles VALUES (501, 'Swift', 'Murugan', 1800);
INSERT INTO vehicles VALUES (502, 'Innova', 'Kamala', 3200);
INSERT INTO vehicles VALUES (503, 'Splendor', 'Siva', 600);
INSERT INTO vehicles VALUES (504, 'Creta', 'Latha', 2800);
INSERT INTO vehicles VALUES (505, 'Pulsar', 'Manoj', 900);
COMMIT;
```

Records Inserted

Display Vehicle Records

SELECT * FROM vehicles;

Resultant Query

vehicle_id	vehicle_name	owner_name	rent_per_day
501	Swift	Murugan	1800
502	Innova	Kamala	3200
503	Splendor	Siva	600
504	Creta	Latha	2800
505	Pulsar	Manoj	900

2. EXPLICIT CURSOR

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and to process them one by one.

Syntax

```
DECLARE
   CURSOR cursor_name IS
      SELECT statement;
BEGIN
   OPEN cursor_name;
   LOOP
      FETCH cursor_name INTO variables;
      EXIT WHEN cursor_name%NOTFOUND;
      statements;
   END LOOP;
   CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves the name and the rent of every vehicle from the vehicles table.

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR vehicle_cur IS
      SELECT vehicle_name, rent_per_day
      FROM vehicles;
   v_name vehicles.vehicle_name%TYPE;
   v_rent vehicles.rent_per_day%TYPE;
BEGIN
   OPEN vehicle_cur;
   LOOP
      FETCH vehicle_cur INTO v_name, v_rent;
      EXIT WHEN vehicle_cur%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(
         'Vehicle: ' || v_name || ' | Rent: ₹' || v_rent
      );
   END LOOP;
   CLOSE vehicle_cur;
END;
/
```

Output

Vehicle: Swift | Rent: ₹1800

Vehicle: Innova | Rent: ₹3200

Vehicle: Splendor | Rent: ₹600

Vehicle: Creta | Rent: ₹2800

Vehicle: Pulsar | Rent: ₹900

3. CURSOR WITH CONDITION

The following cursor displays only the vehicles whose rent is greater than ₹1500.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
   CURSOR vehicle_cur IS
      SELECT vehicle_id, vehicle_name, rent_per_day
      FROM vehicles
      WHERE rent_per_day > 1500;
   v_id vehicles.vehicle_id%TYPE;
   v_name vehicles.vehicle_name%TYPE;
   v_rent vehicles.rent_per_day%TYPE;
BEGIN
   OPEN vehicle_cur;
   LOOP
      FETCH vehicle_cur INTO v_id, v_name, v_rent;
      EXIT WHEN vehicle_cur%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(
         'ID: ' || v_id ||
         ' | Vehicle: ' || v_name ||
         ' | Rent: ₹' || v_rent
      );
   END LOOP;
   CLOSE vehicle_cur;
END;
/
```

Resultant Query

vehicle_id	vehicle_name	rent_per_day
501	Swift	1800
502	Innova	3200
504	Creta	2800

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Important Cursor Attributes

Cursor Attribute	Description
%FOUND	Returns TRUE if the last fetch returned a row
%NOTFOUND	Returns TRUE if the last fetch did not return a row
%ROWCOUNT	Returns the number of rows fetched so far
%ISOPEN	Returns TRUE if the cursor is currently open

5. CURSOR USING %ROWCOUNT

The following program displays the total number of vehicles processed by the cursor.

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR vehicle_cur IS
      SELECT vehicle_name, owner_name
      FROM vehicles;
   v_name vehicles.vehicle_name%TYPE;
   v_owner vehicles.owner_name%TYPE;
BEGIN
   OPEN vehicle_cur;
   LOOP
      FETCH vehicle_cur INTO v_name, v_owner;
      EXIT WHEN vehicle_cur%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(
         'Vehicle: ' || v_name || ' | Owner: ' || v_owner
      );
   END LOOP;
   DBMS_OUTPUT.PUT_LINE(
      'Total Vehicles Processed: ' || vehicle_cur%ROWCOUNT
   );
   CLOSE vehicle_cur;
END;
```

```
/
```

Output

```
Vehicle: Swift | Owner: Murugan
Vehicle: Innova | Owner: Kamala
Vehicle: Splendor | Owner: Siva
Vehicle: Creta | Owner: Latha
Vehicle: Pulsar | Owner: Manoj
Total Vehicles Processed: 5
```

6. CURSOR FOR LOOP

A cursor FOR loop opens the cursor, fetches the rows and closes the cursor automatically, so it needs less code.

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR vehicle_cur IS
      SELECT vehicle_name, rent_per_day
      FROM vehicles;
BEGIN
   FOR rec IN vehicle_cur LOOP
      DBMS_OUTPUT.PUT_LINE(
         rec.vehicle_name || ' - ₹' || rec.rent_per_day
      );
   END LOOP;
END;
/
```

Output

```
Swift - ₹1800
Innova - ₹3200
Splendor - ₹600
Creta - ₹2800
Pulsar - ₹900
```

7. PARAMETERIZED CURSOR

A parameterized cursor accepts a value when it is opened. The following cursor displays the vehicles whose rent is not less than the given amount.

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR vehicle_cur (p_min NUMBER) IS
      SELECT vehicle_name, rent_per_day
      FROM vehicles
      WHERE rent_per_day >= p_min;
   v_name vehicles.vehicle_name%TYPE;
   v_rent vehicles.rent_per_day%TYPE;
BEGIN
   OPEN vehicle_cur(2000);
   LOOP
      FETCH vehicle_cur INTO v_name, v_rent;
      EXIT WHEN vehicle_cur%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(v_name || ' - ₹' || v_rent);
   END LOOP;
   CLOSE vehicle_cur;
END;
/
```

Resultant Query

vehicle_name	rent_per_day
Innova	3200
Creta	2800

8. IMPLICIT CURSOR

Oracle creates an implicit cursor automatically for every INSERT, UPDATE and DELETE statement. The attribute SQL%ROWCOUNT shows the number of rows affected. The following program increases the rent of the vehicles that are below ₹1000 by ₹100.

```
SET SERVEROUTPUT ON;
```

```
BEGIN
    UPDATE vehicles
    SET rent_per_day = rent_per_day + 100
    WHERE rent_per_day < 1000;

    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE(
            SQL%ROWCOUNT || ' record(s) updated.'
        );
    ELSE
        DBMS_OUTPUT.PUT_LINE('No record updated.');
    END IF;
    COMMIT;
END;
/
```

Output

2 record(s) updated.

Display Updated Records

```
SELECT * FROM vehicles;
```

Resultant Query

vehicle_id	vehicle_name	owner_name	rent_per_day
501	Swift	Murugan	1800
502	Innova	Kamala	3200
503	Splendor	Siva	700
504	Creta	Latha	2800
505	Pulsar	Manoj	1000

RESULT:

Thus, the Cursors were successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process vehicle records one by one, and the cursor with condition, cursor

attributes, cursor FOR loop, parameterized cursor and implicit cursor were executed successfully.

EX. NO:12 Creation of PL/SQL Exception Handling using bus tickets table

DATE: 16.09.2026

Aim

To implement Exception Handling in PL/SQL and to handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program. Exception Handling is used to handle the runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```
DECLARE
   declarations;
BEGIN
   statements;
EXCEPTION
   WHEN exception_name THEN
      statements;
END;
/
```

Types of Exceptions

Type	Description
Predefined Exception	Built-in exceptions provided by Oracle
User-Defined Exception	Exception created by the programmer

1. CREATE TABLE

```
CREATE TABLE bus_tickets (
   bus_id NUMBER PRIMARY KEY,
   route VARCHAR2(50),
   available_seats NUMBER,
```

fare NUMBER

);

Table Created

Insert Records

INSERT INTO bus_tickets VALUES (1, 'Chennai - Madurai', 12, 650);

INSERT INTO bus_tickets VALUES (2, 'Chennai - Trichy', 5, 420);

INSERT INTO bus_tickets VALUES (3, 'Salem - Coimbatore', 0, 250);

INSERT INTO bus_tickets VALUES (4, 'Madurai - Tirunelveli', 20, 300);

INSERT INTO bus_tickets VALUES (5, 'Chennai - Villupuram', 8, 160);

COMMIT;

Records Inserted

Display Bus Records

SELECT * FROM bus_tickets;

Resultant Query

bus_id	route	available_seats	fare
1	Chennai - Madurai	12	650
2	Chennai - Trichy	5	420
3	Salem - Coimbatore	0	250
4	Madurai - Tirunelveli	20	300
5	Chennai - Villupuram	8	160

2. NO_DATA_FOUND EXCEPTION

Definition

NO_DATA_FOUND occurs when a SELECT INTO statement does not return any record.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_route bus_tickets.route%TYPE;
   v_fare bus_tickets.fare%TYPE;
BEGIN
```

```
 SELECT route, fare
    INTO v_route, v_fare
    FROM bus_tickets
    WHERE bus_id = 99;

    DBMS_OUTPUT.PUT_LINE('Route: ' || v_route);
    DBMS_OUTPUT.PUT_LINE('Fare: ₹' || v_fare);
 EXCEPTION
    WHEN NO_DATA_FOUND THEN
       DBMS_OUTPUT.PUT_LINE('Exception: Bus not found.');
 END;
 /
```

Output

Exception: Bus not found.

Exception Handled Successfully

3. TOO_MANY_ROWS EXCEPTION

Definition

TOO_MANY_ROWS occurs when a SELECT INTO statement returns more than one record.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_route bus_tickets.route%TYPE;
BEGIN
   SELECT route
   INTO v_route
   FROM bus_tickets
   WHERE available_seats > 5;

   DBMS_OUTPUT.PUT_LINE('Route: ' || v_route);
```

```
EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: More than one bus found.'
        );
END;
/
```

Output

Exception: More than one bus found.

Exception Handled Successfully

4. ZERO_DIVIDE EXCEPTION

Definition

ZERO_DIVIDE occurs when a number is divided by zero. In this program, the fare per seat is calculated for a bus that has no available seats.

Program

```
SET SERVEROUTPUT ON;
DECLARE
    v_seats NUMBER;
    v_fare NUMBER;
    v_result NUMBER;
BEGIN
    SELECT available_seats, fare
    INTO v_seats, v_fare
    FROM bus_tickets
    WHERE bus_id = 3;

    v_result := v_fare / v_seats;
    DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
```

```
    DBMS_OUTPUT.PUT_LINE(
        'Exception: Cannot divide by zero.'
    );
END;
/
```

Output

Exception: Cannot divide by zero.

Exception Handled Successfully

5. DUP_VAL_ON_INDEX EXCEPTION

Definition

DUP_VAL_ON_INDEX occurs when a duplicate value is inserted into a column that has a primary key or a unique constraint.

Program

```
SET SERVEROUTPUT ON;
BEGIN
   INSERT INTO bus_tickets
   VALUES (1, 'Trichy - Salem', 10, 280);
   COMMIT;
EXCEPTION
   WHEN DUP_VAL_ON_INDEX THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Bus ID already exists.'
      );
END;
/
```

Output

Exception: Bus ID already exists.

Exception Handled Successfully

6. VALUE_ERROR EXCEPTION

Definition

VALUE_ERROR occurs when a value cannot be converted or assigned to a variable because of a data type or size mismatch.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_number NUMBER;
BEGIN
   v_number := 'ABC';
   DBMS_OUTPUT.PUT_LINE('Value: ' || v_number);
EXCEPTION
   WHEN VALUE_ERROR THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Invalid number conversion.'
      );
END;
/
```

Output

Exception: Invalid number conversion.

Exception Handled Successfully

7. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition. In this example, a ticket can be booked only when the required number of seats is available.

Program

```
SET SERVEROUTPUT ON;
DECLARE
```

```
  v_available bus_tickets.available_seats%TYPE;
    v_required NUMBER := 6;
    seats_not_available EXCEPTION;
BEGIN
    SELECT available_seats
    INTO v_available
    FROM bus_tickets
    WHERE bus_id = 2;

    IF v_required > v_available THEN
        RAISE seats_not_available;
    END IF;

    UPDATE bus_tickets
    SET available_seats = available_seats - v_required
    WHERE bus_id = 2;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Booking Successful.');
EXCEPTION
    WHEN seats_not_available THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Required seats are not available.'
        );
END;
/
```

Output

Exception: Required seats are not available.

User-Defined Exception Handled Successfully

8. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program books three seats when the required seats are available.

```
SET SERVEROUTPUT ON;
DECLARE
   v_available bus_tickets.available_seats%TYPE;
   v_required NUMBER := 3;
BEGIN
   SELECT available_seats
   INTO v_available
   FROM bus_tickets
   WHERE bus_id = 2;

   IF v_required > v_available THEN
      RAISE_APPLICATION_ERROR(
         -20003,
         'Seats not available'
      );
   END IF;

   UPDATE bus_tickets
   SET available_seats = available_seats - v_required
   WHERE bus_id = 2;
   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Booking Successful.');
```

```
 DBMS_OUTPUT.PUT_LINE('Seats Booked: ' || v_required);
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Output

Booking Successful.

Seats Booked: 3

Display Updated Record

```
SELECT * FROM bus_tickets
WHERE bus_id = 2;
```

Resultant Query

bus_id	route	available_seats	fare
2	Chennai - Trichy	2	420

9. WHEN OTHERS EXCEPTION

The WHEN OTHERS handler catches every exception that is not handled by the earlier handlers. SQLCODE returns the error number and SQLERRM returns the error message.

```
SET SERVEROUTPUT ON;
DECLARE
   v_result NUMBER;
BEGIN
   v_result := 100 / 0;
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error Code: ' || SQLCODE);
      DBMS_OUTPUT.PUT_LINE('Error Message: ' || SQLERRM);
END;
/
```

Output

Error Code: -1476

Error Message: ORA-01476: divisor is equal to zero

Common Predefined Exceptions

Exception	Raised When
NO_DATA_FOUND	SELECT INTO returns no row
TOO_MANY_ROWS	SELECT INTO returns more than one row
ZERO_DIVIDE	A number is divided by zero
DUP_VAL_ON_INDEX	A duplicate value is stored in a unique column
VALUE_ERROR	Data type or size mismatch occurs

RESULT:

Thus, the Exception Handling was successfully implemented using PL/SQL. The predefined exceptions NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, DUP_VAL_ON_INDEX, VALUE_ERROR and OTHERS, and a user-defined exception were handled successfully.

EX. NO:13 Creation of Database Normalization using employee project table

DATE: 18.09.2026

Aim

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and to improve data integrity.

Software Used

Oracle SQL / SQL Command Line

Description

Normalization is the process of organizing the data in a database to reduce redundancy and to avoid insertion, deletion and update anomalies.

The major normal forms covered are:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Boyce-Codd Normal Form (BCNF)

1. UNNORMALIZED TABLE

Consider the following Employee Project table.

Unnormalized Table

The table contains repeating groups. For example, the projects column stores more than one value for the employees Ashok and Chola, and the department name Accounts is repeated for several employees.

emp_id	emp_name	projects	dept_id	dept_name
101	Ashok	P1-Billing, P2-Payroll	D1	Accounts
102	Banu	P1-Billing	D2	Technical
103	Chola	P3-Website, P4-Mobile App	D2	Technical
104	Devika	P2-Payroll	D1	Accounts

Problems in the Unnormalized Table

- Multiple values are stored in a single column.
- Department details are repeated for many employees.
- Updating one value requires many rows to be changed.

2. FIRST NORMAL FORM (1NF)

Definition

A relation is in 1NF when:

- Each column contains atomic values.
- There are no repeating groups.
- Each row is uniquely identifiable.

To convert the table into 1NF, every project is stored in a separate row.

1NF Table

emp_id	emp_name	project_id	project_name	dept_id	dept_name
101	Ashok	P1	Billing	D1	Accounts
101	Ashok	P2	Payroll	D1	Accounts
102	Banu	P1	Billing	D2	Technical
103	Chola	P3	Website	D2	Technical
103	Chola	P4	Mobile App	D2	Technical
104	Devika	P2	Payroll	D1	Accounts

Primary Key: emp_id, project_id

3. SECOND NORMAL FORM (2NF)

Definition

A relation is in 2NF when:

- It is already in 1NF.
- There is no partial dependency.
- Every non-key attribute depends on the complete primary key.

In the 1NF table, some attributes depend only on a part of the composite key. For example:

- emp_name depends only on emp_id.
- dept_id and dept_name depend only on emp_id.
- project_name depends only on project_id.

Therefore, the table is divided into separate tables.

Employee Table

emp_id	emp_name	dept_id	dept_name
101	Ashok	D1	Accounts
102	Banu	D2	Technical
103	Chola	D2	Technical
104	Devika	D1	Accounts

Project Table

project_id	project_name
P1	Billing
P2	Payroll
P3	Website
P4	Mobile App

Assignment Table

emp_id	project_id
101	P1
101	P2
102	P1
103	P3
103	P4
104	P2

4. THIRD NORMAL FORM (3NF)

Definition

A relation is in 3NF when:

- It is already in 2NF.
- There is no transitive dependency.
- A non-key attribute does not depend on another non-key attribute.

In the Employee table, dept_name depends on dept_id and dept_id depends on emp_id. Hence emp_id determines dept_name through dept_id, which is a transitive dependency. The Employee table is therefore divided into two tables.

Employee Table

emp_id	emp_name	dept_id
101	Ashok	D1
102	Banu	D2
103	Chola	D2
104	Devika	D1

Department Table

dept_id	dept_name
D1	Accounts
D2	Technical

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in BCNF when, for every functional dependency X -> Y, the attribute X is a super key of the relation. BCNF is a stricter form of 3NF.

Consider the following Class Schedule table. Every teacher teaches only one subject, so teacher -> subject. A student can learn many subjects from different teachers.

Class Schedule Table

student_id	subject	teacher
1	DBMS	Prof. Mani
2	DBMS	Prof. Mani
1	Python	Prof. Latha
3	Python	Prof. Latha
2	Java	Prof. Raman

The candidate key is (student_id, subject). The dependency teacher -> subject violates BCNF because teacher is not a super key. The table is therefore decomposed into two tables.

Teacher Subject Table

teacher	subject
Prof. Mani	DBMS
Prof. Latha	Python
Prof. Raman	Java

Student Teacher Table

student_id	teacher
1	Prof. Mani
2	Prof. Mani
1	Prof. Latha
3	Prof. Latha
2	Prof. Raman

6. ANOMALIES REMOVED BY NORMALIZATION

Anomaly	Problem in the Unnormalized Table	Solution after Normalization
Insertion	A new department cannot be stored until an employee joins it	Department table stores the department separately
Deletion	Deleting the last employee removes the department details	Department details remain in the Department table
Update	Changing a department name needs many rows to be changed	The name is changed only once in the Department table

7. CREATING THE NORMALIZED TABLES

```
CREATE TABLE department (
    dept_id VARCHAR2(5) PRIMARY KEY,
    dept_name VARCHAR2(50)
);


CREATE TABLE employee (
```

```
 emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(50),
    dept_id VARCHAR2(5) REFERENCES department(dept_id)
);

CREATE TABLE project (
    project_id VARCHAR2(5) PRIMARY KEY,
    project_name VARCHAR2(50)
);

CREATE TABLE assignment (
    emp_id NUMBER REFERENCES employee(emp_id),
    project_id VARCHAR2(5) REFERENCES project(project_id),
    PRIMARY KEY (emp_id, project_id)
);
```

Tables Created

Insert Records

```
INSERT INTO department VALUES ('D1', 'Accounts');
INSERT INTO department VALUES ('D2', 'Technical');

INSERT INTO employee VALUES (101, 'Ashok', 'D1');
INSERT INTO employee VALUES (102, 'Banu', 'D2');
INSERT INTO employee VALUES (103, 'Chola', 'D2');
INSERT INTO employee VALUES (104, 'Devika', 'D1');

INSERT INTO project VALUES ('P1', 'Billing');
INSERT INTO project VALUES ('P2', 'Payroll');
INSERT INTO project VALUES ('P3', 'Website');
INSERT INTO project VALUES ('P4', 'Mobile App');
```

```
INSERT INTO assignment VALUES (101, 'P1');
INSERT INTO assignment VALUES (101, 'P2');
INSERT INTO assignment VALUES (102, 'P1');
INSERT INTO assignment VALUES (103, 'P3');
INSERT INTO assignment VALUES (103, 'P4');
INSERT INTO assignment VALUES (104, 'P2');
COMMIT;
```

Records Inserted

Retrieve the Original Data Using Joins

```
SELECT e.emp_id, e.emp_name, d.dept_name, p.project_name
FROM employee e
JOIN department d ON e.dept_id = d.dept_id
JOIN assignment a ON e.emp_id = a.emp_id
JOIN project p ON a.project_id = p.project_id
ORDER BY e.emp_id;
```

Resultant Query

emp_id	emp_name	dept_name	project_name
101	Ashok	Accounts	Billing
101	Ashok	Accounts	Payroll
102	Banu	Technical	Billing
103	Chola	Technical	Website
103	Chola	Technical	Mobile App
104	Devika	Accounts	Payroll

The joins give back the complete information without storing any value repeatedly, which shows that no data is lost by normalization.

Summary of Normal Forms

Normal Form	Condition
1NF	Atomic values and no repeating groups
2NF	1NF and no partial dependency
3NF	2NF and no transitive dependency
BCNF	Every determinant is a super key

RESULT:

Thus, the Database Normalization was studied and implemented successfully. The Employee Project table was normalized from the unnormalized form up to 1NF, 2NF, 3NF and BCNF, which reduced the data redundancy and improved the data integrity.

KHAN ACADEMY

BOOK LIST DATABASE

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating. Now, add three of your favorite books into the table.

Program:

```
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);

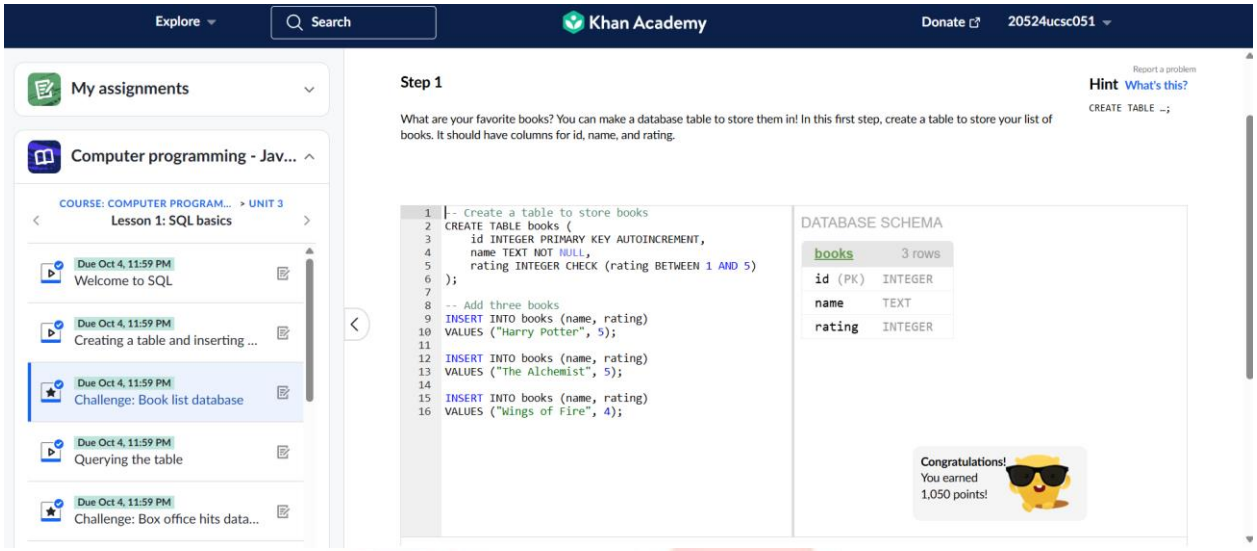
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);

INSERT INTO books VALUES(1,'English',9/10);

INSERT INTO books VALUES(2,'Maths',10/10);

INSERT INTO books VALUES(3,'Tamil',9/10);

SELECT*FROM books;
```



BOX OFFICE HITS DATABASE

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies. Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step.

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

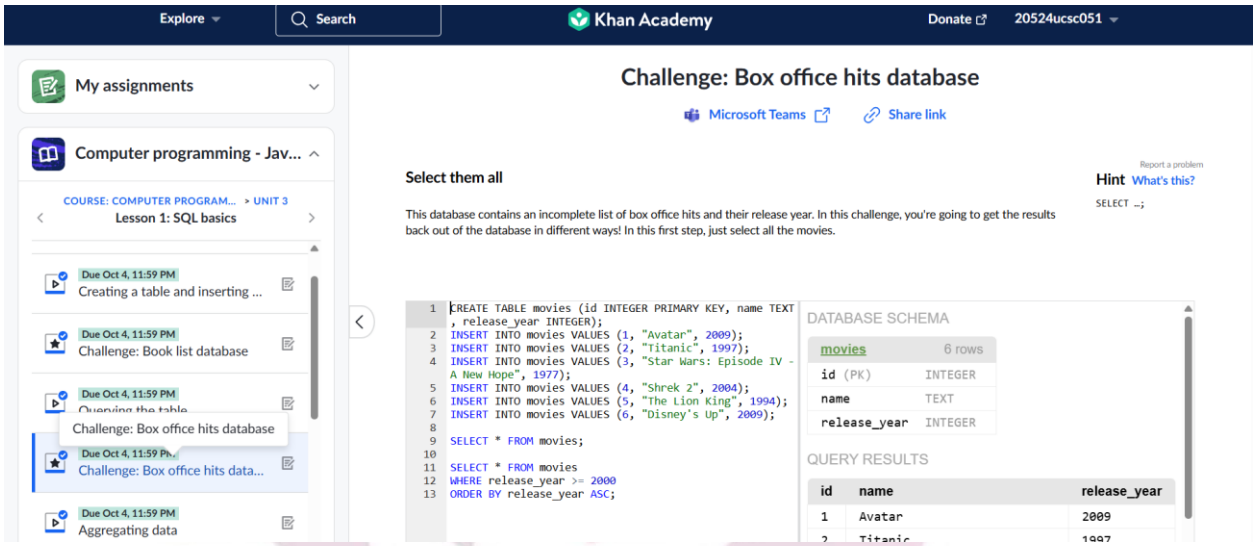
INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);
```

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

SELECT*FROM movies;

SELECT*FROM movies where release_year > 2000 order by release_year;



TODO LISTDATABASE STATS

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take. Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

Program:

CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

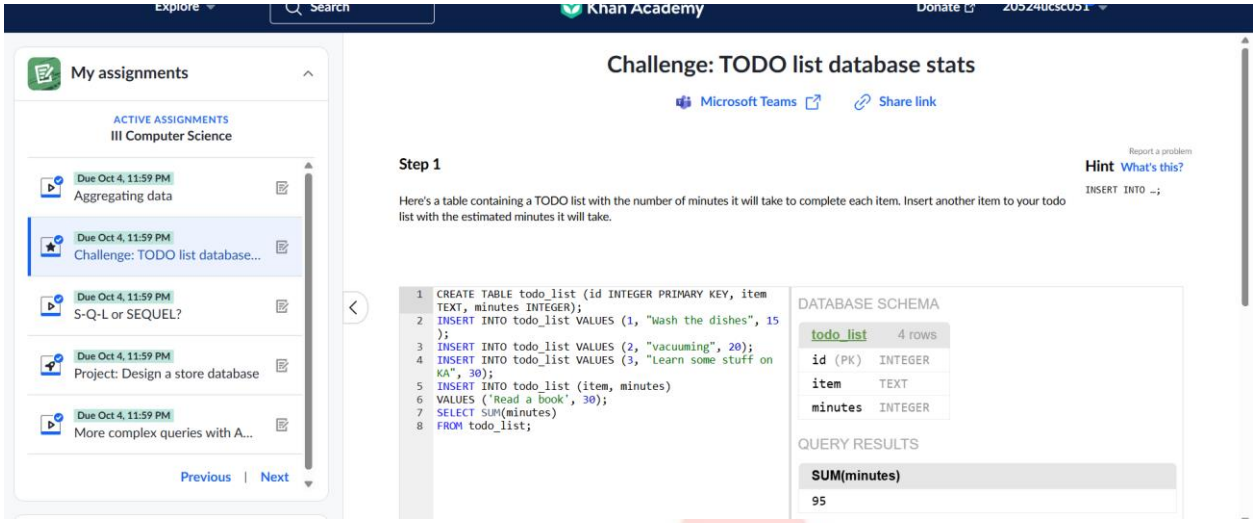
INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Dry your clothes", 40);

SELECT SUM(minutes) FROM todo_list;



PROJECT : DESIGN A STORE DATABASE

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

Program:

```
CREATE TABLE sneakers (
    id INTEGER PRIMARY KEY,
    brand TEXT,
    model TEXT,
    size INTEGER,
    color TEXT,
    price REAL
);
```

```
INSERT INTO sneakers VALUES (1, "Nike", "Air Max", 9, "Black", 120);
INSERT INTO sneakers VALUES (2, "Adidas", "Ultraboost", 10, "White", 180);
INSERT INTO sneakers VALUES (3, "Puma", "RS-X", 8, "Blue", 110);
INSERT INTO sneakers VALUES (4, "New Balance", "574", 9, "Gray", 95);
INSERT INTO sneakers VALUES (5, "Converse", "Chuck Taylor", 10, "Red", 60);
INSERT INTO sneakers VALUES (6, "Vans", "Old Skool", 8, "Black", 70);
INSERT INTO sneakers VALUES (7, "Nike", "Jordan 1", 11, "Red", 200);
INSERT INTO sneakers VALUES (8, "Adidas", "NMD", 9, "Black", 140);
INSERT INTO sneakers VALUES (9, "Puma", "Suede Classic", 10, "Green", 75);
INSERT INTO sneakers VALUES (10, "New Balance", "990", 11, "Gray", 185);
INSERT INTO sneakers VALUES (11, "Converse", "Run Star Hike", 8, "White", 115);
INSERT INTO sneakers VALUES (12, "Vans", "Sk8-Hi", 9, "Blue", 80);
INSERT INTO sneakers VALUES (13, "Nike", "Air Force 1", 10, "White", 130);
INSERT INTO sneakers VALUES (14, "Adidas", "Superstar", 8, "Black", 100);
INSERT INTO sneakers VALUES (15, "Puma", "Future Rider", 9, "Yellow", 90);
SELECT *
FROM sneakers
ORDER BY price DESC;
SELECT
   AVG(price) AS average_price,
   MAX(price) AS highest_price,
   MIN(price) AS lowest_price
FROM sneakers;
```

Output

DATABASE SCHEMA

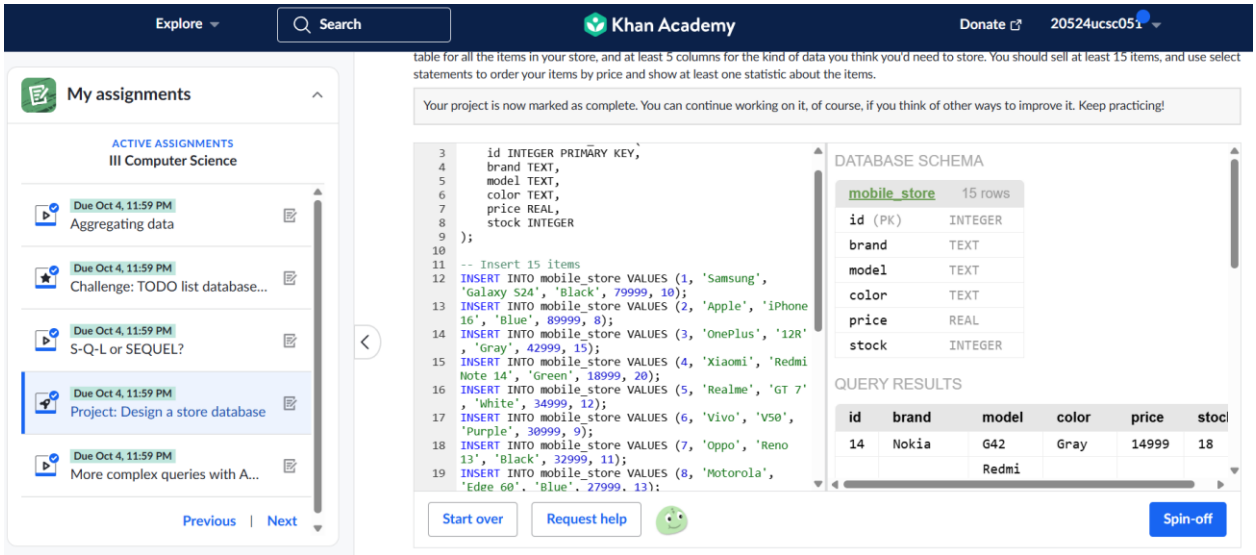
sneakers 15 rows

id (PK) INTEGER
brand TEXT
model TEXT
size INTEGER
color TEXT
price REAL

QUERY RESULTS

id	brand	model	size	color	price
7	Nike	Jordan 1	11	Red	200
10	New Balance	990	11	Gray	185
2	Adidas	Ultraboost	10	White	180
8	Adidas	NMD	9	Black	140
13	Nike	Air Force 1	10	White	130
1	Nike	Air Max	9	Black	120
11	Converse	Run Star Hike	8	White	115
3	Puma	RS-X	8	Blue	110
14	Adidas	Superstar	8	Black	100
4	New Balance	574	9	Gray	95
15	Puma	Future Rider	9	Yellow	90
12	Vans	Sk8-Hi	9	Blue	80
9	Puma	Suede Classic	10	Green	75
6	Vans	Old Skool	8	Black	70
5	Converse	Chuck Taylor	10	Red	60

average_price	highest_price	lowest_price
116.66666666666667	200	60



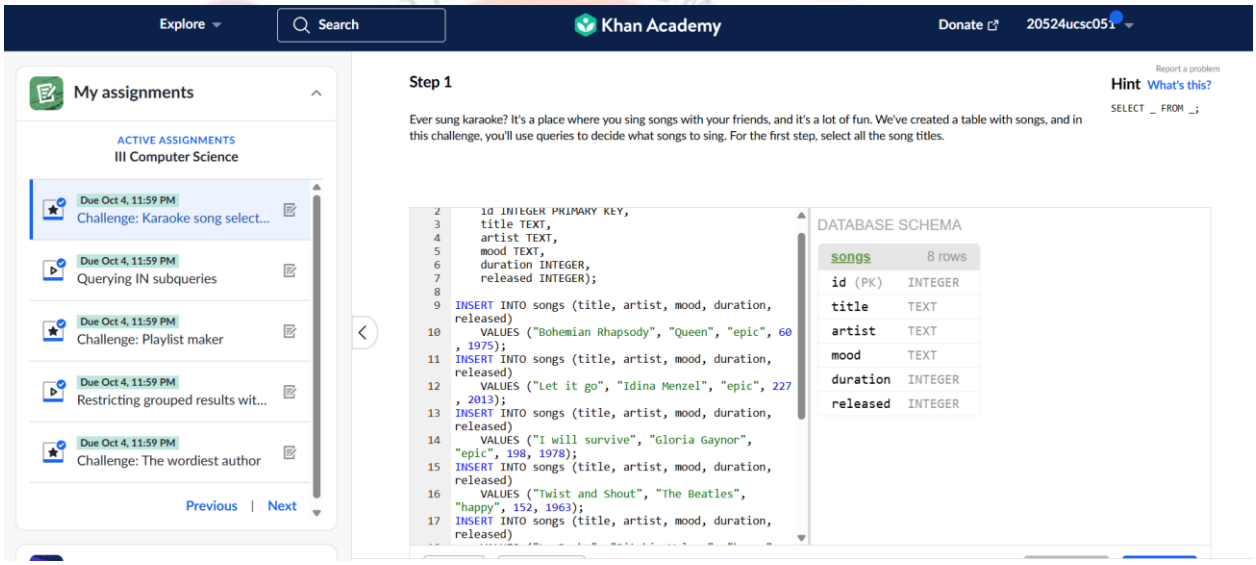
KARAOKE SONG SELECTOR

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles. Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990. People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', and released after 1990, and less than 4 minutes long.

Program:

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
```

```
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);
select title from songs;
select title from songs where mood = 'epic' or released > 1990;
select title from songs where mood = 'epic' and released > 1990 and duration < 240;
```



PLAYLIST MAKER

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the **title** of all the songs by the **artist** named 'Queen'. Now you'll make a 'Pop' playlist. In preparation, select the **name** of all of the artists from the 'Pop' genre. To finish creating the 'Pop' playlist, add another query that will select the **title** of all the songs from the 'Pop' artists. It should use **IN** on a nested subquery that's based on your previous query.

Program:

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT);
INSERT INTO artists (name, country, genre)
    VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
```

```
    VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");
CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);
INSERT INTO songs (artist, title)
```

```
    VALUES ("Taylor Swift", "Shake it off");

INSERT INTO songs (artist, title)

    VALUES ("Rihanna", "Stay");

INSERT INTO songs (artist, title)

    VALUES ("Celine Dion", "My heart will go on");

INSERT INTO songs (artist, title)

    VALUES ("Celine Dion", "A new day has come");

INSERT INTO songs (artist, title)

    VALUES ("Shania Twain", "Party for two");

INSERT INTO songs (artist, title)

    VALUES ("Gloria Estefan", "Conga");

INSERT INTO songs (artist, title)

    VALUES ("Led Zeppelin", "Stairway to heaven");

INSERT INTO songs (artist, title)

    VALUES ("ABBA", "Mamma mia");

INSERT INTO songs (artist, title)

    VALUES ("Queen", "Bicycle Race");

INSERT INTO songs (artist, title)

    VALUES ("Queen", "Bohemian Rhapsody");

INSERT INTO songs (artist, title)
```

```
    VALUES ("Guns N' Roses", "Don't cry");

select title from songs where artist = 'Queen';

select name from artists where genre = 'Pop';

SELECT title

FROM songs

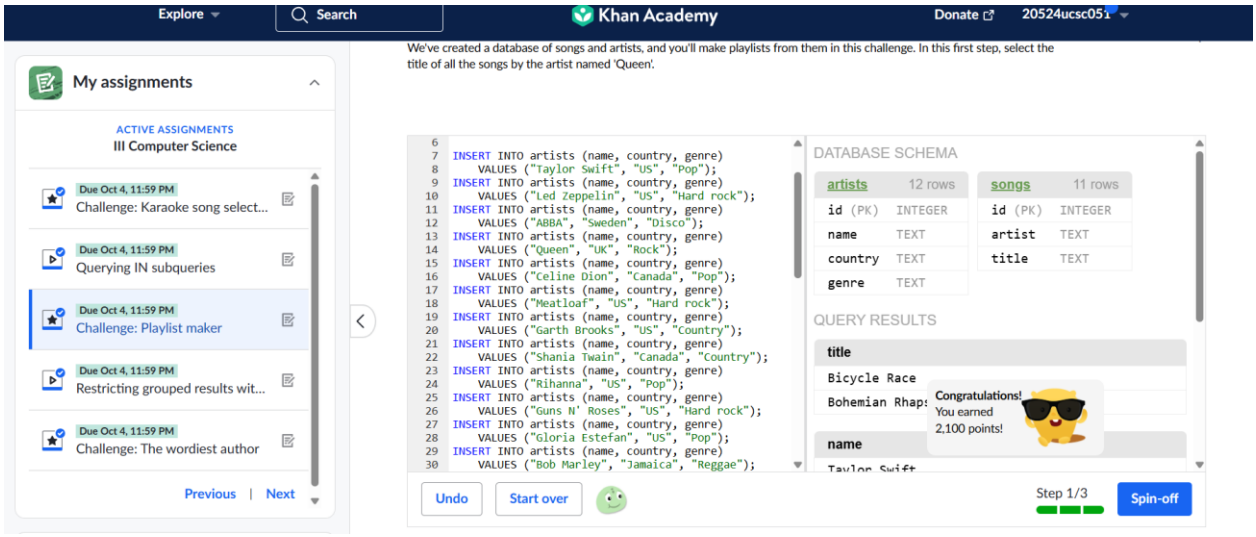
WHERE artist IN (

    SELECT name

    FROM artists

    WHERE genre = 'Pop'

);
```



THE WORDIEST AUTHOR

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column. Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

Program:

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
```

```
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
SELECT author,
       SUM(words) AS total_words
```

```
FROM books

GROUP BY author

HAVING SUM(words) > 1000000;

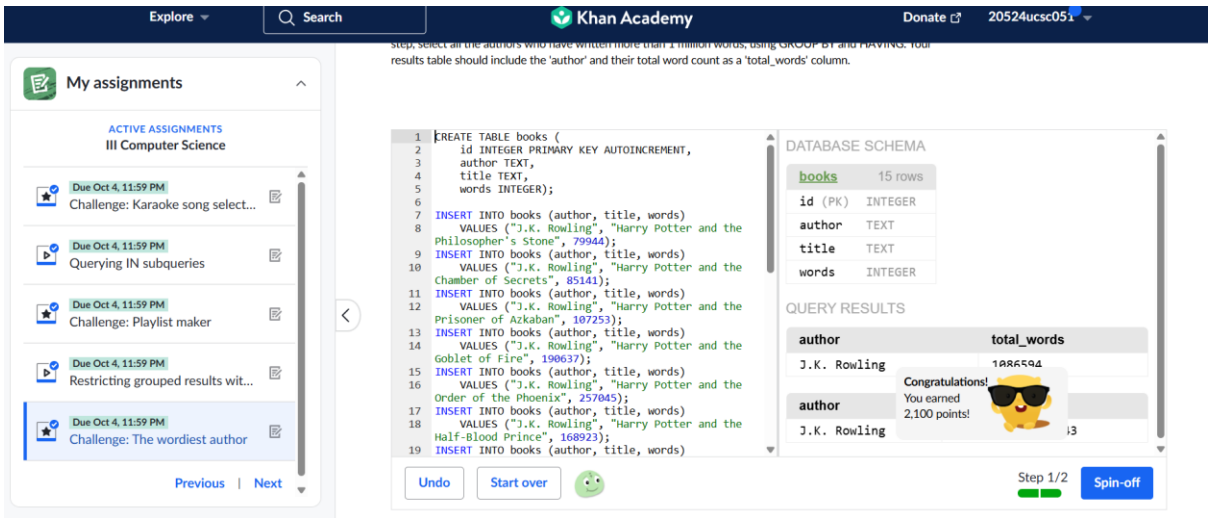
SELECT author,

    AVG(words) AS avg_words

FROM books

GROUP BY author

HAVING AVG(words) > 150000;
```

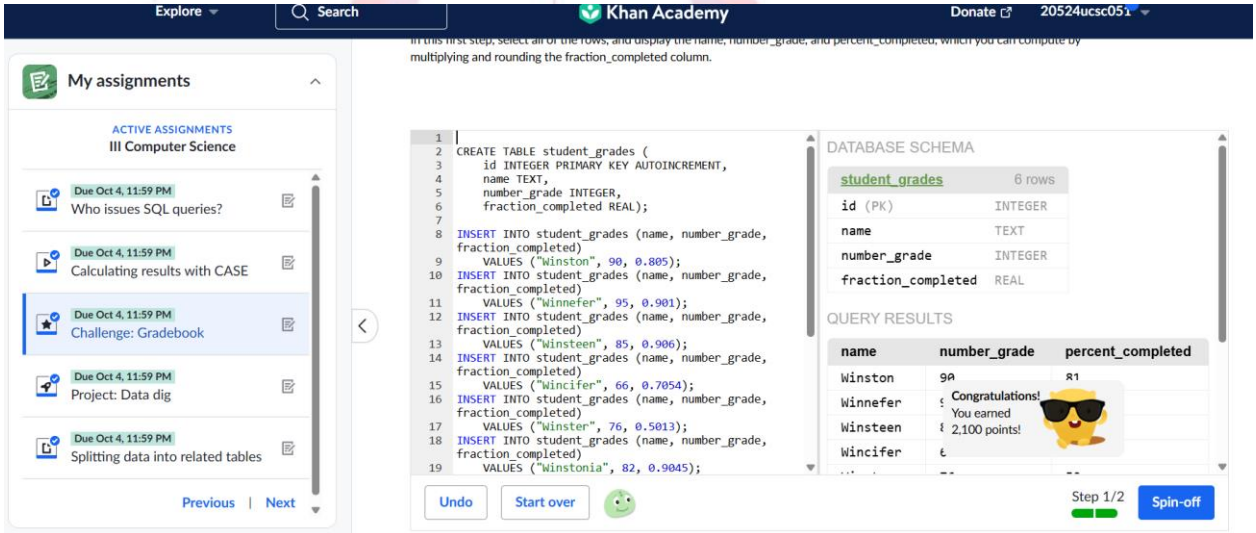


We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column. Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

Program:

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winstonia", 82, 0.9045);
SELECT name,
       number_grade,
       ROUND(fraction_completed * 100) AS percent_completed
```

```
FROM student_grades;
SELECT COUNT(*),
    CASE
        WHEN number_grade > 90 THEN 'A'
        WHEN number_grade > 80 THEN 'B'
        WHEN number_grade > 70 THEN 'C'
        ELSE 'F'
    END AS "letter_grade"
FROM student_grades
GROUP BY
    CASE
        WHEN number_grade > 90 THEN 'A'
        WHEN number_grade > 80 THEN 'B'
        WHEN number_grade > 70 THEN 'C'
        ELSE 'F'
    END;
```



DATA DIG

What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?

Program:

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY,
    name TEXT,
    type TEXT,
    hp INTEGER,
    attack INTEGER
);
INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);
SELECT
    AVG(attack) AS avg_attack,
    MAX(attack) AS max_attack,
```

```
 MIN(attack) AS min_attack
FROM pokemon;
SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type;
SELECT


type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 80;
SELECT
    name,
    attack,
    CASE
        WHEN attack >= 100 THEN "Very Strong"
        WHEN attack >= 80 THEN "Strong"
        ELSE "Average"
    END AS strength
FROM pokemon;
SELECT *
FROM pokemon
WHERE (type = "Fire" OR type = "Water")
AND attack > 80;
```

Output

DATABASE SCHEMA

pokemon 8 rows
id (PK) INTEGER
name TEXT
type TEXT
hp INTEGER
attack INTEGER

QUERY RESULTS

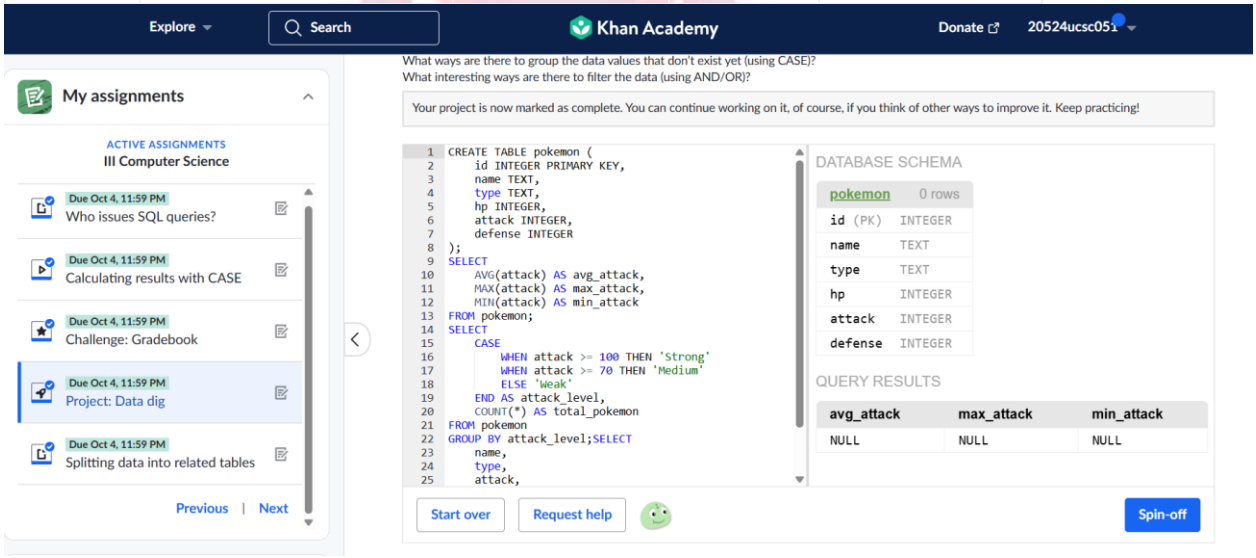
avg_attack	max_attack	min_attack
86.125	125	55

type	avg_attack
Electric	60
Fire	97
Grass	83.5
Water	104

type	avg_attack
Fire	97
Grass	83.5
Water	104

name	attack	strength
Pikachu	55	Average
Charizard	84	Strong
Blastoise	83	Strong
Venusaur	82	Strong
Jolteon	65	Average
Arcanine	110	Very Strong
Gyarados	125	Very Strong
Sceptile	85	Strong

id	name	type	hp	attack
2	Charizard	Fire	78	84
3	Blastoise	Water	79	83
6	Arcanine	Fire	90	110
7	Gyarados	Water	95	125



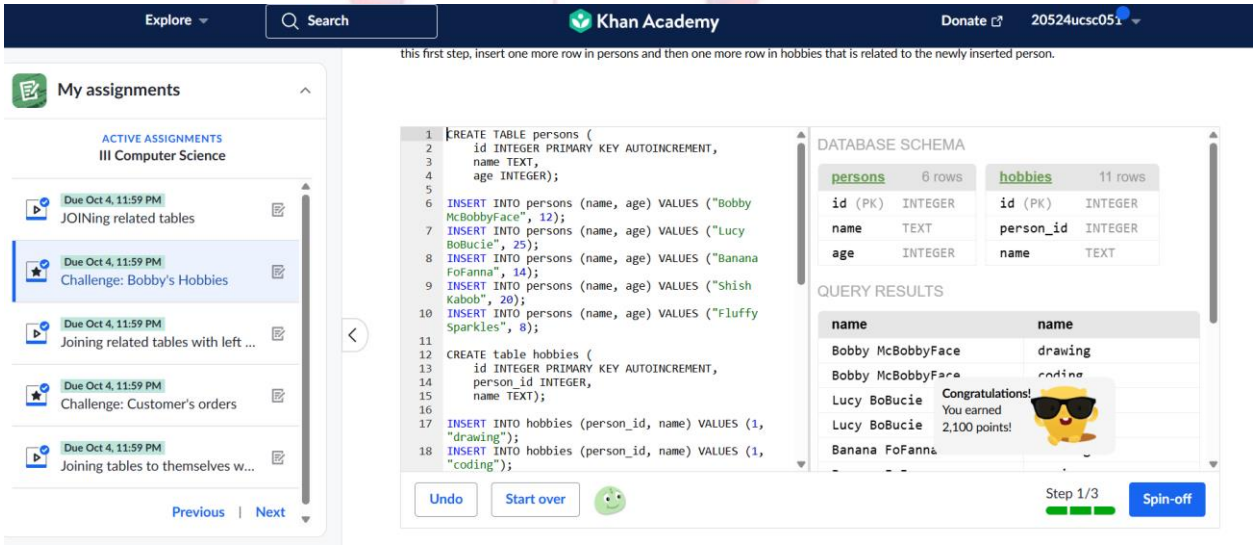
BOBBY'S HOBBIES

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person. Now, select the 2 tables with a join so that you can see each person's name next to their hobby. Now, add an additional query that shows only the name and hobbies of 'Bobby McBobbyFace', using JOIN combined with WHERE.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER);
INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
```

```
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Sam Star", 18);
INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name,
    hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```



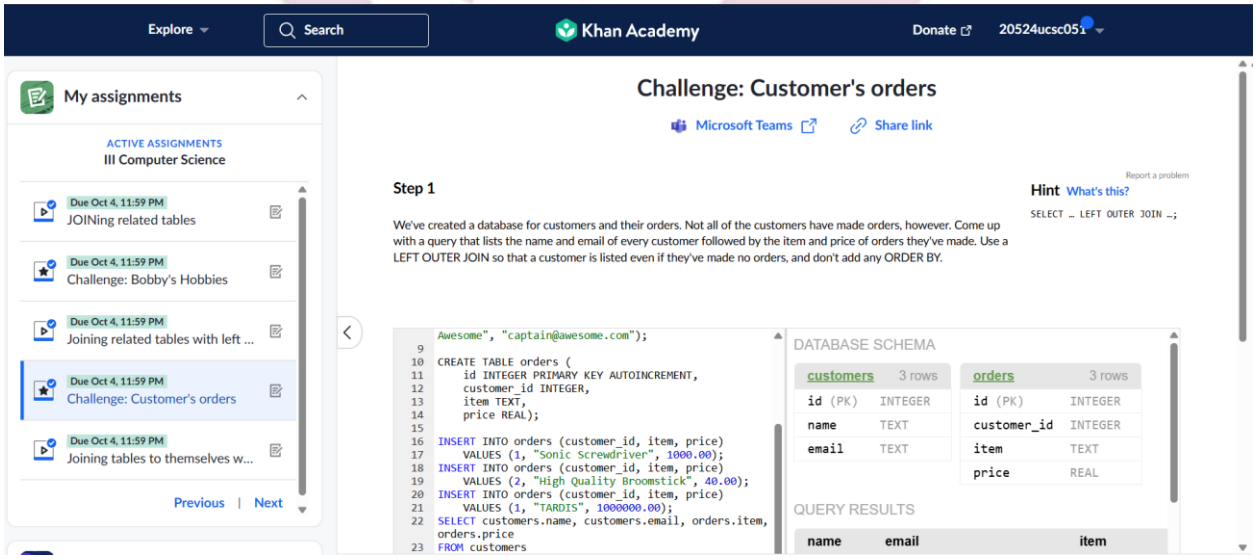
CUSTOMER'S ORDERS

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY. Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.

Program:

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT);
INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);
SELECT customers.name,
      customers.email,
```

```
  orders.item,
      orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT customers.name,
      customers.email,
      SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```



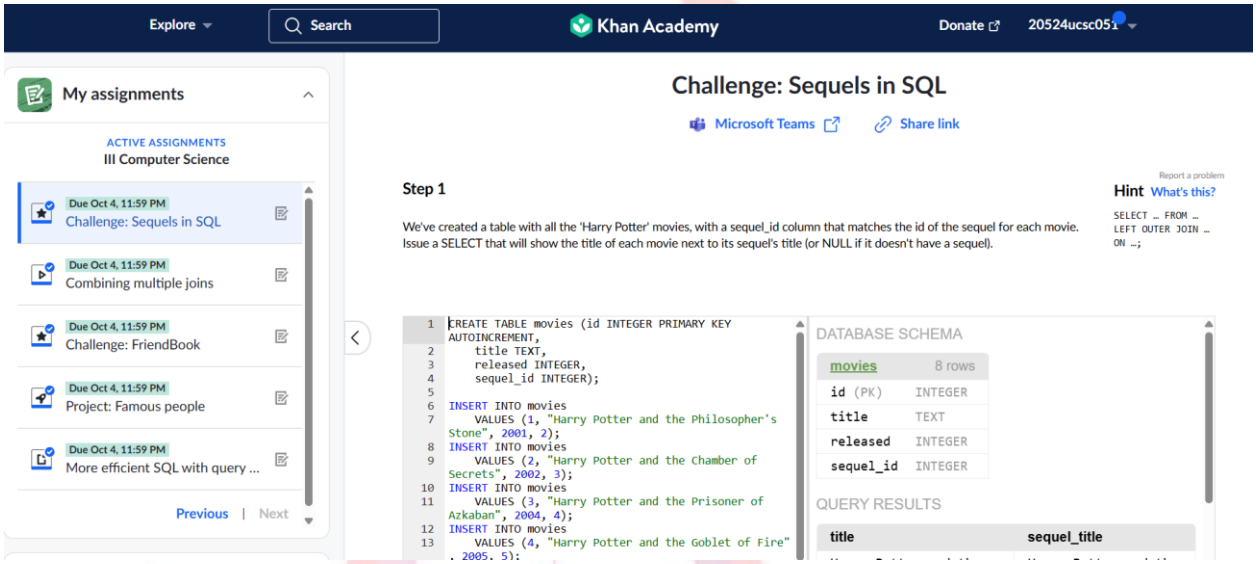
SEQUELS IN SQL

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    released INTEGER,
    sequel_id INTEGER);
INSERT INTO movies
    VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
    VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
    VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
    VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
    VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
    VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
    VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
    VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
SELECT m1.title,
```

```
  m2.title AS sequel_title
FROM movies AS m1
LEFT OUTER JOIN movies AS m2
ON m1.sequel_id = m2.id;
```



FRIENDBOOK

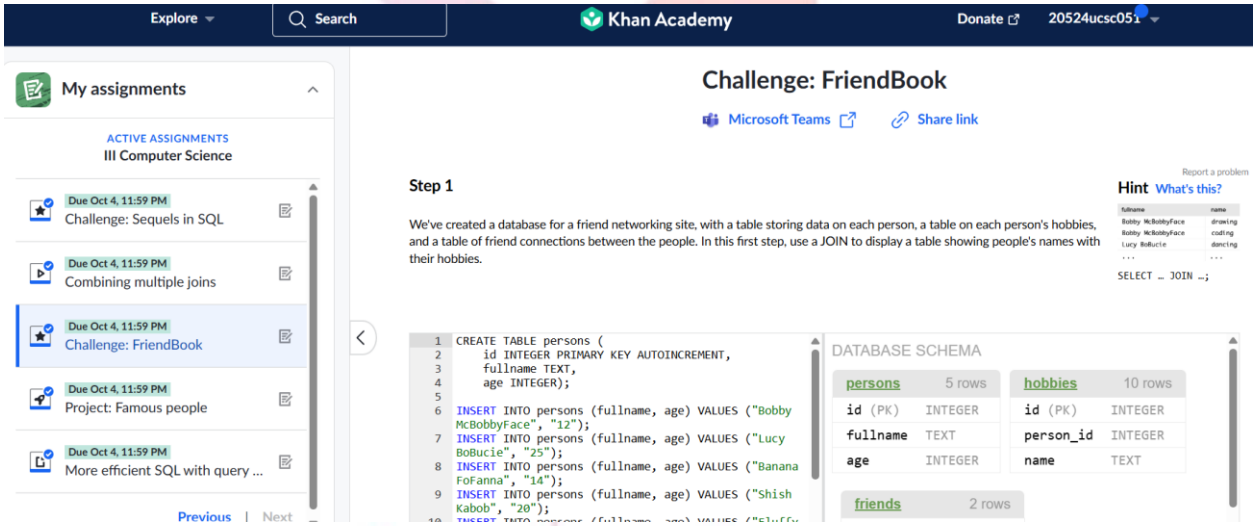
We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies. Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
fullname TEXT,
    age INTEGER);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
CREATE table friends (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person1_id INTEGER,
    person2_id INTEGER);
INSERT INTO friends (person1_id, person2_id)
    VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
    VALUES (2, 3);
```

```
SELECT persons.fullname,
    hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT p1.fullname AS person1,
    p2.fullname AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.idON friends.person2_id = p2.id;
```



FAMOUS PEOPLE

Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?

Program:

```
CREATE TABLE authors (
```

```
 id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);
INSERT INTO authors VALUES (1, "J.K. Rowling", "United Kingdom");
INSERT INTO authors VALUES (2, "George Orwell", "United Kingdom");
INSERT INTO authors VALUES (3, "Mark Twain", "United States");
INSERT INTO authors VALUES (4, "Jane Austen", "United Kingdom");
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER
);
INSERT INTO books VALUES (1, "Harry Potter and the Sorcerer's Stone", 1);
INSERT INTO books VALUES (2, "Harry Potter and the Chamber of Secrets", 1);
INSERT INTO books VALUES (3, "1984", 2);
INSERT INTO books VALUES (4, "Animal Farm", 2);
INSERT INTO books VALUES (5, "The Adventures of Tom Sawyer", 3);
INSERT INTO books VALUES (6, "Pride and Prejudice", 4);
CREATE TABLE author_friends (
    id INTEGER PRIMARY KEY,
    author1_id INTEGER,
    author2_id INTEGER
);

INSERT INTO author_friends VALUES (1, 1, 2);
INSERT INTO author_friends VALUES (2, 3, 4);
SELECT authors.name AS author,
       books.title AS book
```

```
FROM authors
JOIN books
ON authors.id = books.author_id;
SELECT name,
    country
FROM authors;
SELECT a1.name AS author1,
    a2.name AS author2
FROM author_friends
JOIN authors AS a1
ON author_friends.author1_id = a1.id
JOIN authors AS a2
ON author_friends.author2_id = a2.id;
```

Output:

DATABASE SCHEMA

authors4 rows
id (PK)INTEGER
nameTEXT
countryTEXT

books6 rows
id (PK)INTEGER
titleTEXT
author_idINTEGER

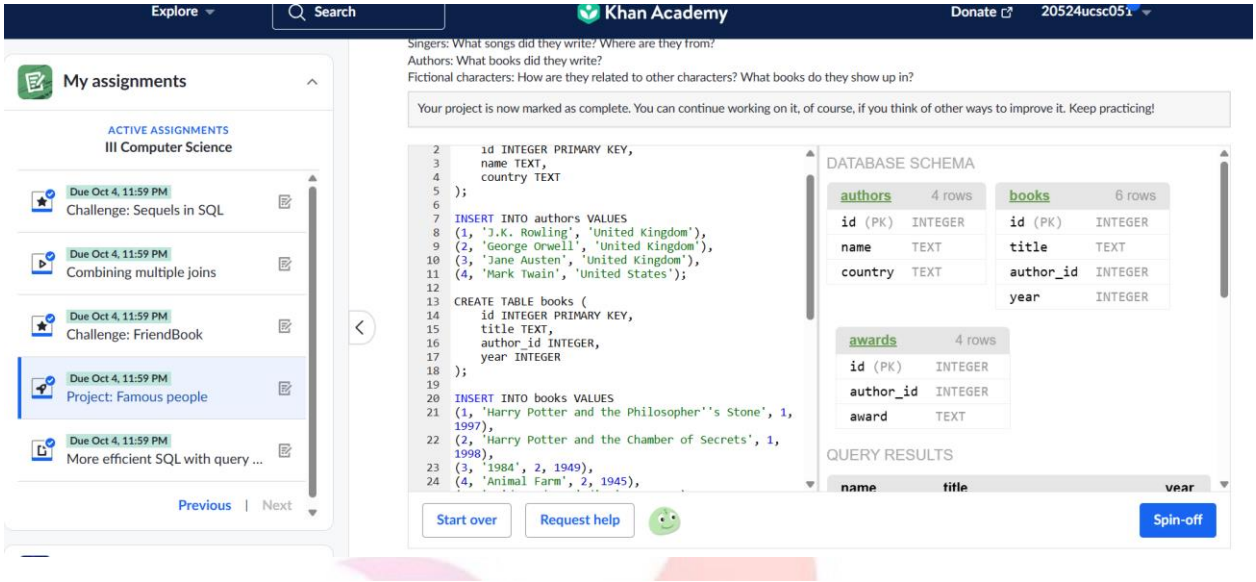
author_friends2 rows
id (PK)INTEGER
author1_idINTEGER
author2_idINTEGER

QUERY RESULTS

author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
J.K. Rowling	Harry Potter and the Chamber of Secrets
George Orwell	1984
George Orwell	Animal Farm
Mark Twain	The Adventures of Tom Sawyer
Jane Austen	Pride and Prejudice

name	country
J.K. Rowling	United Kingdom
George Orwell	United Kingdom
Mark Twain	United States
Jane Austen	United Kingdom

author1	author2
J.K. Rowling	George Orwell
Mark Twain	Jane Austen



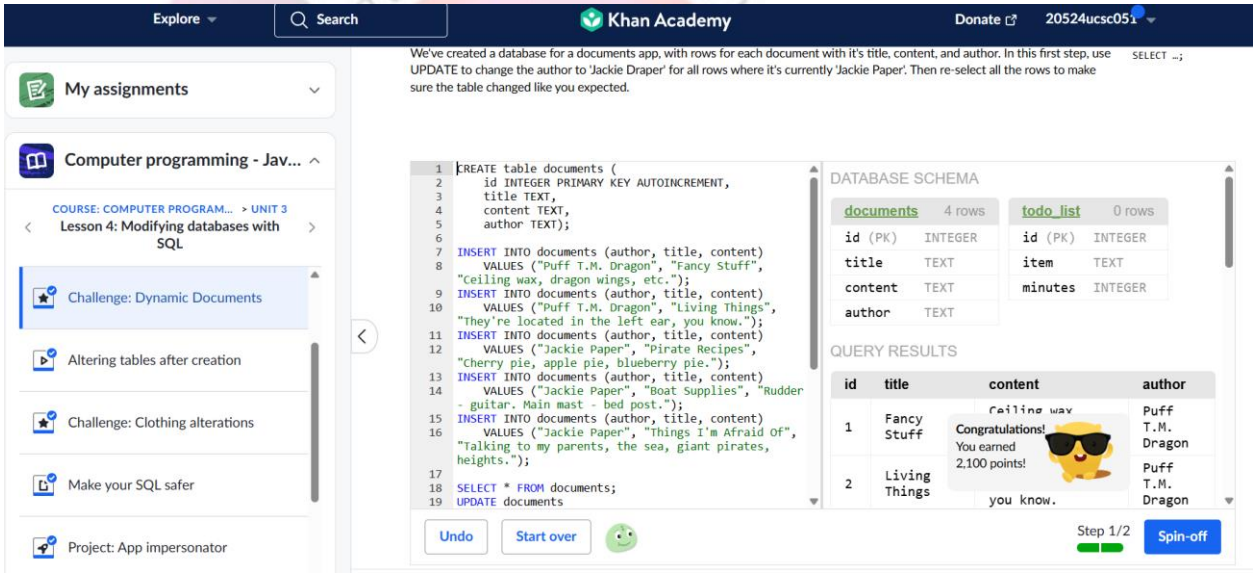
DYNAMIC DOCUMENTS

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected. Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

Program:

```
CREATE table documents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    content TEXT,
    author TEXT);
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
```

```
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");
SELECT * FROM documents;
UPDATE documents
SET author = "Jackie Draper"
WHERE author = "Jackie Paper";
SELECT * FROM documents;
DELETE FROM documents
WHERE title = "Things I'm Afraid Of";
SELECT * FROM documents;
```



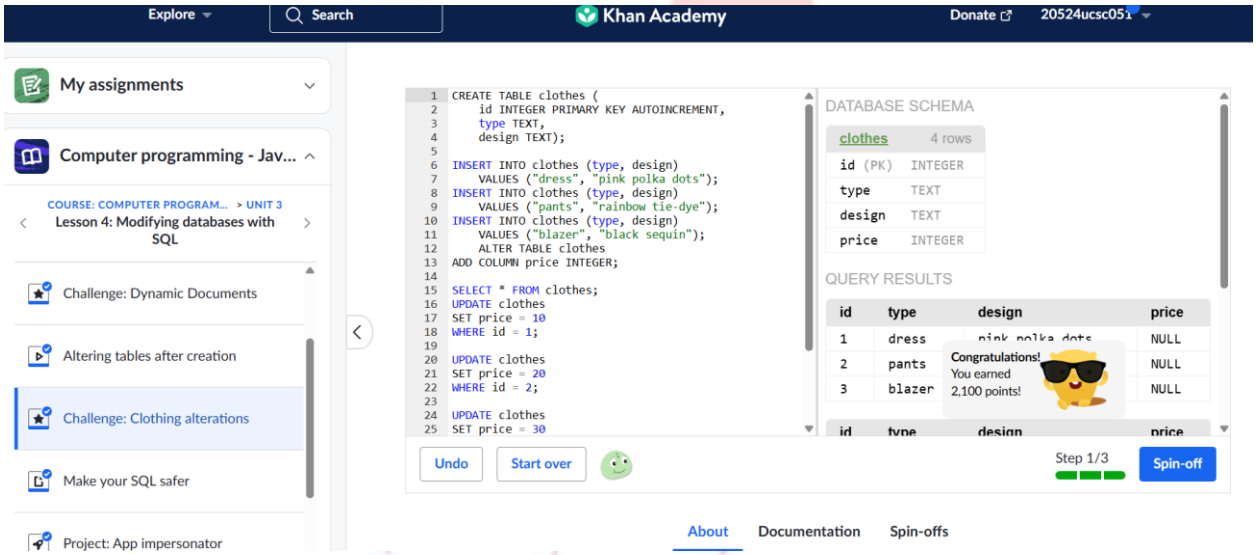
CLOTHING ALTERATIONS

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now. Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all the rows to check that it worked as expected. Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

Program:

```
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    type TEXT,
    design TEXT);
INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");
ALTER TABLE clothes
ADD COLUMN price REAL;
SELECT * FROM clothes;
UPDATE clothes
SET price = 10
WHERE id = 1;
UPDATE clothes
SET price = 20
WHERE id = 2;
```

```
UPDATE clothes
SET price = 30
WHERE id = 3;
SELECT * FROM clothes;
INSERT INTO clothes (type, design, price)
VALUES ("shirt", "blue stripes", 15);
SELECT * FROM clothes;
```



PROJECT: APP IMPERSONATO

CREATE a table to store the data.
INSERT a few example rows in the table.
Use an UPDATE to emulate what happens when you edit data in the app.
Use a DELETE to emulate what happens when you delete data in the app.

Program:

```
CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    content TEXT,
    likes INTEGER
);
INSERT INTO posts (username, content, likes)
VALUES ("alex", "Just finished my homework!", 12);
INSERT INTO posts (username, content, likes)
VALUES ("sam", "Playing my favorite game tonight.", 25);
INSERT INTO posts (username, content, likes)
VALUES ("taylor", "Had a great day at the beach.", 18);
SELECT * FROM posts;
UPDATE posts
SET content = "Just finished my homework and studied SQL!"
WHERE id = 1;
SELECT * FROM posts;
DELETE FROM posts
WHERE id = 2;
SELECT * FROM posts;
```

Output:

DATABASE SCHEMA

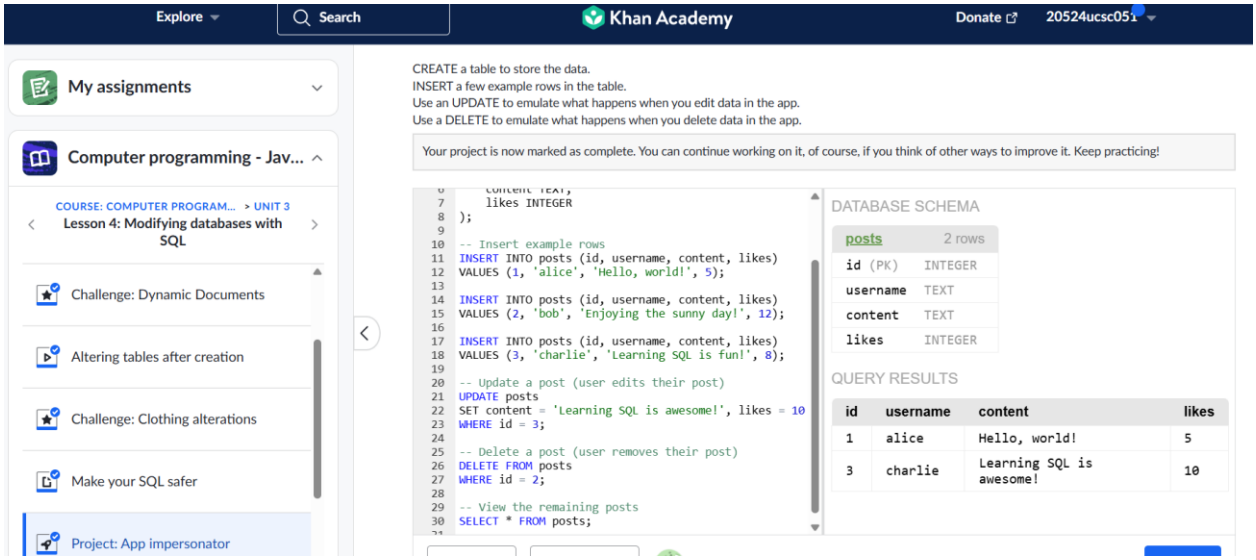
posts2 rows
id (PK)INTEGER
usernameTEXT
contentTEXT
likesINTEGER

QUERY RESULTS

id	username	content	likes
1	alex	Just finished my homework!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
3	taylor	Had a great day at the beach.	18



HACHERRANK

REVISING THE SELECT QUERY I

Query all columns for all American cities in the **CITY** table with populations larger than `100000`. The **CountryCode** for America is `USA`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

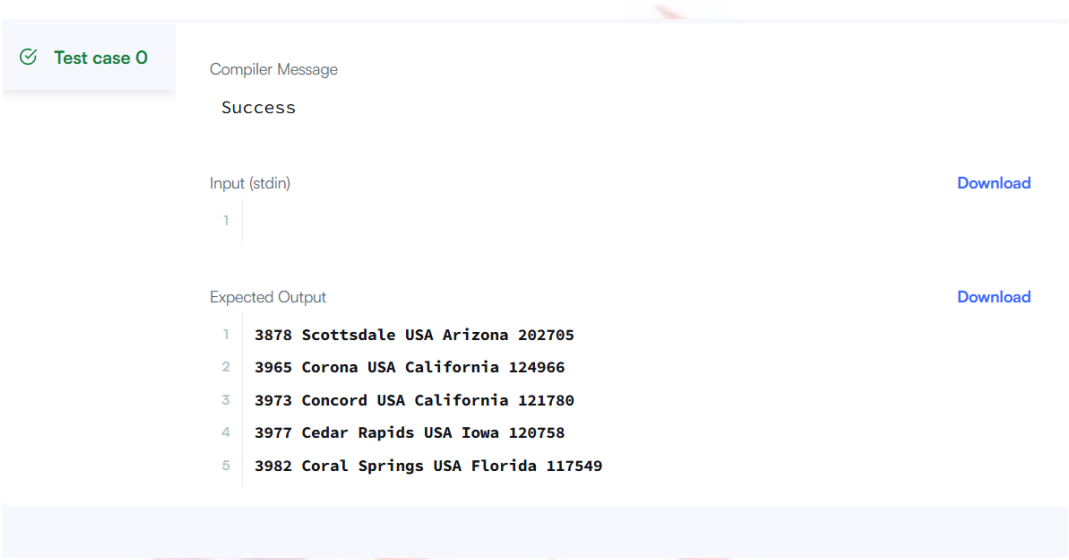
Program

```
SELECT *

FROM CITY

WHERE CountryCode = 'USA'

  AND Population > 100000;
```



REVISING THE SELECT QUERY II

Query the NAME field for all American cities in the CITY table with populations larger than 120000. The *CountryCode* for America is USA.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

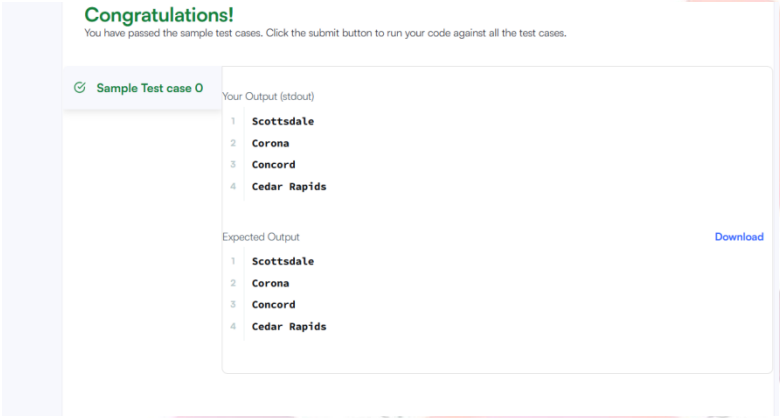
Program

```
SELECT Name

FROM CITY

WHERE CountryCode = 'USA'

  AND Population > 120000;
```



SELECT ALL

Query all columns (attributes) for every row in the CITY table.

The CITY table is described as follows:

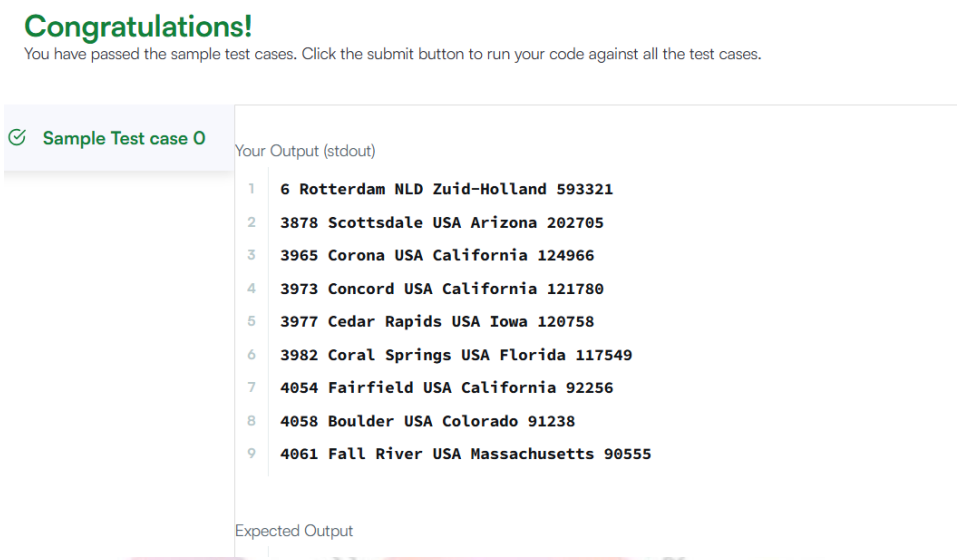
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

```
SELECT *

FROM CITY;
```



SELECT BY ID

Query all columns for a city in CITY with the *ID* 1661.

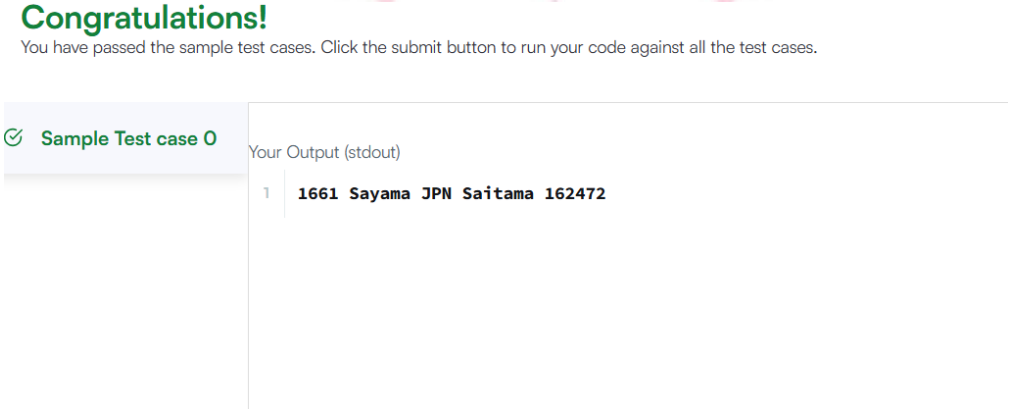
The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

```
SELECT *
FROM CITY
WHERE ID = 1661;
```



JAPANESE CITIES' ATTRIBUTES

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

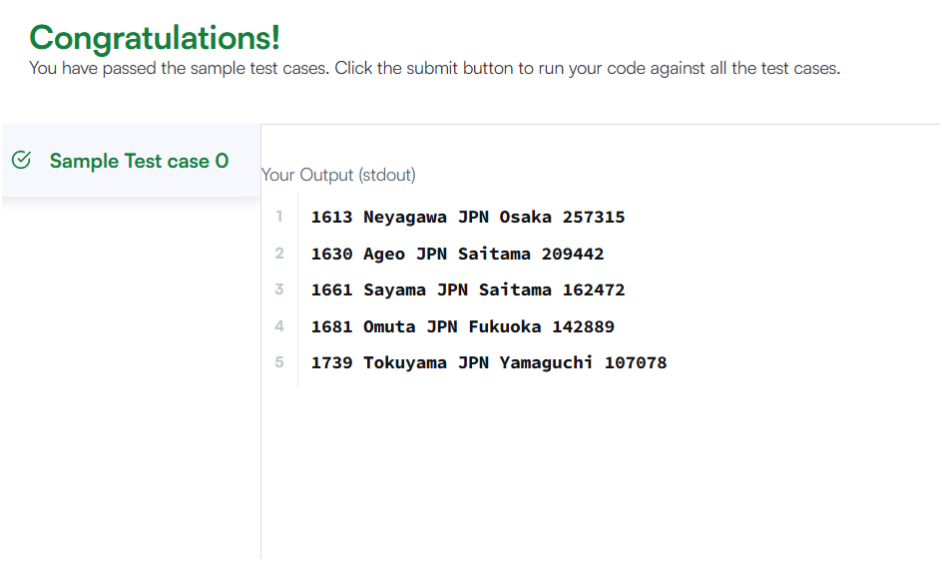
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

```
SELECT *

FROM CITY

WHERE CountryCode = 'JPN';
```



JAPANESE CITIES' NAMES

Query the names of all the Japanese cities in the CITY table. The COUNTRYCODE for Japan is JPN.

The CITY table is described as follows:

CITY

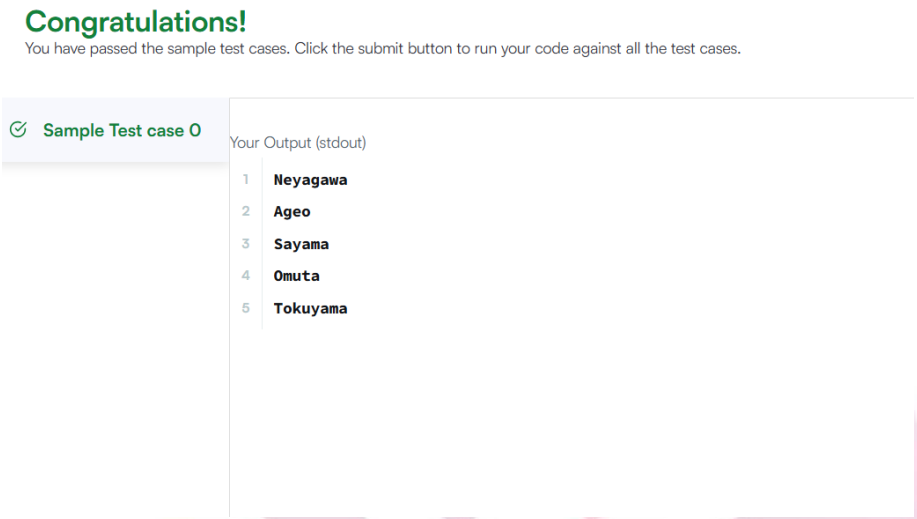
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

```
SELECT Name

FROM CITY
```

WHERE CountryCode = 'JPN';



WEATHER OBSERVATION STATION 1

Query a list of CITY and STATE from the STATION table.

The STATION table is described as follows:

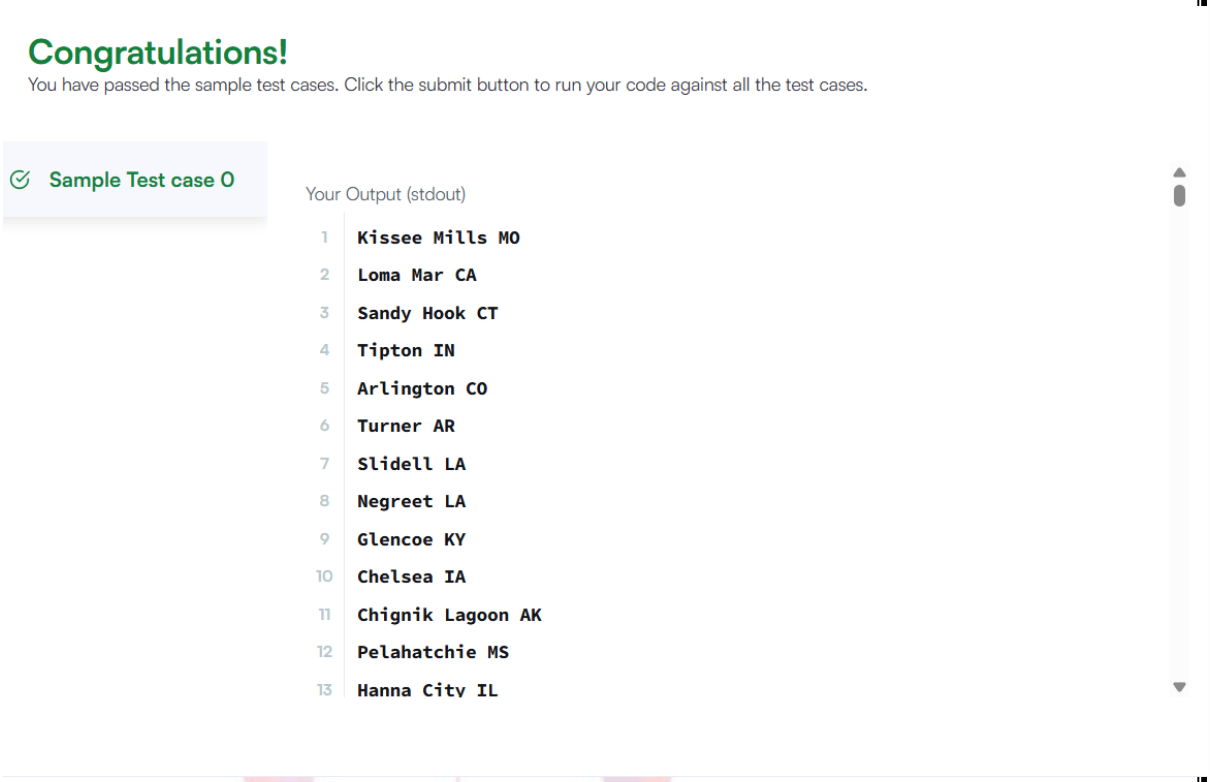
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

Program

```
SELECT CITY, STATE

FROM STATION;
```



WEATHER OBSERVATION STATION 3

Query a list of CITY names from STATION for cities that have an even ID number.

Print the results in any order, but exclude duplicates from the answer.

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

Program

```
SELECT DISTINCT CITY
FROM STATION
WHERE MOD(ID, 2) = 0;
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Kissee Mills
Loma Mar
Tipton
Glencoe
Chignik Lagoon
Albany
Manchester
Cahone
Bowdon
Watkins
Millville
Aguanga
Morenci
```

WEATHER OBSERVATION STATION 4

Find the difference between the total number of CITY entries in the table and the number of distinct CITY entries in the table.

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

Program

```
SELECT COUNT(CITY) - COUNT(DISTINCT CITY) AS Difference
FROM STATION;
```



WEATHER OBSERVATION STATION 5

Query the two cities in STATION with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Sample Input

For example, CITY has four entries: DEF, ABC, PQRS and WXY.

Sample Output

ABC 3

PQRS 4

Explanation

When ordered alphabetically, the CITY names are listed as ABC, DEF, PQRS, and WXY, with lengths and . The longest name is PQRS, but there are options for shortest named city. Choose ABC, because it comes first alphabetically.

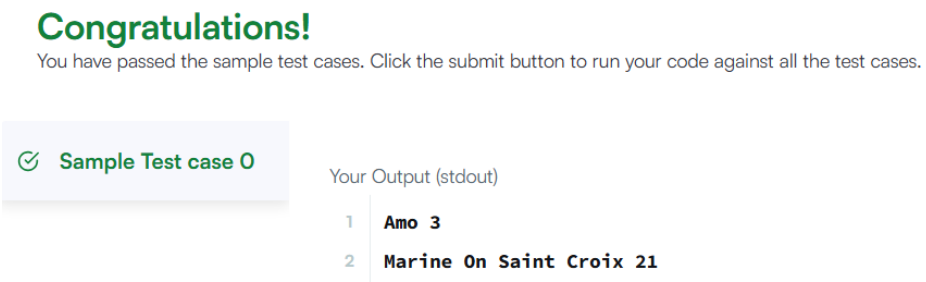
Note

You can write two separate queries to get the desired output. It need not be a single query.

Program

```
SELECT CITY, LENGTH(CITY) AS NameLength
FROM STATION
ORDER BY LENGTH(CITY) ASC, CITY ASC
LIMIT 1;

-- City with longest name
SELECT CITY, LENGTH(CITY) AS NameLength
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY ASC
LIMIT 1;
```



WEATHER OBSERVATION STATION 6

Query the list of *CITY* names starting with vowels (i.e., a, e, i, o, or u) from STATION. Your result *cannot* contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

Program

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY LIKE 'A%'
  OR CITY LIKE 'E%'
  OR CITY LIKE 'I%'
  OR CITY LIKE 'O%'
  OR CITY LIKE 'U%';
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Arlington
Albany
Upperco
Aguanga
Odin
East China
Algonac
Onaway
Irvington
Arrowsmith
Udall
Oakfield
Elkton
```

WEATHER OBSERVATION STATION 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from STATION. Your result *cannot* contain duplicates.

Input Format

The STATION table is described as follows:

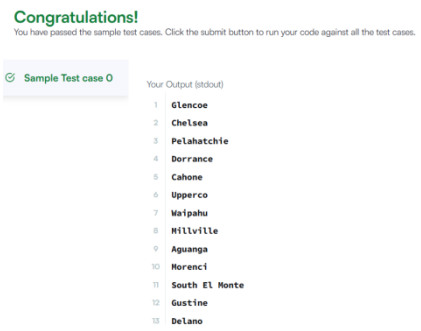
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY LIKE '%a'
  OR CITY LIKE '%e'
  OR CITY LIKE '%i'
  OR CITY LIKE '%o'
  OR CITY LIKE '%u';
```



WEATHER OBSERVATION STATION 8

Query the list of *CITY* names from STATION which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

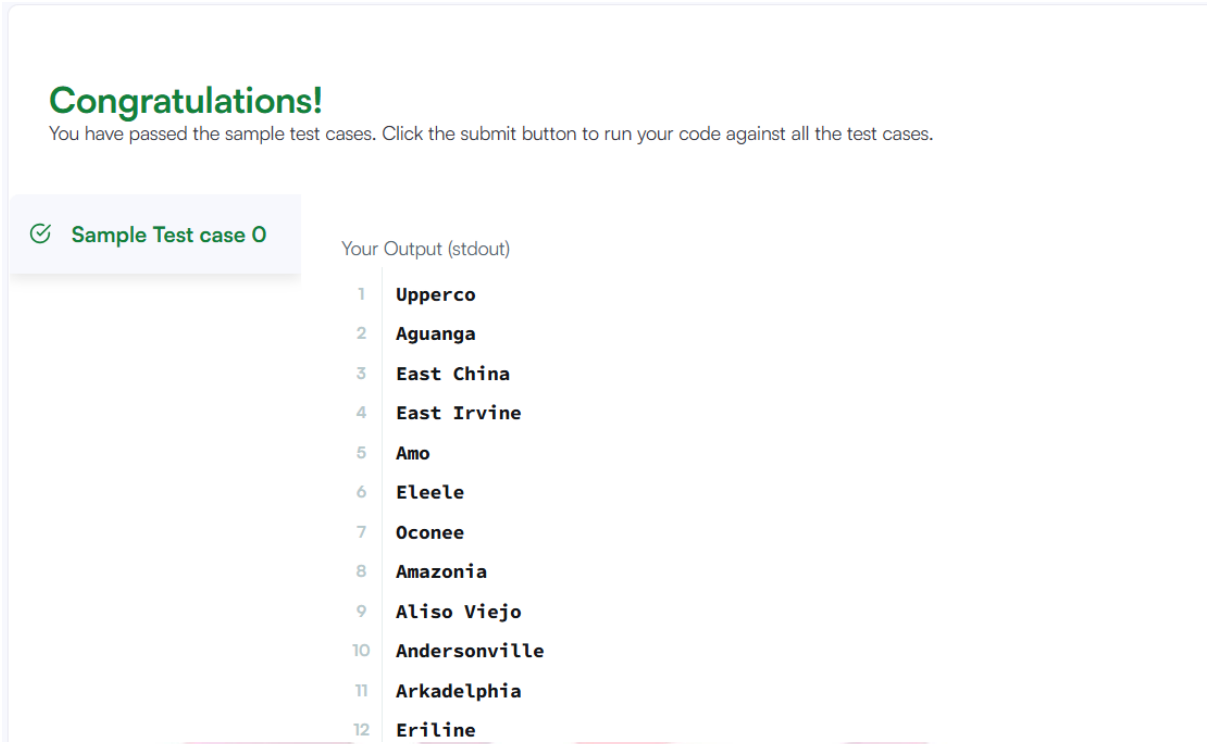
Program

```
SELECT DISTINCT CITY

FROM STATION

WHERE (CITY LIKE 'A%' OR CITY LIKE 'E%' OR CITY LIKE 'I%' OR CITY LIKE 'O%' OR CITY LIKE 'U%')

  AND (CITY LIKE '%A' OR CITY LIKE '%E' OR CITY LIKE '%I' OR CITY LIKE '%O' OR CITY LIKE '%U');
```



WEATHER OBSERVATION STATION 9

Query the list of *CITY* names from STATION that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

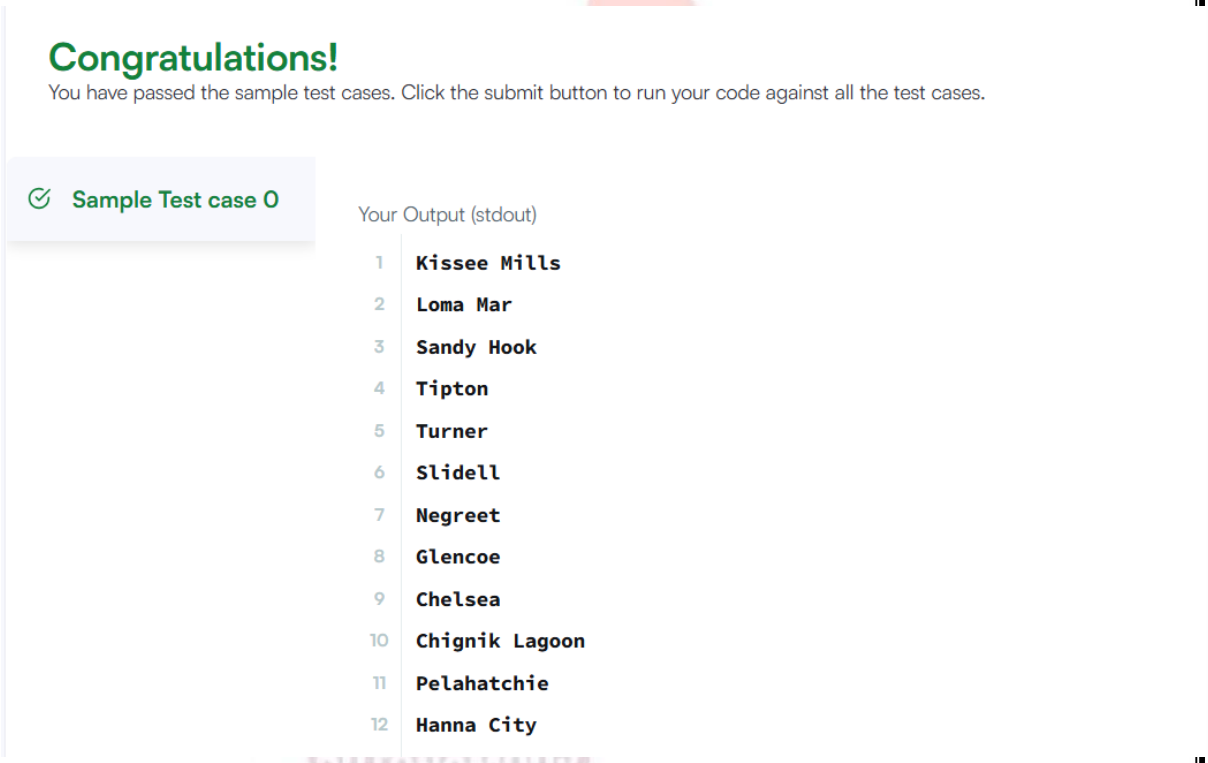
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

Program

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT LIKE 'A%'
 AND CITY NOT LIKE 'E%'
 AND CITY NOT LIKE 'I%'
 AND CITY NOT LIKE 'O%'
 AND CITY NOT LIKE 'U%';
```



WEATHER OBSERVATION STATION 10

Query the list of *CITY* names from STATION that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

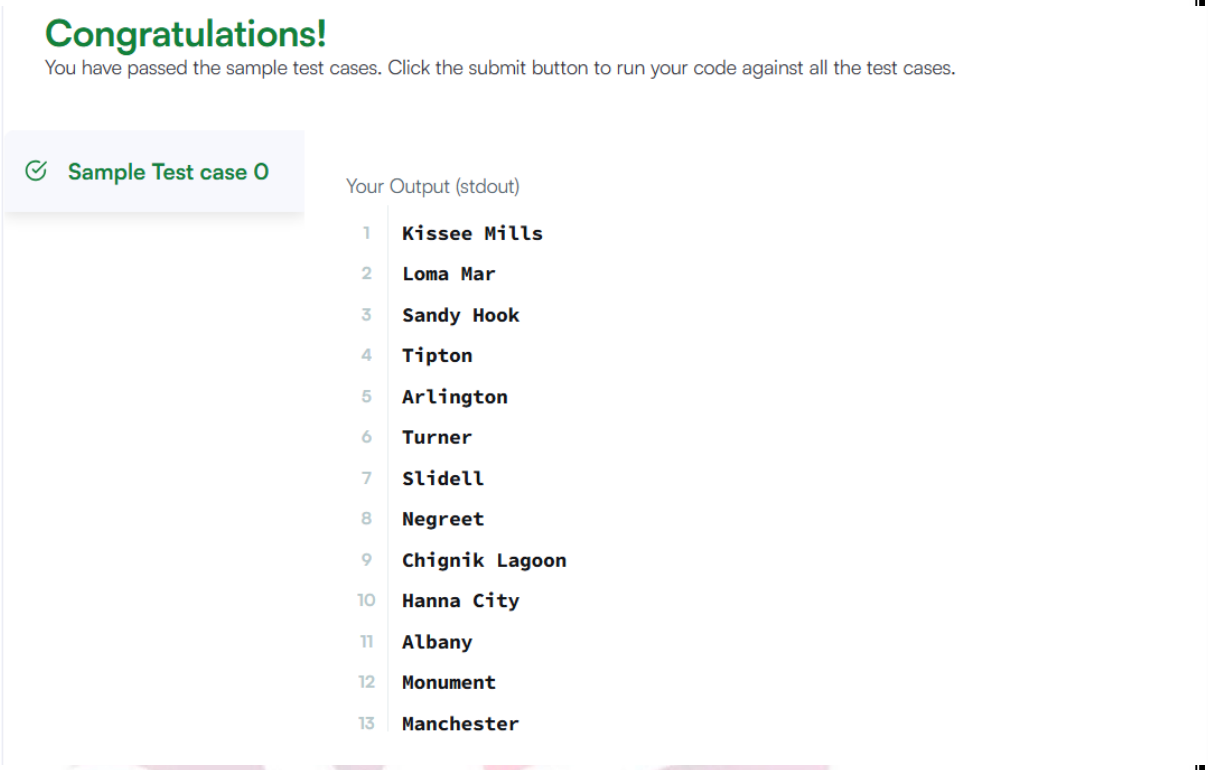
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT LIKE '%A'
  AND CITY NOT LIKE '%E'
  AND CITY NOT LIKE '%I'
  AND CITY NOT LIKE '%O'
  AND CITY NOT LIKE '%U';
```



WEATHER OBSERVATION STATION 11

Query the list of *CITY* names from STATION that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

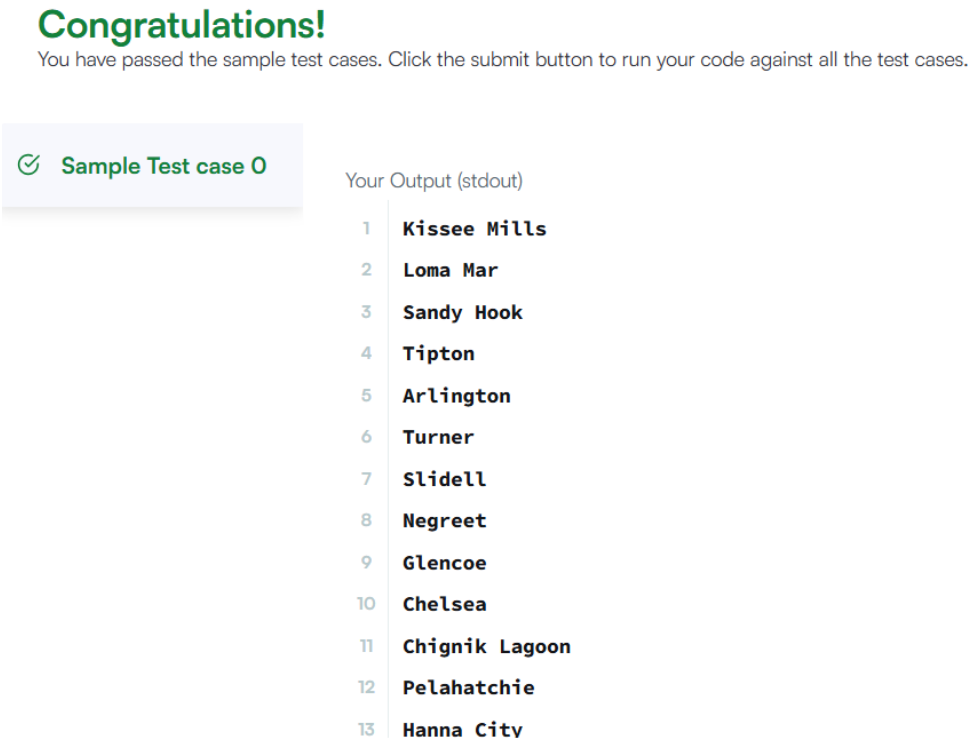
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

Program

```
FROM STATION
WHERE CITY NOT REGEXP '^[AEIOUaeiou].*[AEIOUaeiou]$';
SELECT DISTINCT CITY
```



WEATHER OBSERVATION STATION 12

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

Program

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT REGEXP '^[AEIOUaeiou]'
 AND CITY NOT REGEXP '[AEIOUaeiou]$';
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Kissee Mills
Loma Mar
Sandy Hook
Tipton
Turner
Slidell
Negreet
Chignik Lagoon
Hanna City
Monument
Manchester
Prescott
```

HIGHER THAN 75 MARKS

Query the Name of any student in STUDENTS who scored higher than Marks. Order your output by the last three characters of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending ID.

Input Format

The STUDENTS table is described as follows:

Column	Type
ID	Integer
Name	String
Marks	Integer

The Name column only contains uppercase (A-Z) and lowercase (a-z) letters.

Sample Input

ID	Name	Marks
1	Ashley	81
2	Samantha	75
4	Julia	76
3	Belvet	84

Sample Output

Ashley

Julia

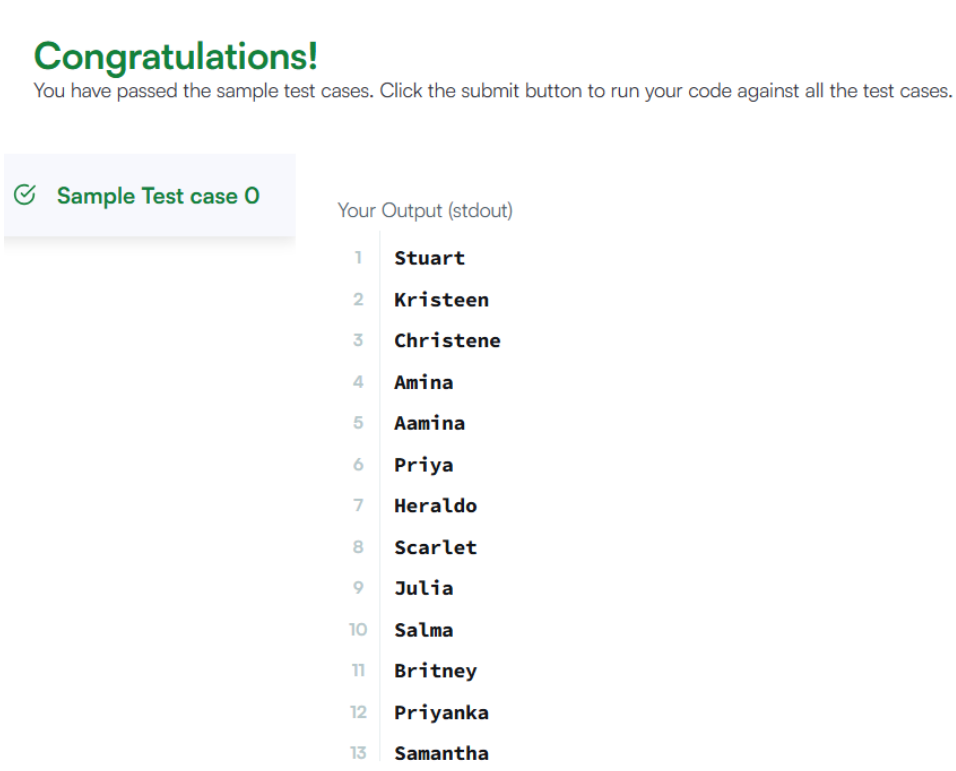
Belvet

Explanation

Only Ashley, Julia, and Belvet have *Marks* > . If you look at the last three characters of each of their names, there are no duplicates and 'ley' < 'lia' < 'vet'.

Program

```
SELECT Name
FROM STUDENTS
WHERE Marks > 75
ORDER BY RIGHT(Name, 3), ID;
```



EMPLOYEE NAMES

Write a query that prints a list of employee names (i.e.: the *name* attribute) from the Employee table in alphabetical order.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

Angela

Bonnie

Frank

Joe

Kimberly

Lisa

Michael

Patrick

Rose

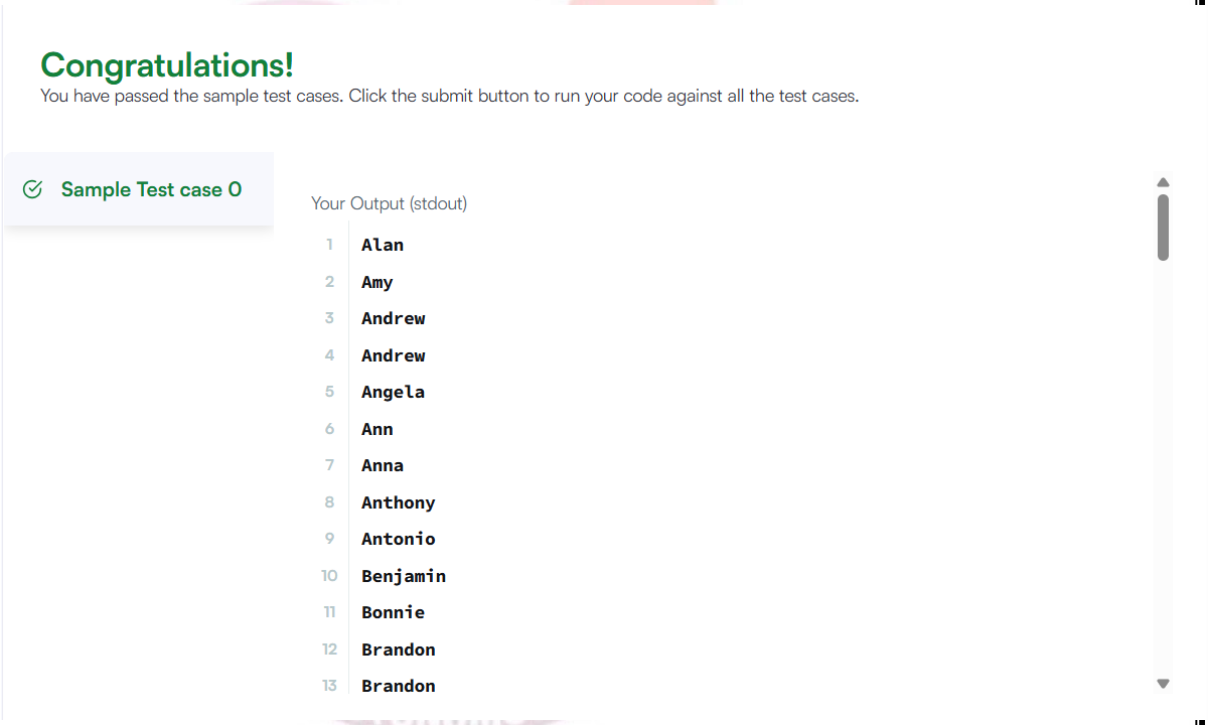
Todd

Program

SELECT name

FROM Employee

ORDER BY name ASC;



EMPLOYEE SALARIES

Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in Employee having a salary greater than per month who have been employees for less than months. Sort your result by ascending *employee_id*.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

Angela

Michael

Todd

Joe

Explanation

Angela has been an employee for month and earns per month.

Michael has been an employee for months and earns per month.

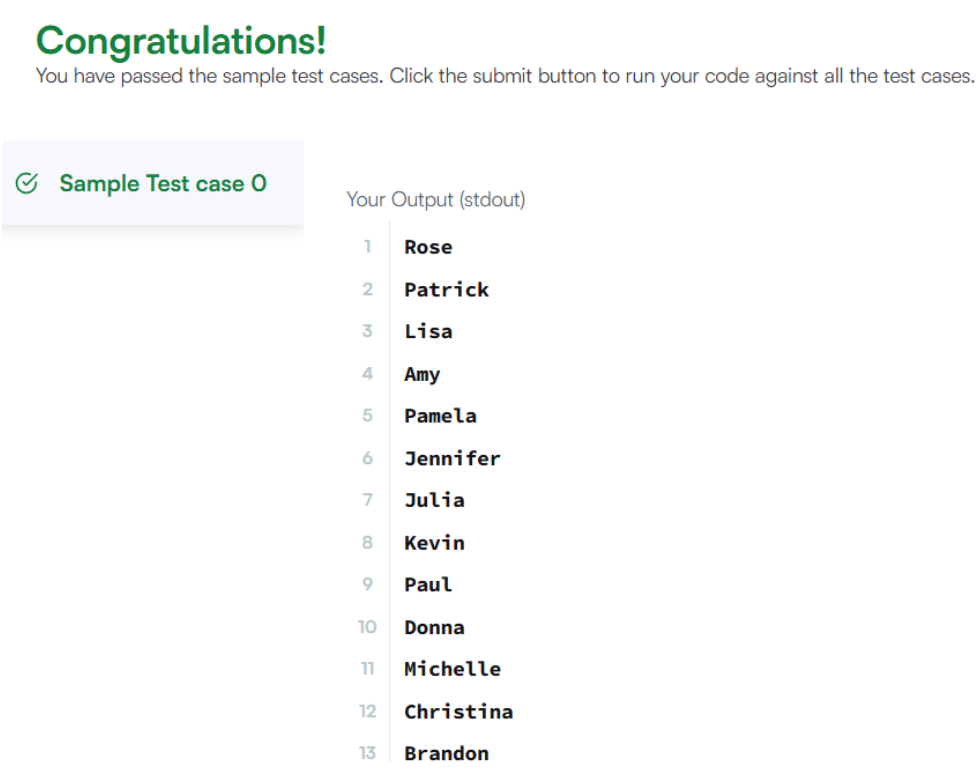
Todd has been an employee for months and earns per month.

Joe has been an employee for months and earns per month.

We order our output by ascending employee_id.

Program

```
SELECT name
FROM Employee
WHERE salary > 2000
  AND months < 10
ORDER BY employee_id ASC;
```



TYPE OF TRIANGLE

Write a query identifying the *type* of each record in the TRIANGLES table using its three side lengths. Output one of the following statements for each record in the table:

- Equilateral: It's a triangle with sides of equal length.
- Isosceles: It's a triangle with sides of equal length.
- Scalene: It's a triangle with sides of differing lengths.
- Not A Triangle: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The TRIANGLES table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

Each row in the table denotes the lengths of each of a triangle's three sides.

Sample Input

A	B	C
20	20	23
20	20	20
20	21	22
13	14	30

Sample Output

Isosceles

Equilateral

Scalene

Not A Triangle

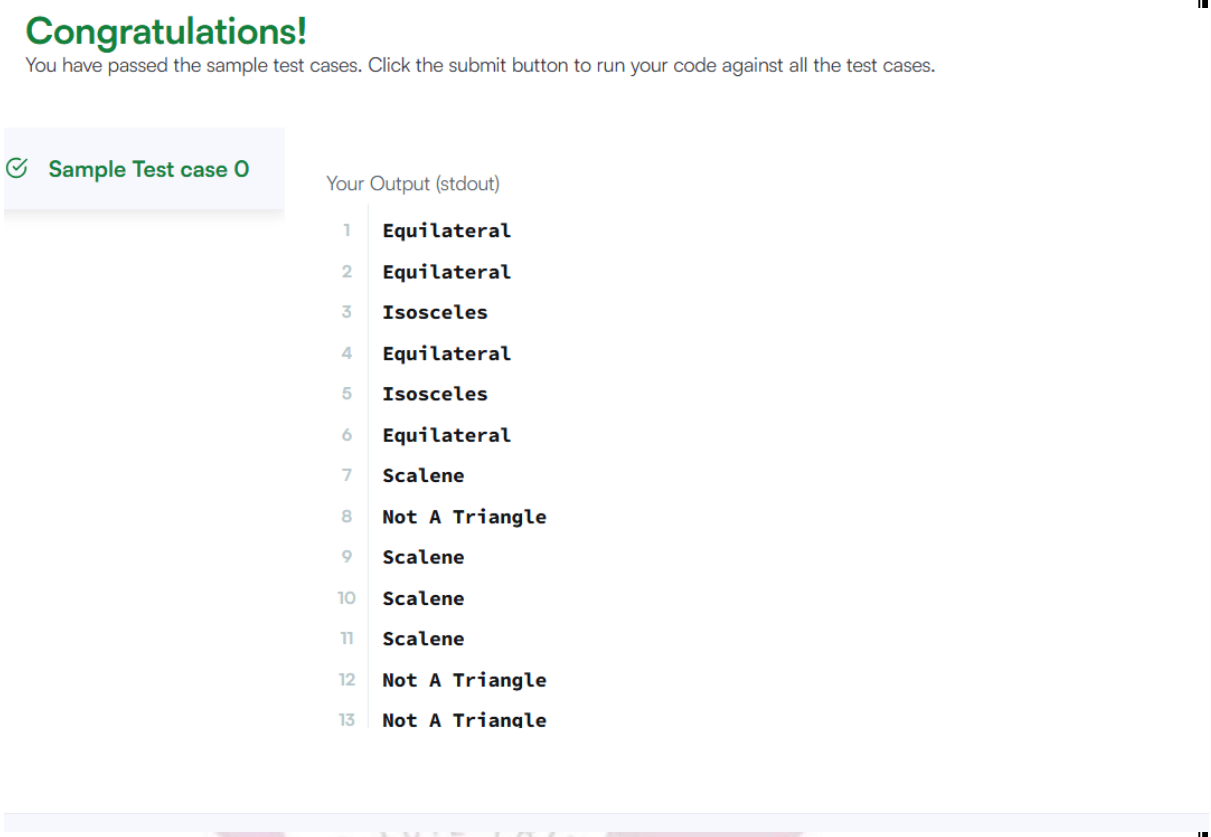
Explanation

Values in the tuple form an Isosceles triangle, because .
Values in the tuple form an Equilateral triangle, because . Values in the tuple form a Scalene triangle, because . Values in the tuple cannot form a triangle because the combined value of sides and is not larger than that of side .

Program

```
SELECT
   CASE
      WHEN A + B <= C OR A + C <= B OR B + C <= A
         THEN 'Not A Triangle'
      WHEN A = B AND B = C
         THEN 'Equilateral'
      WHEN A = B OR B = C OR A = C
         THEN 'Isosceles'
      ELSE 'Scalene'
   END
FROM TRIANGLES;
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS



THE PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).
2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:
3. There are a total of [occupation_count] [occupation]s.

where [occupation_count] is the number of occurrences of an occupation in **OCCUPATIONS** and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Sample Input

An **OCCUPATIONS** table that contains the following records:

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output

Ashely(P)

Christeen(P)

Jane(A)

Jenny(D)

Julia(A)

Ketty(P)

Maria(A)

Meera(S)

Priya(S)

Samantha(D)

There are a total of 2 doctors.

There are a total of 2 singers.

There are a total of 3 actors.

There are a total of 3 professors.

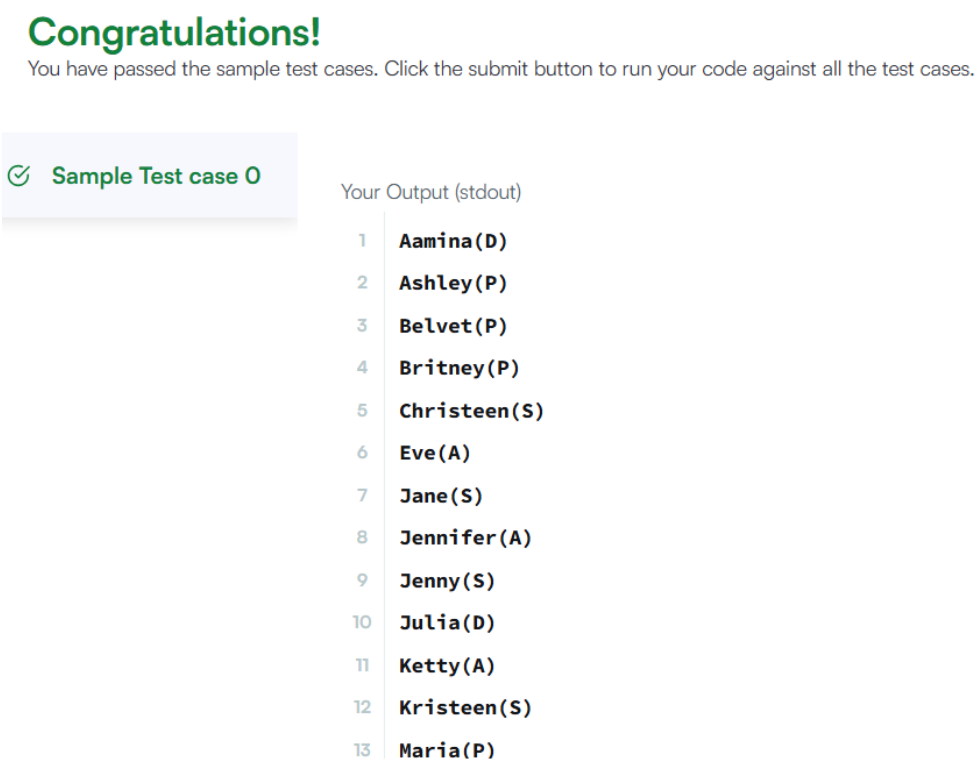
Explanation

The results of the first query are formatted to the problem description's specifications. The results of the second query are ascendingly ordered first by number of names corresponding to each profession (), and then alphabetically by profession (, and).

Program

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;

SELECT CONCAT(
   'There are a total of ',
   COUNT(*),
   ' ',
   LOWER(Occupation),
   's.'
)
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(*), Occupation;
```



OCCUPATIONS

Pivot the *Occupation* column in OCCUPATIONS so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print NULL when there are no more names corresponding to an occupation.

Input Format

The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: Doctor, Professor, Singer or Actor.

Sample Input

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output

Jenny Ashley Meera Jane

Samantha Christeen Priya Julia

NULL Ketty NULL Maria

Explanation

The first column is an alphabetically ordered list of Doctor names.
The second column is an alphabetically ordered list of Professor names.
The third column is an alphabetically ordered list of Singer names.
The fourth column is an alphabetically ordered list of Actor names.
The empty cell data for columns with less than the maximum number of names per occupation (in this case, the Professor and Actor columns) are filled with NULL values.

Language

MySQL

More

1

2

3

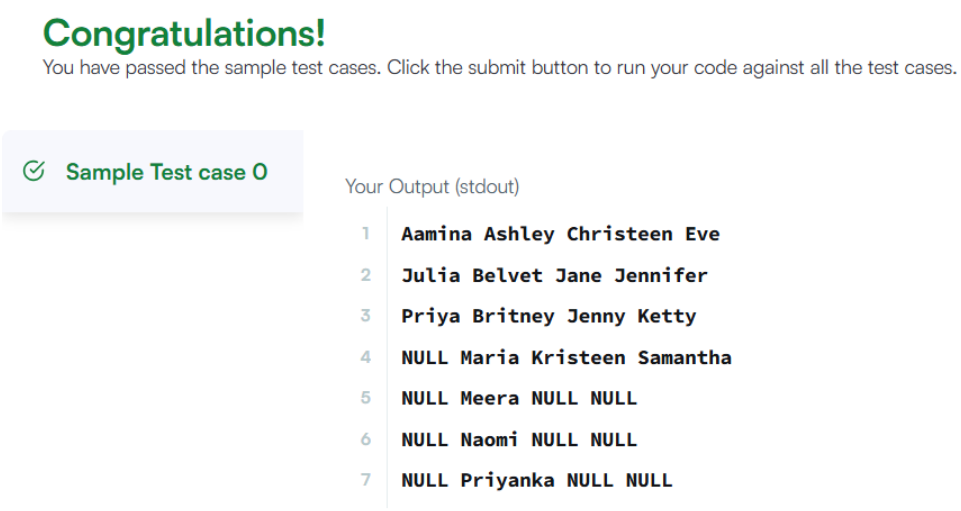
Line: 3 Col: 3

Test against custom input

- Blog
- Scoring
- Environment

Program

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER (
            PARTITION BY Occupation
            ORDER BY Name
        ) AS rn
    FROM OCCUPATIONS
) AS ranked
GROUP BY rn
ORDER BY rn
```



BINARY TREE NODES

You are given a table, *BST*, containing two columns: *N* and *P,* where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	Type
N	Integer
P	Integer

Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Sample Input

N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	null

Sample Output

1 Leaf

2 Inner

3 Leaf

5 Root

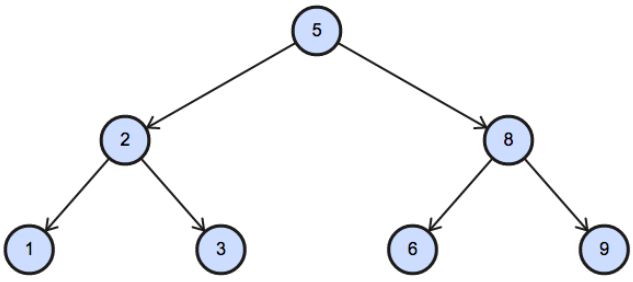
6 Leaf

8 Inner

9 Leaf

Explanation

The *Binary Tree* below illustrates the sample:



Program

```
SELECT
    N,
    CASE
        WHEN P IS NULL THEN 'Root'
        WHEN N NOT IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'
        ELSE 'Inner'
    END AS NodeType
FROM BST
ORDER BY N;
```


Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

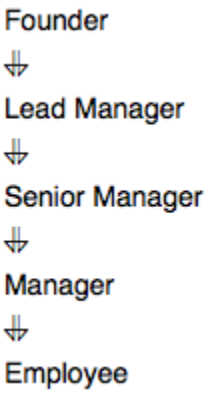
Sample Test case 0

Your Output (stdout)

```
1 Leaf
2 Inner
3 Leaf
4 Inner
5 Leaf
6 Inner
7 Leaf
8 Leaf
9 Inner
10 Leaf
11 Inner
12 Leaf
13 Inner
```

NEW COMPANIES

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

Note:

- The tables may contain duplicate records.
- The company_code is string, so the sorting should not be numeric. For example, if the company_codes are C_1, C_2, and C_10, then the ascending company_codes will be C_1, C_10, and C_2.

Input Format

The following tables contain company data:

- Company: The company_code is the code of the company and founder is the founder of the company.

Column	Type
company_code	String
founder	String

- Lead_Manager: The lead_manager_code is the code of the lead manager, and the company_code is the code of the working company.

Column	Type
lead_manager_code	String
company_code	String

- Senior_Manager: The senior_manager_code is the code of the senior manager, the lead_manager_code is the code of its lead manager, and the company_code is the code of the working company.

Column	Type
senior_manager_code	String
lead_manager_code	String
company_code	String

- Manager: The manager_code is the code of the manager, the senior_manager_code is the code of its senior manager, the lead_manager_code is the code of its lead manager, and the company_code is the code of the working company.

Column	Type
manager_code	String
senior_manager_code	String
lead_manager_code	String
company_code	String

- Employee: The employee_code is the code of the employee, the manager_code is the code of its manager, the senior_manager_code is the code of its senior manager, the lead_manager_code is the code of its lead manager, and the company_code is the code of the working company.

Column	Type
employee_code	String
manager_code	String
senior_manager_code	String
lead_manager_code	String
company_code	String

Sample Input

Company Table:

company_code	founder
C1	Monika
C2	Samantha

Lead_Manager Table:

lead_manager_code	company_code
LM1	C1
LM2	C2

Senior_Manager Table:

senior_manager_code	lead_manager_code	company_code
SM1	LM1	C1
SM2	LM1	C1
SM3	LM2	C2

Manager Table:

manager_code	senior_manager_code	lead_manager_code	company_code
M1	SM1	LM1	C1
M2	SM3	LM2	C2
M3	SM3	LM2	C2

Employee Table:

employee_code	manager_code	senior_manager_code	lead_manager_code	company_code
E1	M1	SM1	LM1	C1
E2	M1	SM1	LM1	C1
E3	M2	SM3	LM2	C2
E4	M3	SM3	LM2	C2

Sample Output

C1 Monika 1 2 1 2

C2 Samantha 1 1 2 2

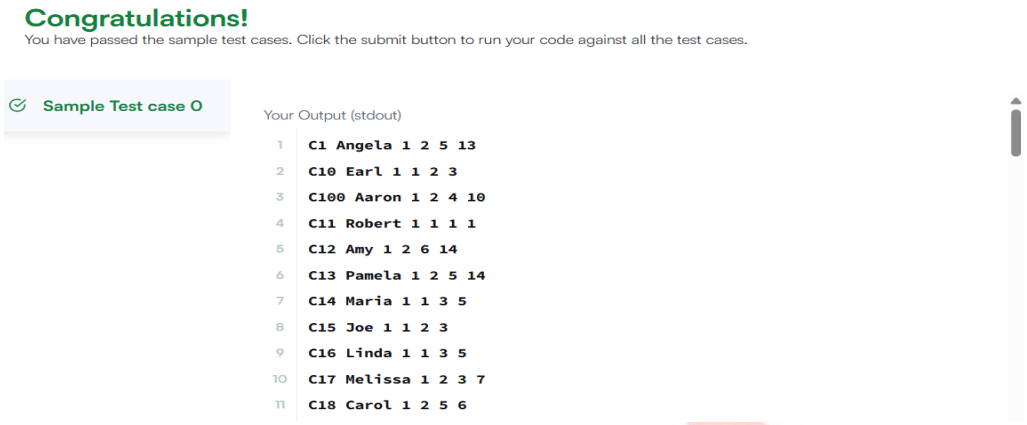
Explanation

In company *C1*, the only lead manager is *LM1*. There are two senior managers, *SM1* and *SM2*, under *LM1*. There is one manager, *M1*, under senior manager *SM1*. There are two employees, *E1* and *E2*, under manager *M1*.

In company *C2*, the only lead manager is *LM2*. There is one senior manager, *SM3*, under *LM2*. There are two managers, *M2* and *M3*, under senior manager *SM3*. There is one employee, *E3*, under manager *M2*, and another employee, *E4*, under manager, *M3*.

Program

```
SELECT
    c.company_code,
    c.founder,
    COUNT(DISTINCT lm.lead_manager_code) AS lead_managers,
    COUNT(DISTINCT sm.senior_manager_code) AS senior_managers,
    COUNT(DISTINCT m.manager_code) AS managers,
    COUNT(DISTINCT e.employee_code) AS employees
FROM Company c
LEFT JOIN Lead_Manager lm
    ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm
    ON c.company_code = sm.company_code
LEFT JOIN Manager m
    ON c.company_code = m.company_code
LEFT JOIN Employee e
    ON c.company_code = e.company_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code;
```



REVISING AGGREGATIONS - THE COUNT FUNCTION

Query a *count* of the number of cities in CITY having a *Population* larger than .

Input Format

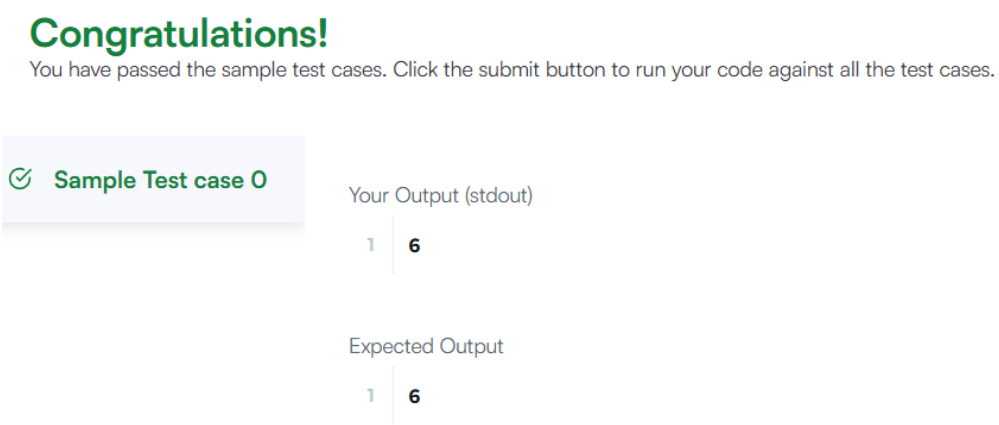
The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

```
SELECT COUNT(*)
FROM CITY
WHERE Population > 100000;
```



REVISING AGGREGATIONS - THE SUM FUNCTION

Query the total population of all cities in CITY where *District* is California.

Input Format

The CITY table is described as follows:

CITY

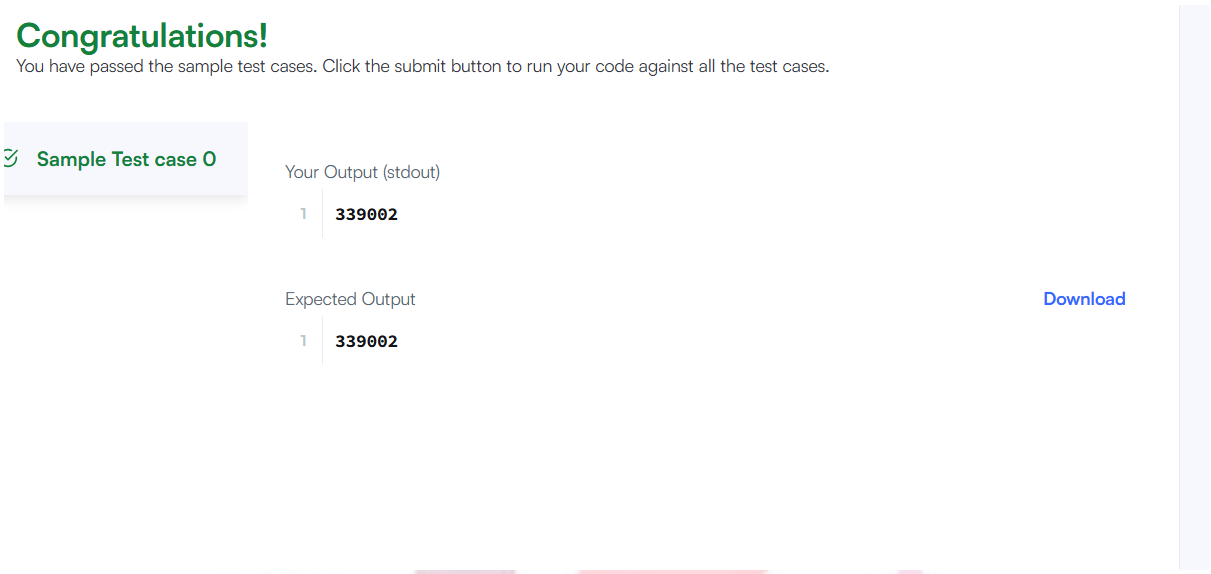
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

SELECT SUM(Population)

FROM CITY

WHERE District = 'California';



REVISING AGGREGATIONS – AVERAGES

Query the average population of all cities in CITY where *District* is California.

Input Format

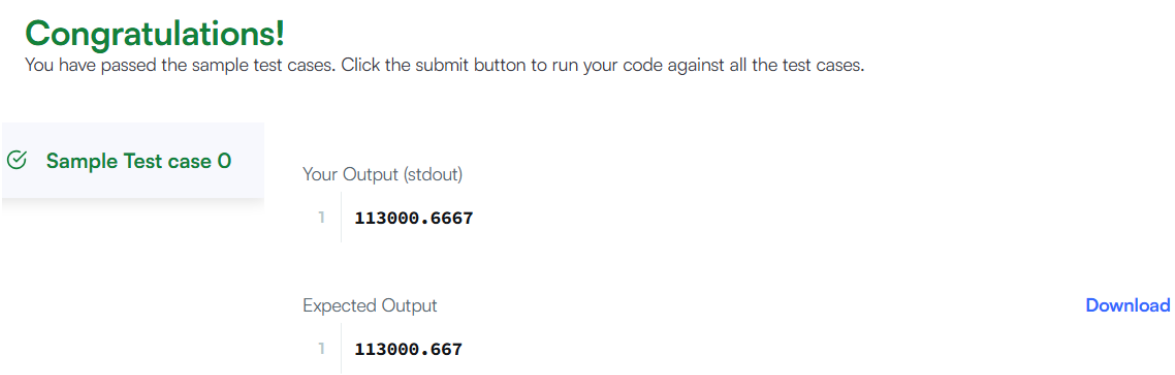
The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

```
SELECT AVG(Population)
FROM CITY
WHERE District = 'California';
```



AVERAGE POPULATION

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

Input Format

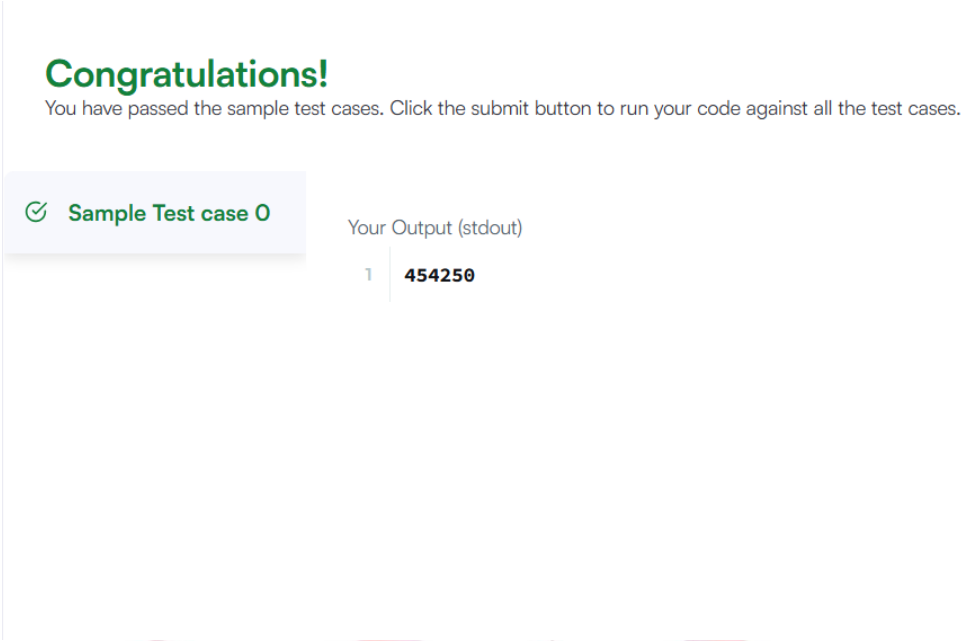
The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

```
SELECT FLOOR(AVG(POPULATION))
 FROM CITY;
```



JAPAN POPULATION

Query the sum of the populations for all Japanese cities in CITY. The *COUNTRYCODE* for Japan is JPN.

Input Format

The CITY table is described as follows

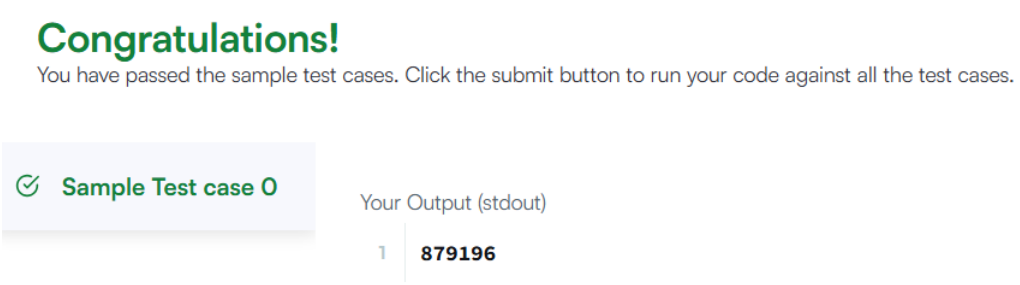
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

:

Program

```
SELECT SUM(POPULATION)
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```



POPULATION DENSITY DIFFERENCE

Query the difference between the maximum and minimum populations in CITY.

Input Format

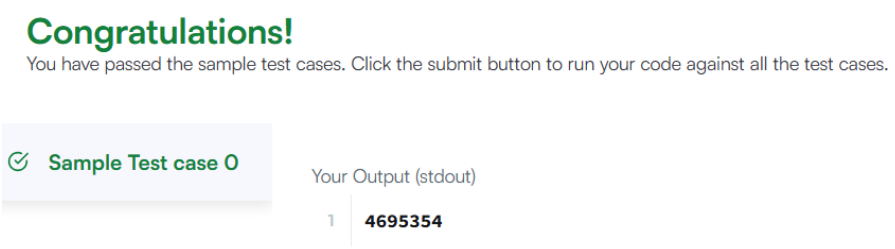
The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Program

```
SELECT MAX(POPULATION) - MIN(POPULATION)
FROM CITY;
```



THE BLUNDER

Samantha was tasked with calculating the average monthly salaries for all employees in the EMPLOYEES table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The EMPLOYEES table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

Note: Salary is per month.

Constraints

.

Sample Input

ID	Name	Salary
1	Kristeen	1420
2	Ashley	2006
3	Julia	2210
4	Maria	3000

Sample Output

2061

Explanation

The table below shows the salaries *without zeros* as they were entered by Samantha:

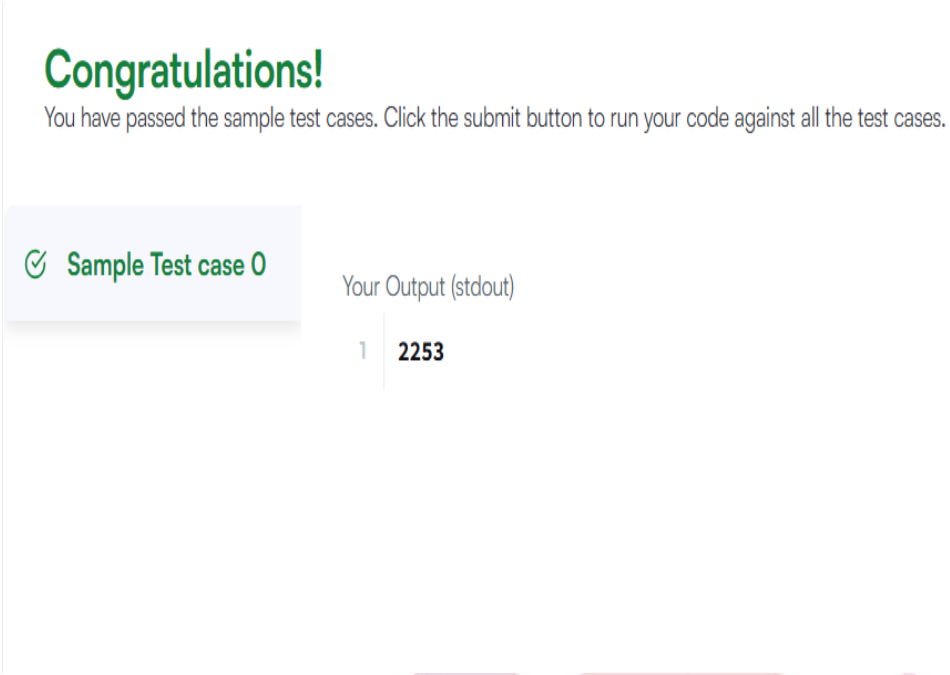
ID	Name	Salary
1	Kristeen	142
2	Ashley	26
3	Julia	221
4	Maria	3

Samantha computes an average salary of . The *actual* average salary is .

The resulting error between the two calculations is . Since it is equal to the integer , it does not get rounded up.

Program

```
SELECT CEIL(AVG(Salary) - AVG(REPLACE(Salary, '0', '')))
FROM EMPLOYEES;
```



TOP EARNERS

We define an employee's total earnings to be their monthly worked, and the maximum total earnings to be the maximum total earnings for any employee in the Employee table. Write a query to find the maximum total earnings for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where employee_id is an employee's ID number, name is their name, months is the total number of months they've been working for the company, and salary is the their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

69952 1

Explanation

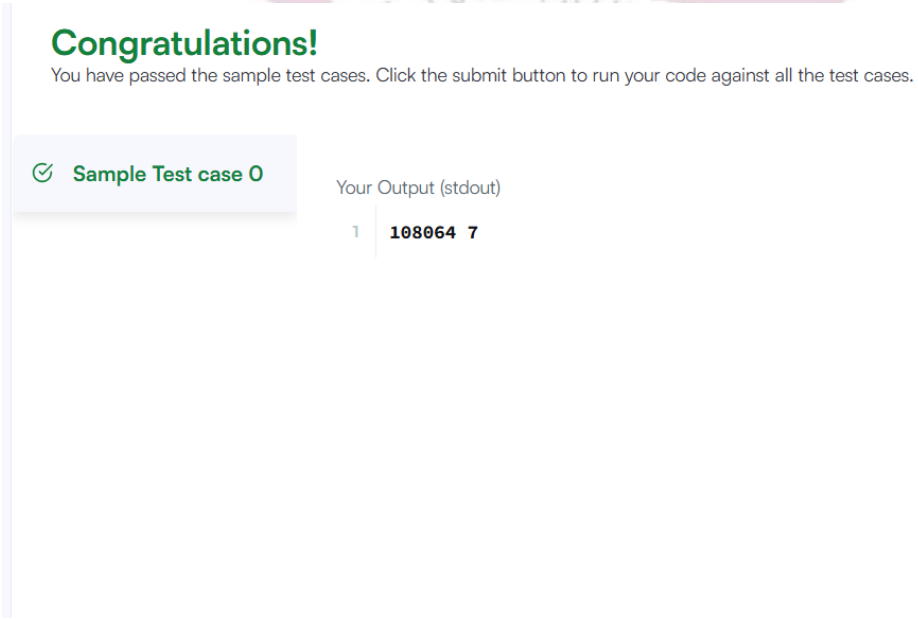
The table and earnings data is depicted in the following diagram:

employee_id	name	months	salary	earnings
12228	Rose	15	1968	29520
33645	Angela	1	3443	3443
45692	Frank	17	1608	27336
56118	Patrick	7	1345	9415
59725	Lisa	11	2330	25630
74197	Kimberly	16	4372	69952
78454	Bonnie	8	1771	14168
83565	Michael	6	2017	12102
98607	Todd	5	3396	16980
99989	Joe	9	3573	32157

The maximum *earnings* value is . The only employee with *earnings* is *Kimberly*, so we print the maximum *earnings* value () and a count of the number of employees who have earned (which is) as two space-separated values.

Program

```
SELECT MAX(months * salary),
      COUNT(*)
FROM Employee
WHERE months * salary = (
   SELECT MAX(months * salary)
   FROM Employee
);
```



WEATHER OBSERVATION STATION 2

Query the following two values from the STATION table:

1. The sum of all values in *LAT_N* rounded to a scale of decimal places.
2. The sum of all values in *LONG_W* rounded to a scale of decimal places.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Output Format

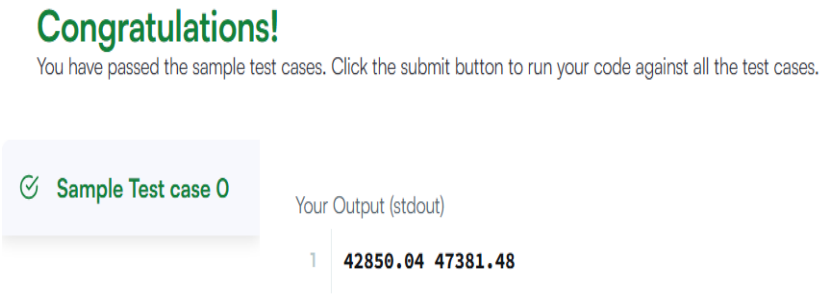
Your results must be in the form:

lat lon

where is the sum of all values in *LAT_N* and is the sum of all values in *LONG_W*. Both results must be rounded to a scale of decimal places.

Program

```
SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2)
FROM STATION;
```



WEATHER OBSERVATION STATION 13

Query the sum of *Northern Latitudes* (*LAT_N*) from STATION having values greater than and less than . Truncate your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

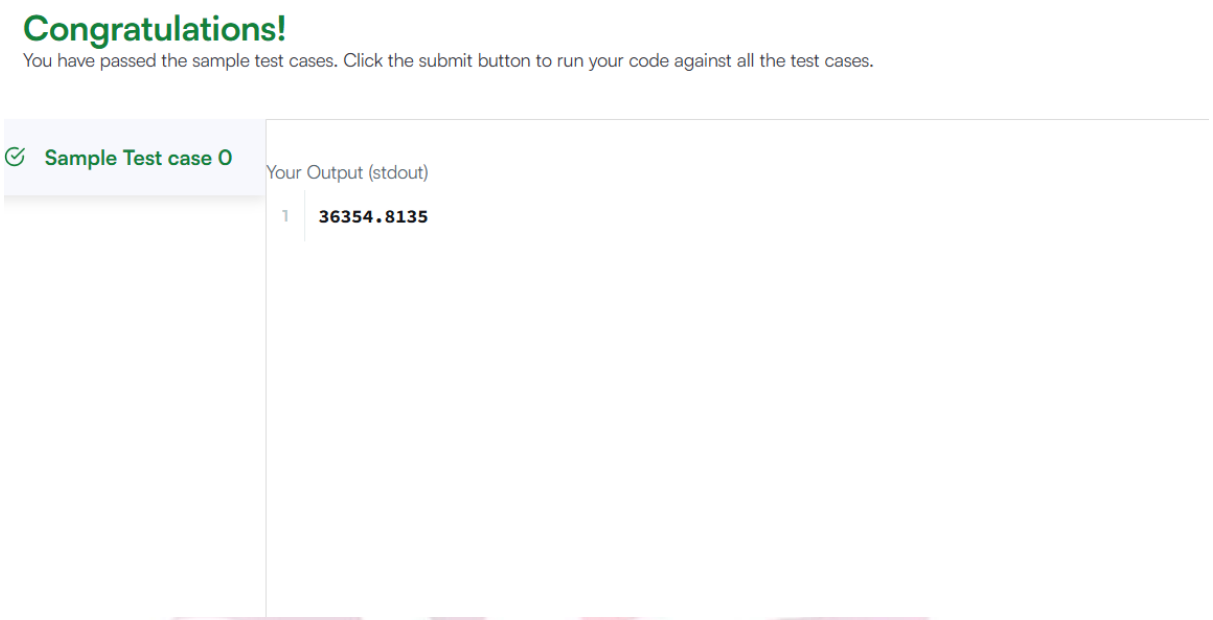
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT TRUNCATE(SUM(LAT_N), 4)
FROM STATION
```

```
WHERE LAT_N > 38.7880
 AND LAT_N < 137.2345;
```



WEATHER OBSERVATION STATION 14

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from STATION that is less than . Truncate your answer to decimal places.

Input Format

The STATION table is described as follows:

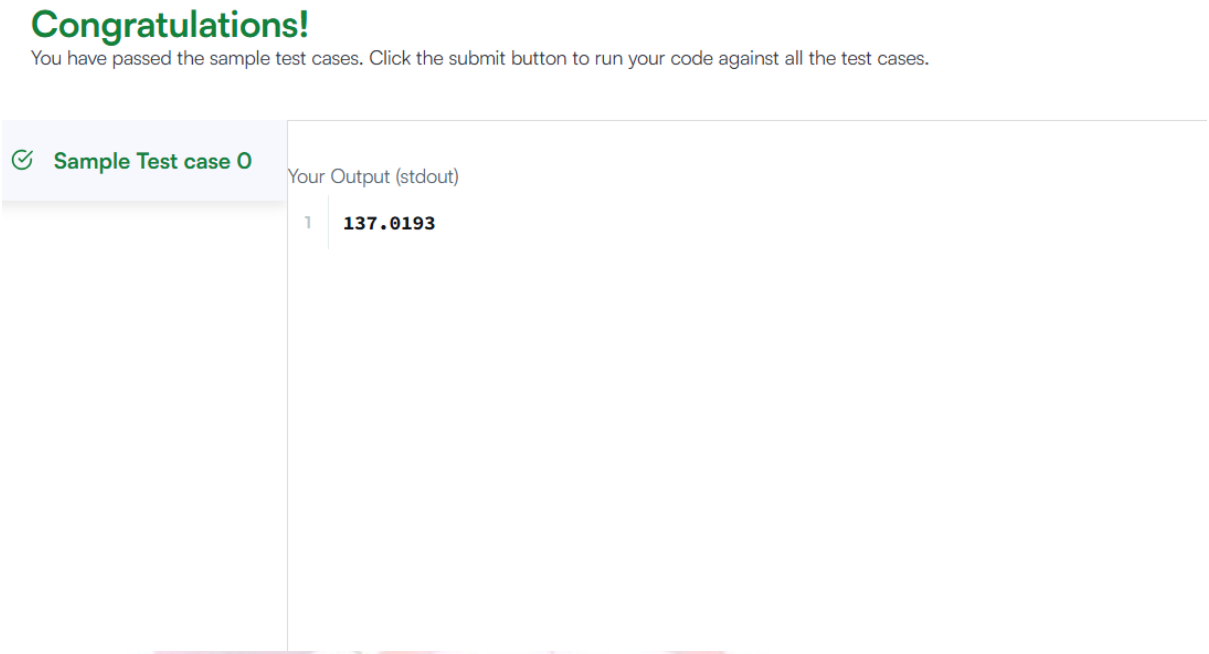
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT TRUNCATE(MAX(LAT_N), 4)
FROM STATION
WHERE LAT_N < 137.2345;
```



WEATHER OBSERVATION STATION 15

Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in STATION that is less than . Round your answer to decimal places.

Input Format

The STATION table is described as follows:

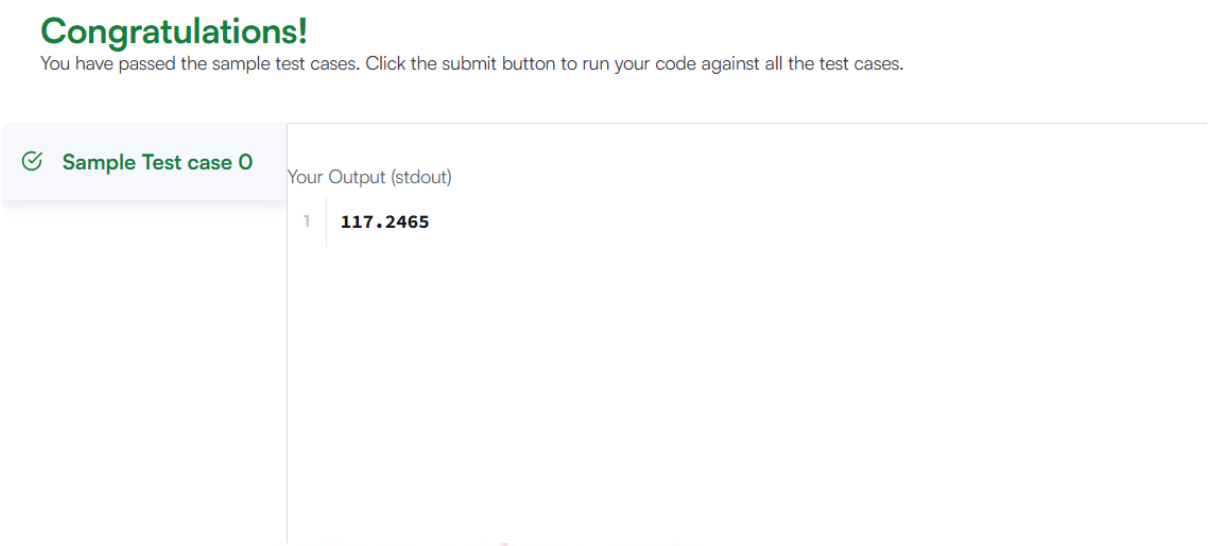
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MAX(LAT_N)
   FROM STATION
   WHERE LAT_N < 137.2345
);
```



Weather Observation Station 16

Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

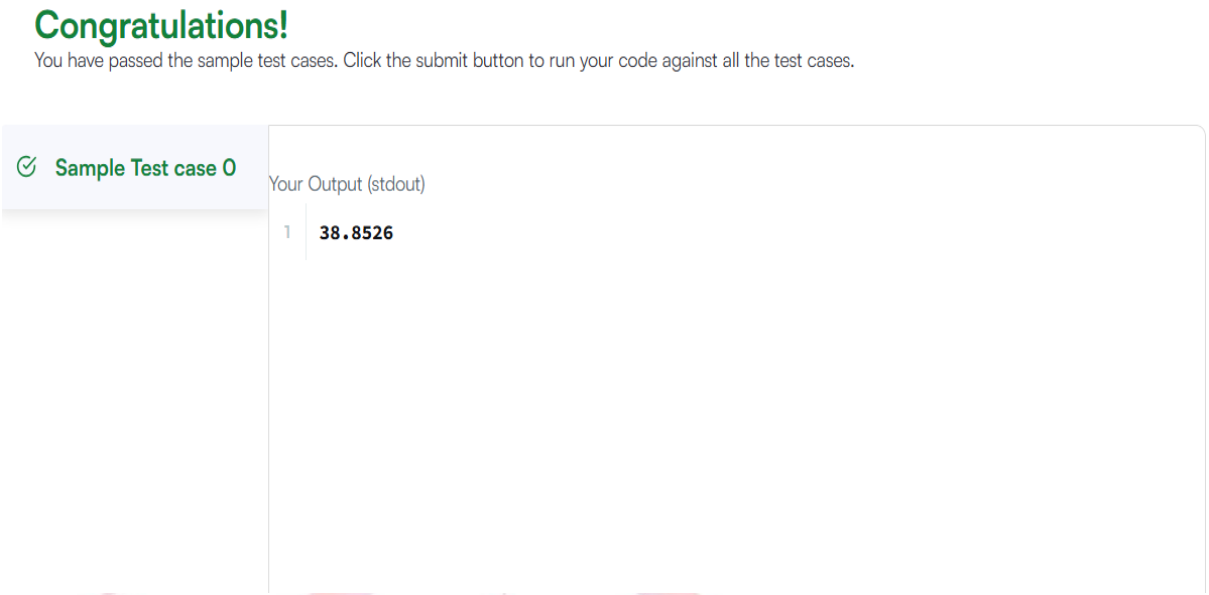
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT ROUND(MIN(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7780;
```



Weather Observation Station 17

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in STATION is greater than . Round your answer to decimal places.

Input Format

The STATION table is described as follows:

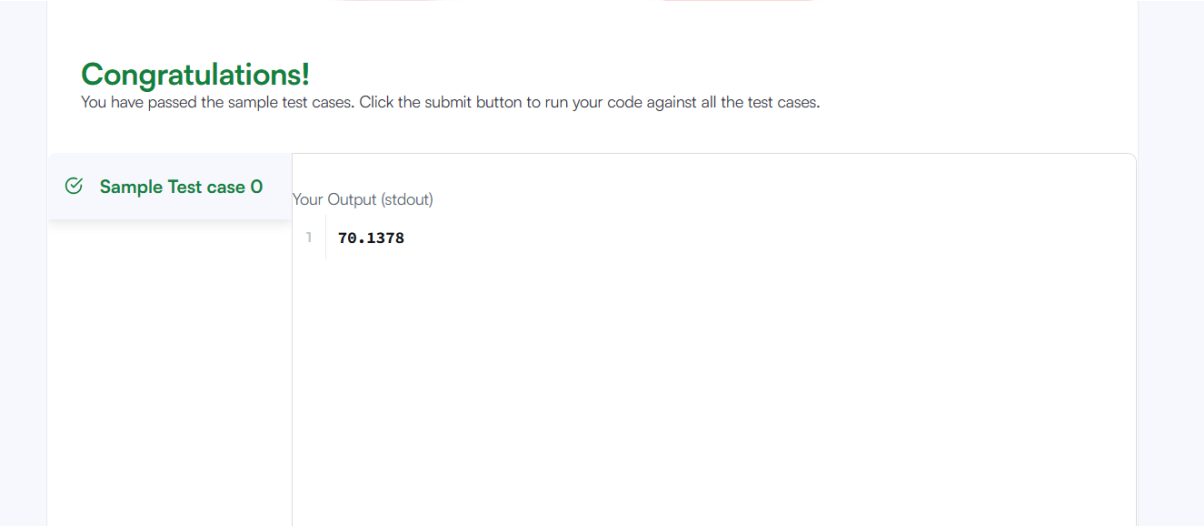
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MIN(LAT_N)
   FROM STATION
   WHERE LAT_N > 38.7780
);
```



Weather Observation Station 18

Consider and to be two points on a 2D plane.

- happens to equal the minimum value in Northern Latitude (LAT_N in STATION).
- happens to equal the minimum value in Western Longitude (LONG_W in STATION).
- happens to equal the maximum value in Northern Latitude (LAT_N in STATION).
- happens to equal the maximum value in Western Longitude (LONG_W in STATION).

Query the Manhattan Distance between points and and round it to a scale of decimal places.

Input Format

The STATION table is described as follows:

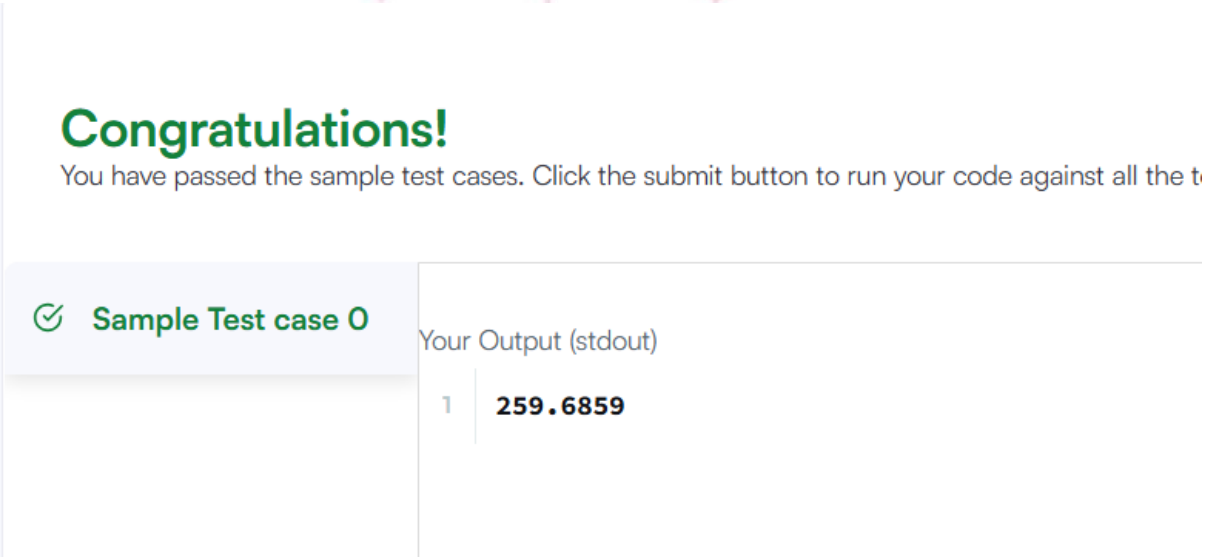
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Program

```
SELECT ROUND(
    (MAX(LAT_N) - MIN(LAT_N)) +
    (MAX(LONG_W) - MIN(LONG_W)),
    4
)
FROM STATION;
```



Weather Observation Station 19

Consider and to be two points on a 2D plane where are the respective minimum and maximum values of Northern Latitude (LAT_N) and are the respective minimum and maximum values of Western Longitude (LONG_W) in STATION.

Query the Euclidean Distance between points and and format your answer to display decimal digits.

Input Format

The STATION table is described as follows:

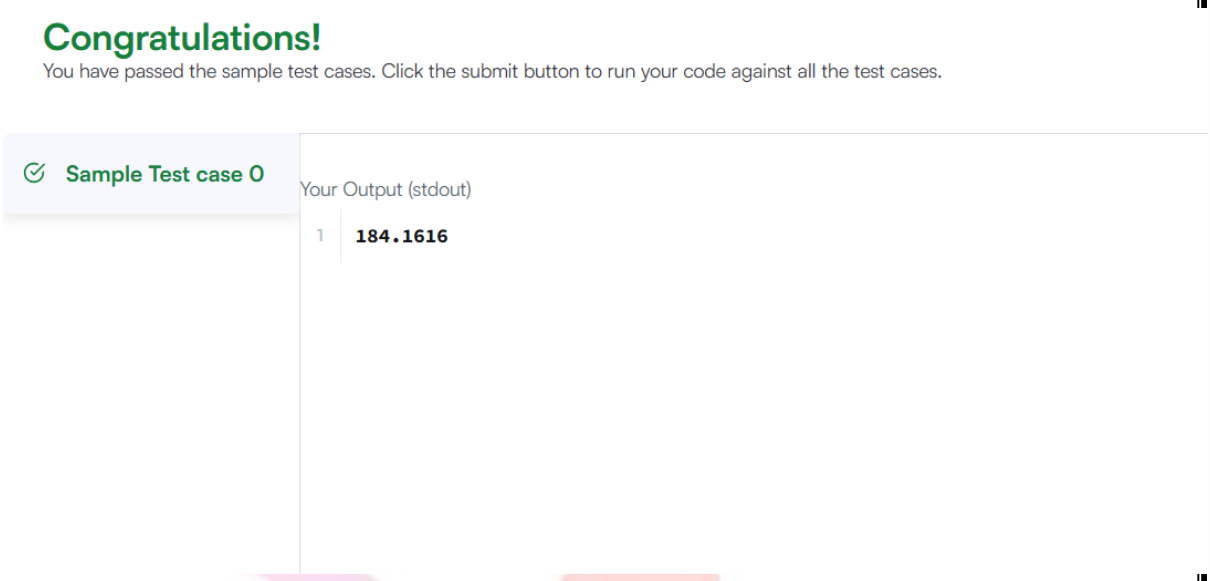
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT ROUND(
   SQRT(
      POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
      POWER(MAX(LONG_W) - MIN(LONG_W), 2)
   ),
   4
)
FROM STATION;
```



WEATHER OBSERVATION STATION 20

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from STATION and round your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Program

```
SELECT ROUND(AVG(LAT_N), 4)
    FROM (
      SELECT
        LAT_N,
        ROW_NUMBER() OVER (ORDER BY LAT_N) AS rn,
        COUNT(*) OVER () AS cnt
      FROM STATION
    ) AS t
    WHERE rn IN (
      FLOOR((cnt + 1) / 2),
      CEIL((cnt + 1) / 2)
    );
```



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  83.8913
```

POPULATION CENSUS

Given the CITY and COUNTRY tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The CITY and COUNTRY tables are described as follows:

CITY

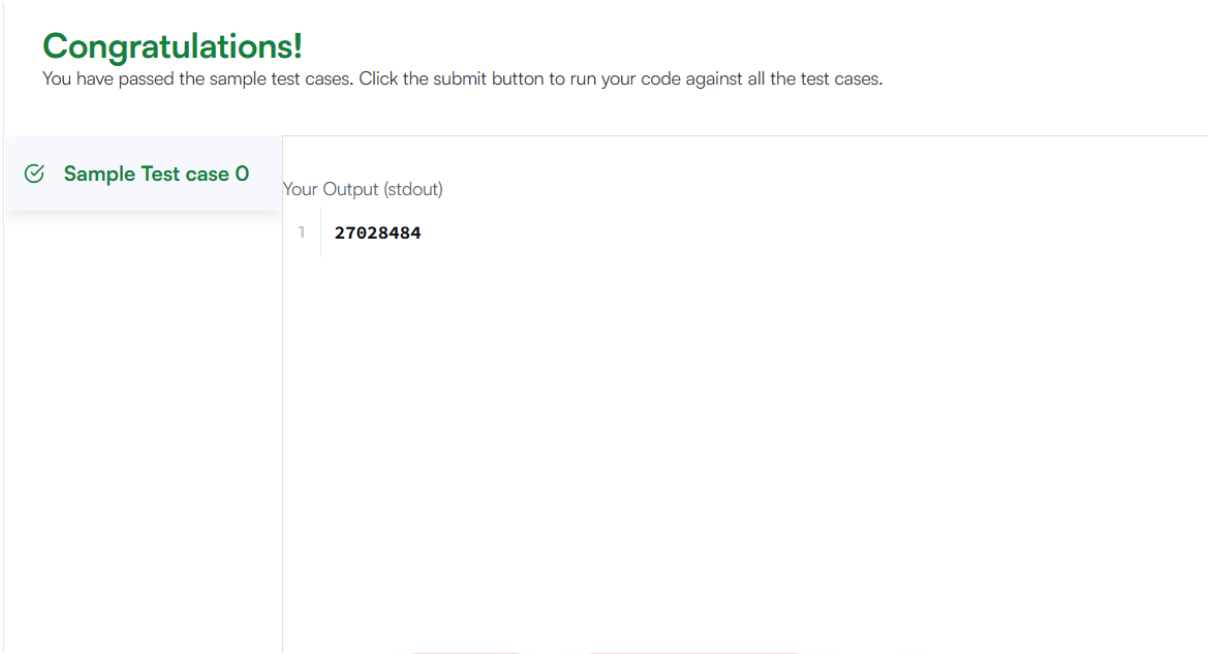
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Program

```
SELECT SUM(CITY.POPULATION)
FROM CITY
JOIN COUNTRY
  ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.CONTINENT = 'Asia';
```



The CITY and COUNTRY tables are described as follows:

CITY

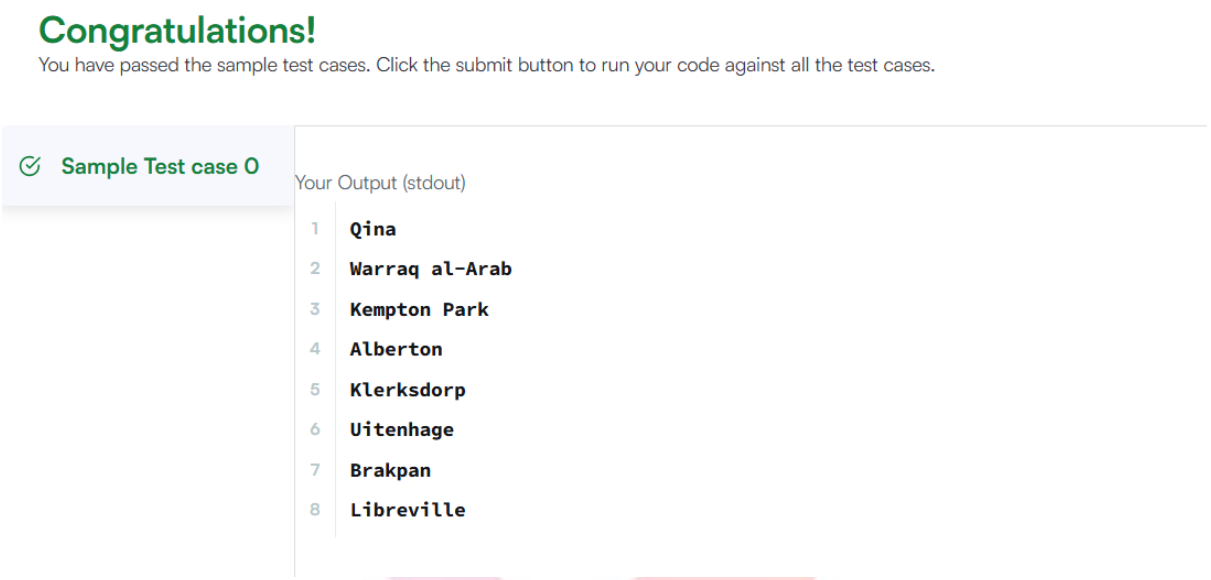
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Program

```
SELECT CITY.NAME
FROM CITY
JOIN COUNTRY
  ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.CONTINENT = 'Africa';
```



AVERAGE POPULATION OF EACH CONTINENT

Given the CITY and COUNTRY tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The CITY and COUNTRY tables are described as follows:

CITY

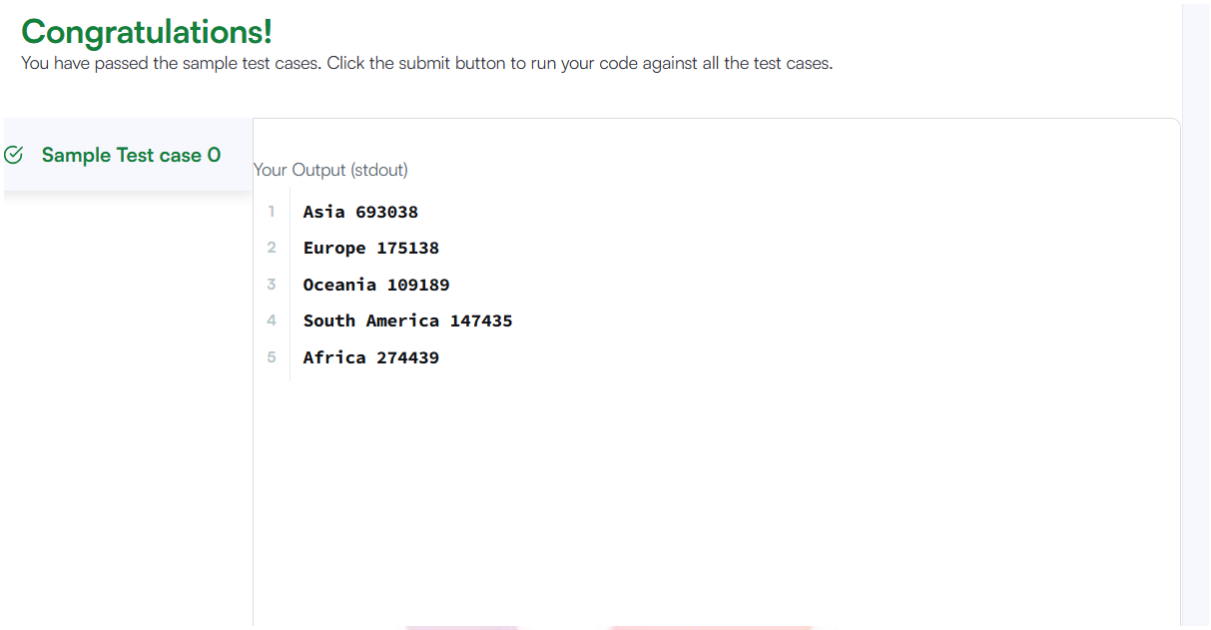
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Program

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY
  ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```



THE REPORT

You are given two tables: Students and Grades. Students contains three columns ID, Name and Marks.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

Sample Input

ID	Name	Marks
1	Julia	88
2	Samantha	68
3	Maria	99
4	Scarlet	78
5	Ashley	63
6	Jane	81

Sample Output

Maria 10 99

Jane 9 81

Julia 9 88

Scarlet 8 78

NULL 7 63

NULL 7 68

Note

Print "NULL" as the name if the grade is less than 8.

Explanation

Consider the following table with the grades assigned to the students:

ID	Name	Marks	Grade
1	Julia	88	9
2	Samantha	68	7
3	Maria	99	10
4	Scarlet	78	8
5	Ashley	63	7
6	Jane	81	9

So, the following students got *8*, *9* or *10* grades:

- *Maria (grade 10)*
- *Jane (grade 9)*
- *Julia (grade 9)*
- *Scarlet (grade 8)*

Program

```
SELECT
   CASE
      WHEN G.Grade < 8 THEN NULL
      ELSE S.Name
   END AS Name,
   G.Grade,
   S.Marks
FROM Students S
JOIN Grades G
   ON S.Marks BETWEEN G.Min_Mark AND G.Max_Mark
ORDER BY
```

```
G.Grade DESC,
  CASE
    WHEN G.Grade >= 8 THEN S.Name
    ELSE NULL
  END ASC,
  CASE
    WHEN G.Grade < 8 THEN S.Marks
    ELSE NULL
  END ASC;
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Britney 10 95
Heraldo 10 94
Julia 10 96
Kristeen 10 100
Stuart 10 99
Amina 9 89
Christene 9 88
Salma 9 81
Samantha 9 87
Scarlet 9 80
Vivek 9 84
Aamina 8 77
```

TOP COMPETITORS

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective hacker_id and name of hackers who achieved full scores for more than one challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending hacker_id.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Difficulty:* The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum score that can be achieved for a challenge at that difficulty level.

Column	Type
difficulty_level	Integer
score	Integer

- *Challenges:* The *challenge_id* is the id of the challenge, the *hacker_id* is the id of the hacker who created the challenge, and *difficulty_level* is the level of difficulty of the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer
difficulty_level	Integer

- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge that the submission belongs to, and *score* is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Sample Input

Hackers Table:

hacker_id	name
5580	Rose
8439	Angela
27205	Frank
52243	Patrick
52348	Lisa
57645	Kimberly
77726	Bonnie
83082	Michael
86870	Todd
90411	Joe

Difficulty Table:

difficulty_level	score
1	20
2	30
3	40
4	60
5	80
6	100
7	120

Challenges Table:

challenge_id	hacker_id	difficulty_level
4810	77726	4
21089	27205	1
36566	5580	7
66730	52243	6
71055	52243	2

Submissions Table:

submission_id	hacker_id	challenge_id	score
68628	77726	36566	30
65300	77726	21089	10
40326	52243	36566	77
8941	27205	4810	4
83554	77726	66730	30
43353	52243	66730	0
55385	52348	71055	20
39784	27205	71055	23
94613	86870	71055	30
45788	52348	36566	0
93058	86870	36566	30
7344	8439	66730	92
2721	8439	4810	36
523	5580	71055	4
49105	52348	66730	0
55877	57645	66730	80
38355	27205	66730	35
3924	8439	36566	80
97397	90411	66730	100
84162	83082	4810	40
97431	90411	71055	30

Sample Output

90411 Joe

Explanation

Hacker 86870 got a score of 30 for challenge 71055 with a difficulty level of 2, so 86870 earned a full score for this challenge.

Hacker 90411 got a score of 30 for challenge 71055 with a difficulty level of 2, so 90411 earned a full score for this challenge.

Hacker 90411 got a score of 100 for challenge 66730 with a difficulty level of 6, so 90411 earned a full score for this challenge.

Only hacker 90411 managed to earn a full score for more than one challenge, so we print the their hacker_id and name as space-separated values.

Program

```
SELECT h.hacker_id, h.name
FROM Hackers h
JOIN Submissions s
   ON h.hacker_id = s.hacker_id
JOIN Challenges c
   ON s.challenge_id = c.challenge_id
JOIN Difficulty d
   ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1
ORDER BY COUNT(DISTINCT s.challenge_id) DESC,
      h.hacker_id ASC;
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
27232 Phillip
28614 Willie
15719 Christina
43892 Roy
14246 David
14372 Michelle
18330 Lawrence
26133 Jacqueline
26253 John
30128 Brandon
35583 Norma
13944 Victor
```

OLLIVANDER'S INVENTORY

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each non-evil wand of high power and age. Write a query to print the id, age, coins_needed, and power of the wands that Ron's interested in, sorted in order of descending power. If more than one wand has same power, sort the result in order of descending age.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- Wands: The id is the id of the wand, code is the code of the wand, coins_needed is the total number of gold galleons needed to buy the wand, and power denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

- Wands_Property: The code is the code of the wand, age is the age of the wand, and is_evil denotes whether the wand is good for the dark arts. If the value of is_evil is 0, it means that the wand is not evil. The mapping between code and age is one-one,

Column	Type
code	Integer
age	Integer
is_evil	Integer

meaning that if there are two pairs, and , then and .

Sample Input

Wands Table:

id	code	coins_needed	power
1	4	3688	8
2	3	9365	3
3	3	7187	10
4	3	734	8
5	1	6020	2
6	2	6773	7
7	3	9873	9
8	3	7721	7
9	1	1647	10
10	4	504	5
11	2	7587	5
12	5	9897	10
13	3	4651	8
14	2	5408	1
15	2	6018	7
16	4	7710	5
17	2	8798	7
18	2	3312	3
19	4	7651	6
20	5	5689	3

Wands_Property Table:

code	age	is_evil
1	45	0
2	40	0
3	4	1
4	20	0
5	17	0

Sample Output

9 45 1647 10

12 17 9897 10

1 20 3688 8

15 40 6018 7

19 20 7651 6

11 40 7587 5

10 20 504 5

18 40 3312 3

20 17 5689 3

5 45 6020 2

14 40 5408 1

Explanation

id	age	coins_needed	power
5	45	6020	2
9	45	1647	10

The data for wands of age 45 (code 1):

- The minimum number of galleons needed for
- The minimum number of galleons needed for

Program

```
SELECT w.id, wp.age, w.coins_needed, w.power
FROM Wands w
JOIN Wands_Property wp
    ON w.code = wp.code
WHERE wp.is_evil = 0
 AND w.coins_needed = (
     SELECT MIN(w2.coins_needed)
     FROM Wands w2
     JOIN Wands_Property wp2
         ON w2.code = wp2.code
     WHERE wp2.is_evil = 0
      AND wp2.age = wp.age
      AND w2.power = w.power
 )
ORDER BY w.power DESC, wp.age DESC;
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1038 496 4789 10
1130 494 9439 10
1315 492 4126 10
9 491 7345 10
858 483 4352 10
1164 481 9831 10
1288 464 4952 10
861 462 8302 10
412 455 5625 10
996 451 8884 10
1608 446 8351 10
1376 443 1735 10
```

CHALLENGES

Julia asked her students to create some coding challenges. Write a query to print the hacker_id, name, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by hacker_id. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- Challenges: The challenge_id is the id of the challenge, and hacker_id is the id of the student who created the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer

Sample Input 0

Hackers Table:

hacker_id	name
5077	Rose
21283	Angela
62743	Frank
88255	Patrick
96196	Lisa

Challenges Table:

challenge_id	hacker_id
61654	5077
58302	21283
40587	88255
29477	5077
1220	21283
69514	21283
46561	62743
58077	62743
18483	88255
76766	21283
52382	5077
74467	21283
33625	96196
26053	88255
42665	62743
12859	62743
70094	21283
34599	88255
54680	88255
61881	5077

Sample Output 0

21283 Angela 6

88255 Patrick 5

96196 Lisa 1

Sample Input 1

Hackers Table:

hacker_id	name
12299	Rose
34856	Angela
79345	Frank
80491	Patrick
81041	Lisa

Challenges Table:

challenge_id	hacker_id
63963	81041
63117	79345
28225	34856
21989	12299
4653	12299
70070	79345
36905	34856
61136	80491
17234	12299
80308	79345
40510	34856
79820	80491
22720	12299
21394	12299
36261	34856
15334	12299
71435	79345
23157	34856
54102	34856
69065	80491

Sample Output 1

12299 Rose 6

34856 Angela 6

79345 Frank 4

80491 Patrick 3

81041 Lisa 1

Explanation

For Sample Case 0, we can get the following details:

hacker_id	name	challenges_created
21283	Angela	6
88255	Patrick	5
5077	Rose	4
62743	Frank	4
96196	Lisa	1

Students and both created challenges, but the maximum number of challenges created is so these students are excluded from the result.

For Sample Case 1, we can get the following details:

hacker_id	name	challenges_created
12299	Rose	6
34856	Angela	6
79345	Frank	4
80491	Patrick	3
81041	Lisa	1

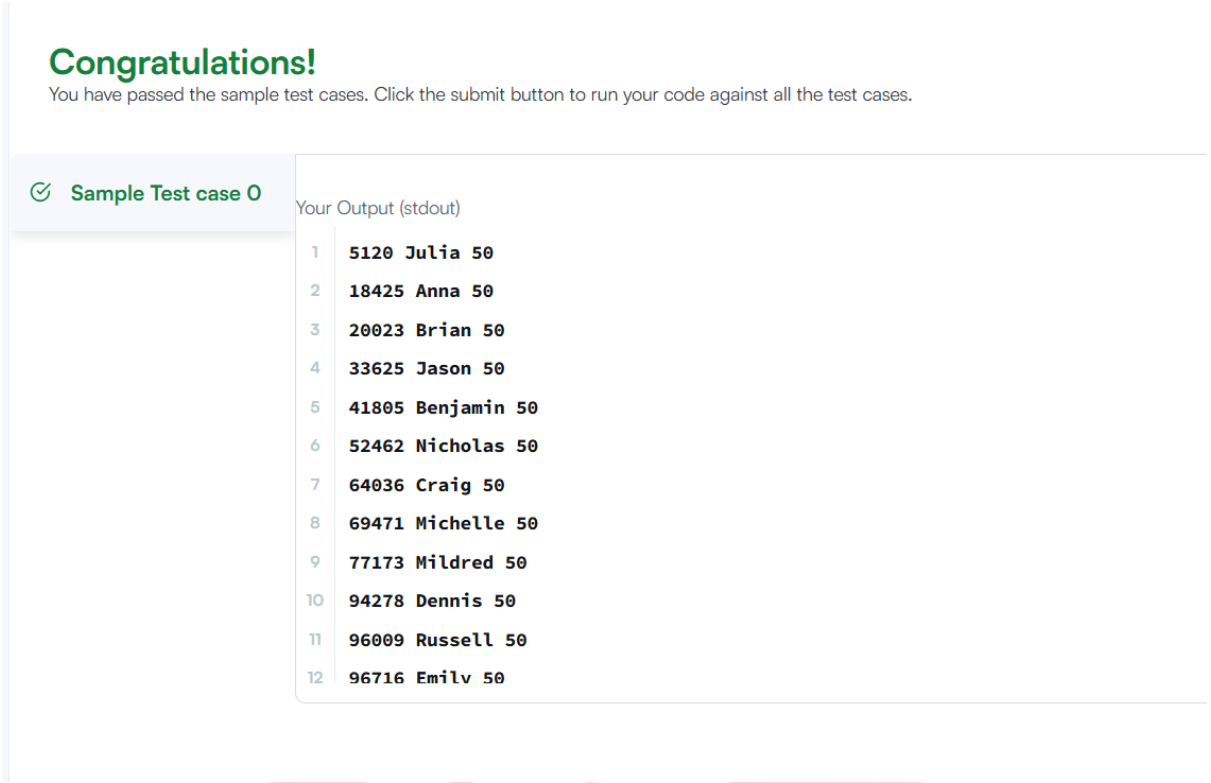
Students and both created challenges. Because is the maximum number of challenges created, these students are included in the result.

Program

```
SELECT h.hacker_id, h.name, COUNT(c.challenge_id) AS total_challenges
FROM Hackers h
JOIN Challenges c
    ON h.hacker_id = c.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_challenges = (
        SELECT MAX(challenge_count)
```

```
FROM (
            SELECT COUNT(*) AS challenge_count
            FROM Challenges
            GROUP BY hacker_id
        ) AS counts
    )
   OR total_challenges IN (
        SELECT challenge_count
        FROM (
            SELECT COUNT(*) AS challenge_count
            FROM Challenges
            GROUP BY hacker_id
        ) AS counts2
        GROUP BY challenge_count
        HAVING COUNT(*) = 1
    )
ORDER BY total_challenges DESC, h.hacker_id ASC;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS



CONTEST LEADERBOARD

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the hacker_id, name, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending hacker_id. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- Submissions: The submission_id is the id of the submission, hacker_id is the id of the hacker who made the submission, challenge_id is the id of the challenge for which the

submission belongs to, and score is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Sample Input

Hackers Table:

hacker_id	name
4071	Rose
4806	Angela
26071	Frank
49438	Patrick
74842	Lisa
80305	Kimberly
84072	Bonnie
87868	Michael
92118	Todd
95895	Joe

submission_id	hacker_id	challenge_id	score
67194	74842	63132	76
64479	74842	19797	98
40742	26071	49593	20
17513	4806	49593	32
69846	80305	19797	19
41002	26071	89343	36
52826	49438	49593	9
31093	26071	19797	2
81614	84072	49593	100
44829	26071	89343	17
75147	80305	49593	48
14115	4806	49593	76
6943	4071	19797	95
12855	4806	25917	13
73343	80305	49593	42
84264	84072	63132	0
9951	4071	49593	43
45104	49438	25917	34
53795	74842	19797	5
26363	26071	19797	29
10063	4071	49593	96

Submissions Table:

Sample Output

4071 Rose 191

74842 Lisa 174

84072 Bonnie 100

4806 Angela 89

26071 Frank 85

80305 Kimberly 67

49438 Patrick 43

Explanation

Hacker *4071* submitted solutions for challenges *19797* and *49593*, so the total score .

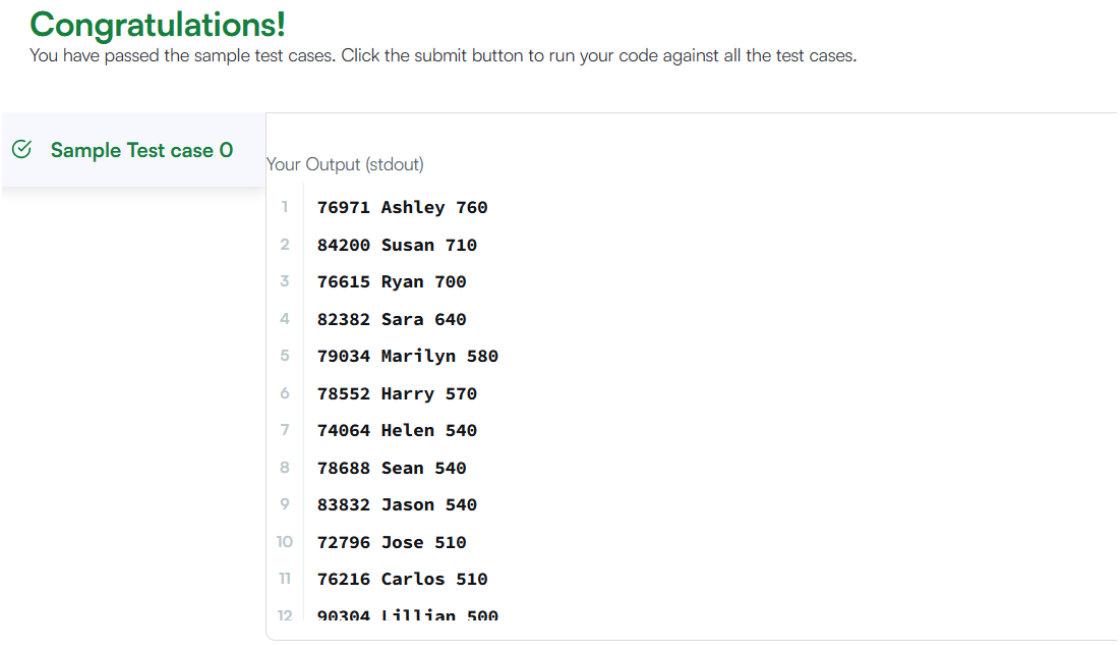
Hacker *74842* submitted solutions for challenges *19797* and *63132*, so the total score

Hacker *84072* submitted solutions for challenges *49593* and *63132*, so the total score .

The total scores for hackers *4806*, *26071*, *80305*, and *49438* can be similarly calculated.

Program

```
SELECT h.hacker_id, h.name, SUM(m.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id, challenge_id, MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) m
    ON h.hacker_id = m.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_score > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```



SQL PROJECT PLANNING

You are given a table, Projects, containing three columns: Task_ID, Start_Date and End_Date. It is guaranteed that the difference between the End_Date and the Start_Date is equal to 1 day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

If the End_Date of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

Sample Output

2015-10-28 2015-10-29

2015-10-30 2015-10-31

2015-10-13 2015-10-15

2015-10-01 2015-10-04

Explanation

The example describes following four projects:

- Project 1: Tasks 1, 2 and 3 are completed on consecutive days, so these are part of the project. Thus start date of project is 2015-10-01 and end date is 2015-10-04, so it took 3 days to complete the project.
- Project 2: Tasks 4 and 5 are completed on consecutive days, so these are part of the project. Thus, the start date of project is 2015-10-13 and end date is 2015-10-15, so it took 2 days to complete the project.
- Project 3: Only task 6 is part of the project. Thus, the start date of project is 2015-10-28 and end date is 2015-10-29, so it took 1 day to complete the project.
- Project 4: Only task 7 is part of the project. Thus, the start date of project is 2015-10-30 and end date is 2015-10-31, so it took 1 day to complete the project.

Program

```
WITH project_groups AS (
    SELECT
        Start_Date,
        End_Date,
        DATE_SUB(
            Start_Date,
            INTERVAL ROW_NUMBER() OVER (ORDER BY Start_Date) DAY
        ) AS grp
    FROM Projects
)
SELECT
    MIN(Start_Date) AS Start_Date,
    MAX(End_Date) AS End_Date
FROM project_groups
GROUP BY grp
ORDER BY
    DATEDIFF(MAX(End_Date), MIN(Start_Date)) ASC,
    MIN(Start_Date) ASC;
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
2015-10-15 2015-10-16
2015-10-17 2015-10-18
2015-10-19 2015-10-20
2015-10-21 2015-10-22
2015-11-01 2015-11-02
2015-11-17 2015-11-18
2015-10-11 2015-10-13
2015-11-11 2015-11-13
2015-10-01 2015-10-05
2015-11-04 2015-11-08
2015-10-25 2015-10-31
```

SYMMETRIC PAIRS

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	Type
X	Integer
Y	Integer

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

Sample Input

X	Y
20	20
20	20
20	21
23	22
22	23
21	20

Sample Output

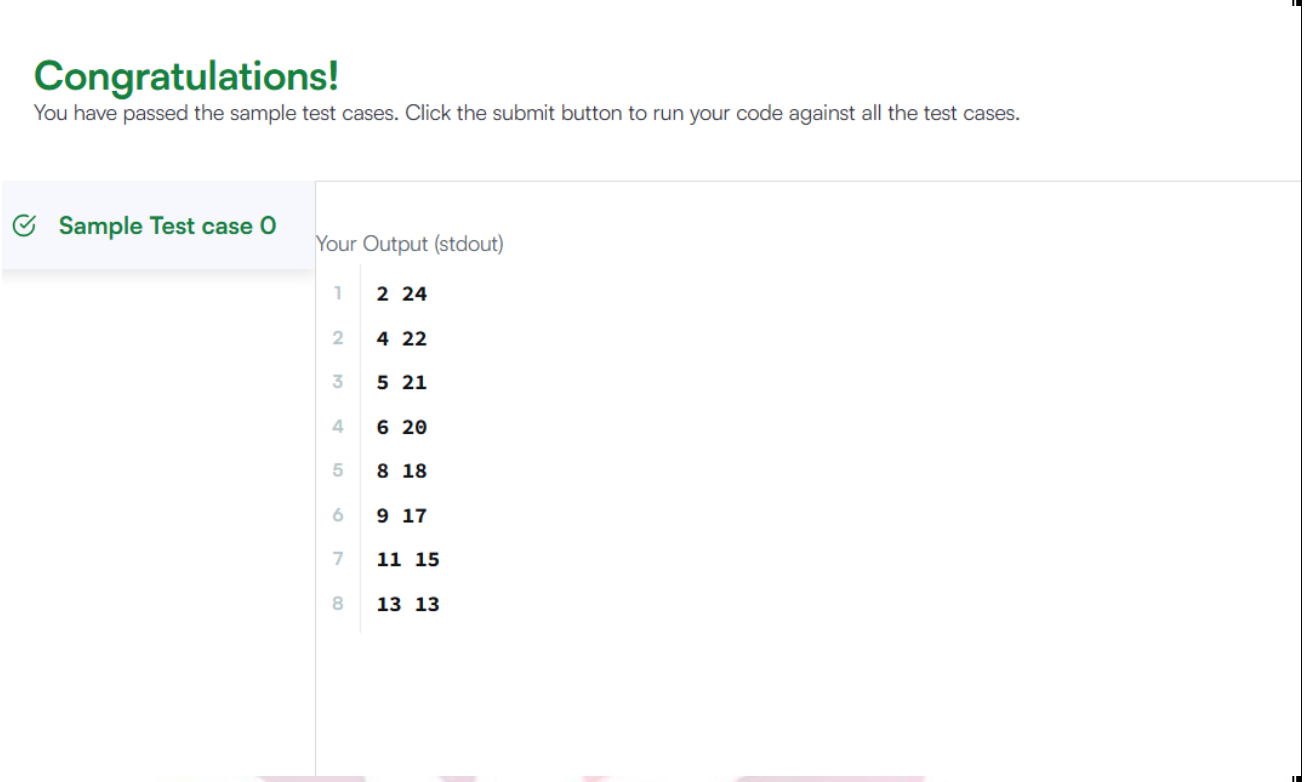
20 20

20 21

22 23

Program

```
SELECT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2
   ON f1.X = f2.Y
  AND f1.Y = f2.X
WHERE f1.X <= f1.Y
GROUP BY f1.X, f1.Y
HAVING f1.X < f1.Y
   OR COUNT(*) > 1
ORDER BY f1.X, f1.Y;
```



INTERVIEWS

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the contest_id, hacker_id, name, and the sums of total_submissions, total_accepted_submissions, total_views, and total_unique_views for each contest sorted by contest_id. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- Contests: The contest_id is the id of the contest, hacker_id is the id of the hacker who created the contest, and name is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- Colleges: The college_id is the id of the college, and contest_id is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- Challenges: The challenge_id is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and college_id is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- View_Stats: The challenge_id is the id of the challenge, total_views is the number of times the challenge was viewed by candidates, and total_unique_views is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- Submission_Stats: The challenge_id is the id of the challenge, total_submissions is the number of submissions for the challenge, and total_accepted_submission is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Sample Input

Contests Table:

contest_id	hacker_id	name
66406	17973	Rose
66556	79153	Angela
94828	80275	Frank

Colleges Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

Challenges Table:

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

View_Stats Table:

challenge_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

Submission_Stats Table:

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

Sample Output

66406 17973 Rose 111 39 156 56

66556 79153 Angela 0 0 11 10

94828 80275 Frank 150 38 41 15

Explanation

The contest is used in the college . In this college , challenges and are asked, so from the *view* and *submission* stats:

- Sum of total submissions
- Sum of total accepted submissions
- Sum of total views
- Sum of total unique views

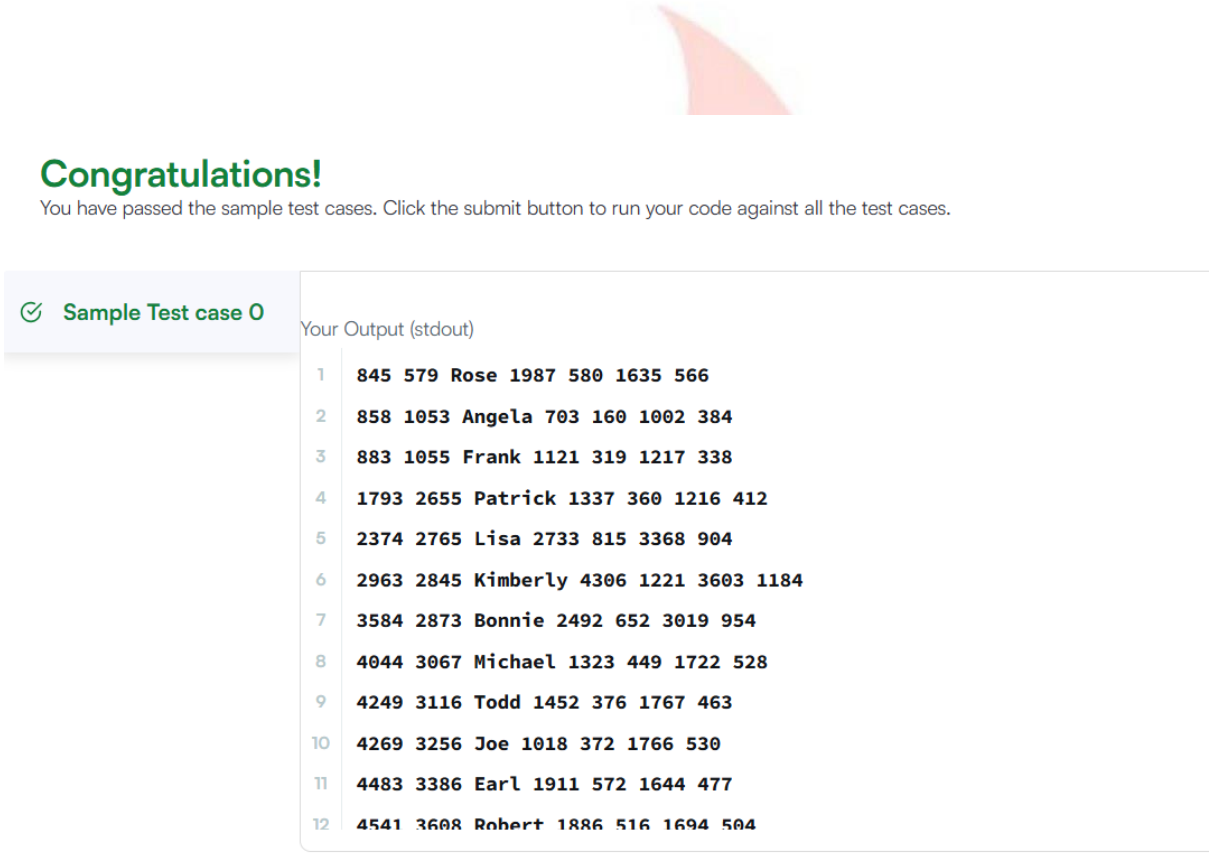
Similarly, we can find the sums for contests and .

Program

```
SELECT
    c.contest_id,
    c.hacker_id,
```

```
  c.name,
    COALESCE(SUM(ss.total_submissions), 0) AS total_submissions,
    COALESCE(SUM(ss.total_accepted_submissions), 0) AS total_accepted_submissions,
    COALESCE(SUM(vs.total_views), 0) AS total_views,
    COALESCE(SUM(vs.total_unique_views), 0) AS total_unique_views
FROM Contests c
JOIN Colleges co
    ON c.contest_id = co.contest_id
JOIN Challenges ch
    ON co.college_id = ch.college_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) ss
    ON ch.challenge_id = ss.challenge_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_views) AS total_views,
        SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) vs
    ON ch.challenge_id = vs.challenge_id
GROUP BY c.contest_id, c.hacker_id, c.name
```

```
HAVING
    COALESCE(SUM(ss.total_submissions), 0)
  + COALESCE(SUM(ss.total_accepted_submissions), 0)
  + COALESCE(SUM(vs.total_views), 0)
  + COALESCE(SUM(vs.total_unique_views), 0) > 0
ORDER BY c.contest_id;
```



15 DAYS OF LEARNING SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

Hackers Table:

hacker_id	name
15758	Rose
20703	Angela
36396	Frank
38289	Patrick
44065	Lisa
53473	Kimberly
62529	Bonnie
79722	Michael

Submissions Table:

submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Sample Output

2016-03-01 4 20703 Angela

2016-03-02 2 79722 Michael

2016-03-03 2 20703 Angela

2016-03-04 2 20703 Angela

2016-03-05 1 36396 Frank

2016-03-06 1 20703 Angela

Explanation

On *March 01, 2016* hackers , , , and made submissions. There are unique hackers who made at least one submission each day. As each hacker made one submission, is considered to be the hacker who made maximum number of submissions on this day. The name of the hacker is *Angela*.

On *March 02, 2016* hackers , , and made submissions. Now and were the only ones to submit every day, so there are unique hackers who made at least one submission each day. made submissions, and name of the hacker is *Michael*.

On *March 03, 2016* hackers , , and made submissions. Now and were the only ones, so there are unique hackers who made at least one submission each day. As each hacker made one submission so is considered to be the hacker who made maximum number of submissions on this day. The name of the hacker is *Angela*.

On *March 04, 2016* hackers , , , and made submissions. Now and only submitted each day, so there are unique hackers who made at least one submission each day. As each hacker made one submission so is considered to be the hacker who made maximum number of submissions on this day. The name of the hacker is *Angela*.

On *March 05, 2016* hackers , , and made submissions. Now only submitted each day, so there is only unique hacker who made at least one submission each day. made submissions and name of the hacker is *Frank*.

On *March 06, 2016* only made submission, so there is only unique hacker who made at least one submission each day. made submission and name of the hacker is *Angela*.

Program

```
WITH RECURSIVE contest_days AS (
    SELECT DATE('2016-03-01') AS submission_date
    UNION ALL
    SELECT DATE_ADD(submission_date, INTERVAL 1 DAY)
    FROM contest_days
    WHERE submission_date < '2016-03-15'
),
```

```
daily_counts AS (
    SELECT
        submission_date,
        hacker_id,
        COUNT(*) AS submission_count
    FROM Submissions
    WHERE submission_date BETWEEN '2016-03-01' AND '2016-03-15'
    GROUP BY submission_date, hacker_id
),
top_hackers AS (
    SELECT d1.submission_date, d1.hacker_id
    FROM daily_counts d1
    LEFT JOIN daily_counts d2
        ON d1.submission_date = d2.submission_date
       AND (
            d2.submission_count > d1.submission_count
            OR (
                d2.submission_count = d1.submission_count
                AND d2.hacker_id < d1.hacker_id
            )
       )
    WHERE d2.hacker_id IS NULL
)
SELECT
    cd.submission_date,
    (
        SELECT COUNT(*)
        FROM (
            SELECT hacker_id
```

```
    FROM Submissions
        WHERE submission_date BETWEEN '2016-03-01' AND cd.submission_date
        GROUP BY hacker_id
        HAVING COUNT(DISTINCT submission_date) =
            DATEDIFF(cd.submission_date, '2016-03-01') + 1
    ) AS consistent_hackers
) AS unique_hackers,
th.hacker_id,
h.name
FROM contest_days cd
JOIN top_hackers th
    ON cd.submission_date = th.submission_date
JOIN Hackers h
    ON th.hacker_id = h.hacker_id
ORDER BY cd.submission_date;
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
2016-03-01 112 81314 Denise
2016-03-02 59 39091 Ruby
2016-03-03 51 18105 Roy
2016-03-04 49 533 Patrick
2016-03-05 49 7891 Stephanie
2016-03-06 49 84307 Evelyn
2016-03-07 35 80682 Deborah
2016-03-08 35 10985 Timothy
2016-03-09 35 31221 Susan
2016-03-10 35 43192 Bobby
2016-03-11 35 3178 Melissa
2016-03-12 35 54967 Kenneth
```

DRAW THE TRIANGLE 1

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

* * * * *

* * * *

* * *

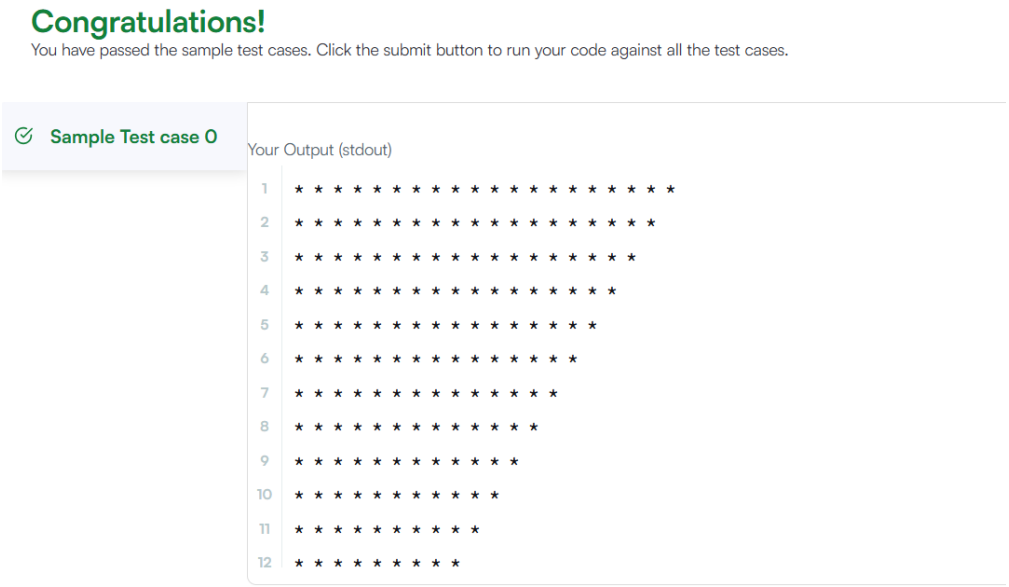
* *

*

Write a query to print the pattern P(20).

Program

```
WITH RECURSIVE numbers AS (
   SELECT 20 AS n
   UNION ALL
   SELECT n - 1
   FROM numbers
   WHERE n > 1
)
SELECT REPEAT('* ', n)
FROM numbers;
```



DRAW THE TRIANGLE 2

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

*

* *

* * *

* * * *

* * * * *

Write a query to print the pattern P(20).

Program

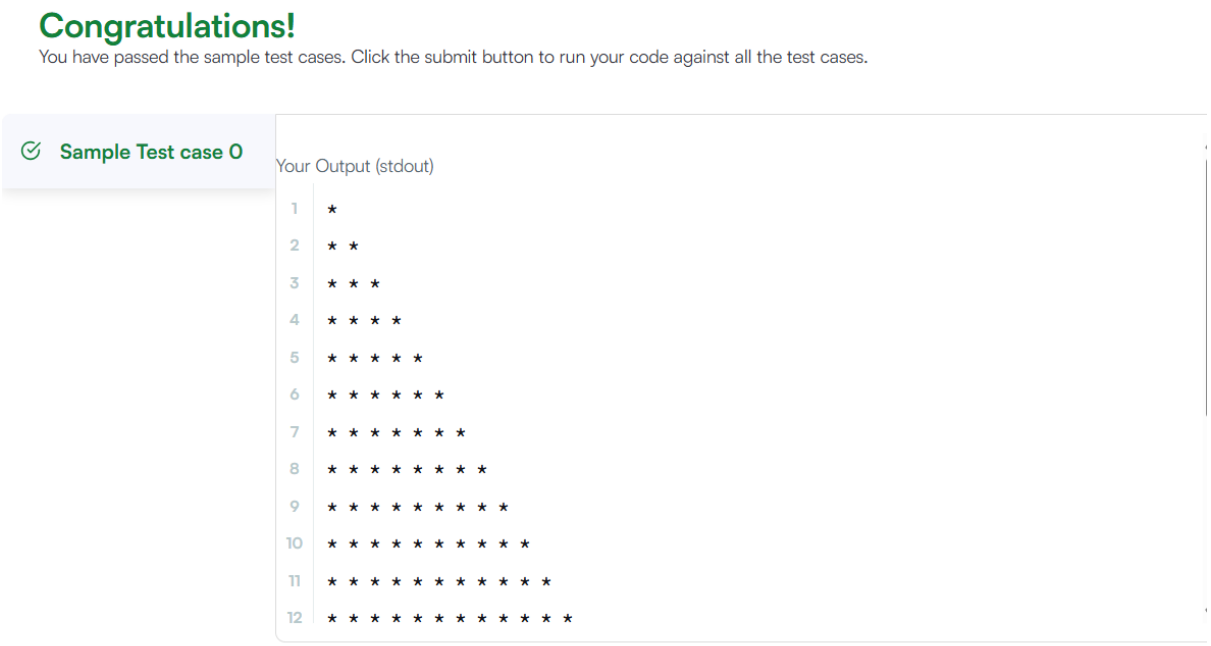
```
SET NOCOUNT ON;


SET NOCOUNT ON;


WITH Numbers AS
(
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1
```

```
 FROM Numbers
    WHERE n < 20
)
SELECT REPLICATE('* ', n)
FROM Numbers
OPTION (MAXRECURSION 20);

go
```



PRINT PRIME NUMBERS

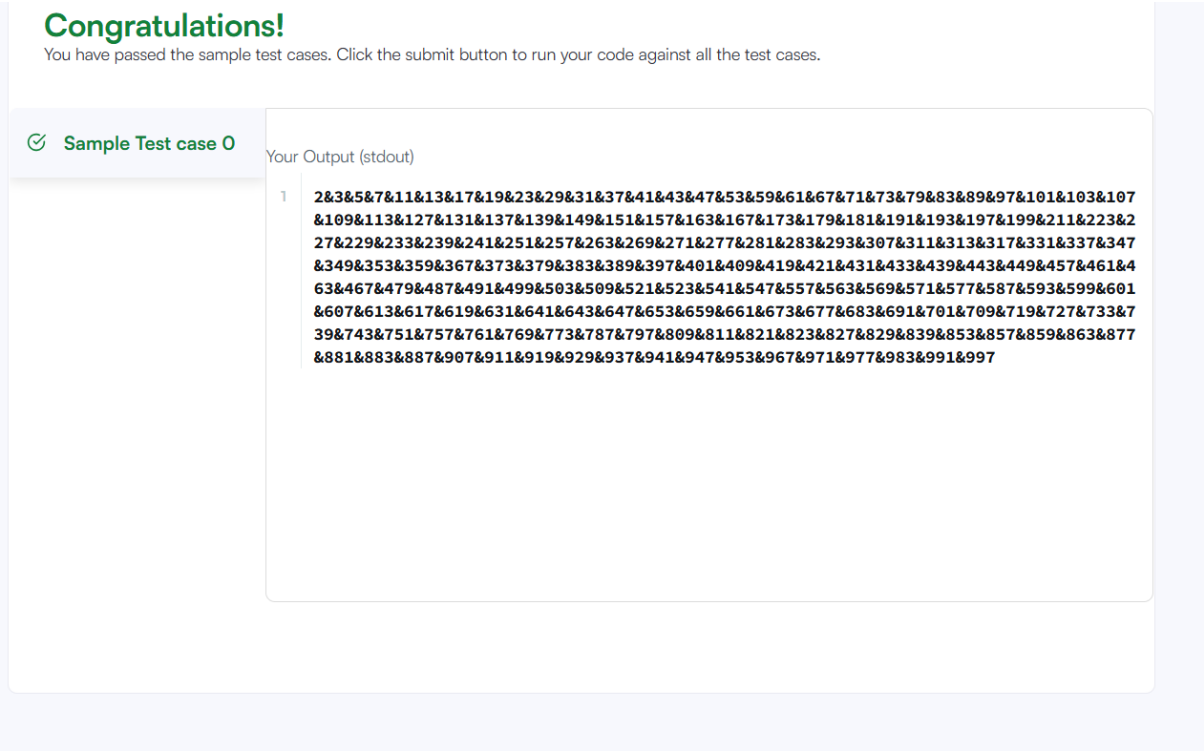
Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

For example, the output for all prime numbers would be:

2&3&5&7

Program

```
WITH RECURSIVE nums AS (
   SELECT 2 AS n
   UNION ALL
   SELECT n + 1
   FROM nums
   WHERE n < 1000
),
primes AS (
   SELECT n
   FROM nums p
   WHERE NOT EXISTS (
      SELECT 1
      FROM nums d
      WHERE d.n <= SQRT(p.n)
       AND p.n % d.n = 0
   )
)
SELECT GROUP_CONCAT(n ORDER BY n SEPARATOR '&')
FROM primes;
```



CODECHEF

BASIC COMMANDS

ALL PRODUCTS

Task

Write a query to output Products table.

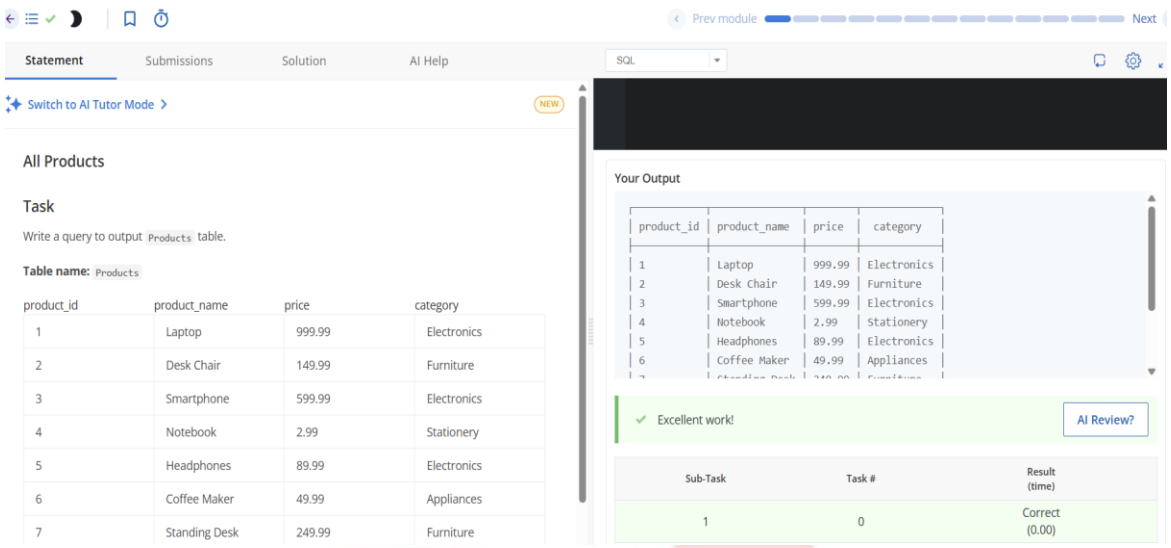
Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture

product_id	product_name	price	category
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

CODE:

```
SELECT * FROM Products;
```



HIGH PRICE OF PRODUCTS

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics

9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

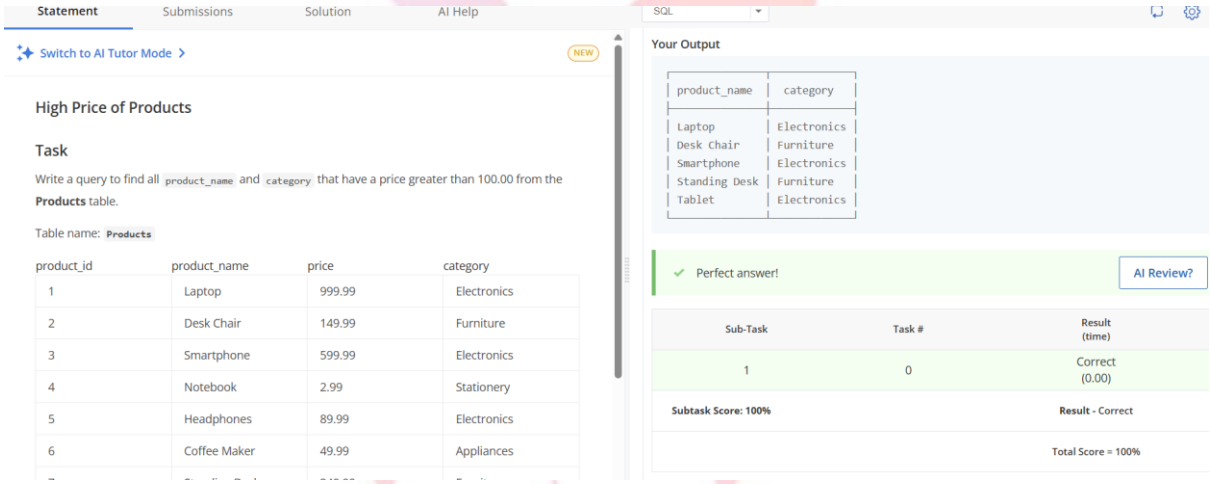
Table name: Products

Expected columns

- product_name
- category

CODE:

```
SELECT product_name, category FROM Products WHERE price > 100.00;
```



AVERAGE SALARY

Task

Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

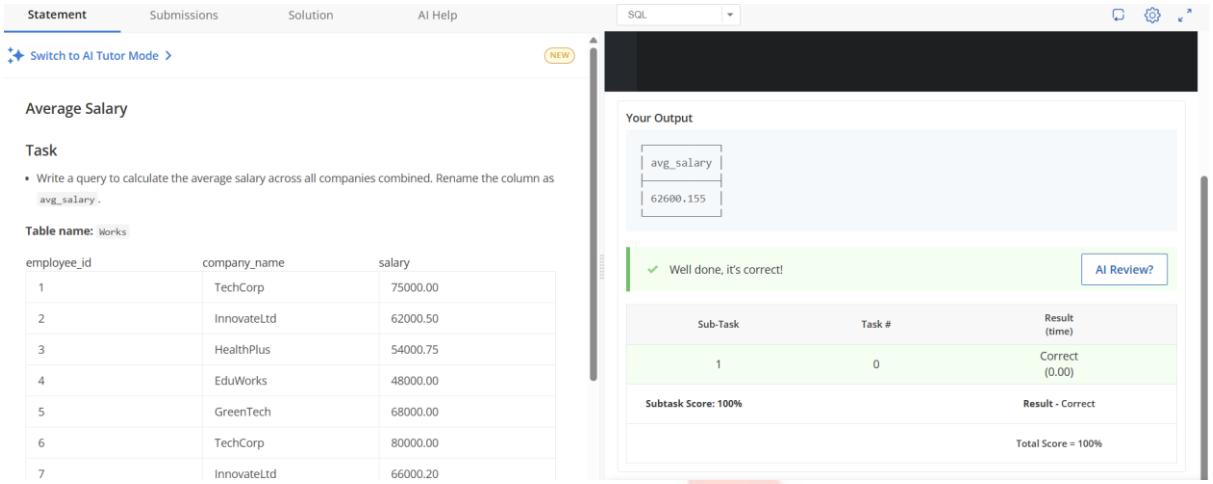
employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected column

- avg_salary

CODE:

```
SELECT AVG(salary) AS avg_salary FROM Works;
```



LOCATE PEOPLE

Task

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles

10	Legal	Boston

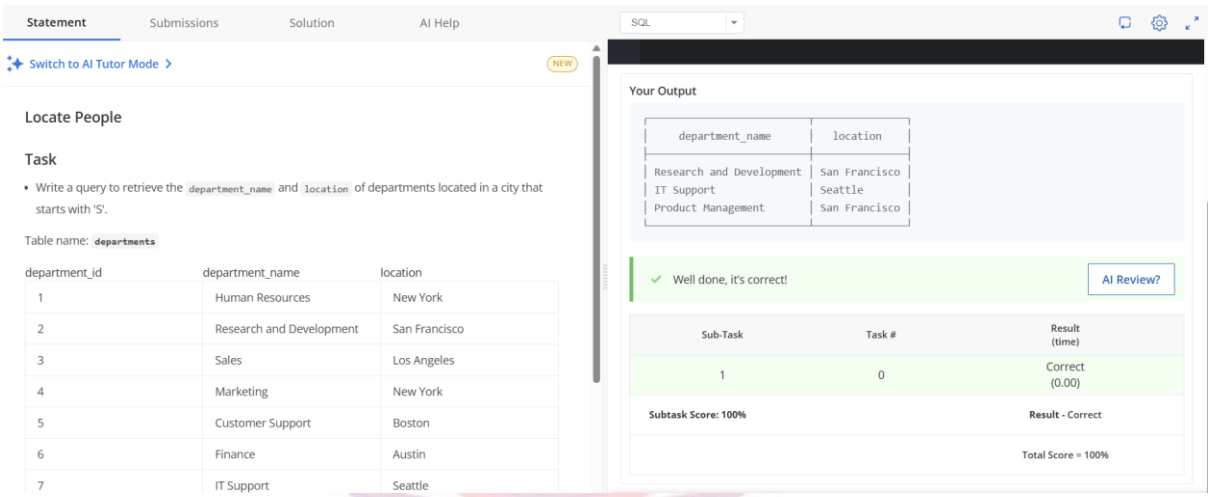
Table name: **departments**

Expected column

- department_name
- location

CODE:

ELECT department_name, location FROM departments WHERE location LIKE 'S%';



DISTINCT COMPANIES

Task

Write a query to select all the distinct companies (company_name) in the Works table.

Table name: Works

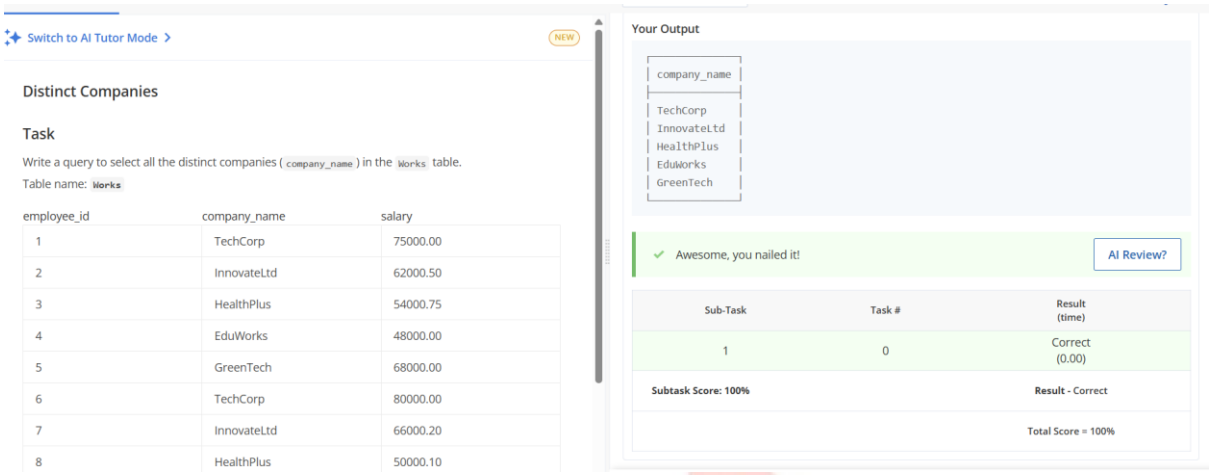
employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column

- company_name

CODE:

```
SELECT DISTINCT company_name FROM Works;
```



FICTION COLLECTION SIZE

Task

Write a query to find the total count of books whose genre is Fiction.

Note: Output column name should be fiction_count.

Table name: Books

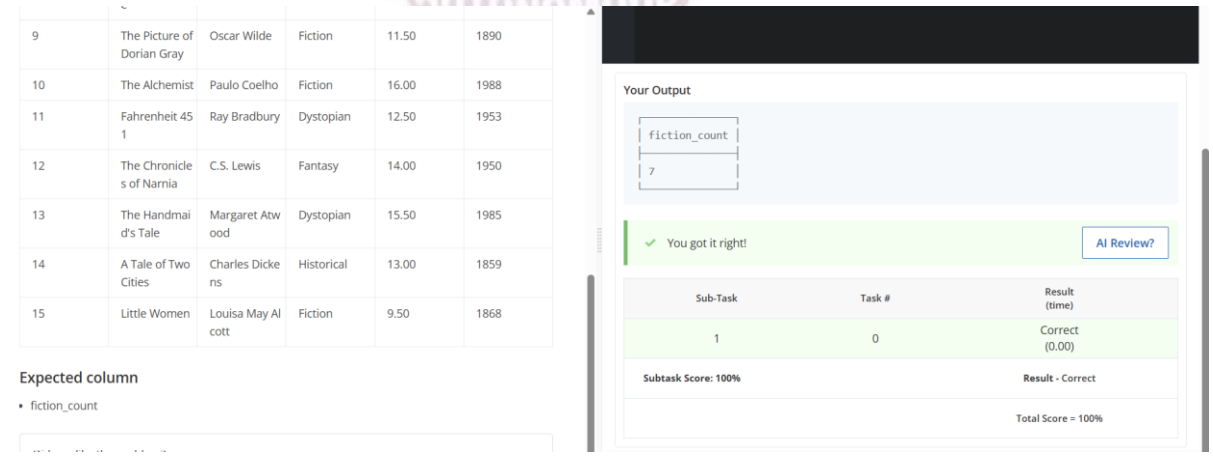
id title author genre price published_year

Expected column

fiction_count

CODE:

```
SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';
```



LIST OF MOVIES WITH RATINGS

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: Cinema

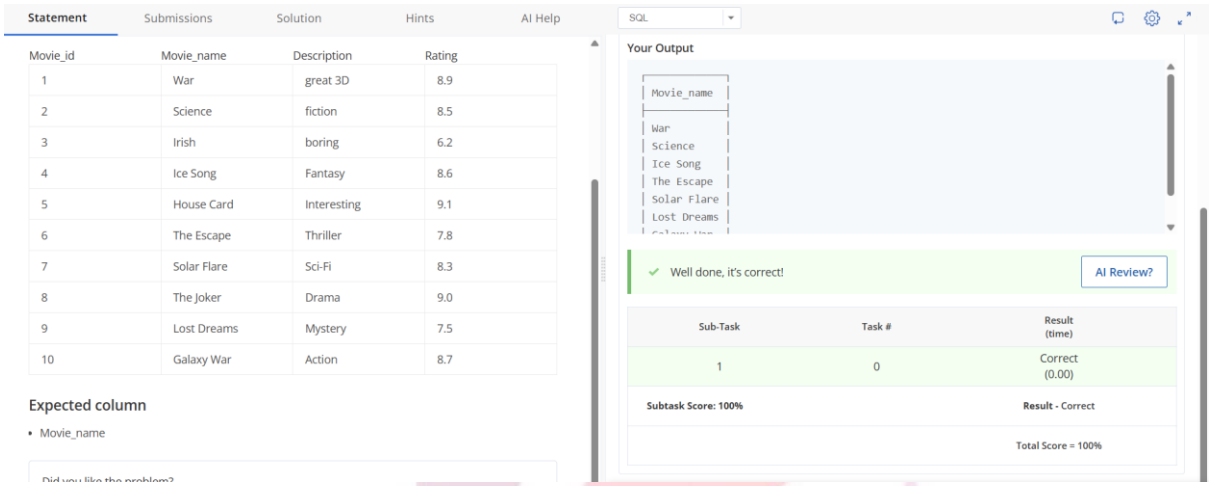
Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

Expected column

Movie_name

CODE:

SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;



HANDLING NULL VALUES

Task

Write a query to retrieve book_id, title, author and published_year of the books which have NULL rating for their books.

Table name: Library

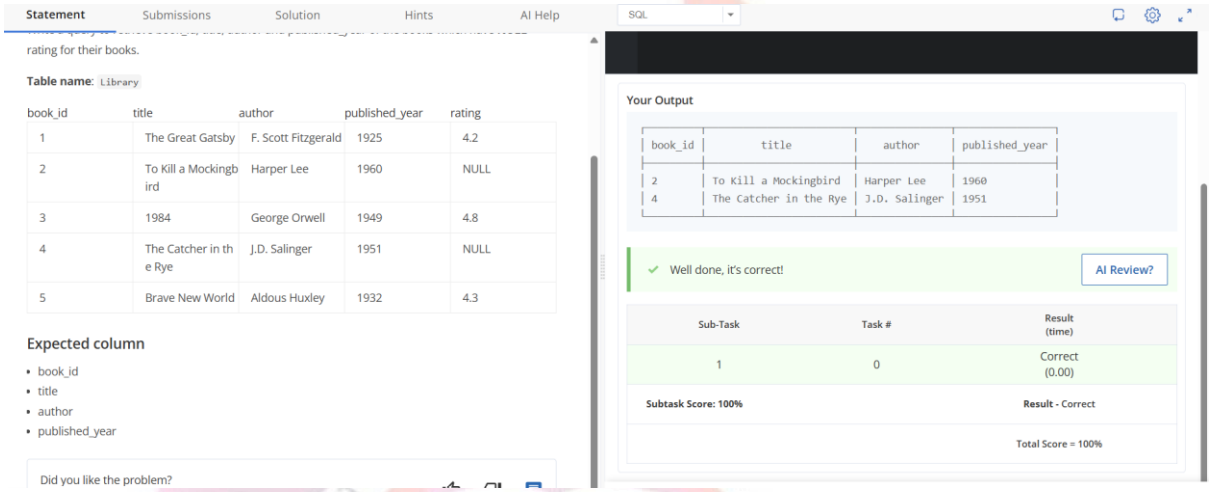
book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

Expected column

- book_id
- title
- author
- published_year

CODE:

SELECT book_id, title, author, published_year FROM Library WHERE rating IS NULL;



HANDLING NULL VALUES

Task

Write a query to retrieve book_id, title, author and published_year of the books which have NULL rating for their books.

Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL

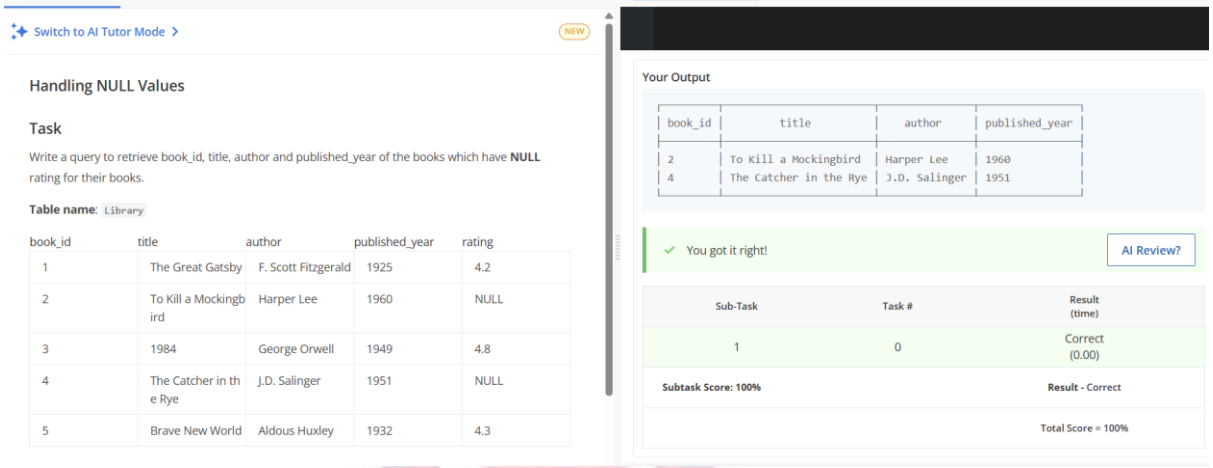
book_id	title	author	published_year	rating
5	Brave New World	Aldous Huxley	1932	4.3

Expected column

- book_id
- title
- author
- published_year

CODE:

```
SELECT book_id, title, author, published_year FROM Library WHERE rating IS NULL;
```



SALARY OF EMPLOYEES

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in descending order

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000

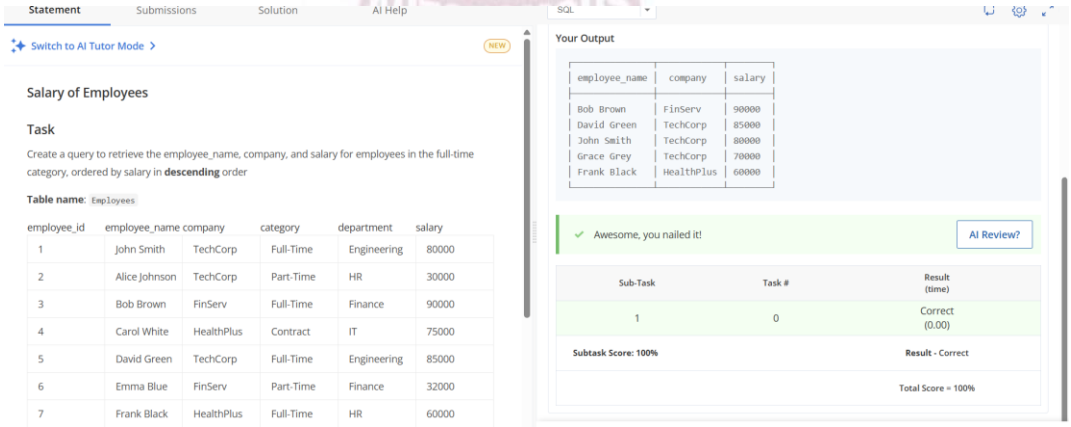
employee_id	employee_name	company	category	department	salary
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected column

- employee_name
- company
- salary

CODE:

SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;



DEPARTMENT OF EACH EMPLOYEE

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

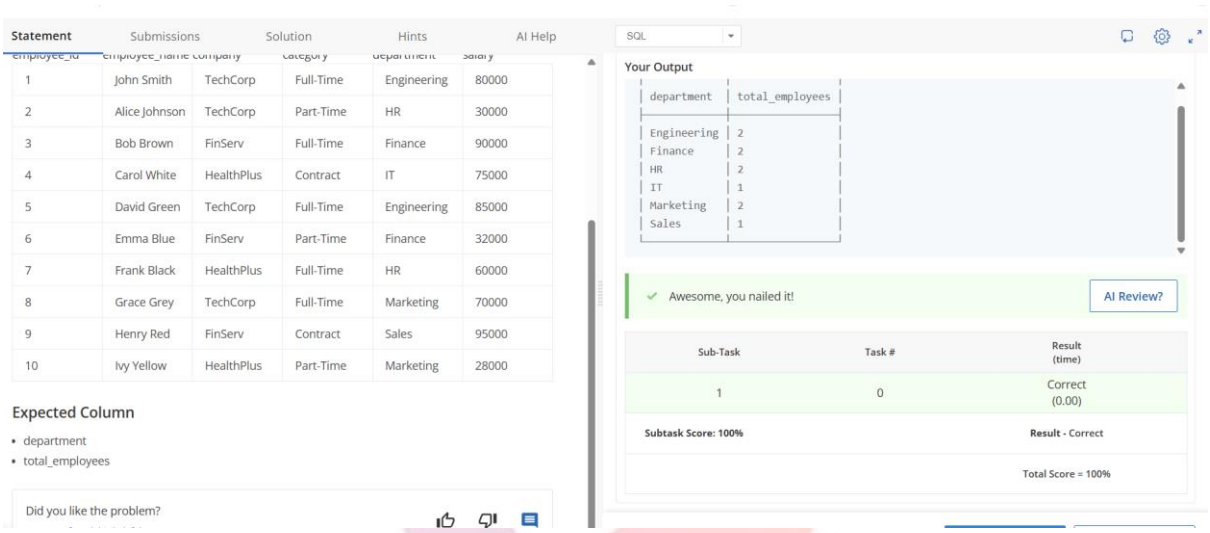
employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected Column

- department
- total_employees

CODE:

SELECT department, COUNT(*) AS total_employees FROM Employees GROUP BY department;



ARTICLE VIEWS

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in ascending order.

Table name: Views

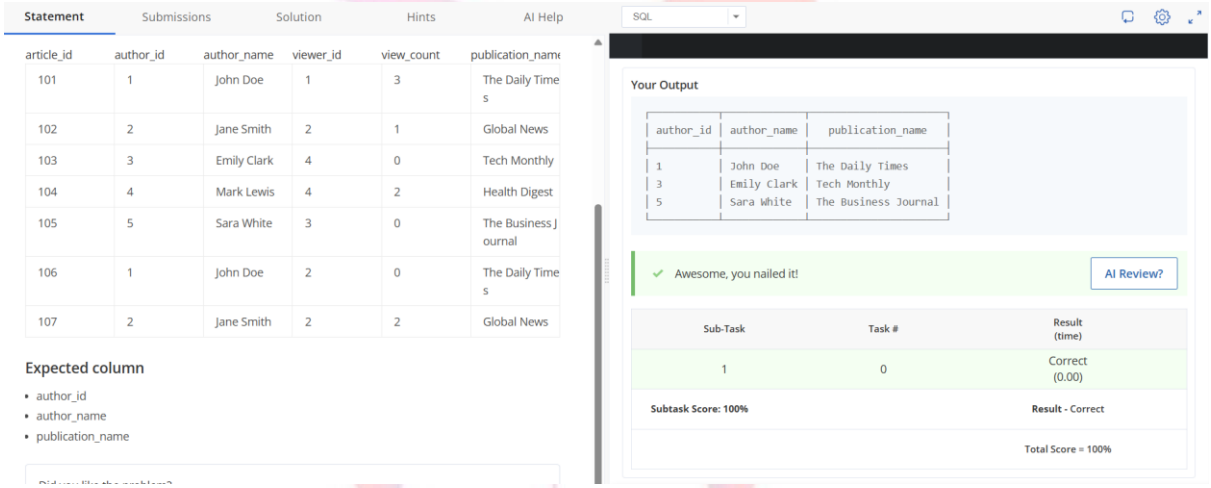
article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

Expected column

- author_id
- author_name
- publication_name

CODE:

```
SELECT DISTINCT author_id, author_name, publication_name FROM Views WHERE
view_count = 0 ORDER BY author_id ASC;
```



PLAYER PERFORMANCE INSIGHTS

Task

Write a query to find the names of the top 3 distinct players by highest score who have won matches, including their scores.

Table 1: Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4

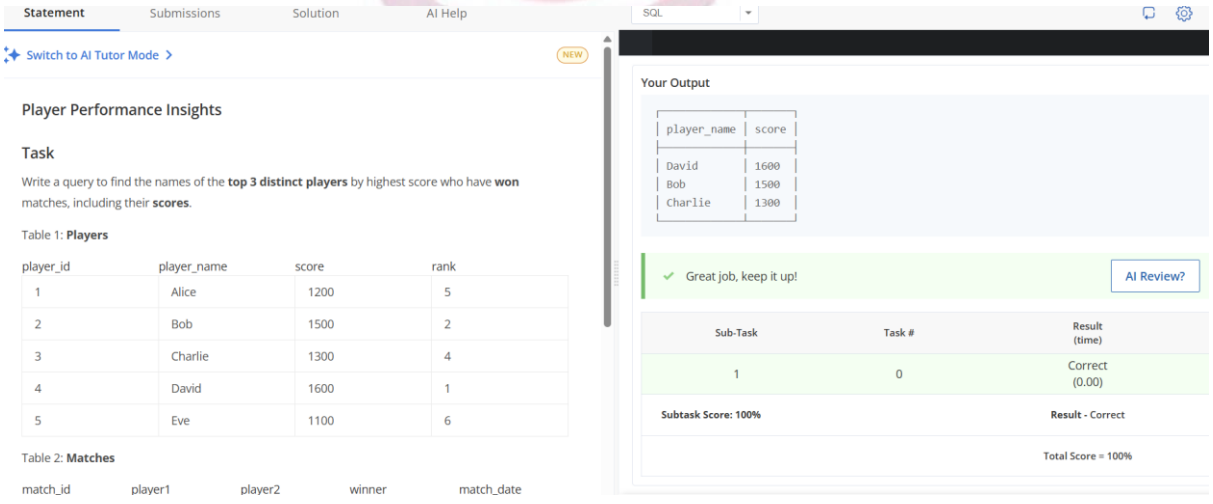
player_id	player_name	score	rank
4	David	1600	1
5	Eve	1100	6

Table 2: Matches

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

CODE:

```
SELECT DISTINCT player_name, score
FROM Players JOIN Matches WHERE winner=player_name
ORDER BY score DESC
LIMIT 3;
```



PLAYER DETAILS

Task

Write a query to retrieve the details of the last five matches played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named Players and Matches.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

Expected Output Columns:

match_id

player_1

player_2

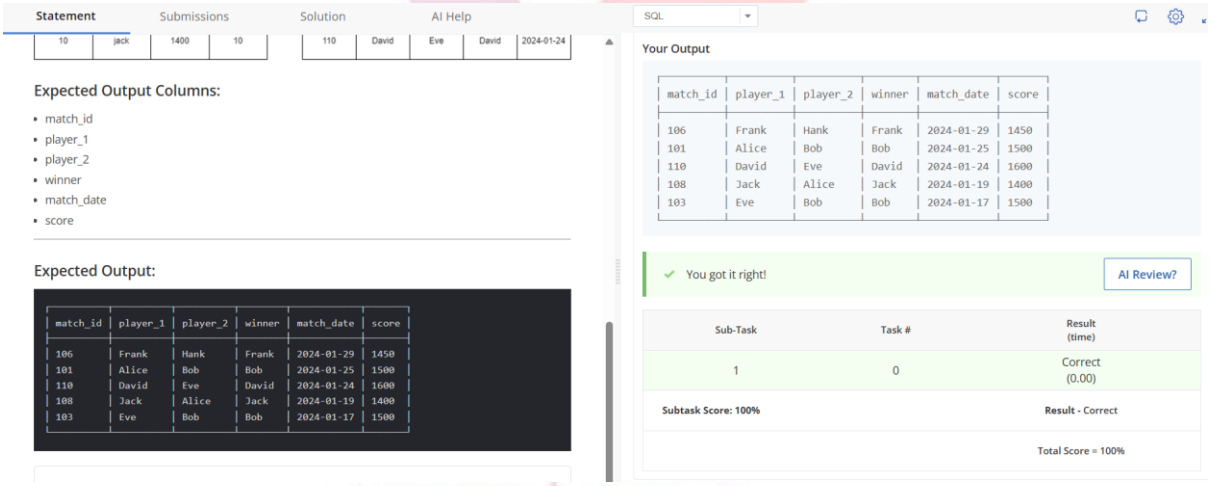
winner

match_date

score

CODE:

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```



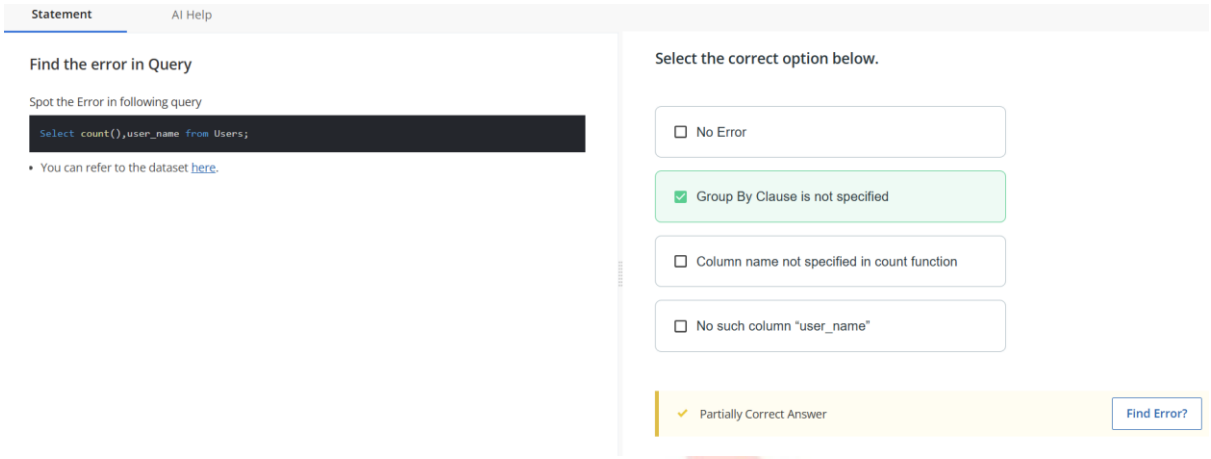
CASE STUDY - CODECHEF DATABASE

FIND THE ERROR IN QUERY

pot the Error in following query

Select count(),user_name from Users;

You can refer to the dataset here.



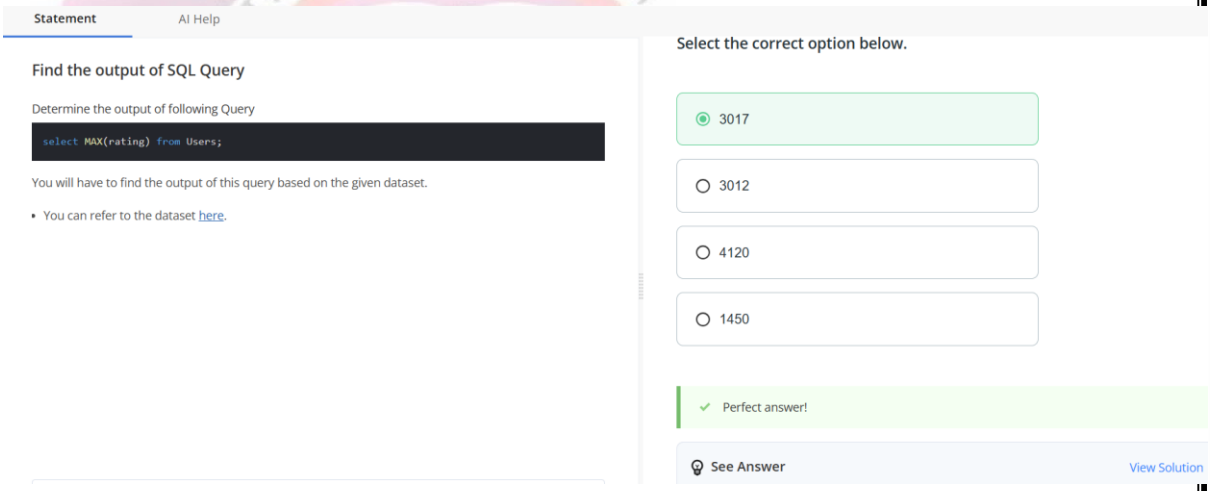
FIND THE OUTPUT OF SQL QUERY

Determine the output of following Query

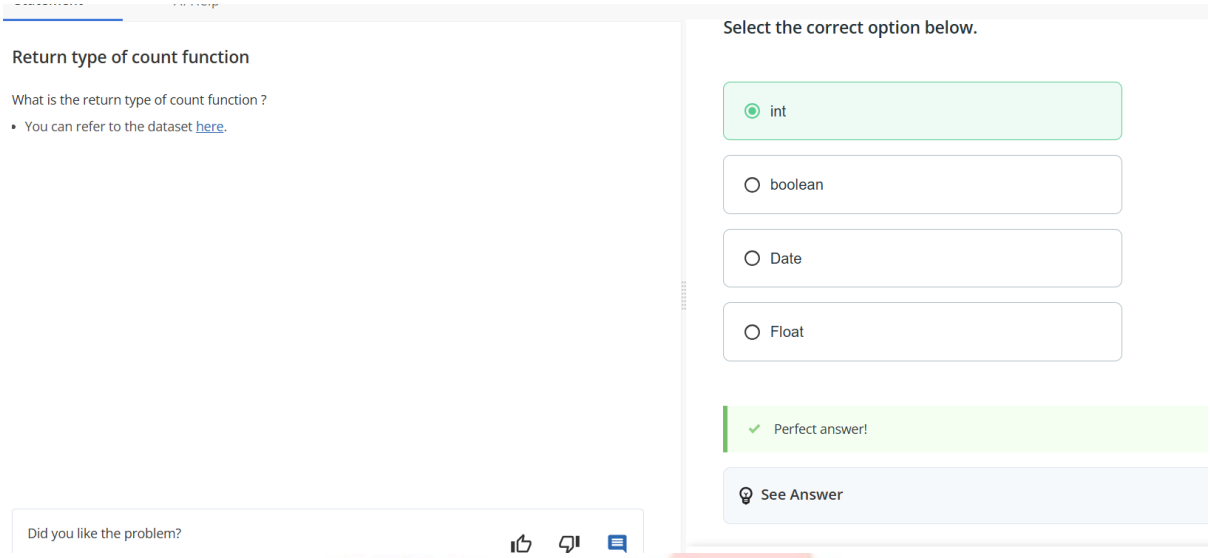
select MAX(rating) from Users;

You will have to find the output of this query based on the given dataset.

- You can refer to the dataset here.



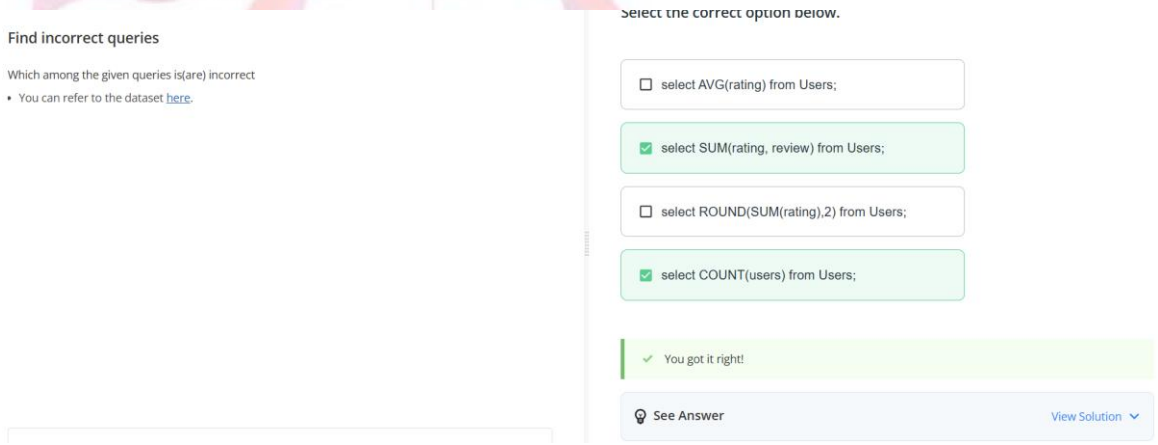
RETURN TYPE OF COUNT FUNCTION



FIND INCORRECT QUERIES

hich among the given queries is(are) incorrect

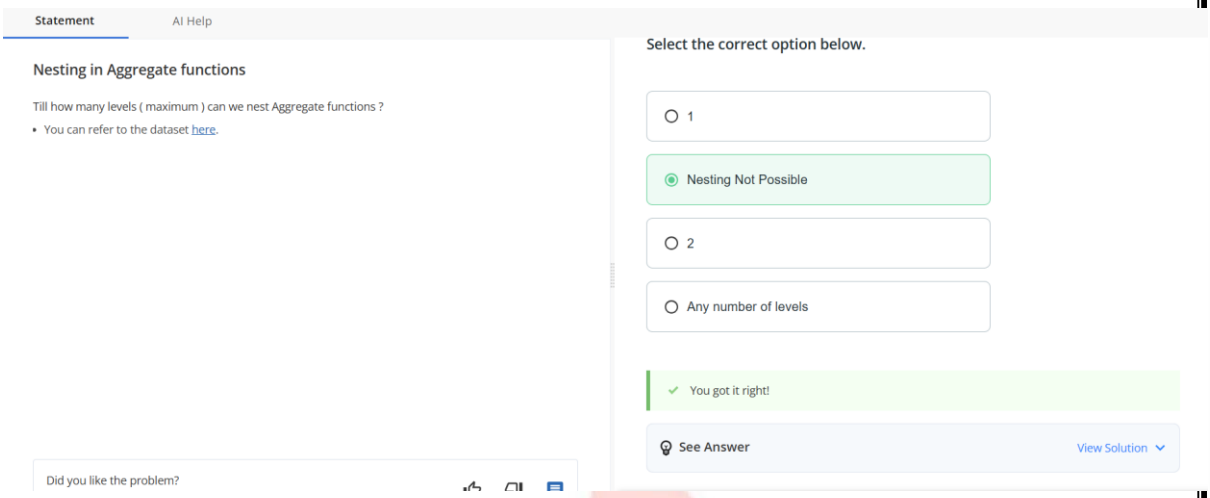
- You can refer to the dataset here.



NESTING IN AGGREGATE FUNCTIONS

Till how many levels (maximum) can we nest Aggregate functions ?

- You can refer to the dataset here.

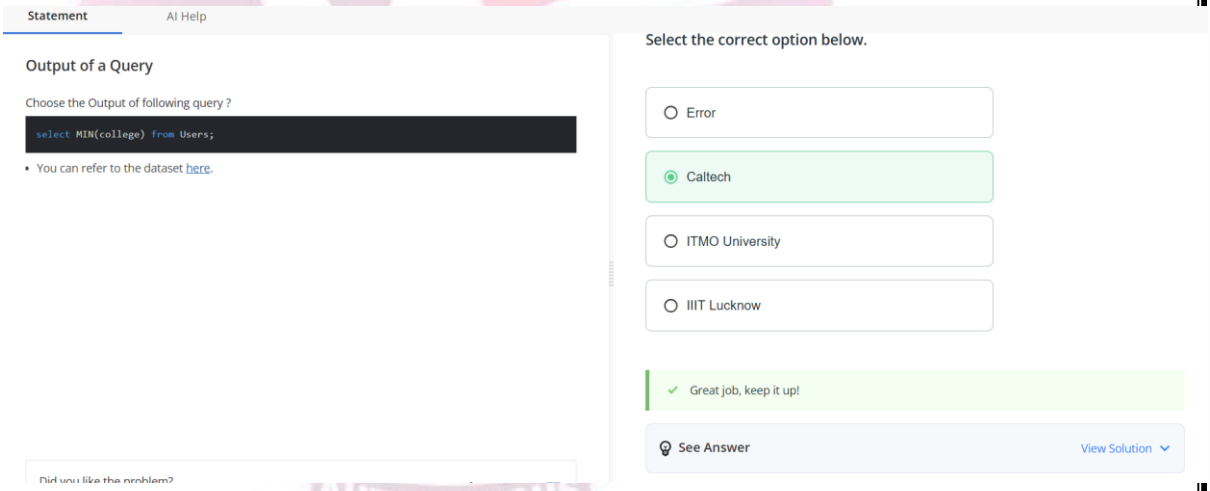


OUTPUT OF A QUERY

Choose the Output of following query ?

select MIN(college) from Users;

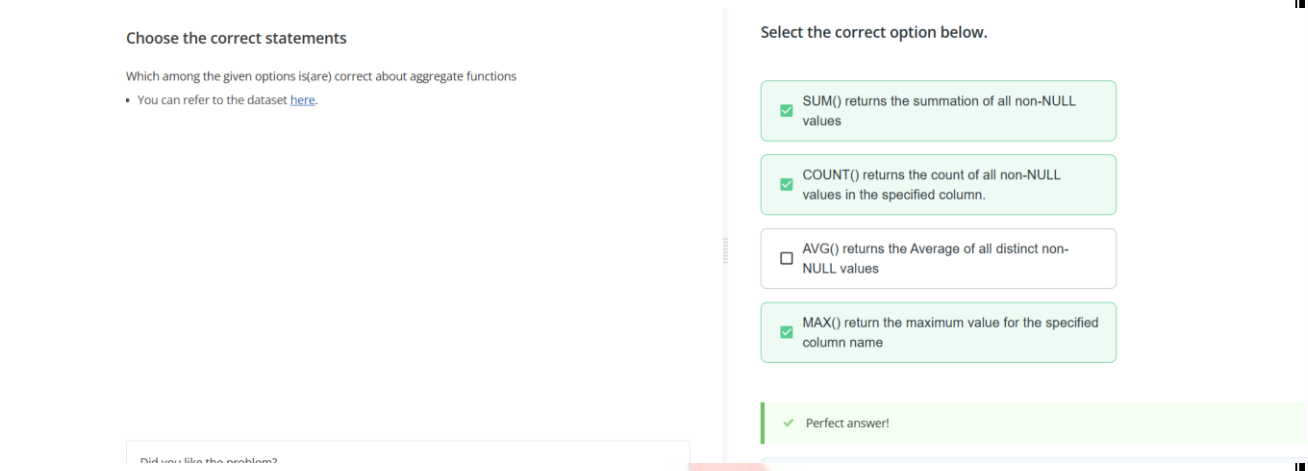
- You can refer to the dataset here.



CHOOSE THE CORRECT STATEMENTS

Which among the given options is(are) correct about aggregate functions

- You can refer to the dataset here.



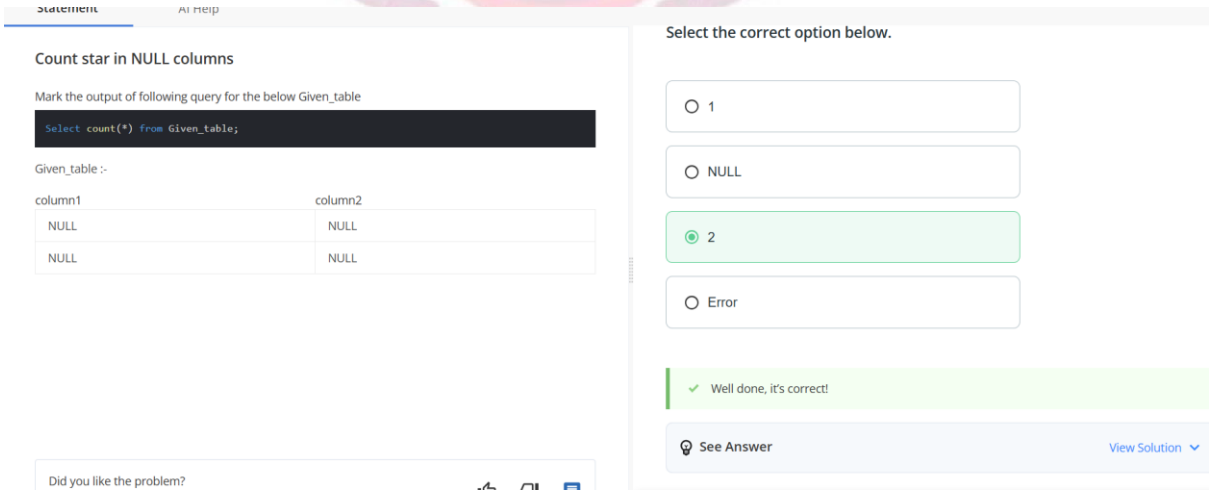
COUNT STAR IN NULL COLUMNS

Mark the output of following query for the below Given_TABLE

Select count(*) from Given_table;

Given_table :-

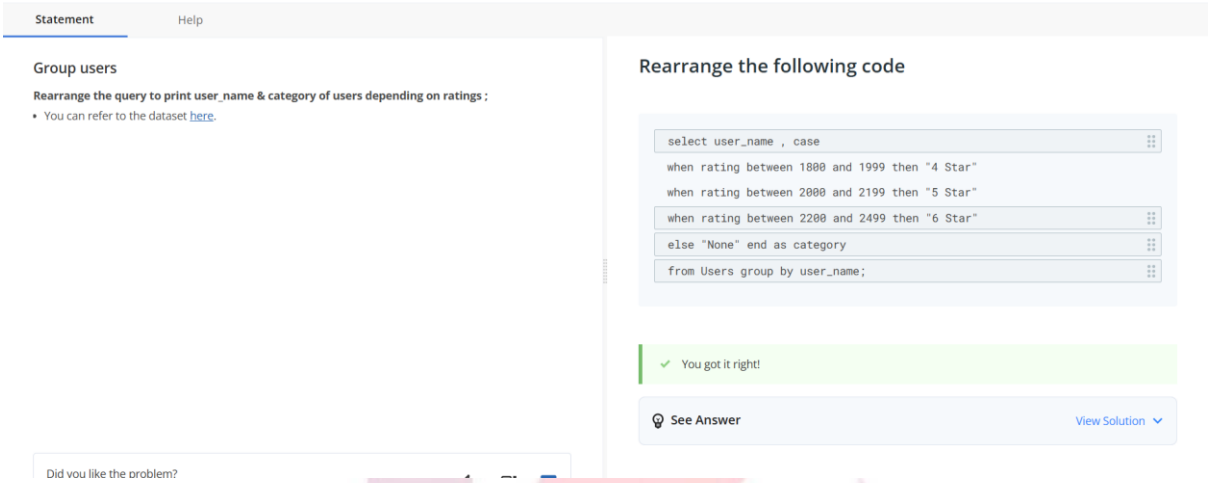
column1	column2
NULL	NULL
NULL	NULL



GROUP USERS

Rearrange the query to print user_name & category of users depending on ratings ;

- You can refer to the dataset here.



CASE STUDY - ABC CAB BOOKING APP

Introduction to cab booking dataset:

This case study scenario involves user complaints and ride information associated with ABC Cab-booking App. You have been given a dataset containing multiple tables associated with the case study.

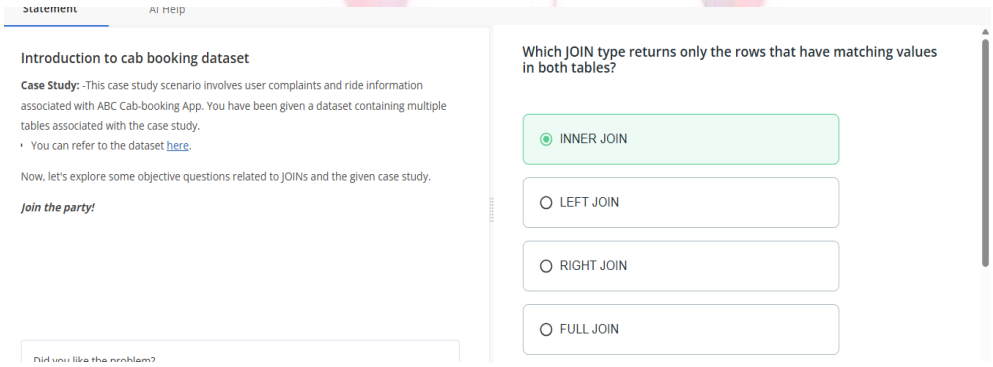
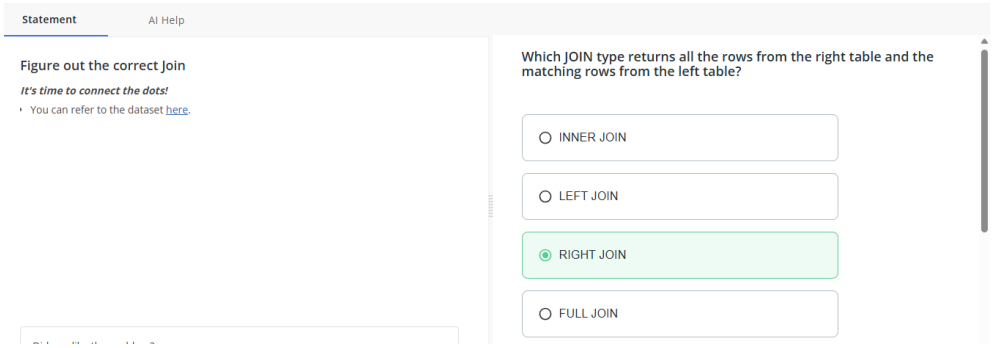


FIGURE OUT THE CORRECT JOIN:

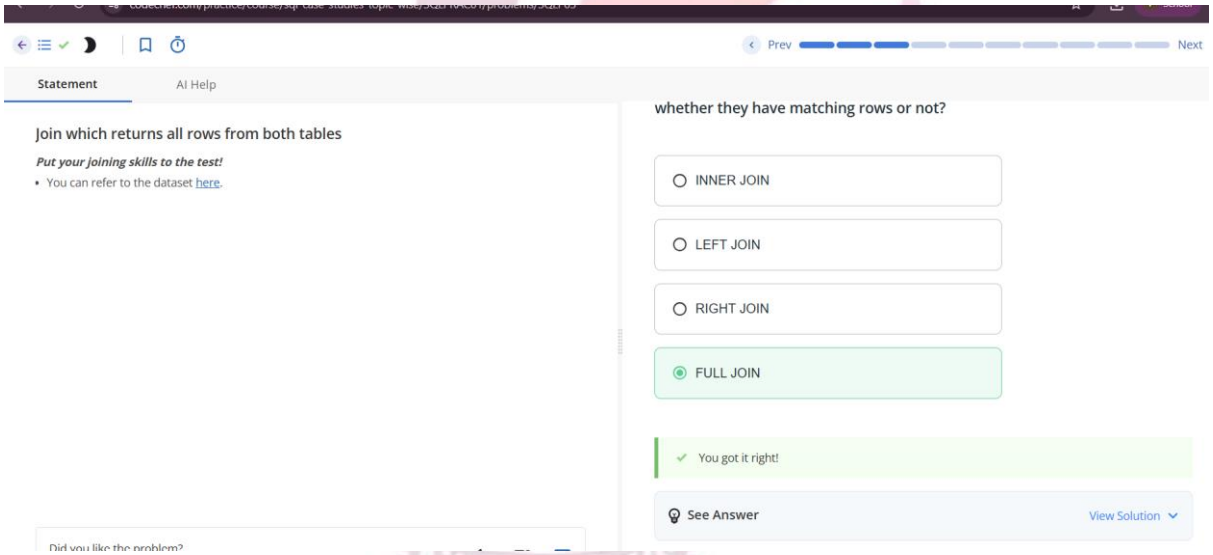
It's time to connect the dots!



JOIN WHICH RETURNS ALL ROWS FROM BOTH TABLES

Put your joining skills to the test!

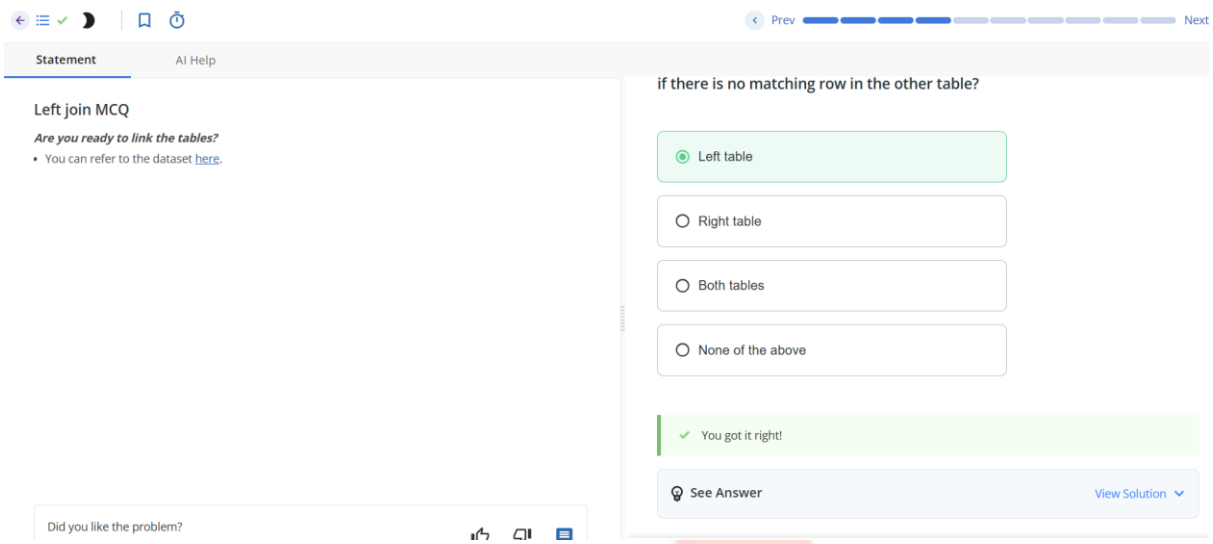
- You can refer to the dataset here.



LEFT JOIN MCQ

Are you ready to link the tables?

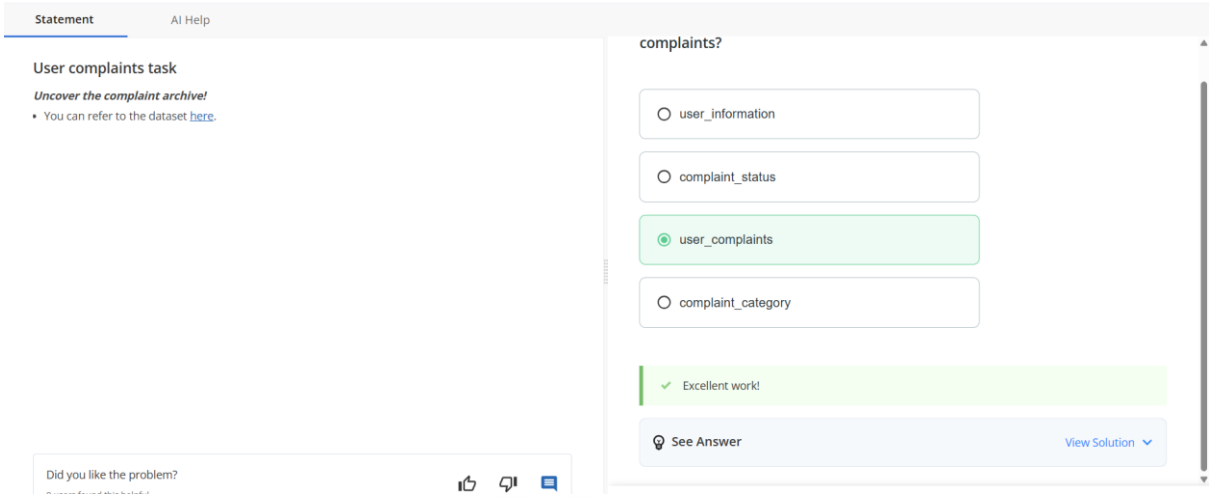
- You can refer to the dataset here.



USER COMPLAINTS TASK

Uncover the complaint archive!

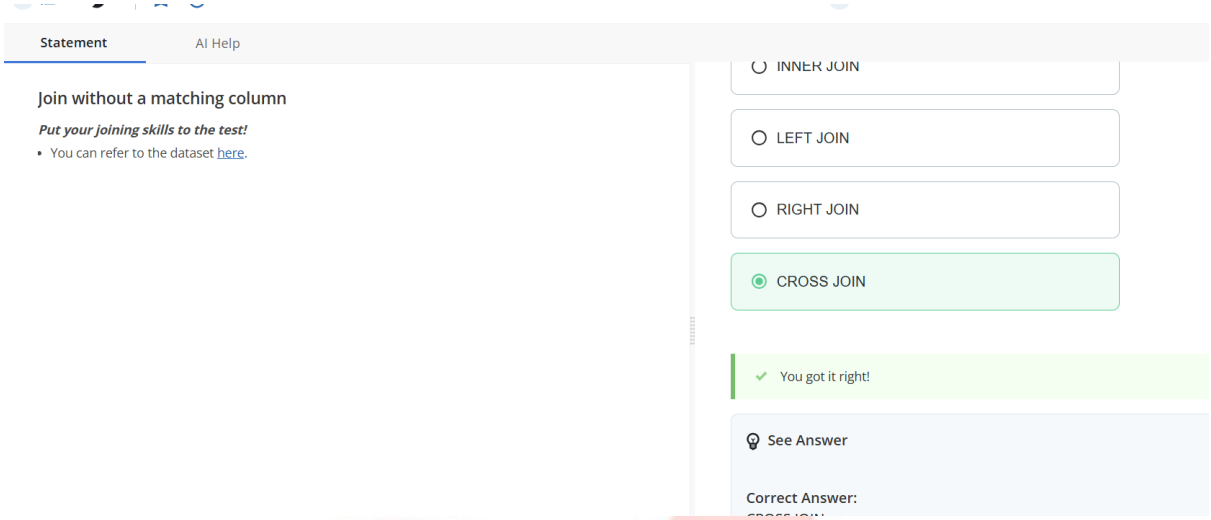
- You can refer to the dataset here.



JOIN WITHOUT A MATCHING COLUMN

Put your joining skills to the test!

You can refer to the dataset here.



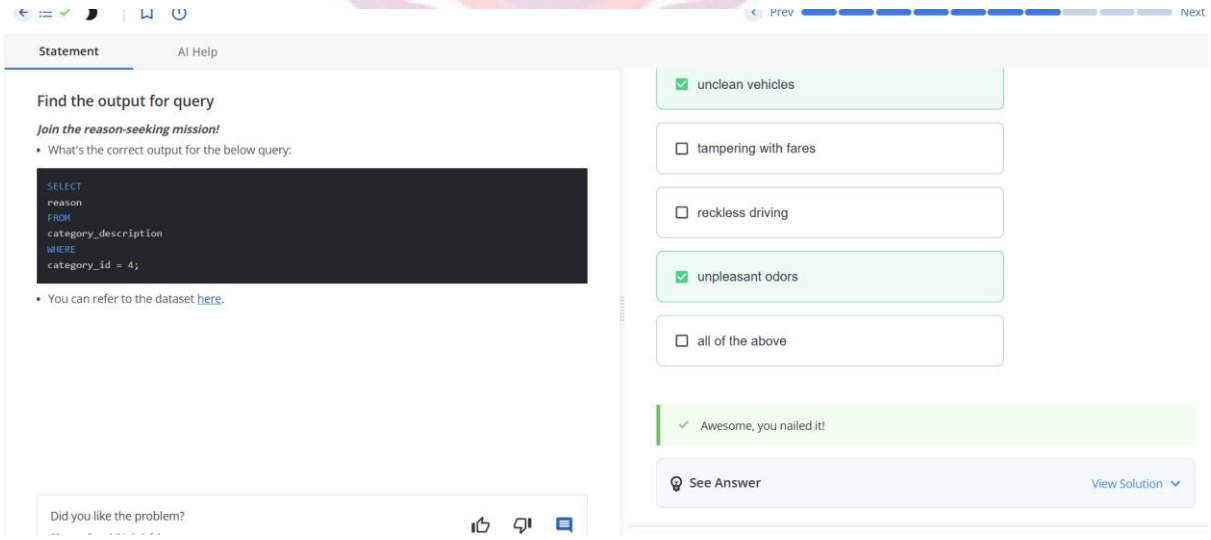
FIND THE OUTPUT FOR QUERY

Join the reason-seeking mission!

- What's the correct output for the below query:

```
SELECT
reason
FROM
category_description
WHERE
category_id = 4;
```

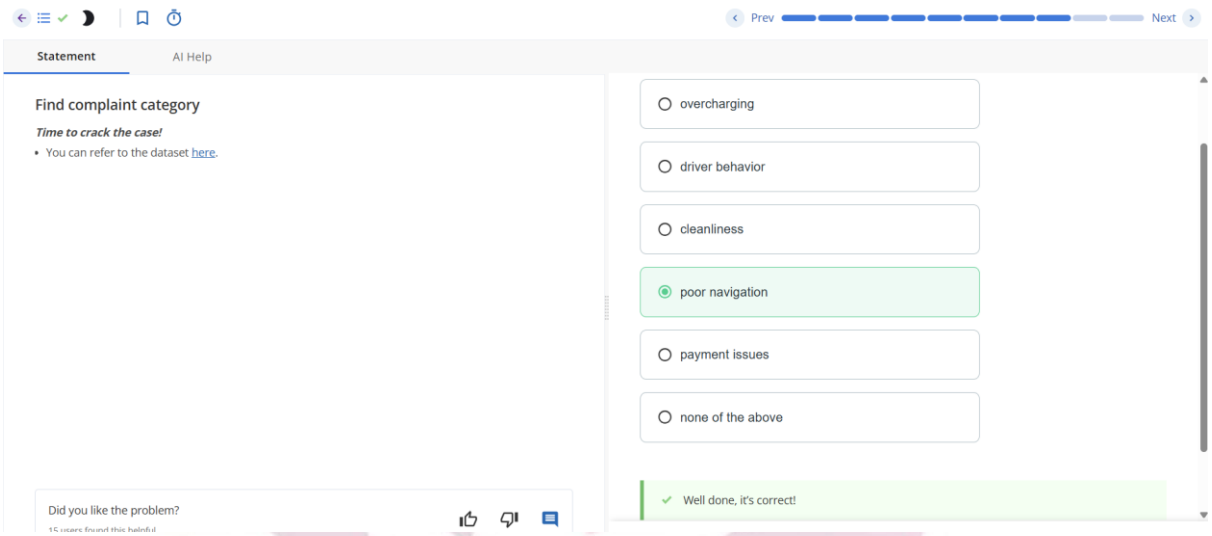
- You can refer to the dataset here.



FIND COMPLAINT CATEGORY

Time to crack the case!

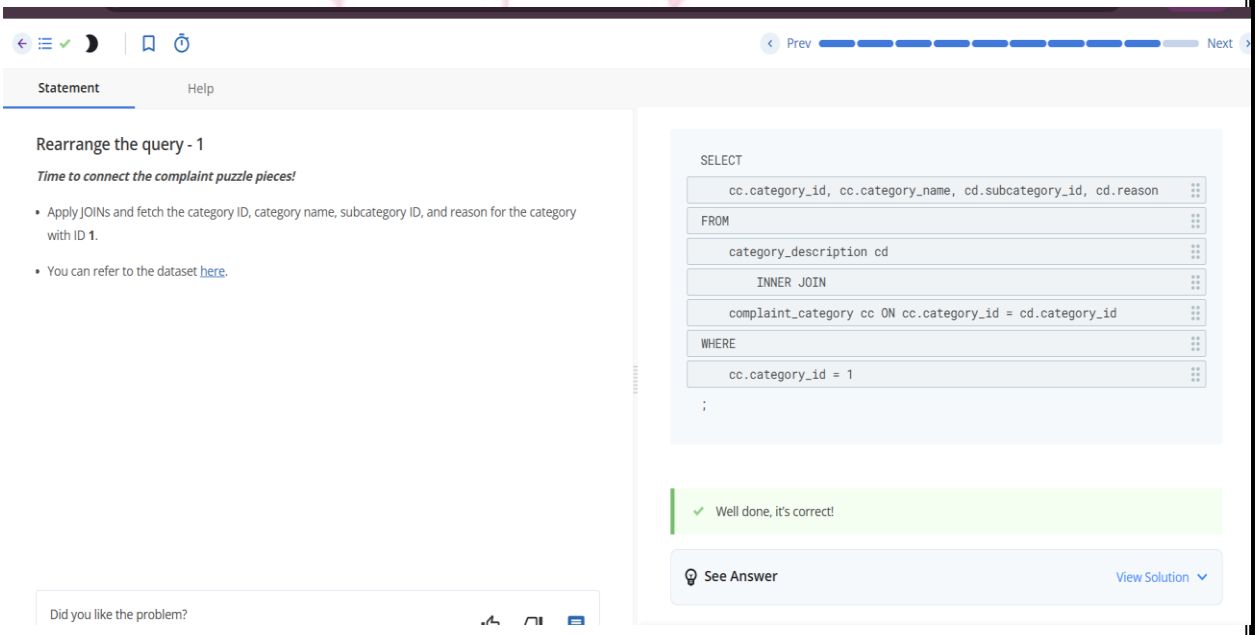
- You can refer to the dataset here.



REARRANGE THE QUERY - 1

Time to connect the complaint puzzle pieces!

- Apply JOINs and fetch the category ID, category name, subcategory ID, and reason for the category with ID 1.
- You can refer to the dataset here.

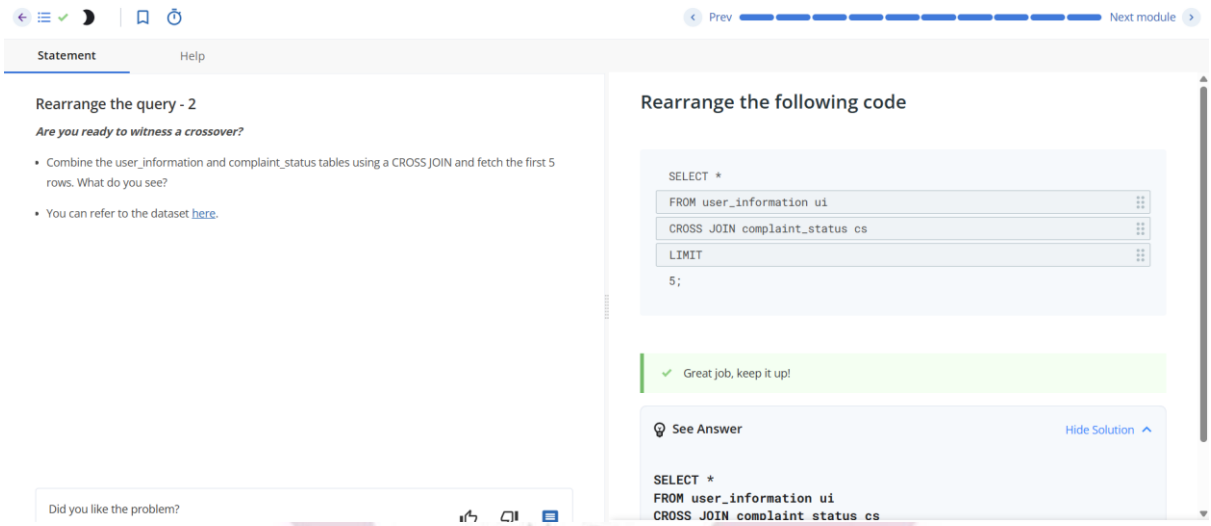


REARRANGE THE QUERY - 2

Are you ready to witness a crossover?

Combine the user_information and complaint_status tables using a CROSS JOIN andfetch the first 5 rows. What do you see?

You can refer to the dataset here.



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CASE STUDY - HAPPY HAVEN HOUSE RENTAL APP

INTRODUCTION TO RENTAL APP

This case study scenario revolves around the house rental app called HappyHaven.
The purpose of this case study is to analyze the data and address various scenarios and challenges faced by users of the app.

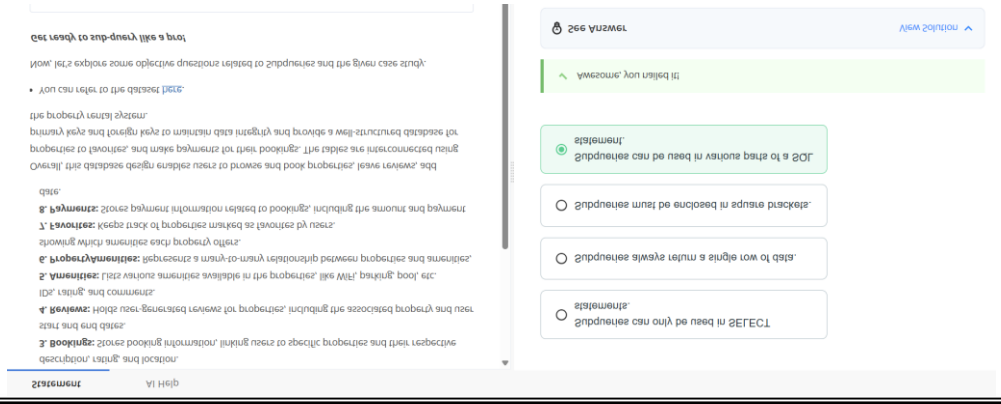
- The dataset includes 8 tables:
  1. Users: Stores information about the platform's users, such as their name, email, phone number, and address.
  2. Properties: Contains details about the available properties for rental, including their title, description, rating, and location.
  3. Bookings: Stores booking information, linking users to specific properties and their respective start and end dates.
  4. Reviews: Holds user-generated reviews for properties, including the associated property and user IDs, rating, and comments.
  5. Amenities: Lists various amenities available in the properties, like WiFi, parking, pool, etc.
  6. PropertyAmenities: Represents a many-to-many relationship between properties and amenities, showing which amenities each property offers.
  7. Favorites: Keeps track of properties marked as favorites by users.
  8. Payments: Stores payment information related to bookings, including the amount and payment date.

Overall, this database design enables users to browse and book properties, leave reviews, add properties to favorites, and make payments for their bookings. The tables are interconnected using primary keys and foreign keys to maintain data integrity and provide a well-structured database for the property rental system.

- You can refer to the dataset here.

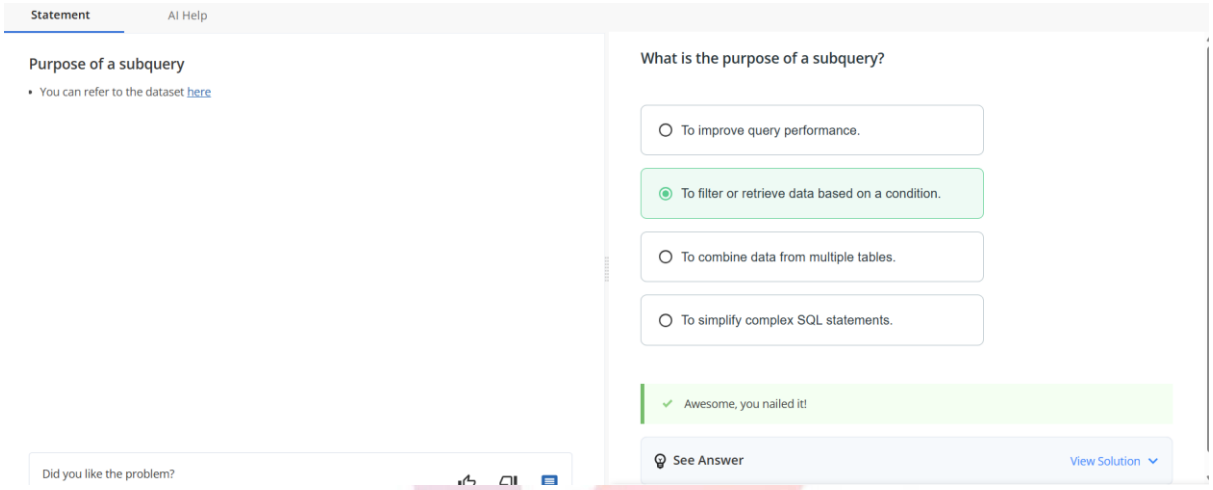
Now, let's explore some objective questions related to Subqueries and the given case study.

Get ready to sub-query like a pro!



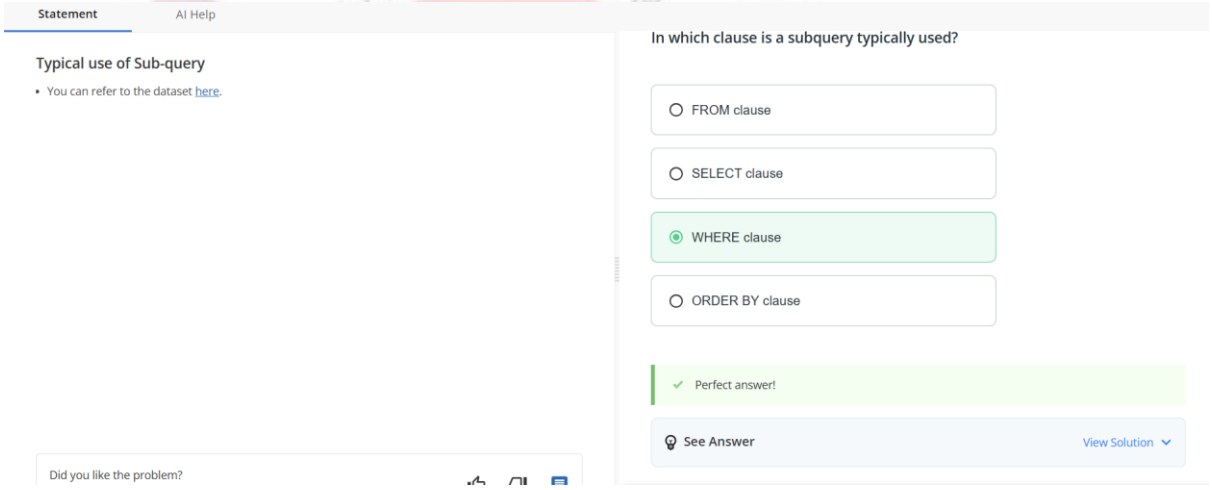
PURPOSE OF A SUBQUERY

You can refer to the dataset here



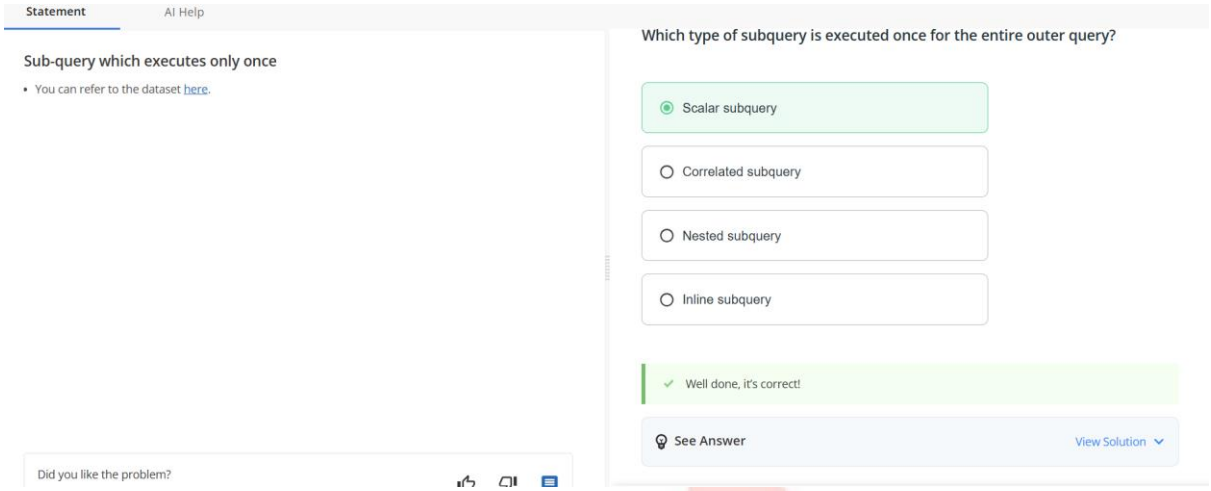
TYPICAL USE OF SUB-QUERY

- You can refer to the dataset here.



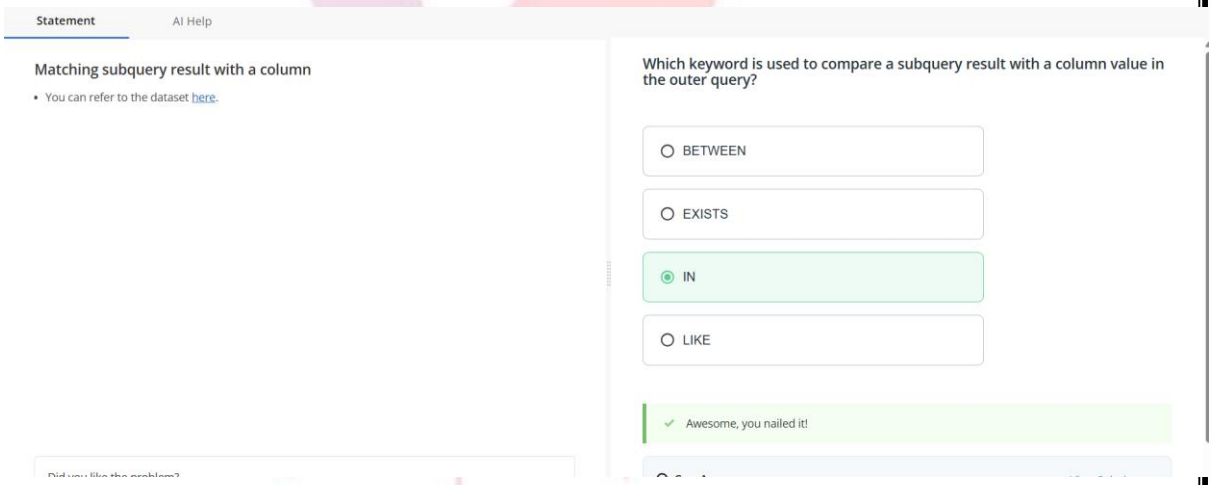
SUB-QUERY WHICH EXECUTES ONLY ONCE

- You can refer to the dataset here.



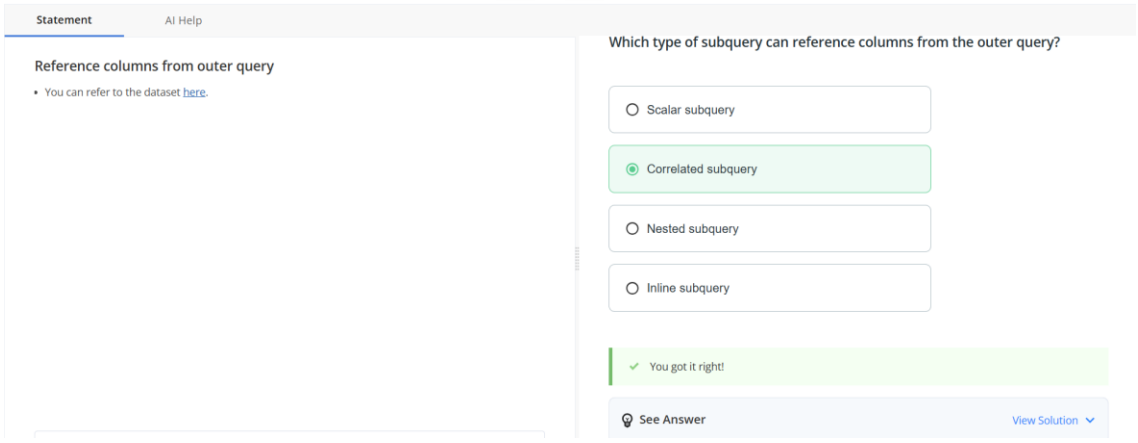
MATCHING SUBQUERY RESULT WITH A COLUMN

- You can refer to the dataset here.



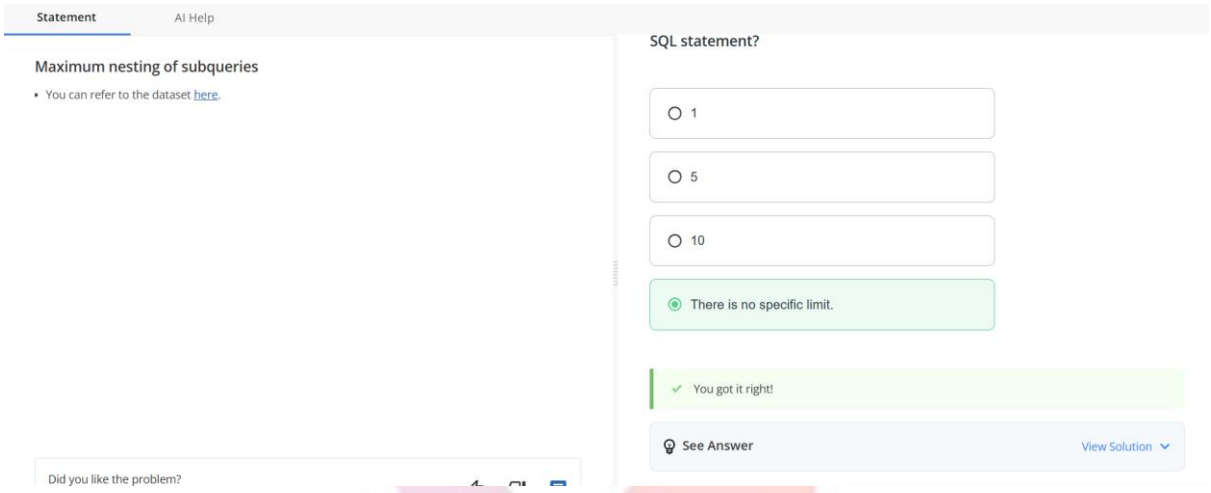
REFERENCE COLUMNS FROM OUTER QUERY

- You can refer to the dataset here.



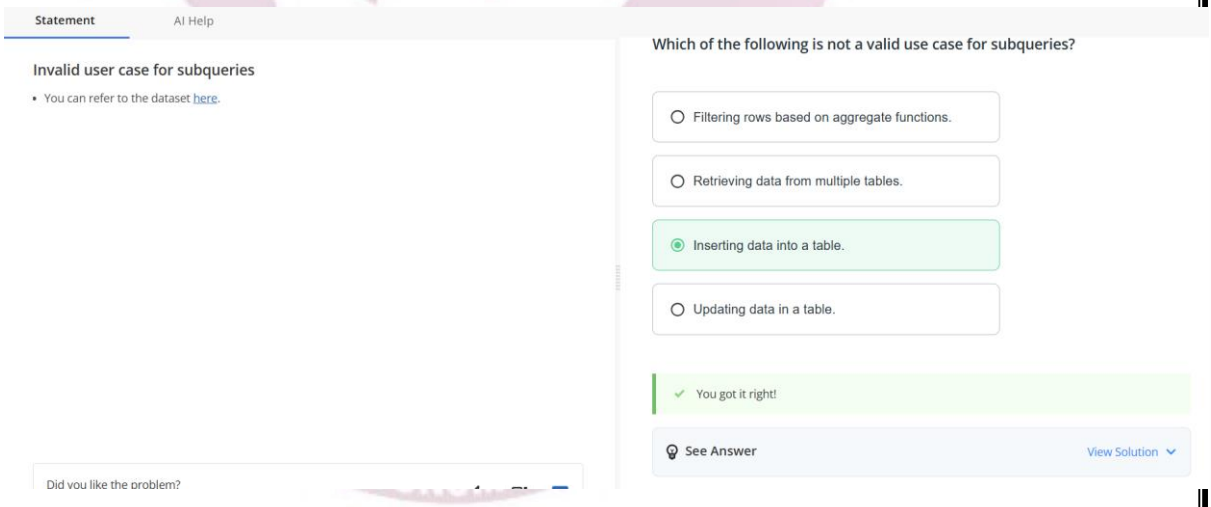
MAXIMUM NESTING OF SUBQUERIES

- You can refer to the dataset here.



INVALID USER CASE FOR SUBQUERIES

- You can refer to the dataset here.

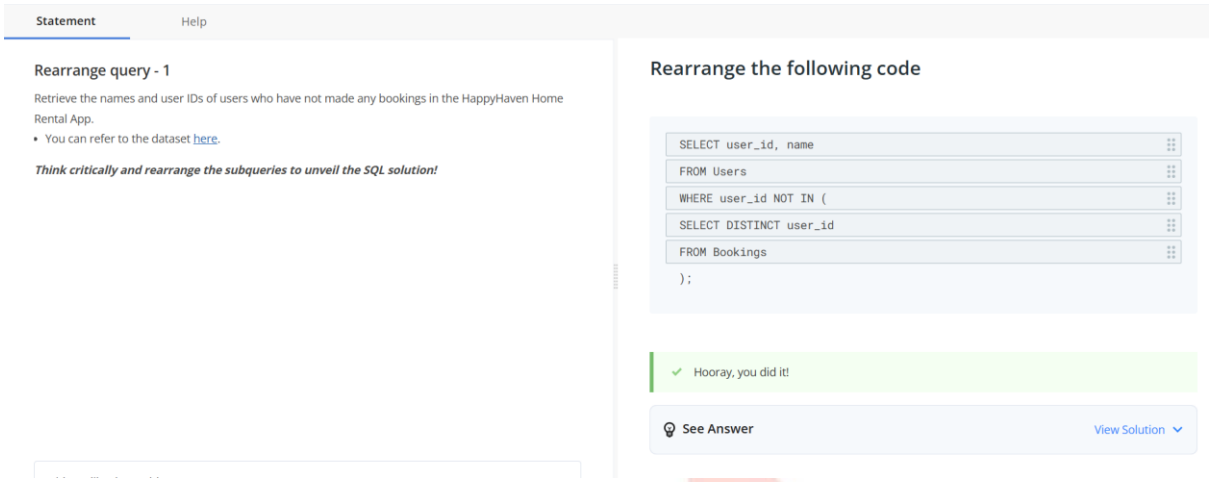


REARRANGE QUERY - 1

Retrieve the names and user IDs of users who have not made any bookings in the HappyHaven Home Rental App.

- You can refer to the dataset here.

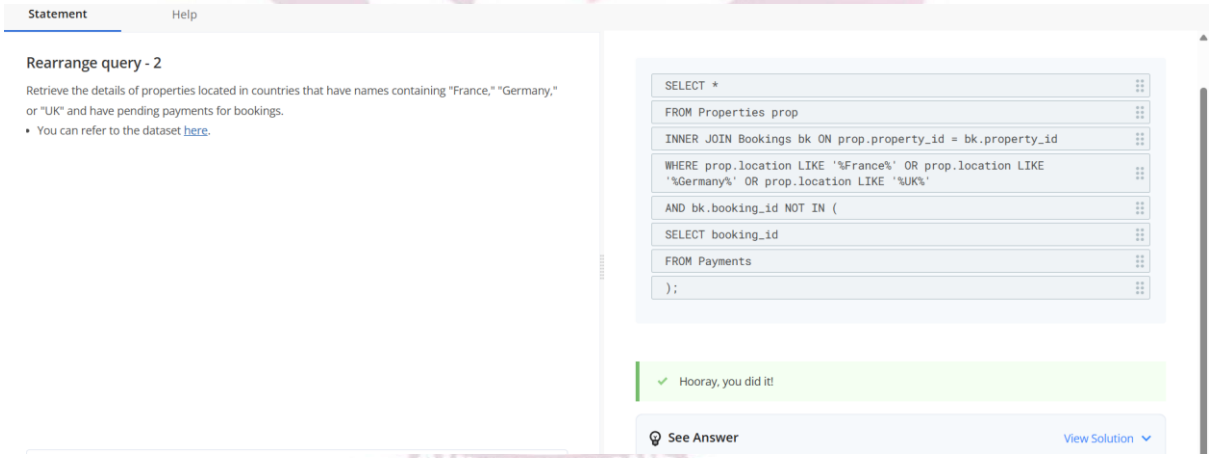
Think critically and rearrange the subqueries to unveil the SQL solution!



REARRANGE QUERY - 2

Retrieve the details of properties located in countries that have names containing "France," "Germany," or "UK" and have pending payments for bookings.

- You can refer to the dataset here.



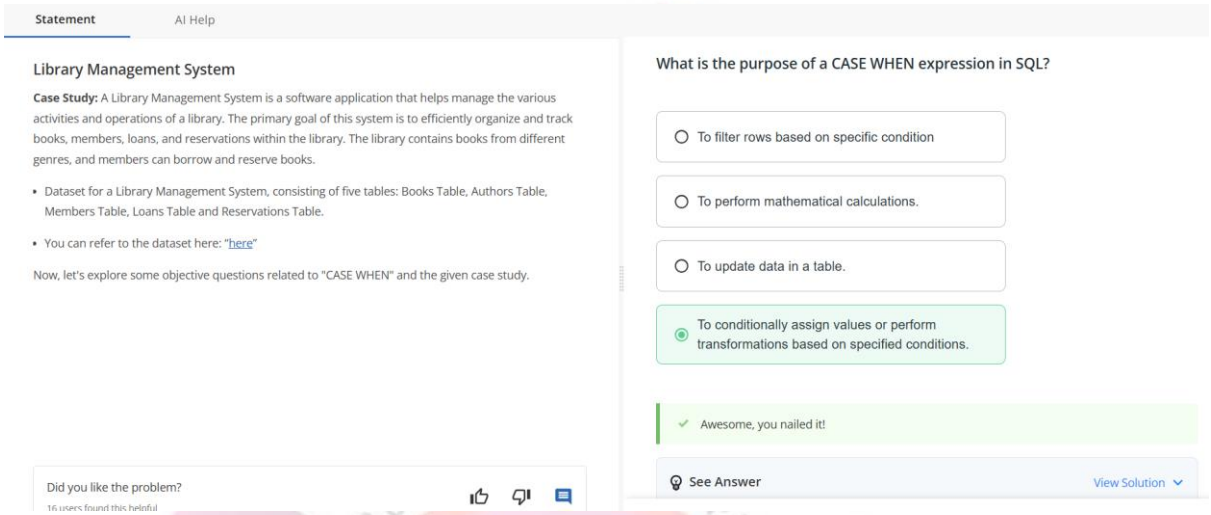
CASE STUDY - LIBRARY MANAGEMENT SYSTEM

LIBRARY MANAGEMENT SYSTEM

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Dataset for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

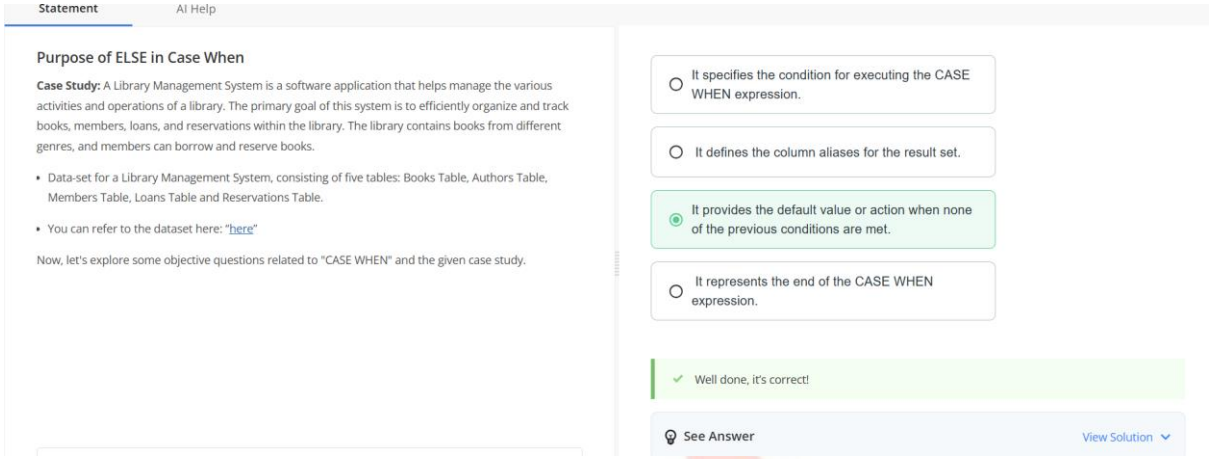


PURPOSE OF ELSE IN CASE WHEN

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

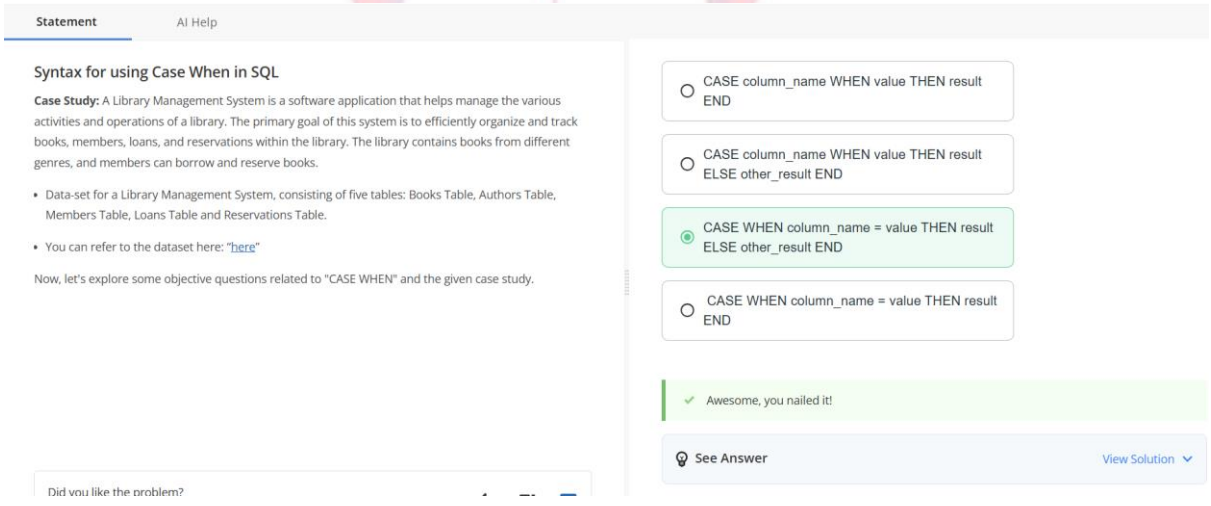


SYNTAX FOR USING CASE WHEN IN SQL

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

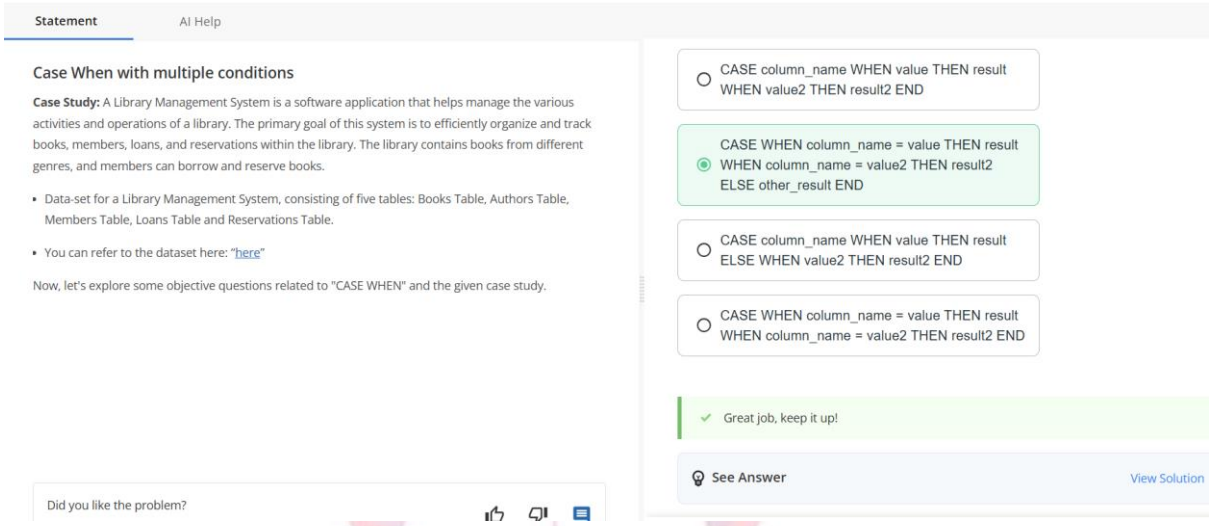


CASE WHEN WITH MULTIPLE CONDITIONS

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

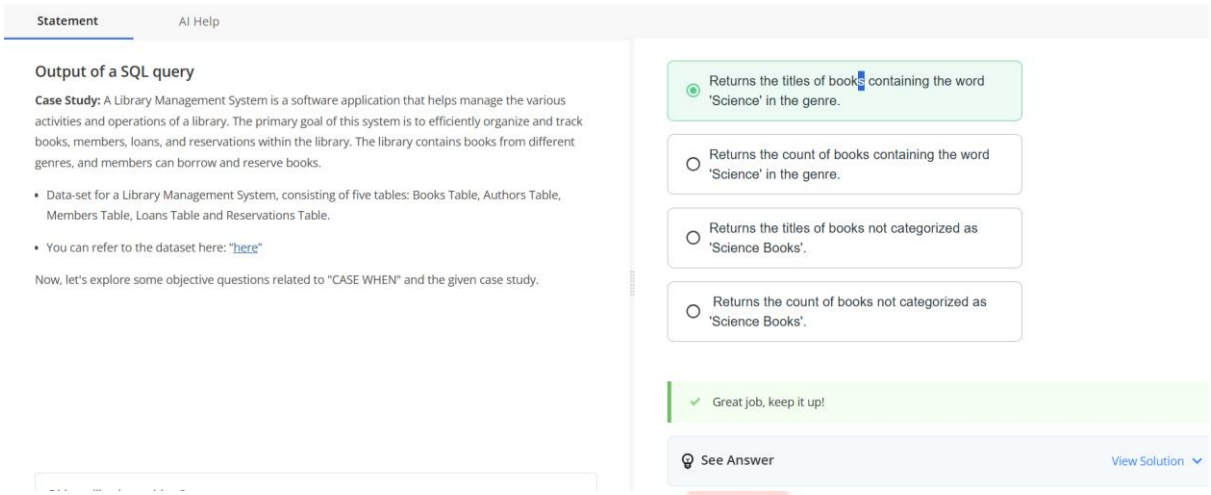


OUTPUT OF A SQL QUERY

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

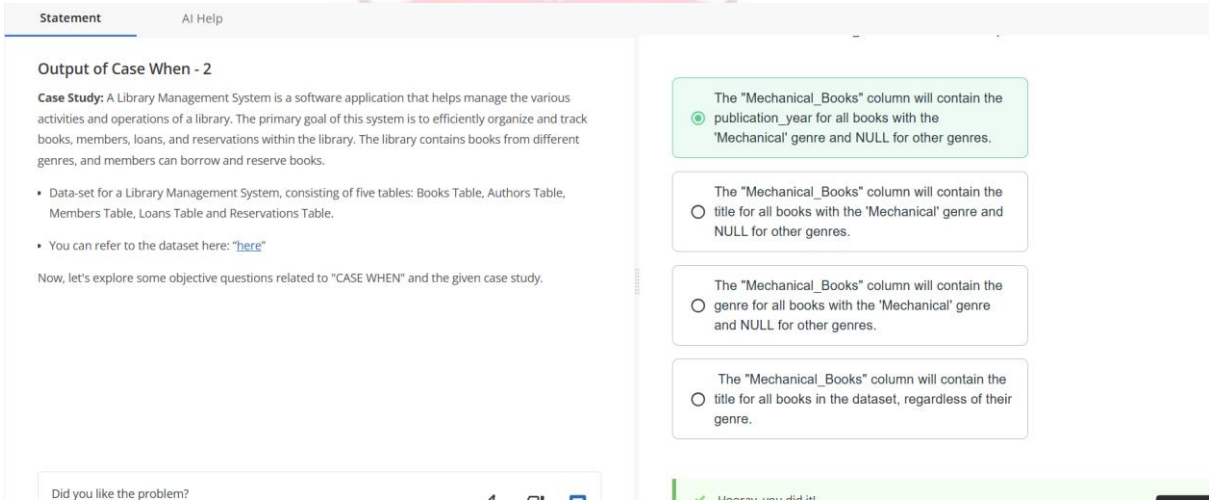


OUTPUT OF CASE WHEN - 2

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

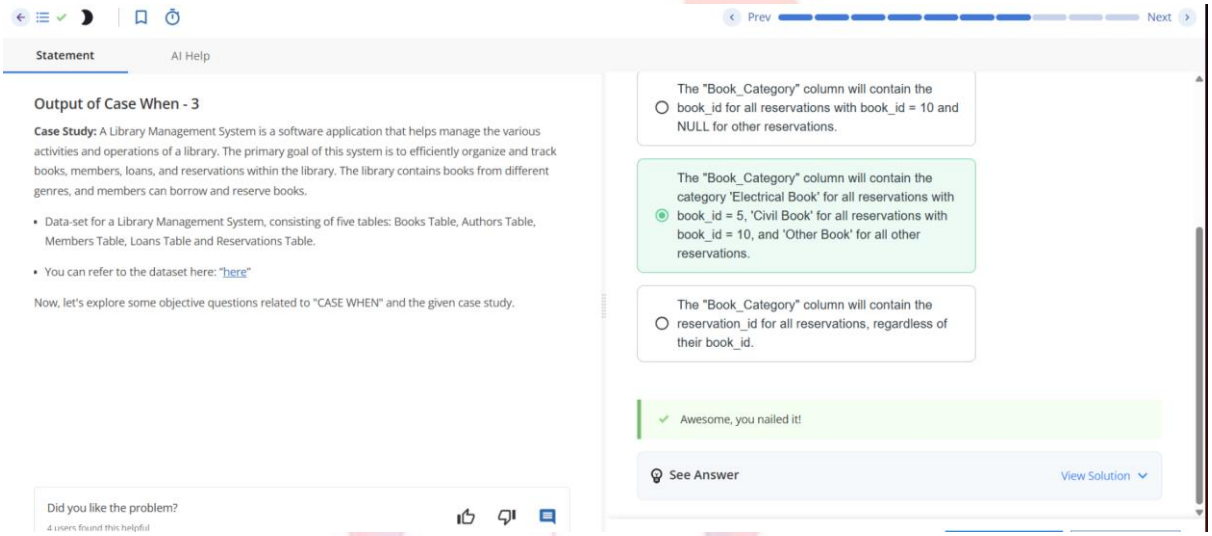


OUTPUT OF CASE WHEN - 3

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

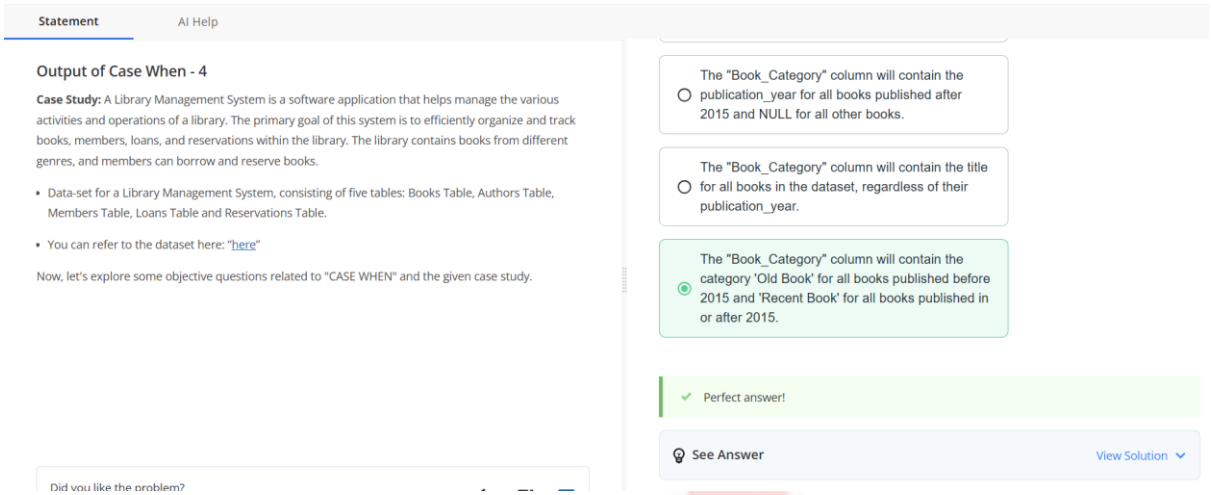


OUTPUT OF CASE WHEN - 4

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

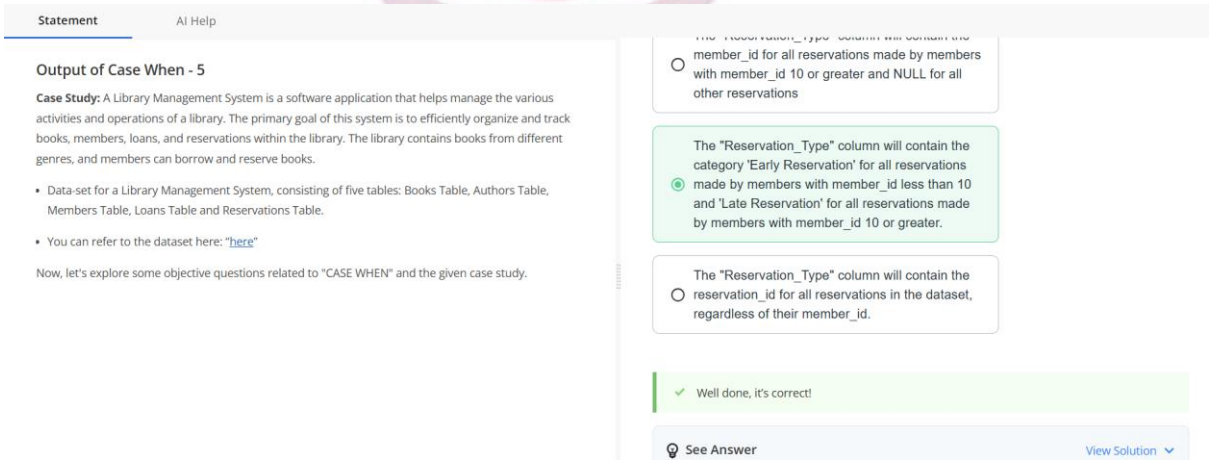


OUTPUT OF CASE WHEN - 5

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.

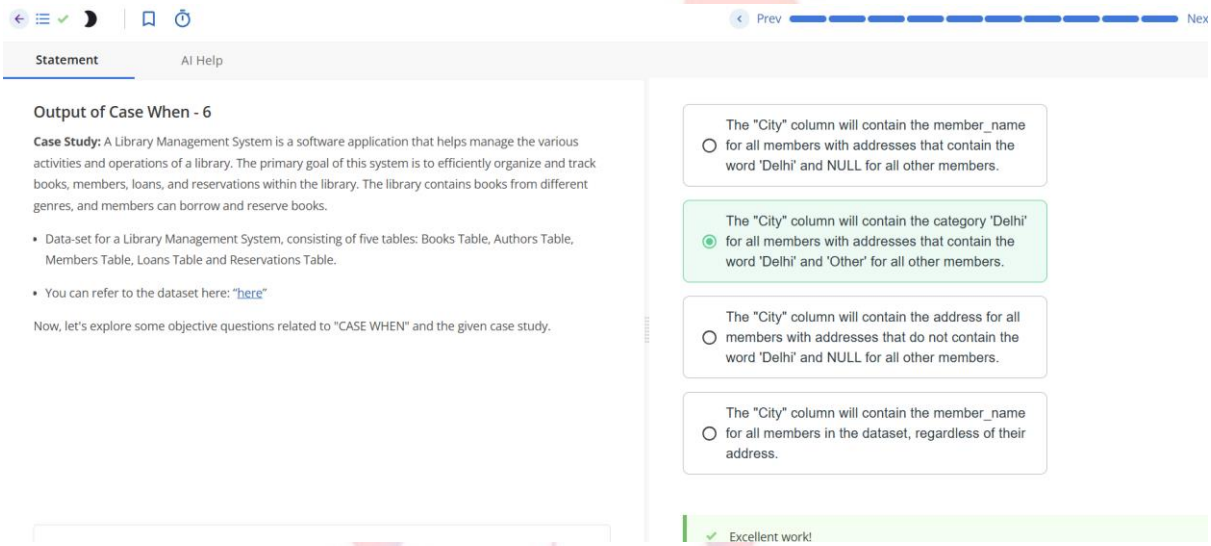


OUTPUT OF CASE WHEN - 6

Case Study: A Library Management System is a software application that helps manage the various activities and operations of a library. The primary goal of this system is to efficiently organize and track books, members, loans, and reservations within the library. The library contains books from different genres, and members can borrow and reserve books.

- Data-set for a Library Management System, consisting of five tables: Books Table, Authors Table, Members Table, Loans Table and Reservations Table.
- You can refer to the dataset here: "here"

Now, let's explore some objective questions related to "CASE WHEN" and the given case study.



PRACTICAL PROGRAMS

1. Consider the following tables CABHUB and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

Create CABHUB Table

CREATE TABLE CABHUB(Vcode NUMBER(3),VehicleName VARCHAR2(20),Make VARCHAR2(20),Color VARCHAR2(15),Capacity NUMBER(2),Charges NUMBER(5));

Results Explain Describe Saved SQL History

```
Table created.
```

0.28 seconds

INSERT INTO CABHUB VALUES (100, 'Innova', 'Toyota', 'WHITE', 5, 15);

INSERT INTO CABHUB VALUES (102, 'SX4', 'Suzuki', 'BLUE', 4, 14);

INSERT INTO CABHUB VALUES (104, 'C-Class', 'Mercedes', 'RED', 4, 35);

INSERT INTO CABHUB VALUES (105, 'A-Star', 'Suzuki', 'WHITE', 3, 14);

INSERT INTO CABHUB VALUES (108, 'Indigo', 'Tata', 'SILVER', 3, 12);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.05 seconds

Create CUSTOMER Table

CREATE TABLE CUSTOMER(Code NUMBER(2),CName VARCHAR2(30),VCode NUMBER(3));

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

INSERT INTO CUSTOMER VALUES (1, 'HemantSaul', 101);

INSERT INTO CUSTOMER VALUES (2, 'Raj Lal', 108);

INSERT INTO CUSTOMER VALUES (3, 'Feroza Shah', 105);

INSERT INTO CUSTOMER VALUES (4, 'Ketan Dhal', 104);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.05 seconds
```

(i) To display the names of all the white colored vehicles.

SELECT VehicleName FROM CABHUB WHERE Color = 'WHITE'

Results Explain Describe Saved SQL History

VEHICLENAME
Innova
A-Star

2 rows returned in 0.04 seconds

(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

SELECT VehicleName, Make, Capacity FROM CABHUB ORDER BY Capacity ASC;

Results Explain Describe Saved SQL History

VEHICLENAME	MAKE	CAPACITY
Indigo	Tata	3
A-Star	Suzuki	3
SX4	Suzuki	4
C-Class	Mercedes	4
Innova	Toyota	5

5 rows returned in 0.00 seconds

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

SELECT MAX(Charges) FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.00 seconds

(iv) To display the customer and the corresponding name of the vehicle hired by them.

SELECT CUSTOMER.CName, CABHUB.VehicleName FROM CUSTOMER INNER JOIN CABHUB ON CUSTOMER.VCode =CABHUB.VCode;

Results Explain Describe Saved SQL History

CNAME	VEHICLENAME
Ketan Dhal	C-Class
Feroza Shah	A-Star
Raj Lal	Indigo

3 rows returned in 0.05 seconds

(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds

(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds

(iii) SELECT COUNT(*), Make FROM CABHUB;

Results Explain Describe Saved SQL History

MAKE	COUNT(*)
Suzuki	2
Mercedes	1
Toyota	1
Tata	1

4 rows returned in 0.02 seconds

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

Results Explain Describe Saved SQL History

VEHICLENAME
SX4
C-Class

2 rows returned in 0.02 seconds

2. Given the following CO2_EMISSION relation, write SQL commands for questions (a) to (f).

Create CO2_EMISSION table:

CREATE TABLE CO2_EMISSION(VEHICLEID NUMBER(3), COMPANY VARCHAR2(20), MODEL VARCHAR2(20), FUELTYPE VARCHAR2(10), ENGINESIZE NUMBER(3,1),CO2EMISSION NUMBER(5));

Results Explain Describe Saved SQL History

Table created.

0.00 seconds

INSERT INTO CO2_EMISSION VALUES (101, 'Toyota', 'Corolla', 'Petrol', 1.8, 165);

INSERT INTO CO2_EMISSION VALUES (102, 'Hyundai', 'Creta', 'Diesel', 1.5, 145);

INSERT INTO CO2_EMISSION VALUES (103, 'Honda', 'City', 'Petrol', 1.5, 152);

INSERT INTO CO2_EMISSION VALUES (104, 'Tata', 'Nexon', 'Diesel', 1.5, 138);

INSERT INTO CO2_EMISSION VALUES (105, 'Maruti', 'Swift', 'Petrol', 1.2, 125)

INSERT INTO CO2_EMISSION VALUES (106, 'Kia', 'Seltos', 'Diesel', 1.5, 150);

INSERT INTO CO2_EMISSION VALUES (107, 'Mahindra', 'XUV700', 'Diesel', 2.2, 185);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.05 seconds
```

(a) Display all the details of vehicles that use **Petrol** as the fuel type.

SELECT * FROM CO2_EMISSION WHERE FUELTYPE = 'Petrol';

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
105	Maruti	Swift	Petrol	1.2	125

3 rows returned in 0.05 seconds CSV Export

(b) Display the **Company** and **Model** of vehicles whose **EngineSize** is greater than **1.5 litres**.

SELECT COMPANY, MODEL FROM CO2_EMISSION WHERE ENGINESIZE > 1.5

Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.00 seconds CSV Export

(c) Display the **Company**, **Model**, and **CO2Emission** in **ascending order of CO2Emission**.

SELECT COMPANY, MODEL, CO2EMISSIONFROM CO2_EMISSION ORDER BY CO2EMISSION ASC;

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	125
Tata	Nexon	138
Hyundai	Creta	145
Kia	Seltos	150
Honda	City	152
Toyota	Corolla	165
Mahindra	XUV700	185

7 rows returned in 0.00 seconds CSV Export

(d) Display the details of vehicles whose **CO2Emission** is greater than **150 g/km**.

SELECT * FROM CO2_EMISSION WHERE CO2EMISSION > 150;

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
107	Mahindra	XUV700	Diesel	2.2	185

3 rows returned in 0.00 seconds CSV Export

(e) Count the number of vehicles that use **Diesel** as the fuel type.

SELECT COUNT(*) FROM CO2_EMISSION WHERE FUELTYPE = 'Diesel';

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

(f) Insert the following record into the **CO2_EMISSION** table.

108, 'MG', 'Hector', 'Petrol', 1.5, 160

INSERT INTO CO2_EMISSION VALUES (108, 'MG', 'Hector', 'Petrol', 1.5, 160);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.01 seconds
```

SELECT * FROM CO2_EMISSION;

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
102	Hyundai	Creta	Diesel	1.5	145
103	Honda	City	Petrol	1.5	152
104	Tata	Nexon	Diesel	1.5	138
105	Maruti	Swift	Petrol	1.2	125
106	Kia	Seltos	Diesel	1.5	150
107	Mahindra	XUV700	Diesel	2.2	185
108	MG	Hector	Petrol	1.5	160

8 rows returned in 0.00 seconds CSV Export

(g) Write the output of the following SQL commands.

(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;
SELECT MAX(CO2EMISSION) FROM CO2_EMISSION;

Results Explain Describe Saved SQL History

MAX(CO2EMISSION)
185

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;
SELECT MIN(ENGINESIZE) FROM CO2_EMISSION;

Results Explain Describe Saved SQL History

MIN(ENGINESIZE)
1.2

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';
SELECT AVG(CO2EMISSION) FROM CO2_EMISSIONWHERE FUELTYPE = 'Petrol';

Results Explain Describe Saved SQL History

AVG(CO2EMISSION)
150.5

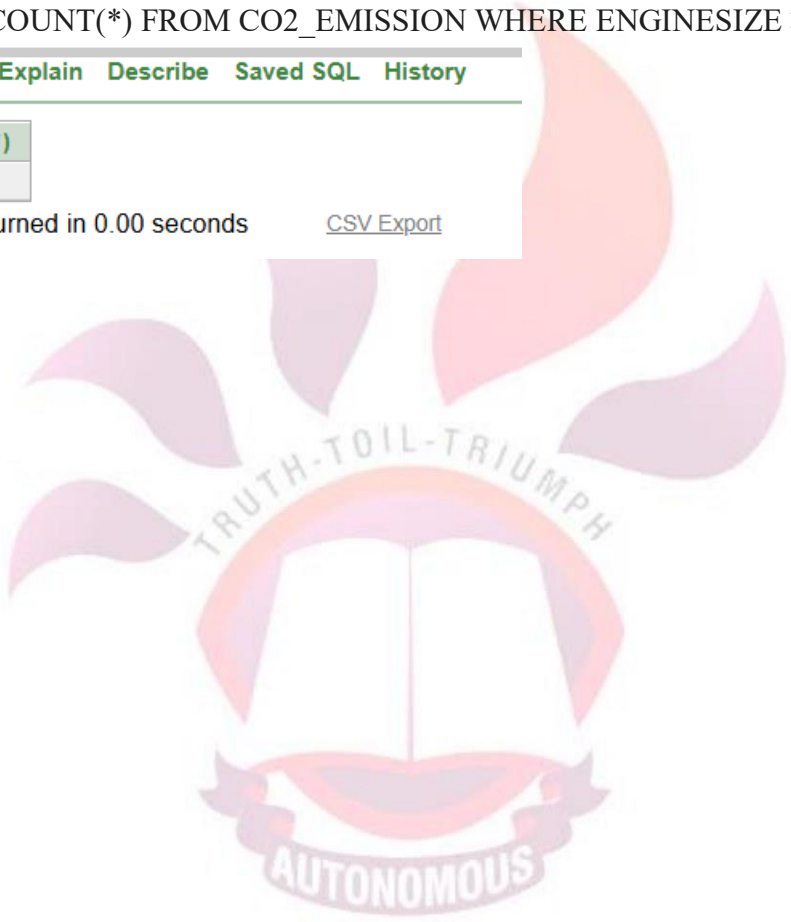
1 rows returned in 0.00 seconds CSV Export

(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;
SELECT COUNT(*) FROM CO2_EMISSION WHERE ENGINESIZE >= 1.5;

Results Explain Describe Saved SQL History

COUNT(*)
7

1 rows returned in 0.00 seconds CSV Export



3. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables APPLICANTS and COURSES.

Create APPLICANTS Table:

CREATE TABLE APPLICANTS(NO NUMBER(4),NAME VARCHAR2(20),FEE NUMBER(6),GENDER CHAR(1),C_ID VARCHAR2(3),JOINYEAR NUMBER(4))

Results Explain Describe Saved SQL History

Table created.

0.00 seconds

INSERT INTO APPLICANTS VALUES(1012,'Amandeep',30000,'M','A01',2012);

INSERT INTO APPLICANTS VALUES(1102,'Avisha',25000,'F','A02',2009);

INSERT INTO APPLICANTS VALUES(1103,'Ekant',30000,'M','A02',2011);

INSERT INTO APPLICANTS VALUES (1049,'Arun',30000,'M','A03',2009);

INSERT INTO APPLICANTS VALUES(1025,'Amber',40000,'M','A02',2011);

INSERT INTO APPLICANTS VALUES(1106,'Ela',40000,'F','A05',2010);

INSERT INTO APPLICANTS VALUES(1107,'Nikita',35000,'F','A03',2012);

INSERT INTO APPLICANTS VALUES(1108,'Arleena',30000,'F','A03',2012);

INSERT INTO APPLICANTS VALUES(2109,'Shakti',35000,'M','A04',2011);

INSERT INTO APPLICANTS VALUES(1101,'Kirat',25000,'M','A01',2012);

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.01 seconds

Create COURSES Table:

CREATE TABLE COURSES(C_ID VARCHAR2(3),COURSE VARCHAR2(30));



Table created.

0.00 seconds

INSERT INTO COURSES VALUES ('A01','Fashion Design');

INSERT INTO COURSES VALUES ('A02','Networking');

INSERT INTO COURSES VALUES ('A03','Hotel Management');

INSERT INTO COURSES VALUES ('A04','Event Management');

INSERT INTO COURSES VALUES ('A05','Office Management');

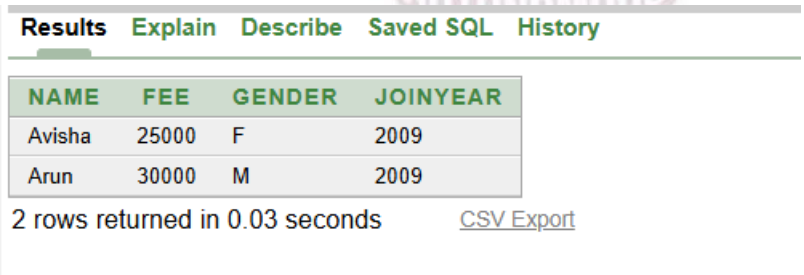


1 row(s) inserted.

0.01 seconds

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

SELECT NAME, FEE, GENDER, JOINYEAR FROM APPLICANTS WHERE JOINYEAR < 2010;



(b) To display the names of applicants, who are paying fee more than 30000.

SELECT NAME FROM APPLICANTS WHERE FEE > 30000;

Results Explain Describe Saved SQL History

NAME
Amber
Ela
Nikita
Shakti

4 rows returned in 0.00 seconds CSV Export

(c) To display name of all applicants in ascending order of their joinyear.

SELECT NAME FROM APPLICANTS ORDER BY JOINYEAR ASC;

Results Explain Describe Saved SQL History

NAME
Arun
Avisha
Ela
Ekant
Amber
Shakti
Nikita
Arleena
Amandeep
Kirat

10 rows returned in 0.00 seconds CSV Export

(d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.

SELECT JOINYEAR, COUNT(*) FROM APPLICANTS GROUP BY JOINYEAR;

Results Explain Describe Saved SQL History

JOINYEAR	COUNT(*)
2009	2
2010	1
2011	3
2012	4

4 rows returned in 0.05 seconds CSV Export

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

SELECT C_ID, COUNT(*) FROM APPLICANTS GROUP BY C_ID;

Results Explain Describe Saved SQL History

C_ID	COUNT(*)
A03	3
A04	1
A02	3
A05	1
A01	2

5 rows returned in 0.00 seconds CSV Export

(f) To display the applicant's name with their respective course's name from the tables APPLICANTS and COURSES.

SELECT APPLICANTS.NAME, COURSES.COURSE FROM APPLICANTS JOIN COURSES ON APPLICANTS.C_ID = COURSES.C_ID;

Results Explain Describe Saved SQL History

C_ID	COUNT(*)
A03	3
A04	1
A02	3
A05	1
A01	2

5 rows returned in 0.00 seconds CSV Export

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';

SELECT NAME, JOINYEAR FROM APPLICANTS WHERE GENDER='F' AND C_ID='A02';

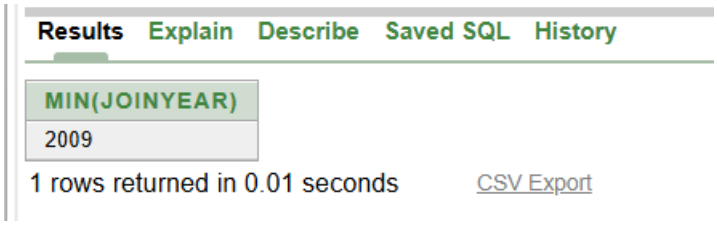
Results Explain Describe Saved SQL History

NAME	JOINYEAR
Avisha	2009

1 rows returned in 0.02 seconds CSV Export

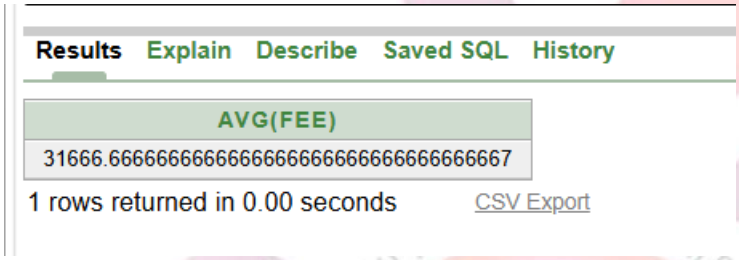
(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= ‘M’;

SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE GENDER='M';



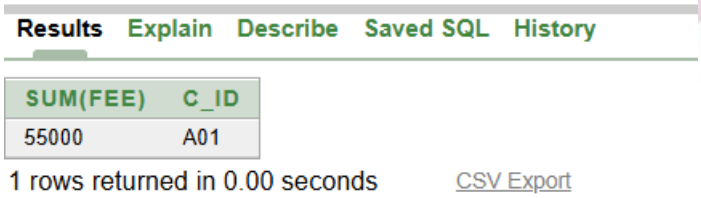
(g3) SELECT AVG(FEE) FROM APPLICANTS WHERE C_ID= ‘A01’ OR C_ID= ‘A05’;

SELECT AVG(FEE) FROM APPLICANTS WHERE C_ID='A01' OR C_ID='A05';



(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

SELECT SUM(FEE), C_ID FROM APPLICANTS GROUP BY C_ID HAVING COUNT(*) =2;



4. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables **PRODUCTS AND SUPPLIERS**

Create PRODUCTS Table:

CREATE TABLE PRODUCTS(PID NUMBER(3), PNAME VARCHAR2(30),QTY NUMBER(4), PRICE NUMBER(6),COMPANY VARCHAR2(30),SUPCODE VARCHAR2(3));



Table created.

0.00 seconds

INSERTINTO PRODUCTS VALUES(101,'DIGITALCAMERA14X',120,12000,'RENIX','S01');

INSERT INTO PRODUCTS VALUES(102,'DIGITAL PAD 11i',100,22000,'DIGI POP','S02');

INSERT INTO PRODUCTS VALUES(104,'PENDRIVE’16GB',500,1100,'STOREKING','S01');

INSERT INTO PRODUCTS VALUES(106,'LED SCREEN',70,28000,'DEXPERTS','S02');

INSERT INTO PRODUCTS VALUES(105,'CAR GPS SYSSTEM',60,12000,'MOVEON','S03');



1 row(s) inserted.

0.01 seconds

Create SUPPLIERS table:

CREATE TABLE SUPPLIERS1(SUPCODE VARCHAR2(3),SNAME VARCHAR2(30),CITY VARCHAR2(20));

Results Explain Describe Saved SQL History

Table created.

0.00 seconds

INSERT INTO SUPPLIERS1 VALUES('S01','GET ALL INC','KOLKATA');

INSERT INTO SUPPLIERS1 VALUES('S03','EASY MARKET CORP','DELHI');

INSERT INTO SUPPLIERS1 VALUES('S02','DIGI BUSY GROUP','CHENNAI');

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.01 seconds

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

SELECT * FROM PRODUCTS ORDER BY PNAME ASC;

Results Explain Describe Saved SQL History

PID	PNAME	QTY	PRICE	COMPANY	SUPCODE
105	CAR GPS SYSSTEM	60	12000	MOVEON	S03
101	DIGITAL CAMERA14X	120	12000	RENIX	S01
102	DIGITAL PAD 11i	100	22000	DIGI POP	S02
106	LED SCREEN	70	28000	DEXPERTS	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01

5 rows returned in 0.00 seconds CSV Export

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

SELECT PNAME, PRICE FROM PRODUCTS WHERE PRICE BETWEEN 10000 AND 15000;

Results Explain Describe Saved SQL History

PNAME	PRICE
DIGITAL CAMERA14X	12000
CAR GPS SYSSTEM	12000

2 rows returned in 0.02 seconds CSV Export

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be

S01 2

S02 2

S03 1

SELECT SUPCODE, COUNT(*) FROM PRODUCTS GROUP BY SUPCODE;

Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	2
S03	1

3 rows returned in 0.01 seconds CSV Export

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

SELECT PRICE, PNAME, QTY FROM PRODUCTS WHERE QTY > 100;

Results Explain Describe Saved SQL History

PRICE	PNAME	QTY
12000	DIGITAL CAMERA14X	120
1100	PEN DRIVE 16 GB	500

2 rows returned in 0.01 seconds CSV Export

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

SELECT SNAME FROM SUPPLIERS1 WHERE CITY IN ('DELHI','CHENNAI');

Results Explain Describe Saved SQL History

SNAME
EASY MARKET CORP
DIGI BUSY GROUP

2 rows returned in 0.02 seconds CSV Export

(f) To display the name of the companies and the name of the products in descending order of company names.

SELECT COMPANY, PNAME FROM PRODUCTS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	PNAME
STOREKING	PEN DRIVE 16 GB
RENIX	DIGITAL CAMERA14X
MOVEON	CAR GPS SYSSTEM
DIGI POP	DIGITAL PAD 11i
DEXPERTS	LED SCREEN

5 rows returned in 0.01 seconds CSV Export

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

SELECT DISTINCT SUPCODE FROM PRODUCTS;

Results Explain Describe Saved SQL History

SUPCODE
S01
S02
S03

3 rows returned in 0.02 seconds CSV Export

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

SELECT MAX(PRICE), MIN(PRICE) FROM PRODUCTS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
28000	1100

1 rows returned in 0.00 seconds CSV Export

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

SELECT PRICE * QTY AS AMOUNT FROM PRODUCTS WHERE PID = 104;

Results Explain Describe Saved SQL History

AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P.SUPCODE = S.SUPCODE AND QTY > 100;

Results Explain Describe Saved SQL History

PNAME	SNAME
PEN DRIVE 16 GB	GET ALL INC
DIGITAL CAMERA14X	GET ALL INC

2 rows returned in 0.02 seconds CSV Export

5. Answer the questions (a) and (b) on the basis of the following tables SHOPPE and ACCESSORIES

Create SHOPPE Table:

CREATE TABLE SHOPPE(ID VARCHAR(5), SName VARCHAR(30), Area VARCHAR(20));

Results Explain Describ

```
Table created.
```

0.02 seconds

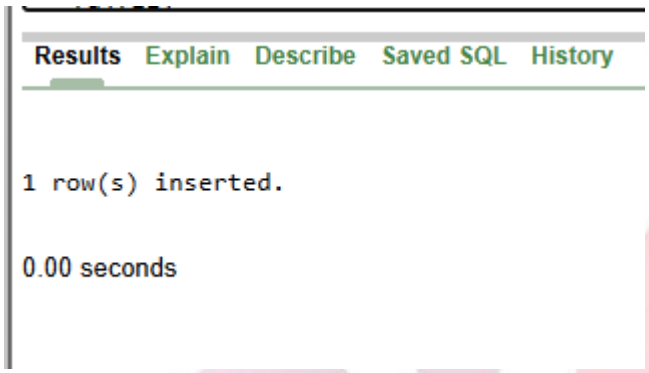
INSERT INTO SHOPPE VALUES ('S0001','ABC Computeronics','CP');

INSERT INTO SHOPPE VALUES ('S0002','All Infotech Media','GK II');

INSERT INTO SHOPPE VALUES ('S0003','Tech Shoppe','CP');

INSERT INTO SHOPPE VALUES ('S0004','Greeks Techno Soft','Nehru Place');

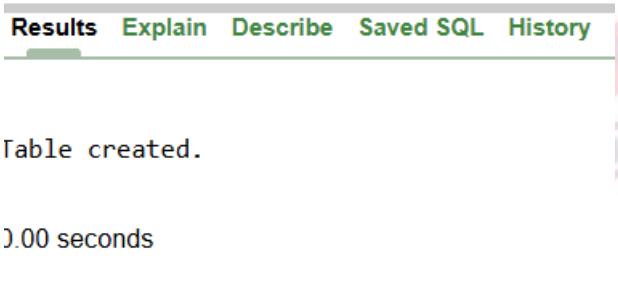
INSERT INTO SHOPPE VALUES ('S0005','Hitech Tech Store','Nehru Place');



Create ACCESSORIES Table:

CREATE TABLE ACCESSORIES(No VARCHAR(5), Name VARCHAR(30), Price INT,

ID VARCHAR(5));



INSERT INTO ACCESSORIES VALUES ('A01','Mother Board',12000,'S01');

INSERT INTO ACCESSORIES VALUES ('A02','Hard Disk',5000,'S01');

INSERT INTO ACCESSORIES VALUES ('A03','Keyboard',500,'S02');

INSERT INTO ACCESSORIES VALUES ('A04','Mouse',300,'S01');

INSERT INTO ACCESSORIES VALUES ('A05','Mother Board',13000,'S02');

INSERT INTO ACCESSORIES VALUES ('A06','Keyboard',400,'S03');

INSERT INTO ACCESSORIES VALUES ('A07','LCD',6000,'S04');

INSERT INTO ACCESSORIES VALUES ('T08','LCD',5500,'S05');

INSERT INTO ACCESSORIES VALUES ('T09','Mouse',350,'S05');

INSERT INTO ACCESSORIES VALUES ('T10','Hard Disk',4500,'S03');

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.01 seconds

(a) Write the SQL queries:

(i) To display Name and Price of all the accessories in ascending order of their Price.

SELECT Name, Price FROM ACCESSORIES ORDER BY Price ASC;

Results Explain Describe Saved SQL History

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.02 seconds CSV Export

(ii) To display Id and SName of all Shoppe in Nehru Place.

SELECT ID, Sname FROM SHOPPE WHERE Area='Nehru Place';

Results Explain Describe Saved SQL History

ID	SNAME
S0004	Greeks Techno Soft
S0005	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

(iii) To display Minimum and Maximum Price of each Name of accessories.

SELECT Name, MIN(Price), MAX(Price) FROM ACCESSORIES GROUP BY Name;

Results Explain Describe Saved SQL History

NAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.02 seconds CSV Export

(iv) To display Name, Price of all accessories and their respective SName where they are available.

SELECT A.NAME, A.PRICE, S.SNAME FROM ACCESSORIES A, SHOPPE SWHERE A.ID = S.ID;

Results Explain Describe Saved SQL History

NAME	PRICE	SNAME
Keyboard	450	ABC Electronics
Headphone	1500	Digital World
Webcam	800	Modern Accessories
Mouse	600	Modern Accessories

4 rows returned in 0.01 seconds SQL Exit

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

SELECT DISTINCT NAME FROM ACCESSORIES WHERE PRICE >= 500;

Results Explain Describe Saved SQL History

NAME
Mother Board
LCD
Keyboard
Hard Disk

4 rows returned in 0.00 seconds CSV Export

(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;

SELECT AREA, COUNT(*) FROM SHOPPE GROUP BY AREA;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

SELECT COUNT(DISTINCT AREA) FROM SHOPPE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN (‘S02, ‘S03’);

SELECT NAME, PRICE * 0.05 AS DISCOUNT FROM ACCESSORIESWHERE ID IN ('S02', 'S03')

Results Explain Describe Saved SQL History

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

6. Write SQL commands for (a) to (f) and Write the outputs for (g) on the basis of table HOSPITAL

Create HOSPITAL Table:

CREATE TABLE HOSPITAL(No INT, Name VARCHAR(20), Age INT, Department VARCHAR(30),DateOfAdm DATE,Charges INT,Gender CHAR(1));

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

INSERT INTO HOSPITAL VALUES(1,'Arpit',62,'Surgery','1998-01-21',300,'M');

INSERT INTO HOSPITAL VALUES(2,'Zarina',22,'ENT','1997-12-12',250,'F');

INSERT INTO HOSPITAL VALUES(3,'Kareem',32,'Orthopedic','1998-02-19',200,'M');

INSERT INTO HOSPITAL VALUES(4,'Arun',12,'Surgery','1998-01-11',300,'M');

INSERT INTO HOSPITAL VALUES(5,'Zubin',30,'ENT','1998-02-24',250,'M');

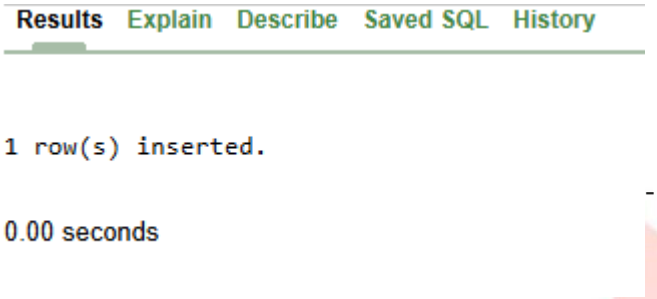
INSERT INTO HOSPITAL VALUES(6,'Ketaki',16,'ENT','1998-01-12',250,'M');

INSERT INTO HOSPITAL VALUES(7,'Ankita',29,'Cardiology','1998-02-20',800,'F');

INSERT INTO HOSPITAL VALUES(8,'Zareen',45,'Gynecology','1998-02-22',300,'F');

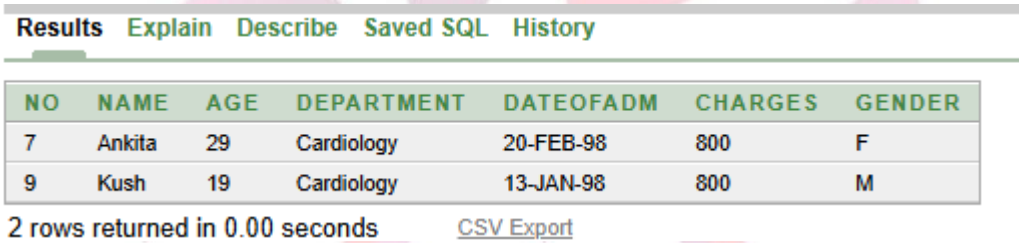
INSERT INTO HOSPITAL VALUES(9,'Kush',19,'Cardiology','1998-01-13',800,'M');

INSERT INTO HOSPITAL VALUES(10,'Shilpa',23,'Nuclear Medicine','1998-02-21',400,'F');



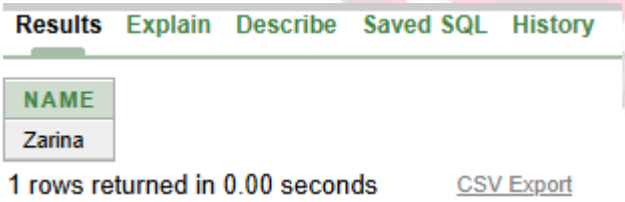
(a) To select all the information of patients of cardiology department.

SELECT * FROM HOSPITAL WHERE Department='Cardiology';



(b) To list the names of female patients who are in ENT department.

SELECT Name FROM HOSPITAL WHERE Gender='F' AND Department='ENT';



(c) To list name of all patients with their date of admission in ascending order.

SELECT Name, DateOfAdm FROM HOSPITAL ORDER BY DateOfAdm ASC;

Results Explain Describe Saved SQL History

NAME	DATEOFADM
Zarina	12-DEC-97
Arun	11-JAN-98
Ketaki	12-JAN-98
Kush	13-JAN-98
Arpit	21-JAN-98
Kareem	19-FEB-98
Ankita	20-FEB-98
Shilpa	21-FEB-98
Zareen	22-FEB-98
Zubin	24-FEB-98

10 rows returned in 0.00 seconds CSV Export

(d) To display Patient's Name, Charges, Age for only female patients.

SELECT Name, Charges, Age FROM HOSPITAL WHERE Gender='F';

Results Explain Describe Saved SQL History

NAME	CHARGES	AGE
Zarina	250	22
Ankita	800	29
Zareen	300	45
Shilpa	400	23

4 rows returned in 0.00 seconds CSV Export

(e) To count the number of patients with Age<30.

SELECT COUNT(*) FROM HOSPITAL WHERE Age < 30;

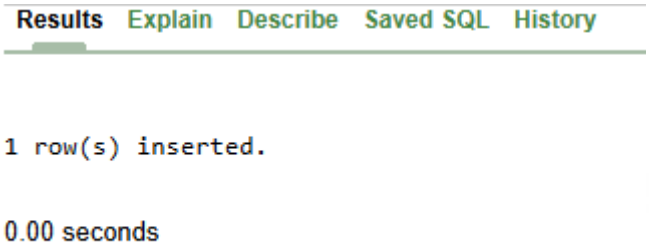
Results Explain Describe Saved SQL History

COUNT(*)
6

1 rows returned in 0.00 seconds CSV Export

(f) To inset in a new row in the HOSPITAL table with the following data: 11, “Aftab”, 24, “Surgery”, {25/02/98}, 300, “M”

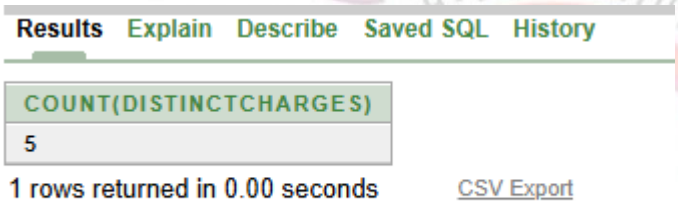
INSERT INTO HOSPITALVALUES(11,'Aftab',24,'Surgery','25-FEB-98',300,'M');



(g) Give the output of following SQL statements:

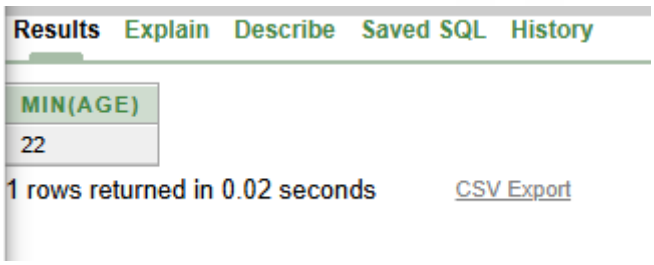
(i) Select COUNT (DISTINCT charges) from HOSPITAL;

SELECT COUNT(DISTINCT Charges) FROM HOSPITAL;



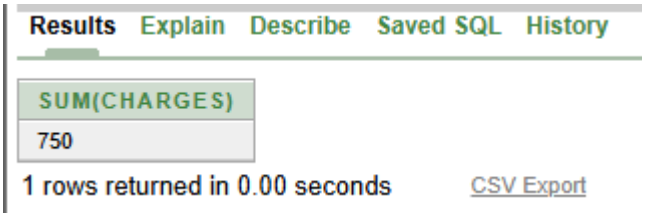
(ii) Select MIN (Age) from HOSPITAL where Gender = “F”

SELECT MIN(Age) FROM HOSPITAL WHERE Gender='F';



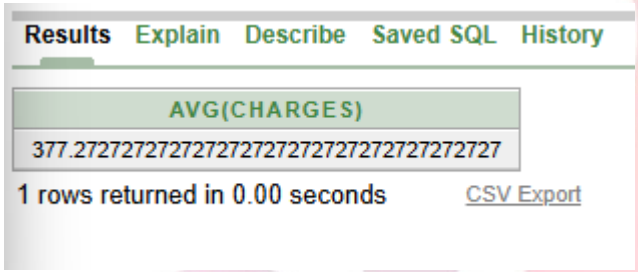
(iii) Select SUM (Charges) from HOSPITAL where Department = “ENT”

SELECT SUM(Charges) FROM HOSPITAL WHERE Department='ENT';



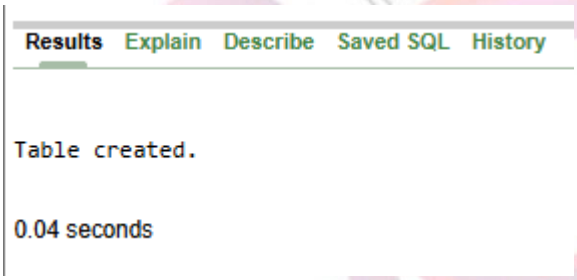
(iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

SELECT AVG(Charges) FROM HOSPITAL WHERE DateOfAdm < '12-AUG-98';



7. Answer the questions (a) and (b) on the basis of the following tables **STORE** and **ITEM**

CREATE TABLE STORE(SNo VARCHAR(5), SName VARCHAR(30), Area VARCHAR(20));



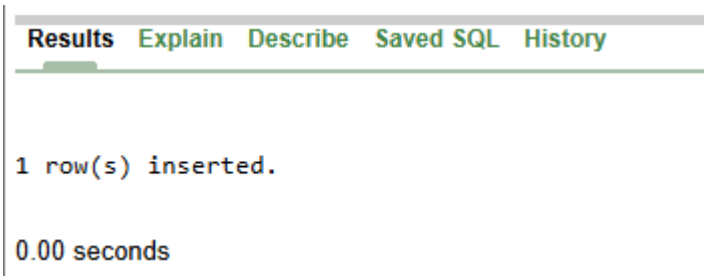
INSERT INTO STORE VALUES ('S01','ABC Computertronics','GK II');

INSERT INTO STORE VALUES ('S02','All InfoTech Media','CP');

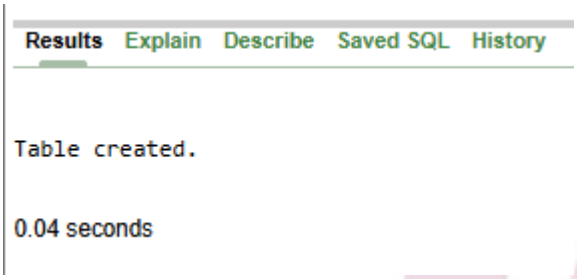
INSERT INTO STORE VALUES ('S03','Tech Shoppe','Nehru Place');

INSERT INTO STORE VALUES ('S04','Greeks Techno Soft','Nehru Place');

INSERT INTO STORE VALUES ('S05','Hitech Tech Store','CP');



CREATE TABLE ITEM(INo VARCHAR(5), IName VARCHAR(30), Price INT, SNo VARCHAR(5));



INSERT INTO ITEM VALUES ('T01','Mother Board',12000,'S01');

INSERT INTO ITEM VALUES ('T02','Hard Disk',5000,'S01');

INSERT INTO ITEM VALUES ('T03','Keyboard',500,'S02');

INSERT INTO ITEM VALUES ('T04','Mouse',300,'S01');

INSERT INTO ITEM VALUES ('T05','Mother Board',13000,'S02');

INSERT INTO ITEM VALUES ('T06','Keyboard',400,'S03');

INSERT INTO ITEM VALUES ('T07','LCD',6000,'S04');

INSERT INTO ITEM VALUES ('T08','LCD',5500,'S05');

INSERT INTO ITEM VALUES ('T09','Mouse',350,'S05');

INSERT INTO ITEM VALUES ('T10','Hard Disk',4500,'S03');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

SELECT IName, Price FROM ITEM1 ORDER BY Price ASC;

Results Explain Describe Saved SQL History

INAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.00 seconds CSV Export

(ii) To display SNo and SName of all store location in CP.

SELECT SNo, Sname FROM STORES WHERE Area='CP';

Results Explain Describe Saved SQL History

SNO	SNAME
S02	All InfoTech Media
S05	Hitech Tech Store

2 rows returned in 0.02 seconds CSV Export

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

SELECT Iname, MIN(Price), MAX(Price) FROM ITEM1 GROUP BY Iname;

Results Explain Describe Saved SQL Histor

INAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

(iv) To display IName, Price of all items and their respective SName where they are available.

SELECT IName, Price, Sname FROM ITEM1 I, STORES S WHERE I.Sno=S.Sno;

Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Mother Board	12000	ABC Computertronics
Hard Disk	5000	ABC Computertronics
Keyboard	500	All InfoTech Media
Mouse	300	ABC Computertronics
Mother Board	13000	All InfoTech Media
Keyboard	400	Tech Shoppe
LCD	6000	Greeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.00 seconds CSV Export

(b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

SELECT DISTINCT Iname FROM ITEM1 WHERE Price>=5000;

Results Explain Describe Saved SQL History

INAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.02 seconds CSV Export

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;

SELECT Area, COUNT(*) FROM STORES GROUP BY Area

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (DISTINCT Area) FROM STORE:

SELECT COUNT(DISTINCT Area) FROM STORES;

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN (‘S02’, ‘S03’);

SELECT IName, Price*0.05 DISCOUNT FROM ITEM1 WHERE SNo IN ('S02','S03');

Results Explain Describe Saved SQL History

INAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

8. Write SQL commands for (a) to (f) and write output for (g) on the basis of Teacher relation given below:

CREATE TABLE TEACHER(No INT,Name VARCHAR(20), Age INT,Department VARCHAR(20), DateOfJoin DATE, Salary INT, Gender CHAR(1));

Results Explain Describe Saved SQL History

```
Table created.

0.05 seconds
```

INSERT INTO TEACHER VALUES(1,'Jugal',34,'Computer','10-JAN-97',12000,'M');

INSERT INTO TEACHER VALUES(2,'Sharmila',31,'History','24-MAR-98',20000,'F');

INSERT INTO TEACHER VALUES(3,'Sandeep',32,'Maths','12-DEC-96',30000,'M');

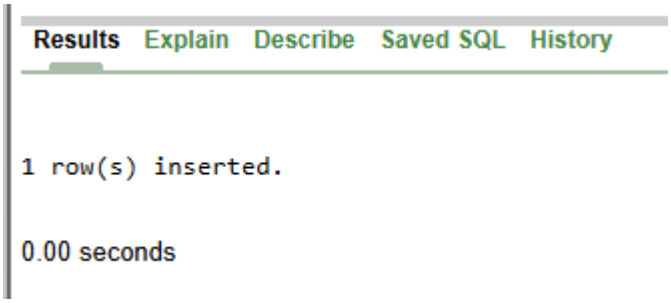
INSERT INTO TEACHER VALUES(4,'Sangeeta',35,'History','01-JUL-99',40000,'F');

INSERT INTO TEACHER VALUES(5,'Rakesh',42,'Maths','05-SEP-97',25000,'M');

INSERT INTO TEACHER VALUES(6,'Shyam',50,'History','27-JUN-98',30000,'M');

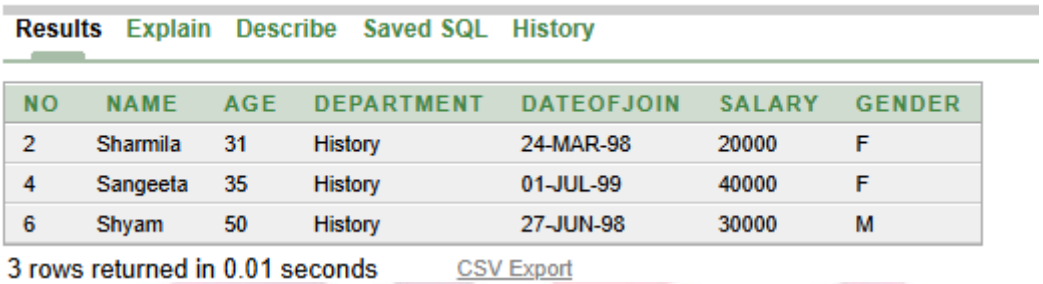
INSERT INTO TEACHER VALUES(7,'Shiv Om',44,'Computer','25-FEB-97',21000,'M');

INSERT INTO TEACHER VALUES(8,'Shalakha',33,'Maths','31-JUL-97',20000,'F');



(a) To show all information about the teacher of history department

SELECT * FROM TEACHER WHERE Department='History';



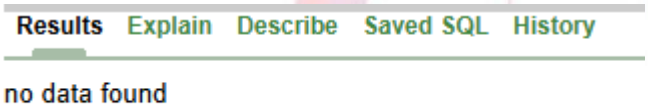
Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFJOIN	SALARY	GENDER
2	Sharmila	31	History	24-MAR-98	20000	F
4	Sangeeta	35	History	01-JUL-99	40000	F
6	Shyam	50	History	27-JUN-98	30000	M

3 rows returned in 0.01 seconds CSV Export

(b) To list the names of female teacher who are in Hindi department

SELECT Name FROM TEACHER WHERE Gender='F' AND Department='Hindi';



(c) To list names of all teachers with their date of joining in ascending order.

SELECT Name, DateOfJoin FROM TEACHE ORDER BY DateOfJoin ASC;

Results Explain Describe Saved SQL History

NAME	DATEOFJOIN
Sandeep	12-DEC-96
Jugal	10-JAN-97
Jugal	10-JAN-97
Shiv Om	25-FEB-97
Shalakha	31-JUL-97
Rakesh	05-SEP-97
Sharmila	24-MAR-98
Shyam	27-JUN-98
Sangeeta	01-JUL-99

9 rows returned in 0.00 seconds CSV Export

(d) To display teacher's Name, Salary, Age for male teacher only

SELECT Name, Salary, Age FROM TEACHER WHERE Gender='M';

Results Explain Describe Saved SQL History

NAME	SALARY	AGE
Jugal	12000	34
Jugal	12000	34
Sandeep	30000	32
Rakesh	25000	42
Shyam	30000	50
Shiv Om	21000	44

6 rows returned in 0.00 seconds CSV Export

(e) To count the number of teachers with Age>23.

SELECT COUNT(*) FROM TEACHER WHERE Age > 23;

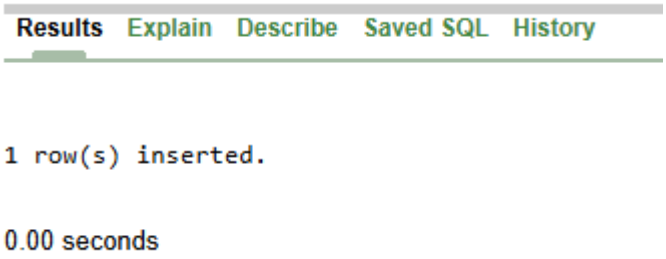
Results Explain Describe Saved SQL History

COUNT(*)
9

1 rows returned in 0.00 seconds CSV Export

(f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"

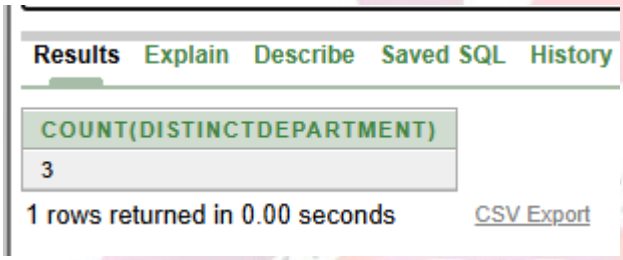
INSERT INTO TEACHER VALUES (9,'Raja',26,'Computer','13-MAY-95',2300,'M');



(g) Give the output of following SQL statements:

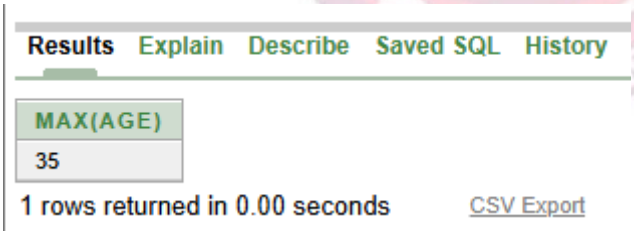
(i) Select COUNT (distinct department) from TEACHER;

SELECT COUNT(DISTINCT Department) FROM TEACHER;



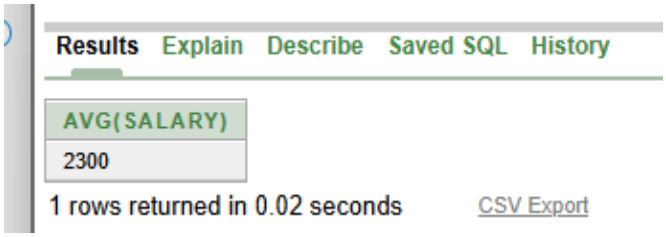
(II) Select MAX (Age) from TEACHER where Gender= “F”

SELECT MAX(Age) FROM TEACHER WHERE Gender='F';



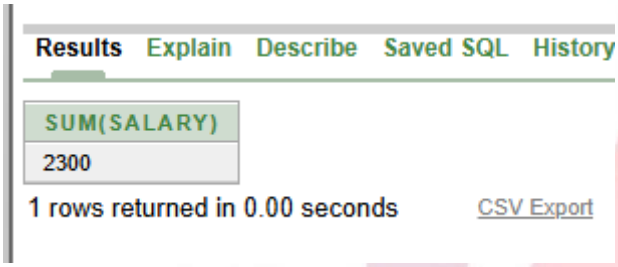
(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};

SELECT AVG(Salary) FROM TEACHER WHERE DateOfJoin < '12-JUL-96';



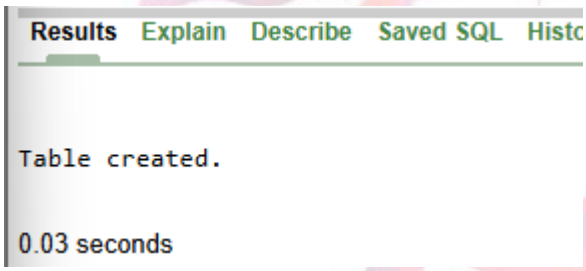
(iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

SELECT SUM(Salary) FROM TEACHER WHERE DateOfJoin < '12-JUL-96';\



9. Given the following tables for a database LIBRARY:

CREATE TABLE BOOKS(Book_Id VARCHAR(5), Book_Name VARCHAR(30), Author_Name VARCHAR(30),Publishers VARCHAR(20)Price INT, Type VARCHAR(20), Qty INT);



INSERT INTO BOOKS VALUES('F0001','TheTears','WilliamHopkins','First Publ.',750,'Fiction',10);

INSERT INTO BOOKS VALUES('F0002','Thunderbolts','AnnaRoberts','First Publ.',700,'Fiction',5);

INSERT INTO BOOKS VALUES('T0001','My First C++','Brian & Brooke','EPB',250,'Text',10);

INSERT INTO BOOKS VALUES('T0002','C++ Brainworks','A. W. Rossaine','TDH',325,'Text',5);

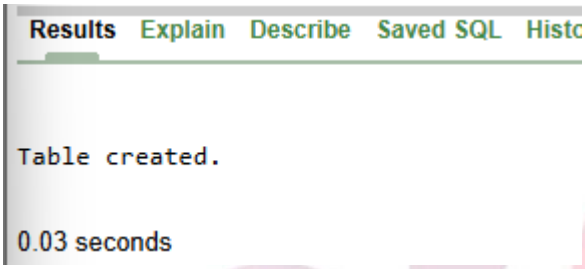
INSERT INTO BOOKS VALUES('C0001','Fast Cook','LataKapoor','EPB',350,'Cookery',8);




```
1 row(s) inserted.

0.00 seconds
```

CREATE TABLE ISSUED(Book_Id VARCHAR(5), Quantity_Issued INT);




```
Table created.

0.03 seconds
```

INSERT INTO ISSUED VALUES ('F0001',3);

INSERT INTO ISSUED VALUES ('T0001',1);

INSERT INTO ISSUED VALUES ('C0001',5);




```
1 row(s) inserted.

0.00 seconds
```

Write SQL queries for (a) to (f):

(a) To show Book name, Author name and Price of books of EPB publishers.

SELECT Book_Name, Author_Name, Price FROM BOOKS WHERE Publishers='EPB';

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME	PRICE
My First C++	Brian & Brooke	250
My First C++	Brian & Brooke	250
Fast Cook	LataKapoor	350

3 rows returned in 0.02 seconds CSV Export

(b) To list the names of the books of Fiction Type.

SELECT Book_Nam FROM BOOKS WHERE Type='Fiction';

Results Explain Describe Saved SQL History

BOOK_NAME
The Tears
Thunderbolts
Thunderbolts

3 rows returned in 0.00 seconds CSV Export

(c) To display the names and price of the books in descending order of their price.

SELECT Book_Name, Price FROM BOOKS ORDER BY Price DESC;

Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
The Tears	750
Thunderbolts	700
Thunderbolts	700
Fast Cook	350
C++ Brainworks	325
My First C++	250
My First C++	250

7 rows returned in 0.00 seconds CSV Export

(d) To increase the price of all books of first publisher by 50.

UPDATE BOOKS SET Price = Price + 50 WHERE Publishers='First Publ.';

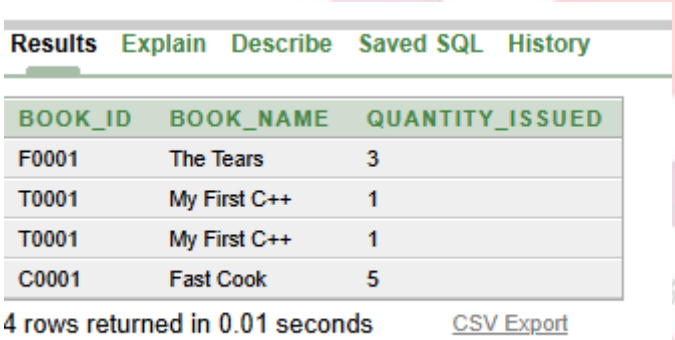



```
3 row(s) updated.

0.01 seconds
```

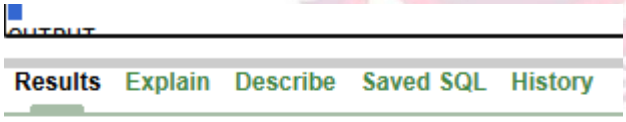
(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).

SELECT B.Book_Id, B.Book_Name, I.Quantity_Issued FROM BOOKS B, ISSUED I WHERE B.Book_Id = I.Book_Id;



(f) To insert a new row in the table Issued following the data: “F0002”,4

INSERT INTO ISSUED VALUES ('F0002',4);




```
1 row(s) inserted.

0.00 seconds
```

(g) Give the output of the following queries based on the above tables:

(i) SELECT COUNT(DISTINCT Publishers) FROM Books;

SELECT COUNT(DISTINCT Publishers) FROM BOOKS;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPUBLISHERS)
3

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

SELECT SUM(Price) FROM BOOKS WHERE Qty > 5;

Results Explain Describe Saved SQL History

SUM(PRICE)
1650

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;

SELECT Book_Name, Author_Name FROM BOOKS WHERE Price < 500;

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME
My First C++	Brian & Brooke
My First C++	Brian & Brooke
C++ Brainworks	A. W. Rossaine
Fast Cook	LataKapoor

4 rows returned in 0.00 seconds CSV Export

(iv) SELECT COUNT (*) FROM Books;

SELECT COUNT(*) FROM BOOKS;

Results Explain Describe Saved SQL History

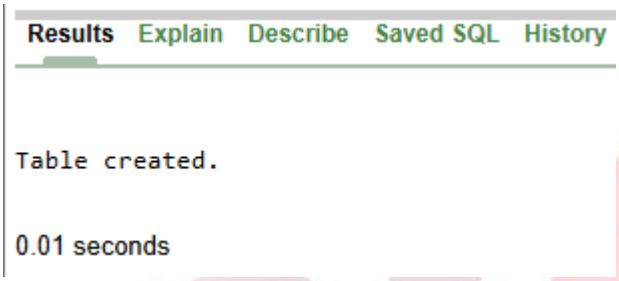
COUNT(*)
7

1 rows returned in 0.00 seconds CSV Export

10. Consider the following tables CARDEN and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

CREATE TABLE CARDEN(Ccode INT, CarName VARCHAR(20), Make VARCHAR(20), Color VARCHAR(10), Capacity INT, Charges INT);



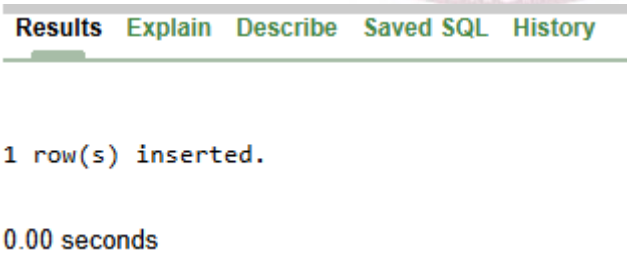
INSERT INTO CARDEN VALUES(501,'A-star','Suzuki','RED',3,14);

INSERT INTO CARDEN VALUES(503,'Indigo','Tata','SILVER',3,12);

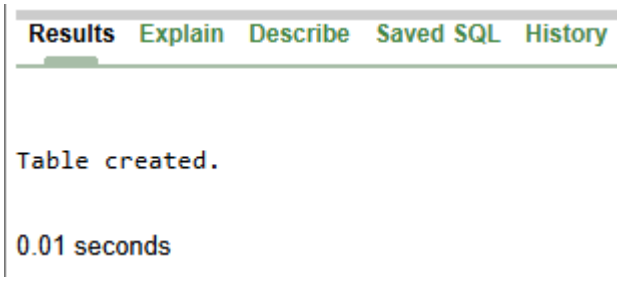
INSERT INTO CARDEN VALUES(502,'Innova','Toyota','WHITE',7,15);

INSERT INTO CARDEN VALUES(509,'SX4','Suzuki','SILVER',4,14);

INSERT INTO CARDEN VALUES(510,'C-Class','Mercedes','RED',4,35);



CREATE TABLE CUSTOMER(CCode INT, CName VARCHAR(30), Ccode_CarCode INT);

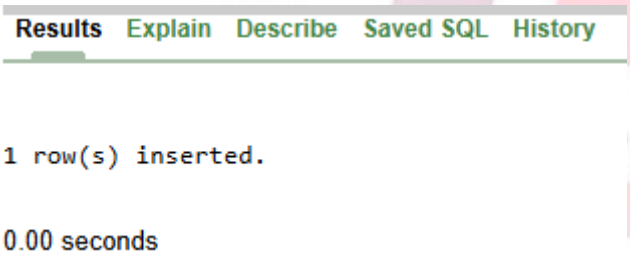


INSERT INTO CUSTOMER VALUES(1001,'HamantSahu',501);

INSERT INTO CUSTOMER VALUE(1002,'Raj Lal',509);

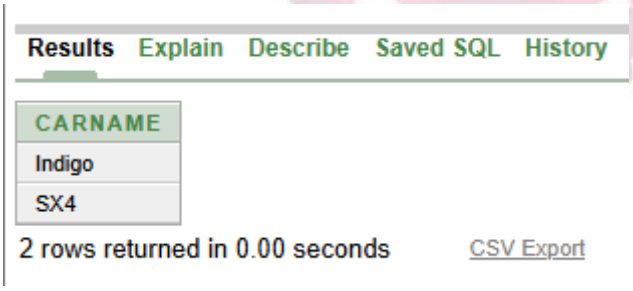
INSERT INTO CUSTOMER VALUES(1003,'Feroja Shah',503);

INSERT INTO CUSTOMER VALUES(1004,'Ketan Dhal',502);



(i) To display the name of all the SILVER colored cars.

SELECT CarName FROM CARDEN WHERE Color='SILVER';



(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;

Results Explain Describe Saved SQL History

CARNAME	MAKE	CAPACITY
Innova	Toyota	7
C-Class	Mercedes	4
SX4	Suzuki	4
Indigo	Tata	3
A-star	Suzuki	3

5 rows returned in 0.00 seconds CSV Export

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

SELECT MAX(Charges) FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.00 seconds CSV Export

(iv) To display the customer name and the corresponding name of the cards hired by them.

SELECT CName, CarName FROM CUSTOMER, CARDE WHERE CUSTOMER.Ccode_CarCode = CARDEN.Ccode;

Results Explain Describe Saved SQL History

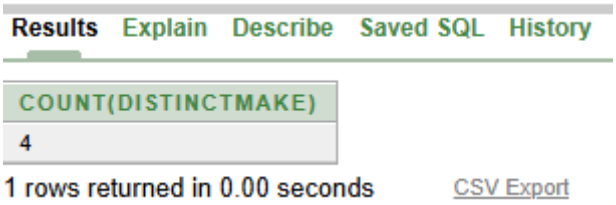
CNAME	CARNAME
HamantSahu	A-star
Feroja Shah	Indigo
Ketan Dhal	Innova
Raj Lal	SX4

4 rows returned in 0.00 seconds CSV Export

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

SELECT COUNT(DISTINCT Make) FROM CARDEN;



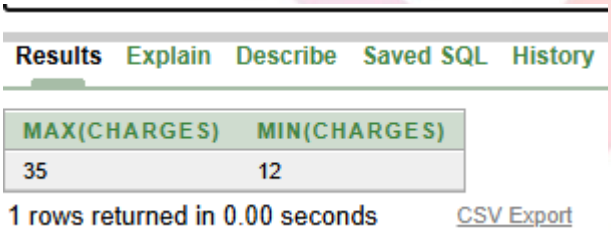
Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

SELECT MAX(Charges), MIN(Charges) FROM CARDEN;



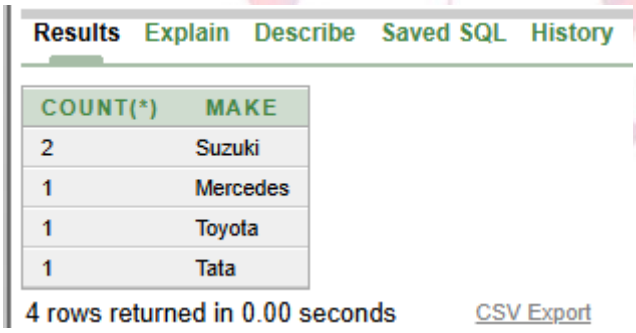
Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (*), Make FROM CARDEN;

SELECT COUNT(*), Make FROM CARDEN GROUP BY Make;



Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.00 seconds CSV Export

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

SELECT CarNam FROM CARDEN WHERE Capacity = 4;

Results Explain Describe Saved SQL History

CARNAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

11. Consider the following tables EMPLOYEE and SALGRADE and answer (a) and (b) parts of this question:

CREATE TABLE EMPLOYEE(ECODE INT, NAME VARCHAR(30), DESIG VARCHAR(20), SGRADE VARCHAR(5), DOJ DATE, DOB DATE);

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

INSERT INTO EMPLOYEE VALUES(101,'AbdulAhmad','EXECUTIVE','S03','23-MAR-2003','13-JAN-1980');

INSERT INTO EMPLOYEE VALUES(102,'RaviChander','HEAD-IT','S02','12-FEB-2010','22-JUL-1987');

INSERT INTO EMPLOYEE VALUES(103,'JohnKen','Receptionist','S03','24-JUN-2009','24-FEB-1983');

INSERT INTO EMPLOYEE VALUES(105,'NazarAmeen','GM','S02','11-AUG-2006','03-MAR-1984');

INSERT INTO EMPLOYEE VALUE(108,'PriyamSen','CEO','S01','29-DEC-2004','19-JAN-1982');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

CREATE TABLE SALGRADE (SGRADE VARCHAR(5), SALARY INT, HRA INT);

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

INSERT INTO SALGRADE VALUES ('S01',56000,18000);

INSERT INTO SALGRADE VALUES('S02',32000,12000);

INSERT INTO SALGRADE VALUES('S03',24000,8000);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

SELECT * FROM EMPLOYEE ORDER BY DOJ DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
102	Ravi Chander	HEAD-IT	S02	12-FEB-10	22-JUL-87
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	NazarAmeen	GM	S02	11-AUG-06	03-MAR-84
108	PriyamSen	CEO	S01	29-DEC-04	19-JAN-82
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-03	13-JAN-80

5 rows returned in 0.01 seconds CSV Export

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

SELECT NAME, DESIG FROM EMPLOYEE WHERE SGRADE IN ('S02','S03');

Results Explain Describe Saved SQL History

NAME	DESIG
Abdul Ahmad	EXECUTIVE
Ravi Chander	HEAD-IT
John Ken	Receptionist
NazarAmeen	GM

4 rows returned in 0.01 seconds CSV Export

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between ’09-FEB-2006’ and ’08-AUG-2009’.

SELECT * FROM EMPLOYEE WHERE DOJ BETWEEN '09-FEB-2006' AND '08-AUG-2009';

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	NazarAmeen	GM	S02	11-AUG-06	03-MAR-84

2 rows returned in 0.02 seconds CSV Export

(iv) TO add a new row in the EMPLOYEE table with the following data:

109, ‘Harish Roy’, ‘HEAD-IT’, ‘S02’, ’09-SEP-2007’, ’21-APR-1983’.

INSERT INTO EMPLOYEE VALUES (109,'Harish Roy','HEAD-IT','S02','09-SEP-2007','21-APR-1983');

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.00 seconds

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

SELECT COUNT(SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
2	S03
3	S02
1	S01

3 rows returned in 0.00 seconds CSV Export

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

SELECT MIN(DOB), MAX(DOJ) FROM EMPLOYEE;

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
13-JAN-80	12-FEB-10

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S

WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E.SGRADE = S.SGRADE AND E.ECODE < 103;

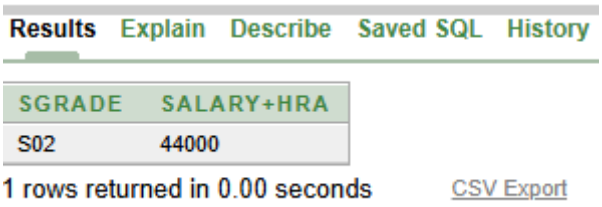
Results Explain Describe Saved SQL History

NAME	SALARY
Ravi Chander	32000
Abdul Ahmad	24000

2 rows returned in 0.01 seconds CSV Export

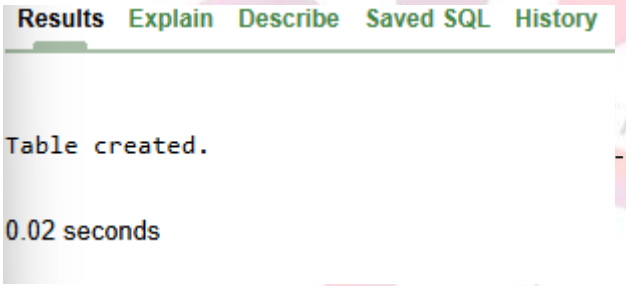
(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = ‘S02’;

SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE='S02';



12. Consider the following tables WORKER and PAYLAVEL and answer (a) and (b) parts of this question

CREATE TABLE WORKER(ECODE INT, NAME VARCHAR(30), DESIGN VARCHAR(20), PLEVEL VARCHAR(5), DOJ DATE,DOB DATE);



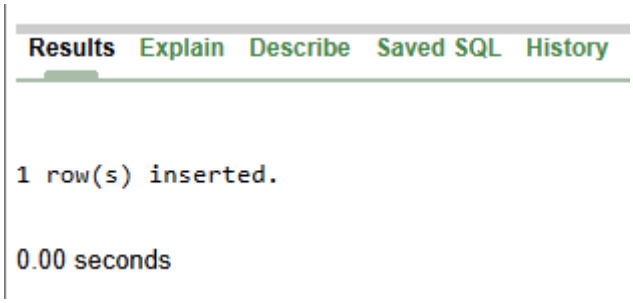
INSERT INTO WORKER VALUES(11,'RadheShyam','Supervisor','P001','13-SEP-2004','23-AUG-1981');

INSERT INTO WORKER VALUES(12,'ChanderNath','Operator','P003','22-FEB-2010','12-JUL-1987');

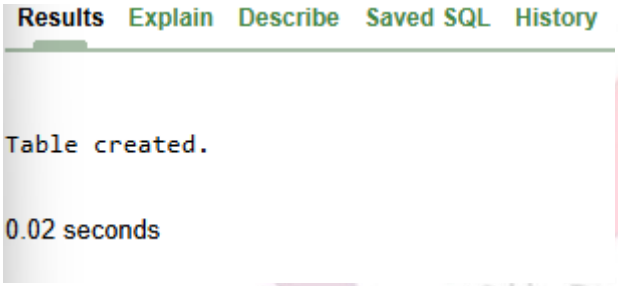
INSERT INTO WORKER VALUES(13,'Fizza','Operator','P003','14-JUN-2009','14-OCT-1983');

INSERT INTO WORKER VALUES(15,'Ahmeen Ahmad','Mechanic','P002','21-AUG-2006','13-MAR-1984');

INSERT INTO WORKER VALUES (18,'Sanya','Clerk','P002','19-DEC-2005','09-JUN-1983');



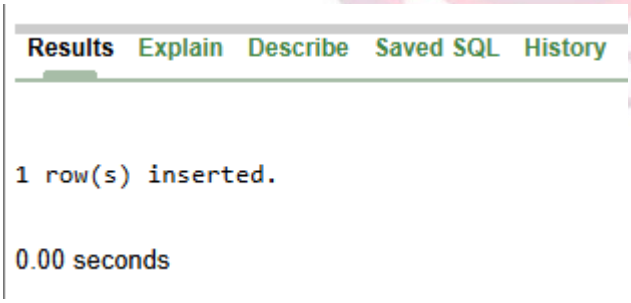
CREATE TABLE PAYLEVEL(PLEVEL VARCHAR(5), PAY INT, ALLOWANCE INT);



INSERT INTO PAYLEVEL VALUES ('P001',26000,12000);

INSERT INTO PAYLEVEL VALUES('P002',22000,10000);

INSERT INTO PAYLEVEL VALUES('P003',12000,6000);



(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.

SELECT * FROM WORKER ORDER BY DOB DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	ChanderNath	Operator	P003	22-FEB-10	12-JUL-87
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84
13	Fizza	Operator	P003	14-JUN-09	14-OCT-83
18	Sanya	Clerk	P002	19-DEC-05	09-JUN-83
11	RadheShyam	Supervisor	P001	13-SEP-04	23-AUG-81

5 rows returned in 0.01 seconds CSV Export

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

SELECT NAME, DESIGN FROM WORKER WHERE PLEVEL IN ('P001','P002');

Results Explain Describe Saved SQL History

NAME	DESIGN
RadheShyam	Supervisor
Ahmeen Ahmad	Mechanic
Sanya	Clerk

3 rows returned in 0.00 seconds CSV Export

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

SELECT FROM WORKER WHERE DOB BETWEEN '19-JAN-1984' AND '18-JAN-1987';

Results Explain Describe Saved SQL History

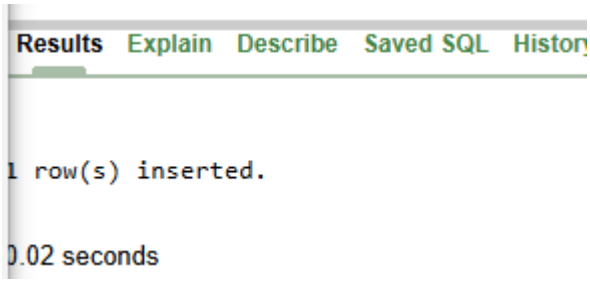
ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84

1 rows returned in 0.00 seconds CSV Export

(iv) To add a new row with the following:

19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

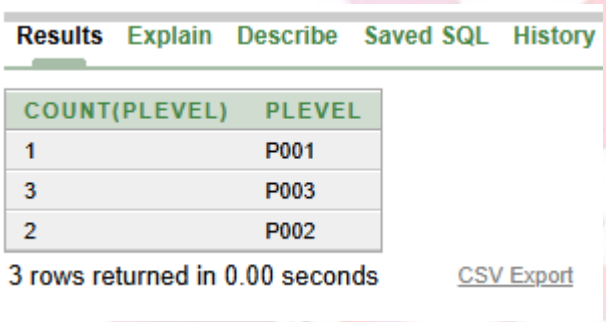
INSERT INTO WORKER VALUES (19,'Daya Kishore','Operator','P003','19-JUN-2008','11-JUL-1984');



(b) Give the output of the following SQL queries:

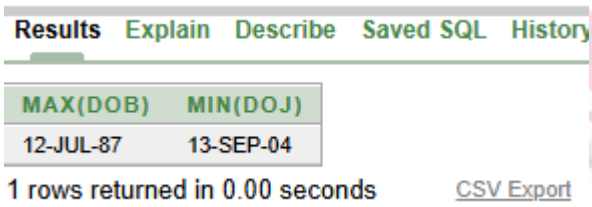
(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

SELECT COUNT(PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;



(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

SELECT MAX(DOB), MIN(DOJ) FROM WORKER;



(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W.PLEVEL = P.PLEVEL AND W.ECODE < 13

Results Explain Describe Saved SQL History

NAME	PAY
RadheShyam	26000
ChanderNath	12000

2 rows returned in 0.00 seconds CSV Export

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL = 'P003';

SELECT PLEVEL, PAY+ALLOWANCE FROM PAYLEVEL WHERE PLEVEL='P003';

Results Explain Describe Saved SQL History

PLEVEL	PAY+ALLOWANCE
P003	18000

1 rows returned in 0.01 seconds CSV Export

13. Consider the following tables STORE and SUPPLIERS and answer (a) and (b) parts of this question:

CREATE TABLE STORE(ItemNo INT, Item VARCHAR(30), Scode INT, Qty INT, Rate INT, LastBuy DATE);

Results Explain Describe Saved SQL History

```
Table created.

0.03 seconds
```

INSERT INTO STORE VALUES (2005,'Sharpener Classic',23,60,8,'31-JUN-09');

INSERT INTO STORE VALUES (2003,'Ball Pen 0.25',22,50,25,'01-FEB-09');

INSERT INTO STORE VALUES (2002,'Gel Pen Premium',21,150,12,'24-FEB-10');

INSERT INTO STORE VALUES (2006,'Gel Pen Classic',21,250,20,'11-MAY-09');

INSERT INTO STORE VALUES (2001,'Eraser Small',22,220,6,'19-JAN-09');

INSERT INTO STORE VALUES (2004,'Eraser Big',22,110,8,'02-DEC-09');

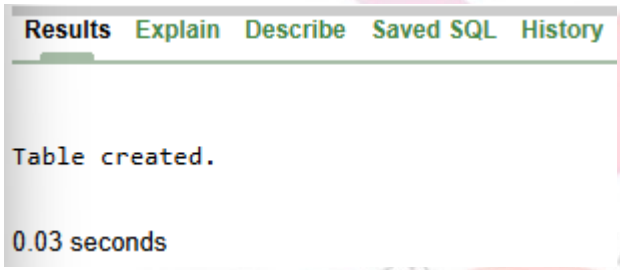
INSERT INTO STORE VALUES (2009,'Ball Pen 0.5',21,180,18,'03-NOV-09');




```
1 row(s) inserted.

0.00 seconds
```

CREATE TABLE SUPPLIERS(Scode INT, Sname VARCHAR(30));



INSERT INTO SUPPLIERS VALUES (21,'Premium Stationers');

INSERT INTO SUPPLIERS VALUES (23,'Soft Plastics');

INSERT INTO SUPPLIERS VALUES (22,'Tetra Supply');




```
1 row(s) inserted.

0.00 seconds
```

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

SELECT * FROM STORE ORDER BY LastBuy ASC;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2006	Gel Pen Classic	21	250	20	11-MAY-09
2005	Sharpener Classic	23	60	8	30-JUN-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09
2004	Eraser Big	22	110	8	02-DEC-09
2002	Gel Pen Premium	21	150	12	24-FEB-10

7 rows returned in 0.00 seconds CSV Export

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

SELECT ItemNo, Item FROM STORE WHERE Rate > 15;

Results Explain Describe Saved SQL History

ITEMNO	ITEM
2003	Ball Pen 0.25
2006	Gel Pen Classic
2009	Ball Pen 0.5

3 rows returned in 0.00 seconds CSV Export

(iii)To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

SELECT * FROM STORE WHERE Scode = 22 OR Qty > 110;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2006	Gel Pen Classic	21	250	20	11-MAY-09
2001	Eraser Small	22	220	6	19-JAN-09
2004	Eraser Big	22	110	8	02-DEC-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

6 rows returned in 0.00 seconds CSV Export

(iv)To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

SELECT Scode, MIN(Rate) FROM STORE GROUP BY Scode;

Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	6
21	12
23	8

3 rows returned in 0.02 seconds CSV Export

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

SELECT COUNT(DISTINCT Scode)FROM STORE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTSCODE)
3

1 rows returned in 0.00 seconds CSV Export

(ii)SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

SELECT Rate * Qty FROM STORE WHERE ItemNo = 2004;

Results Explain Describe Saved SQL History

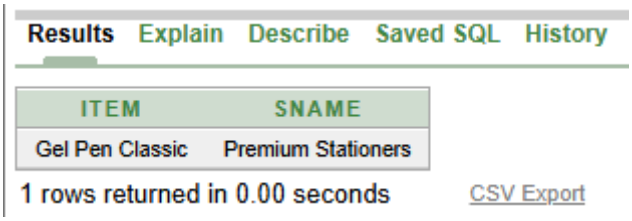
RATE*QTY
880

1 rows returned in 0.00 seconds CSV Export

(iii)SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

SELECT Item, Sname FROM STORE S, SUPPLIERS P WHERE S.Scode = P.Scode

AND ItemNo = 2006;



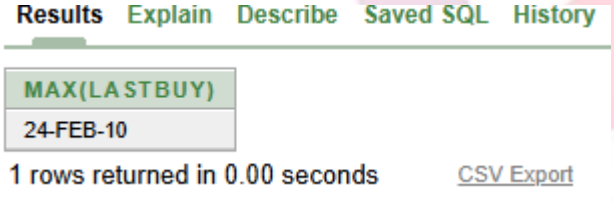
Results Explain Describe Saved SQL History

ITEM	SNAME
Gel Pen Classic	Premium Stationers

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT MAX(LastBuy) FROM STORE;

SELECT MAX(LastBuy) FROM STORE;



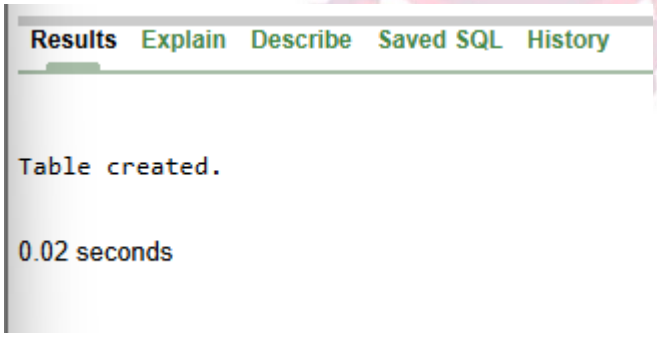
Results Explain Describe Saved SQL History

MAX(LASTBUY)
24-FEB-10

1 rows returned in 0.00 seconds CSV Export

14. Consider the following table GARMENT and FABRIC,

CREATE TABLE GARMENT(GCODE INT, DESCRIPTION VARCHAR(30), PRICE INT, FCODE VARCHAR(5), READYDATE DATE);



Results Explain Describe Saved SQL History

Table created.

0.02 seconds

INSERT INTO GARMENT VALUES(10023,'Pencil Skirt',1150,'F03','19-DEC-08');

INSERT INTO GARMENT VALUES(10001,'Formal Shirt',1250,'F01','12-JAN-08');

INSERT INTO GARMENT VALUES(10012,'Informal Shirt',1550,'F02','06-JUN-08');

INSERT INTO GARMENT VALUES(10024,'Baby Top',750,'F03','07-APR-07');

INSERT INTO GARMENT VALUES(10090,'Tulip Skirt',850,'F02','31-MAR-07');

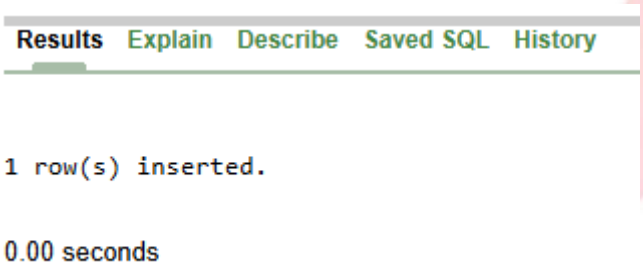
INSERT INTO GARMENT VALUES(10019,'Evening Gown',850,'F03','06-JUN-08');

INSERT INTO GARMENT VALUES(10009,'Informal Pant',1500,'F02','20-OCT-08');

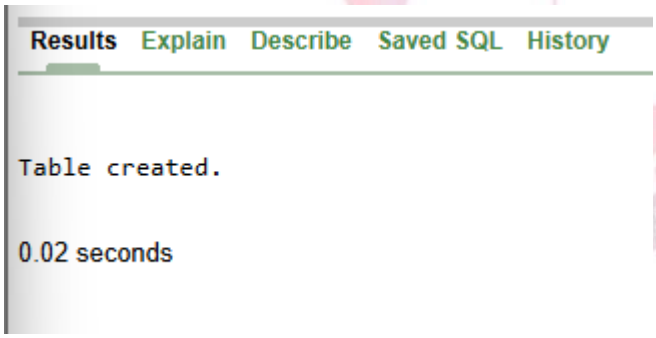
INSERT INTO GARMENT VALUES(10007,'Formal Pant',1350,'F01','09-MAR-08');

INSERT INTO GARMENT VALUES(10020,'Frock',850,'F04','09-SEP-07');

INSERT INTO GARMENT VALUES(10089,'Slacks',750,'F03','20-OCT-08');



CREATE TABLE FABRIC(FCODE VARCHAR(5), TYPE VARCHAR(20));



INSERT INTO FABRIC VALUES('F04','Polyester');

INSERT INTO FABRIC VALUES('F02','Cotton');

INSERT INTO FABRIC VALUES('F03','Silk');

INSERT INTO FABRIC VALUES('F01','Terelene');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

Write SQL commands for the statements (i) to (iv) and give outputs for the SQL queries (v) to (viii).

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

SELECT GCODE, DESCRIPTIONFROM GARMENTORDER BY GCODE DESC;

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
10090	Tulip Skirt
10089	Slacks
10024	Baby Top
10023	Pencil Skirt
10020	Frock
10019	Evening Gown
10012	Informal Shirt
10009	Informal Pant
10007	Formal Pant
10001	Formal Shirt

10 rows returned in 0.00 seconds CSV Export

(ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).

SELECT * FROM GARMENT WHERE READYDATE BETWEEN '08-DEC-07' AND '16-JUN-08';

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Informal Shirt	1550	F02	06-JUN-08
10019	Evening Gown	850	F03	06-JUN-08
10007	Formal Pant	1350	F01	09-MAR-08

4 rows returned in 0.00 seconds CSV Export

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

SELECT AVG(PRICE FROM GARMENT WHERE FCODE='F03';

Results Explain Describe Saved SQL History

AVG(PRICE)
875

1 rows returned in 0.00 seconds CSV Export

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

SELECT FCODE,MAX(PRICE),MIN(PRICE)FROM GARMENTGROUP BY FCODE;

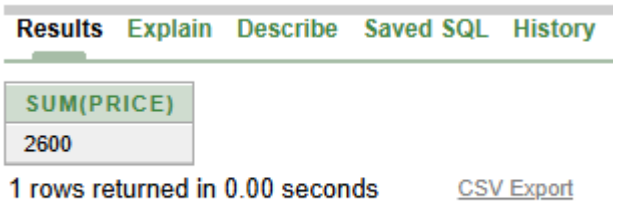
Results Explain Describe Saved SQL History

FCODE	MAX(PRICE)	MIN(PRICE)
F02	1550	850
F04	850	850
F03	1150	750
F01	1350	1250

4 rows returned in 0.02 seconds CSV Export

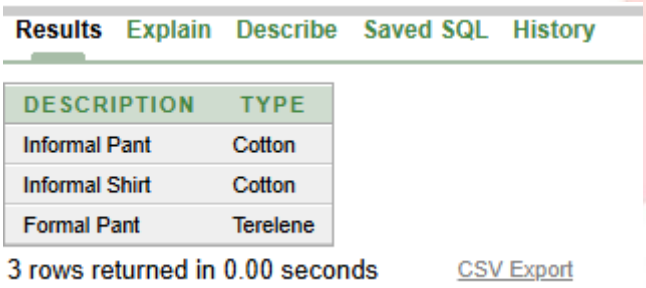
(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = ‘F01’;

SELECT SUM(PRICE) FROM GARMENT WHERE FCODE='F01';



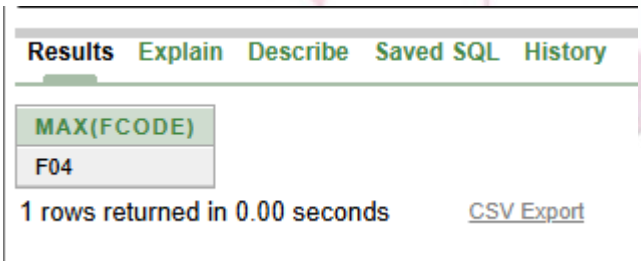
(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT.FCODE = FABRIC.FCODE AND GARMENT.PRICE >= 1260;



(vii) SELECT MAX (FCODE) FROM FABRIC;

SELECT MAX(FCODE) FROM FABRIC;



(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

SELECT COUNT(DISTINCT PRICE) FROM GARMENT;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPRICE)
7

1 rows returned in 0.00 seconds CSV Export

15. Given the following tables for a database FURNITURE : Write SQL command for (a) to (f) and write the outputs for (g) on the bases of tables **FURNITURE** AND **ARRIVALS.**

CREATE TABLEFURNITURE(No INT,ItemName VARCHAR(30),Type VARCHAR(20), DateOfStock DATE, Price INT, Discount INT);

Results Explain Describe Saved SQL History

```
Table created.

0.00 seconds
```

INSERT INTO FURNITURE VALUES (1,'White lotus','Double Bed','23-FEB-02',30000,25);

INSERT INTO FURNITURE VALUES (2,'Pink feather','Baby cot','20-JAN-02',7000,20);

INSERT INTO FURNITURE VALUES (3,'Dolphin','Baby cot','19-FEB-02',9500,20);

INSERT INTO FURNITURE VALUES(4,'Decent','Office Table','01-JAN-02',25000,30);

INSERT INTO FURNITURE VALUES(5,'Comfort zone','Double Bed','12-JAN-02',25000,25);

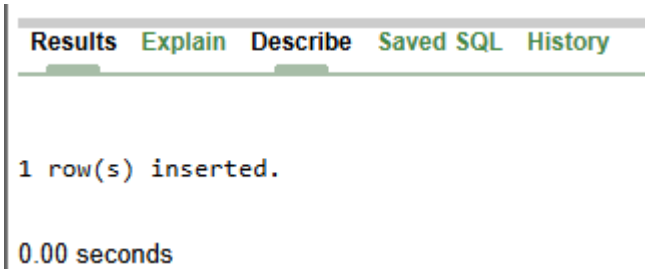
INSERT INTO FURNITURE VALUES(6,'Donald','Baby cot','24-FEB-02',6500,15);

INSERT INTO FURNITURE VALUES(7,'Royal Finish','Office Table','20-FEB-02',18000,30);

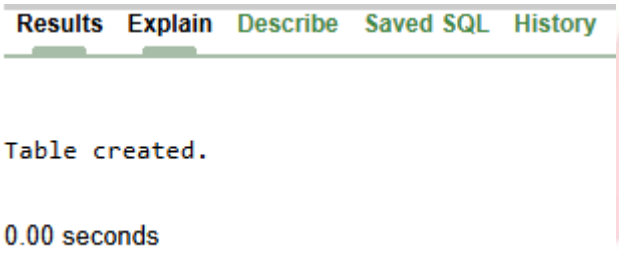
INSERT INTO FURNITURE VALUES(8,'Royal tiger','Sofa','22-FEB-02',31000,30);

INSERT INTO FURNITURE VALUES(9,'Econo sitting','Sofa','13-DEC-01',9500,25);

INSERT INTO FURNITURE VALUES(10,'Eating paradise','Dining Table','19-FEB-02',11500,25);



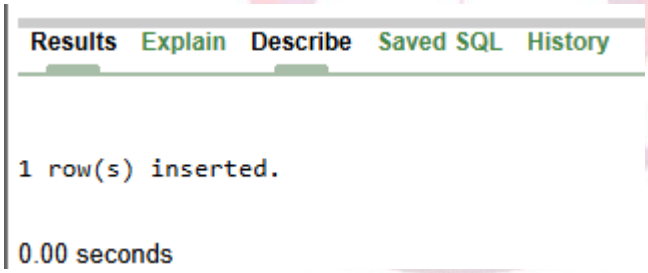
CREATE TABLE ARRIVALS(No INT ItemName VARCHAR(30), Type VARCHAR(20), DateOfStock DATE, Price INT, Discount INT);



INSERT INTO ARRIVALS VALUES (11,'Wood Comfort','Double Bed','23-MAR-03',25000,25);

INSERT INTO ARRIVALS VALUES(12,'Old Fox','Sofa','20-FEB-03',17000,20);

INSERT INTO ARRIVALS VALUES(13,'Micky','Baby cot','21-FEB-02',7500,15);



(a) To show all information about the baby cots from the FURNITURE table.

SELECT * FROM FURNITURE WHERE Type='Baby cot';

Results Explain Describe Saved SQL History

NO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	Pink feather	Baby cot	20-JAN-02	7000	20
3	Dolphin	Baby cot	19-FEB-02	9500	20
6	Donald	Baby cot	24-FEB-02	6500	15

3 rows returned in 0.00 seconds CSV Export

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

SELECT ItemName FROM FURNITURE WHERE Price > 15000;

Results Explain Describe Saved SQL History

ITEMNAME
White lotus
Decent
Comfort zone
Royal Finish
Royal tiger

5 rows returned in 0.00 seconds CSV Export

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

SELECT ItemName, Type FROM FURNITURE WHERE DateOfStock < '22-JAN-02' ORDER BY ItemName DESC;

Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Pink feather	Baby cot
Econo sitting	Sofa
Decent	Office Table
Comfort zone	Double Bed

4 rows returned in 0.00 seconds CSV Export

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

SELECT ItemName, DateOfStock FROM FURNITUR WHERE Discount > 25;

Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Decent	01-JAN-02
Royal Finish	20-FEB-02
Royal tiger	22-FEB-02

3 rows returned in 0.00 seconds CSV Export

(e) To count the number of items, whose TYPE is “Sofa” from FURNITURE table.

SELECT COUNT(*) FROM FURNITURE WHERE Type='Sofa';

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

(f) To insert a new row in the ARRIVALS table with the following data: 14, “Velvet touch”, Double bed”, {25/03/03}, 25000, 30

INSERT INTO ARRIVALS VALUES (14,'Velvet touch','Double Bed','25-MAR-03',25000,30);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

(i) Select COUNT(distinct TYPE) from FURNITURE;

```
SELECT COUNT(DISTINCT Type) FROM FURNITURE;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTTYPE)
5

1 rows returned in 0.02 seconds CSV Export

(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

```
SELECT MAX(Discount FROM FURNITURE;
```

Results Explain Describe Saved SQL History

MAX(DISCOUNT)
30

1 rows returned in 0.00 seconds CSV Export

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = “Baby cot”;

```
SELECT AVG(Discount) FROM FURNITURE WHERE Type='Baby cot';
```

Results Explain Describe Saved SQL History

AVG(DISCOUNT)
18.333333333333333333333333333333333333

1 rows returned in 0.00 seconds CSV Export

(iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

```
SELECT SUM(Price)FROM FURNITUREWHERE DateOfStock < '12-FEB-02';
```

Results Explain Describe Saved SQL History

SUM(PRICE)
66500

1 rows returned in 0.00 seconds CSV Export

16. **Given the following COVID19 relation, write SQL commands for questions (a) to (f).**

CREATE TABLE COVID19(PatientID INT, PatientName VARCHAR(30), State VARCHAR(30), Age INT, Status VARCHAR(20), Vaccinated VARCHAR(5));

Results Explain Describe Saved SQL History

```
Table created.

0.00 seconds
```

INSERT INTO COVID19 VALUES(101,'Arun','Tamil Nadu',35,'Recovered','Yes');

INSERT INTO COVID19 VALUES(102,'Priya','Kerala',42,'Active','Yes');

INSERT INTO COVID19 VALUES(103,'Karthik','Karnataka',28,'Recovered','No');

INSERT INTO COVID19 VALUES(104,'Divya','Tamil Nadu',31,'Active','Yes');

INSERT INTO COVID19 VALUES(105,'Rahul','Andhra Pradesh',55,'Deceased','No');

INSERT INTO COVID19 VALUES(106,'Meena','Kerala',24,'Recovered','Yes');

INSERT INTO COVID19 VALUES(107,'Suresh','Tamil Nadu',48,'Active','No');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

a) Display all details of patients from **Tamil Nadu**.

SELECT * FROM COVID19 WHERE State='Tamil Nadu';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	35	Recovered	Yes
104	Divya	Tamil Nadu	31	Active	Yes
107	Suresh	Tamil Nadu	48	Active	No

3 rows returned in 0.00 seconds CSV Export

b) Display the names of patients whose **Status** is **Recovered**.

SELECT PatientName FROM COVID19 WHERE Status='Recovered';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Karthik
Meena

3 rows returned in 0.02 seconds CSV Export

c) Display **PatientName** and **Age** in ascending order of **Age**.

SELECT PatientName, Age FROM COVID19 ORDER BY Age ASC;

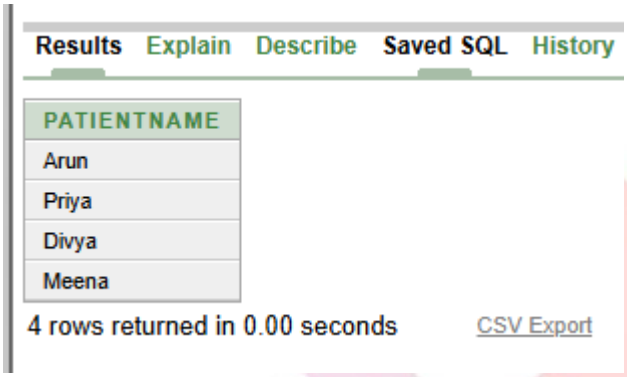
Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Meena	24
Karthik	28
Divya	31
Arun	35
Priya	42
Suresh	48
Rahul	55

7 rows returned in 0.00 seconds CSV Export

d) Display the names of patients who are **Vaccinated = 'Yes'**.

SELECT PatientName FROM COVID19 WHERE Vaccinated='Yes';



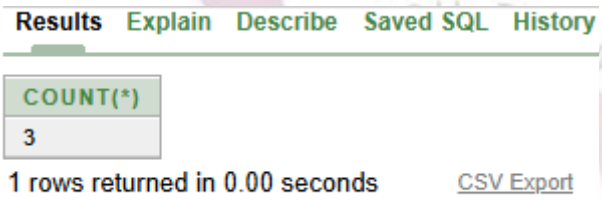
Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Priya
Divya
Meena

4 rows returned in 0.00 seconds CSV Export

e) Count the number of patients whose status is **Active**.

SELECT COUNT(*) FROM COVID19 WHERE Status='Active';



Results Explain Describe Saved SQL History

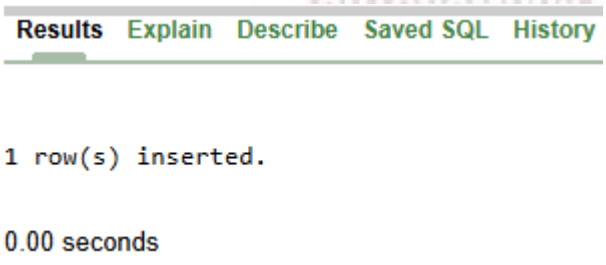
COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

f) Insert the following record into the **COVID19** table:

108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

INSERT INTO COVID19 VALUES (108,'Anitha','Puducherry',39,'Recovered','Yes');



Results Explain Describe Saved SQL History

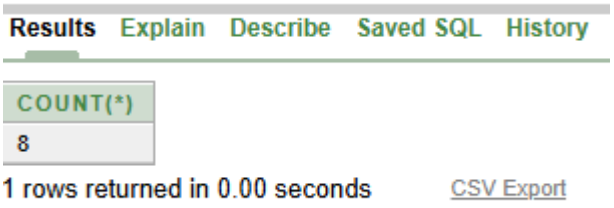
```
1 row(s) inserted.

0.00 seconds
```

(g) Write the output of the following SQL commands.

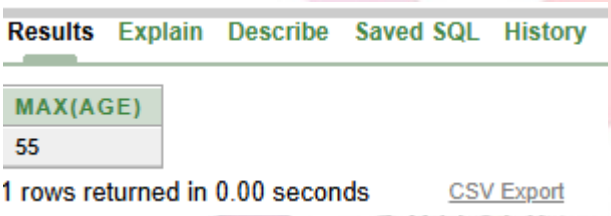
a. SELECT COUNT(*) FROM COVID19;

SELECT COUNT(*) FROM COVID19;



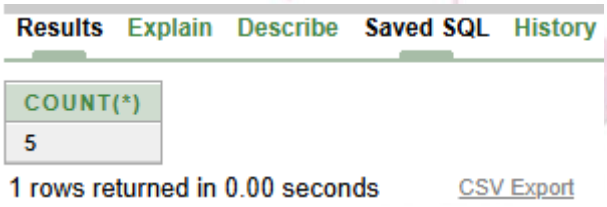
b. SELECT MAX(Age) FROM COVID19;

SELECT MAX(Age) FROM COVID19;



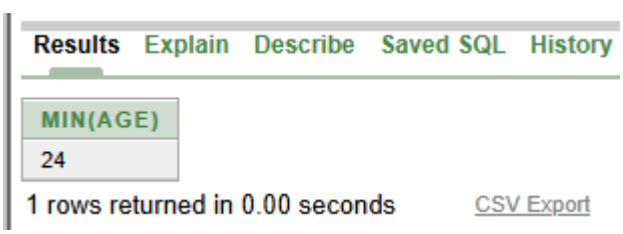
c. SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';

SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';



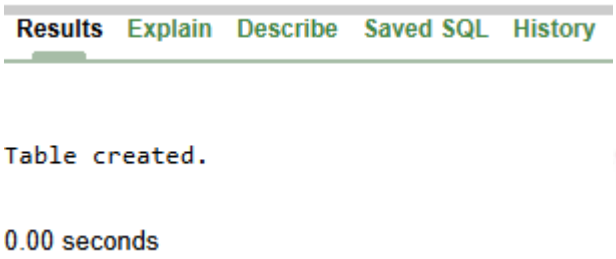
d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

SELECT MIN(Age) FROM COVID19 WHERE Status='Recovered';



17. **Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).**

CREATE TABLE HOUSEPRICE(HouseID INT,Location VARCHAR(30),Bedrooms INT, AreaSqft INT, Price INT, HouseType VARCHAR(20));



INSERT INTO HOUSEPRICE VALUES(201,'Chennai',2,1100,5200000,'Apartment');

INSERT INTO HOUSEPRICE VALUES(202,'Coimbatore',3,1450,6800000,'Villa');

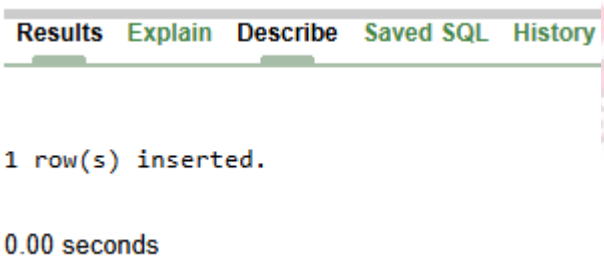
INSERT INTO HOUSEPRICE VALUES(203,'Madurai',2,980,4100000,'Apartment');

INSERT INTO HOUSEPRICE VALUES(204,'Chennai',4,2200,9800000,'Villa');

INSERT INTO HOUSEPRICE VALUES(205,'Trichy',3,1650,7200000,'Independent');

INSERT INTO HOUSEPRICE VALUES(206,'Salem',2,1200,5000000,'Apartment');

INSERT INTO HOUSEPRICE VALUES(207,'Erode',3,1800,7600000,'Independent');



a) Display all houses located in **Chennai**.

SELECT * FROM HOUSEPRICE WHERE Location='Chennai';

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1100	5200000	Apartment
204	Chennai	4	2200	9800000	Villa

2 rows returned in 0.00 seconds CSV Export

b) Display the **Location** and **Price** of houses having **3 Bedrooms**.

SELECT Location, Price FROM HOUSEPRICE WHERE Bedrooms=3;

Results Explain Describe Saved SQL H

LOCATION	PRICE
Coimbatore	6800000
Trichy	7200000
Erode	7600000

3 rows returned in 0.00 seconds CSV E

c) Display **Location**, **AreaSqft**, and **Price** in descending order of **Price**.

SELECT Location, AreaSqft, Price FROM HOUSEPRICE ORDER BY Price DESC;

Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2200	9800000
Erode	1800	7600000
Trichy	1650	7200000
Coimbatore	1450	6800000
Chennai	1100	5200000
Salem	1200	5000000
Madurai	980	4100000

7 rows returned in 0.00 seconds CSV Export

d) Display the details of houses costing more than **₹70,00,000**.

SELECT * FROM HOUSEPRICE WHERE Price > 7000000;

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1650	7200000	Independent
207	Erode	3	1800	7600000	Independent

3 rows returned in 0.00 seconds CSV Export

e) Count the number of **Apartments**.

SELECT COUNT(*) FROM HOUSEPRICE WHERE HouseType='Apartment';

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

f) Insert the following record:

208, 'Vellore', 3, 1700, 7300000, 'Villa'

INSERT INTO HOUSEPRICE VALUES (208,'Vellore',3,1700,7300000,'Villa');

Results Explain Describe Saved SQL History

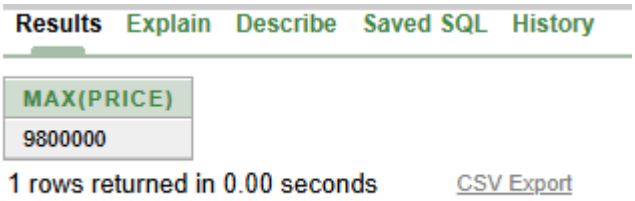
```
1 row(s) inserted.
```

0.00 seconds

(g) Write the output of the following SQL commands.

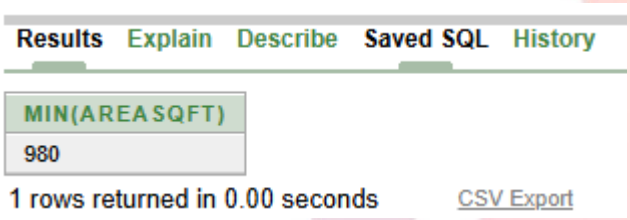
(i) SELECT MAX(Price)FROM HOUSEPRICE;

SELECT MAX(Price) FROM HOUSEPRICE;



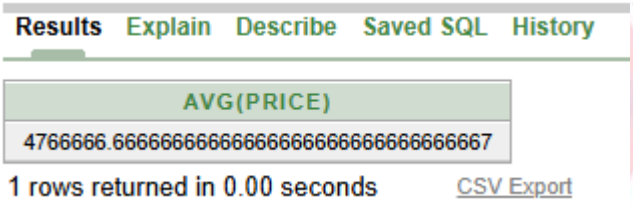
(ii) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

SELECT MIN(AreaSqft) FROM HOUSEPRICE;



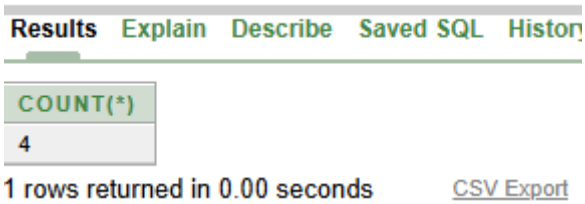
(iii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

SELECT AVG(Price) FROM HOUSEPRICE WHERE HouseType='Apartment';



(iv) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

SELECT COUNT(*) FROM HOUSEPRICE WHERE Bedrooms=3;



18. **Given the following CANCERDATA relation, write SQL commands for questions (a) to (f).**

CREATE TABLE CANCERDATA(PatientID INT,PatientName VARCHAR(30), CancerType VARCHAR(20), Age INT, Stage VARCHAR(5) SurvivalYears INT);

Results Explain Describe Saved SQL History

```
Table created.

0.03 seconds
```

INSERT INTO CANCERDATA VALUES(301,'Kavya','Breast',45,'II',8);

INSERT INTO CANCERDATA VALUES(302,'Arun','Lung',60,'III',4);

INSERT INTO CANCERDATA VALUES(303,'Divya','Breast',38,'I',10);

INSERT INTO CANCERDATA VALUES(304,'Hari','Colon',52,'II',6);

INSERT INTO CANCERDATA VALUES(305,'Meena','Lung',57,'IV',2);

INSERT INTO CANCERDATA VALUES(306,'Ravi','Prostate',64,'III',5);

INSERT INTO CANCERDATA VALUES(307,'Priya','Breast',41,'II',9);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

a) Display all patients diagnosed with **Breast Cancer**.

SELECT * FROM CANCERDATA WHERE CancerType='Breast';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Kavya	Breast	45	II	8
303	Divya	Breast	38	I	10
307	Priya	Breast	41	II	9

3 rows returned in 0.00 seconds CSV Export

b) Display the names of patients whose **Stage** is **III**.

SELECT PatientName FROM CANCERDATA WHERE Stage='III';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Ravi

2 rows returned in 0.00 seconds CSV Export

c) Display **PatientName** and **Age** in ascending order of **Age**.

SELECT PatientName, Age FROM CANCERDATA ORDER BY Age ASC;

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Divya	38
Priya	41
Kavya	45
Hari	52
Meena	57
Arun	60
Ravi	64

7 rows returned in 0.00 seconds CSV Export

d) Display the names of patients having **SurvivalYears greater than 5**.

SELECT PatientName FROM CANCERDATA WHERE SurvivalYears > 5;

Results Explain Describe Saved SQL History

PATIENTNAME
Kavya
Divya
Hari
Priya

4 rows returned in 0.00 seconds CSV Export

e) Count the number of patients diagnosed with **Lung Cancer**.

SELECT COUNT(*) FROM CANCERDATA WHERE CancerType='Lung';

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

f) Insert the following record:

308, 'Anand', 'Leukemia', 50, 'II', 7

INSERT INTO CANCERDATA VALUES (308,'Anand','Leukemia',50,'II',7);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(g) Write the output of the following SQL commands.

(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

Results Explain Describe Saved SQL History

COUNT(DISTINCTCANCERTYPE)
5

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;

SELECT MAX(SurvivalYears) FROM CANCERDATA;

Results Explain Describe Saved SQL History

MAX(SURVIVALYEARS)
10

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

Results Explain Describe Saved SQL History

AVG(AGE)
41.333333333333333333333333333333333333

1 rows returned in 0.01 seconds CSV Export

(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

SELECT MIN(Age)FROM CANCERDATA WHERE Stage='II';

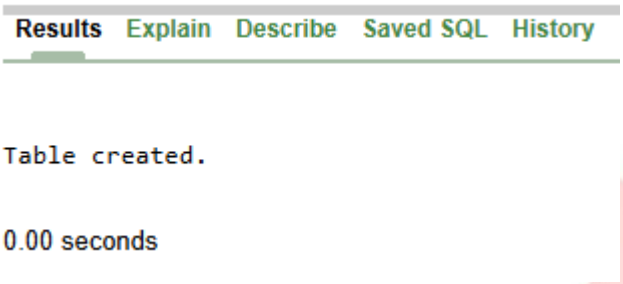
Results Explain Describe Saved SQL History

MIN(AGE)
41

1 rows returned in 0.00 seconds CSV Export

19. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables ITEMS and TRADERS

CREATE TABLE ITEMS(CODE INT, INAME VARCHAR(30), QTY INT, PRICE INT, COMPANY VARCHAR(30), TCODE VARCHAR(5));



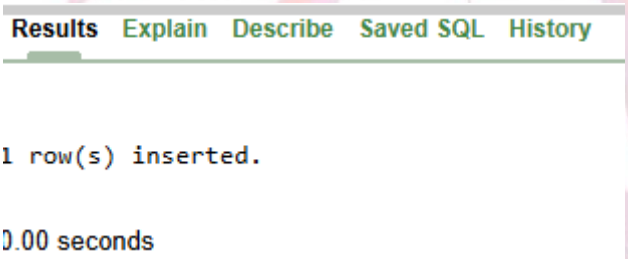
INSERT INTO ITEMS VALUES(1001,'DIGITAL PAD12i',120,11000,'XENITA','T01');

INSERT INTO ITEMS VALUES(1006,'LED SCREEN 40',70,38000,'SANTORA','T02');

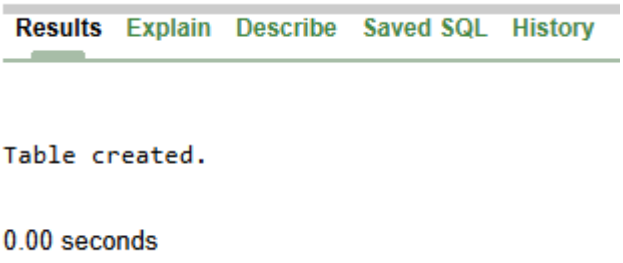
INSERT INTO ITEMS VALUES(1004,'CAR GPS SYSTEM',50,21500,'GEOKNOW','T01');

INSERT INTO ITEMS VALUES(1003,'DIGITAL CAMERA 12X',160,8000,'DIGICLICK','T02');

INSERT INTO ITEMS VALUES(1005,'PEN DRIVE 32 GB',600,1200,'STOREHOME','T03');



CREATE TABLE TRADERS(TCODEVARCHAR(5),TNAMEVARCHAR(30),CITY VARCHAR(20));



INSERT INTO TRADERS VALUES('T01','ELECTRONIC SALES','MUMBAI');

INSERT INTO TRADERS VALUES('T03','BUSY STORE CORP','DELHI');

INSERT INTO TRADERS VALUES('T02','DISP HOUSE INC','CHENNAI');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To display the details of all the items in ascending order of item names (i.e. INAME).

SELECT * FROM ITEMS ORDER BY INAME ASC;

Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T02
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T03

5 rows returned in 0.00 seconds CSV Export

(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;

Results Explain Describe Saved SQL History

INAME	PRICE
DIGITAL PAD12i	11000
CAR GPS SYSTEM	21500

2 rows returned in 0.01 seconds CSV Export

(c) To display the number of items, which are traded by each trader. The expected output of this query should be

T01 2

T02 2

T03 1

SELECT TCODE, COUNT(*) FROM ITEMS GROUP BY TCODE;

Results Explain Describe Saved SQL History

TCODE	COUNT(*)
T01	2
T03	1
T02	2

3 rows returned in 0.01 seconds CSV Export

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY > 150;

Results Explain Describe Saved SQL History

PRICE	INAME	QTY
8000	DIGITAL CAMERA 12X	160
1200	PEN DRIVE 32 GB	600

2 rows returned in 0.00 seconds CSV Export

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

SELECT TNAME FROM TRADERS WHERE CITY IN ('DELHI','MUMBAI');

Results Explain Describe Saved SQL History

TNAME
ELECTRONIC SALES
BUSY STORE CORP

2 rows returned in 0.00 seconds CSV Export

(f) To display the name of the companies and the bane of the items in descending order of company names.

SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	INAME
XENITA	DIGITAL PAD12i
STOREHOME	PEN DRIVE 32 GB
SANTORA	LED SCREEN 40
GEOKNOW	CAR GPS SYSTEM
DIGICLICK	DIGITAL CAMERA 12X

5 rows returned in 0.00 seconds CSV Export

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

SELECT MAX(PRICE), MIN(PRICE) FROM ITEMS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.00 seconds CSV Export

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

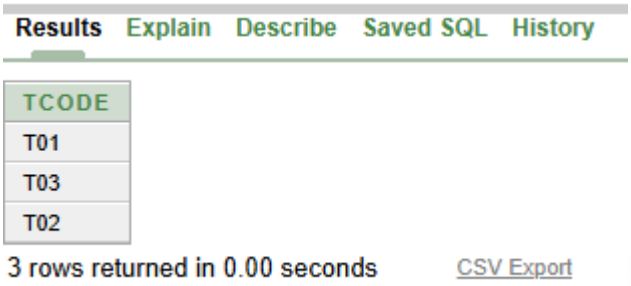
Results Explain Describe Saved SQL History

AMOUNT
1075000

1 rows returned in 0.00 seconds CSV Export

(g3) SELE CT DISTINCT TCODE FROM ITEMS;

SELECT DISTINCT TCODE FROM ITEMS;



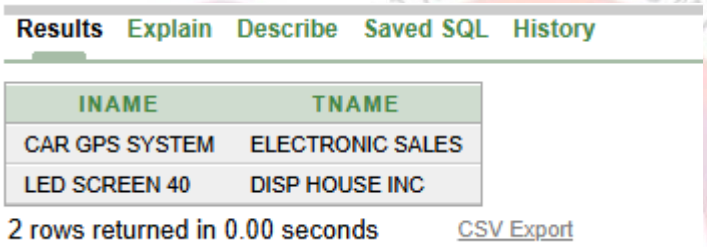
Results Explain Describe Saved SQL History

TCODE
T01
T03
T02

3 rows returned in 0.00 seconds CSV Export

(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;



Results Explain Describe Saved SQL History

INAME	TNAME
CAR GPS SYSTEM	ELECTRONIC SALES
LED SCREEN 40	DISP HOUSE INC

2 rows returned in 0.00 seconds CSV Export

TRUTH-TOIL-TRIUMPH
AUTONOMOUS